LLD Handbook Study Edition — Vol 2: Interview Approach and Case Studies I

OOP · UML · SOLID · Clean Code · Refactoring · Patterns · Concurrency · 45 case studies in Java — every topic, readable length

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It (Study Edition)**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 14 — Case Studies: Everyday Objects (Study Edition)**
   - 14.1 Design a Parking Lot
   - 14.2 Design a Vending Machine
   - 14.3 Design a Coffee Vending Machine
   - 14.4 Design an ATM
   - 14.5 Design an Elevator System
   - 14.6 Design a Traffic Signal Control System
   - 14.7 Design a Shipping Locker System (Amazon Locker)
   - Chapter summary
3. **Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)**
   - Why booking problems are interview favourites
   - 15.1 Design a Library Management System
   - 15.2 Design a Movie Ticket Booking System (BookMyShow)
   - 15.3 Design a Concert Ticket Booking System
   - 15.4 Design a Hotel Management System
   - Chapter summary
4. **Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition**
   - Design a Car Rental System
   - Design a Restaurant Management System
   - Design an Airline Management System
   - Design a Course Registration System
   - Design a Meeting Scheduler
   - Chapter summary
5. **Chapter 17 — Case Studies: Games & Rules — Study Edition**
   - Design Blackjack and a Deck of Cards
   - Design Chess
   - Design Tic-Tac-Toe
   - Design Snake and Ladder
   - Design 2048
   - Design a Jigsaw Puzzle
   - Chapter summary
6. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition**
   - 18.0 What platform problems test
   - 18.1 Design Stack Overflow
   - 18.2 Design Cricinfo
   - 18.3 Design LinkedIn
   - 18.4 Design Facebook
   - 18.5 Chapter summary

Chapter 1 — The LLD Interview & How to Approach It (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full code, outputs and extra examples are in the Full Edition (`README.md`).
>
> **You will learn:** what the LLD round is, a complete small answer, an 11-step framework with a time budget, how to get requirements and classes, a SOLID check, the six common patterns, how to write interview code, a worked parking lot, communication tips and a study plan.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design (LLD)** round: design **one application** while an interviewer watches, in 45–60 minutes (machine coding: 90 min–2 h). You design **the classes inside the program** (parking lot, vending machine, chess, Splitwise), not servers. Analogy: system design is the **city plan**; LLD is the **floor plan of one house**.

Term	Meaning
Class / Object	A named type with fields and methods / one concrete instance of it
Responsibility	The one job a class does, in one sentence
Relationship	How two classes connect: uses, holds, owns, or extends
UML	Standard symbols for drawing classes and relationships
Design pattern	A named, reusable answer to a recurring design problem

The hard part is **explaining aloud** why each class exists and **justifying** how they connect.

What you should be able to do after this handbook

Bound one sentence into requirements plus an out-of-scope list; pick classes with **one job** each; relate every pair correctly; code the core flows until they run; name the pattern you used **and the one you rejected**.

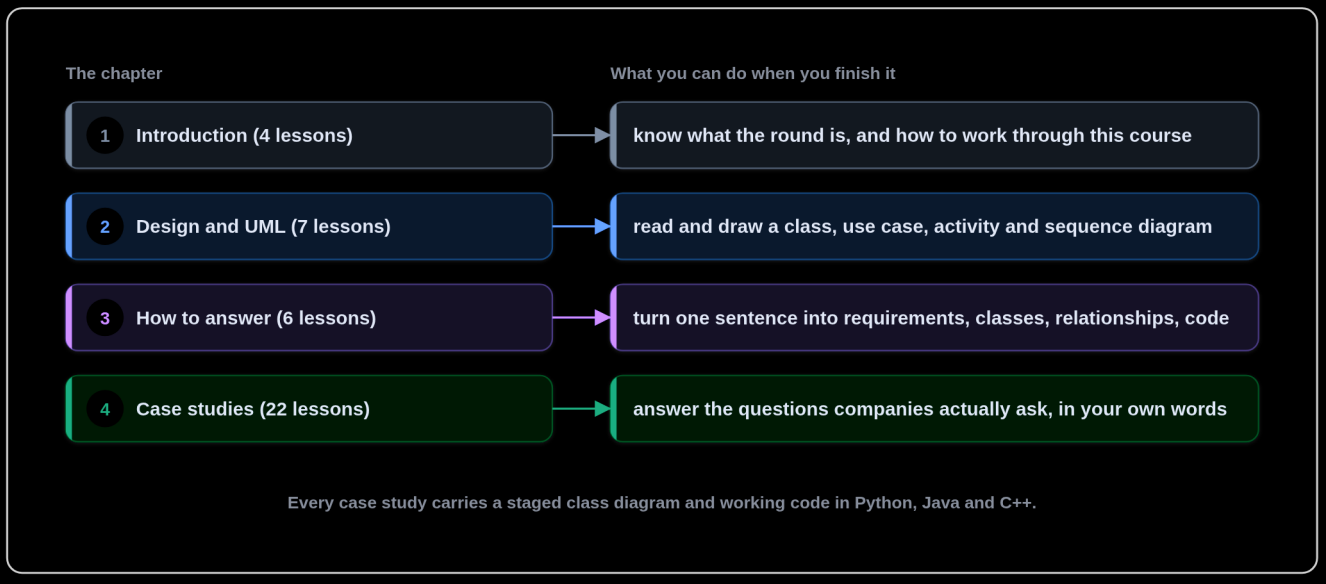


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The handbook follows this arc (round → UML → method → case studies) and adds OOP, SOLID, principles, clean code, refactoring, all 23 GoF patterns and concurrency.

What every case study in this handbook contains · Who this is for · What you need already · What this handbook will not do

- **Case-study template:** requirements with out-of-scope, actors and use cases, a staged class diagram, key flow diagrams, runnable Java with output, patterns **only where really used**, and "Extensions & follow-up questions".
- **For:** anyone facing this round. **Needed:** one OO language (examples use **Java 21**); not UML (Chapter 3), not system design (this round stays **inside one process**), not DSA beyond basic collections.
- **Won't do:** teach a language or give scripts to memorise. **The reusable part is the method.**

TL;DR

Design the classes of one application out loud, then code the main flows. Method beats memorisation.

1.2 How the interview runs

What it is

One sentence ("Design a parking lot"), then silence. Names for the round: **OOD interview** (US big tech), **LLD** (umbrella term), **machine-coding round** (commonly reported at Flipkart, Swiggy, PhonePe: **runnable code is mandatory**, 90 min–2 h).

	LLD	HLD / system design
Scope / vocabulary	One application: classes, interfaces, patterns	Many machines: servers, DBs, caches, queues
Output	Class diagram + code	Architecture + capacity numbers

OOD round vs machine-coding round

	OOD / LLD discussion	Machine coding
Time / output	45–60 min; requirements, diagram, code for 2–3 flows	~90 min coding; a **complete running program**
Storage / interaction	Rarely discussed; continuous conversation	**In-memory**; silent coding, then **demo + walkthrough**
Judged on	Bounding, responsibilities, relationships, trade-offs	Working code, **data models**, **service interfaces**, **extensibility**, readability, edge cases, sometimes thread safety
Common failure	Not converging, coding too early	Not finishing, or one giant class

How it works: the five core steps

1. **Clarify requirements** 2. **Actors and use cases** 3. **Find the classes** 4. **Connect them** (class diagram) 5. **Write the code** for 2–3 main flows.

Order matters more than speed: skip step 1 and step 3 invents classes nobody asked for.

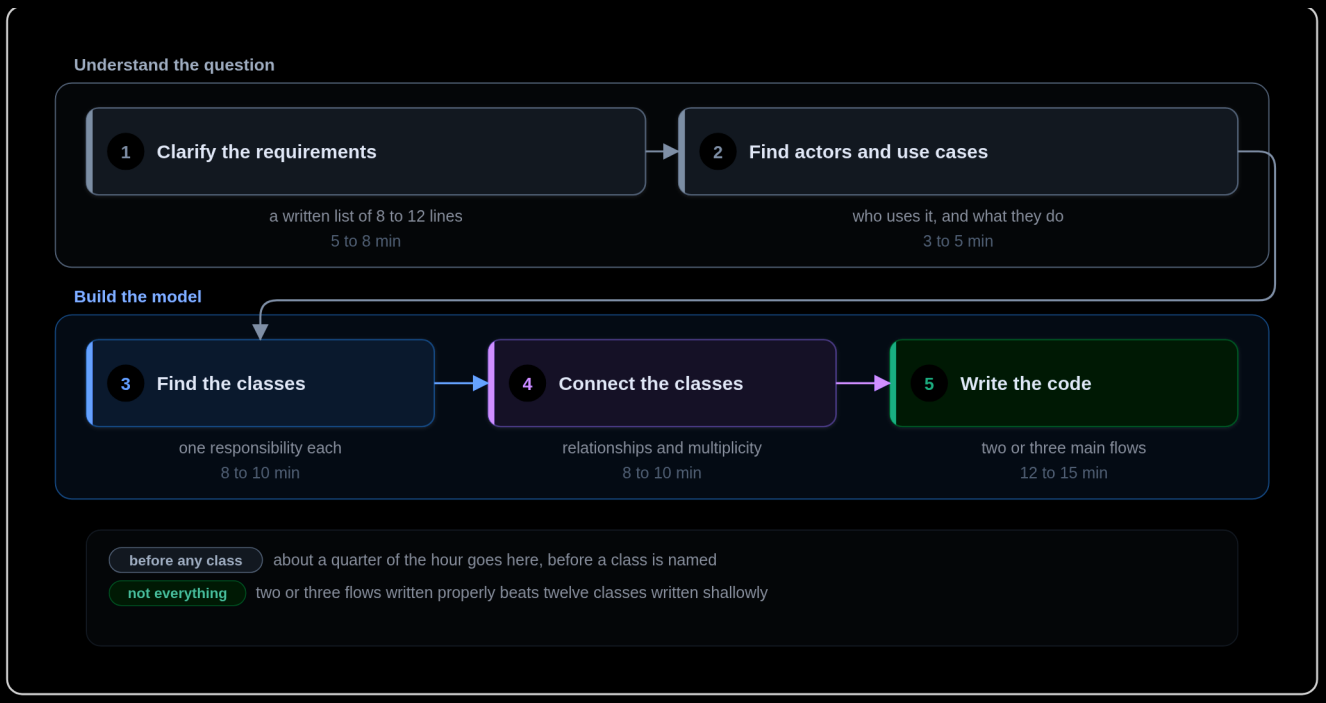


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

About **a quarter of the hour passes before any class is named**. **Nobody finishes the product**; you are judged on the part you built properly.

Time-boxing

45-min OOD step	Time	Output
Requirements	5–8 min	8–12 lines + out-of-scope
Actors & use cases	3–5 min	2–4 actors, 6–10 use cases
Classes	8–10 min	One responsibility each
Connect	8–10 min	Diagram with multiplicity
Code	12–15 min	2–3 main-flow methods

90-min machine coding: requirements/assumptions 5–10 · model (entities, enums, interfaces) 10–15 · core flow end to end 30–40 · remaining features + edge cases 15–20 · polish + demo 5–10 · buffer 5 (never zero). **Get something running by halfway.**

What the interviewer is actually judging

1. Did you **bound** the problem? 2. **One job** per class? 3. **Right relationships**? 4. Does **code match the diagram**? 5. Can you explain a **trade-off**?

Machine coding adds: **runs**, **modular** (new type = new class), readable, handles bad input, thread-safe (senior). Perfect UML notation is **not** graded; the model is.

A worked example of size

Parking lot: ~12 requirements → ~12 classes → ~15 relationships. Code the three that carry the main flows; name the other nine. Costliest mistake: all twelve at equal depth.

> **TL;DR**
>
> Five steps in order, a quarter of the time before naming a class, two or three flows done properly.

1.3 A complete answer in five minutes: "Design a music playlist"

Step 1: requirements

Songs are **ordered**; a listener asks for **the next song**; **shuffle** changes the order. **Out of scope (said aloud):** accounts, downloads, storage, audio.

Step 2: actors and use cases

One **actor** (someone outside the app): the listener (add song, play next, shuffle). On bigger problems **actors reveal which classes must exist**.

Step 3: classes, one sentence each

Class	Responsibility
`Song`	Title, artist, length; knows only itself
`Playlist`	Holds songs in order; stores, does not play
`PlayOrder` (interface)	Picks the next position (several rules exist)
`InOrder` / `Shuffled`	Insertion order / an order fixed up front
`Player`	Holds playlist + order, remembers the position

A rule that can vary becomes its own type. If a responsibility needs "and" to join two jobs, it is two classes.

Step 4: connect them

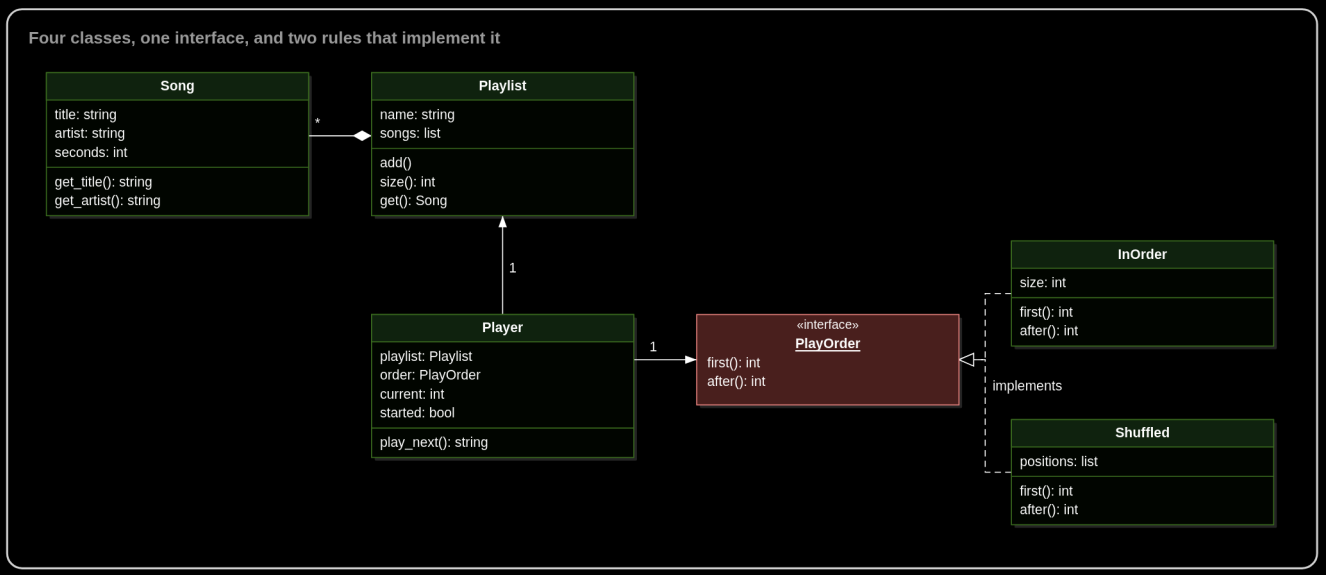
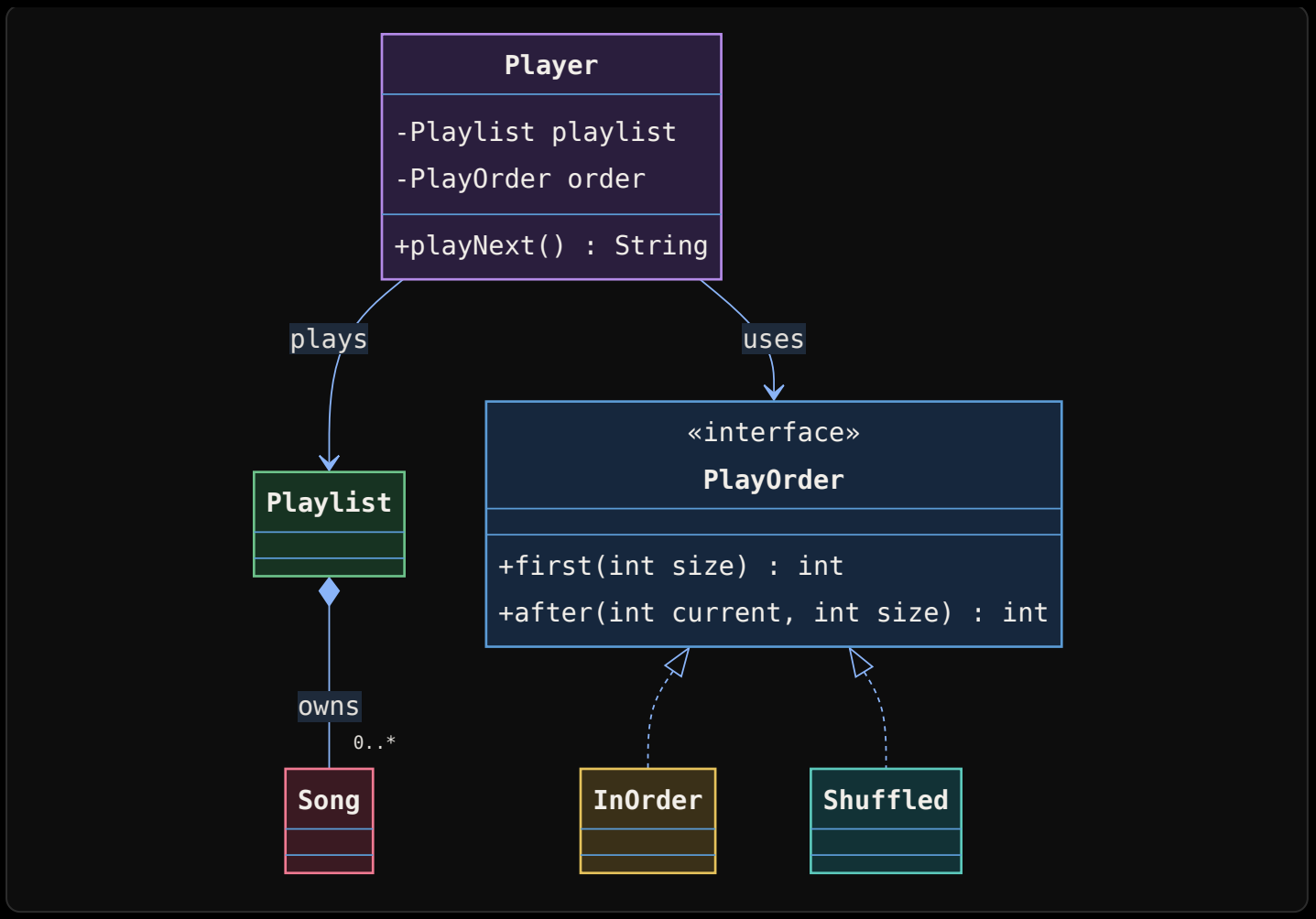


Figure — The playlist design: Playlist owns many Songs, Player uses a PlayOrder, and InOrder and Shuffled implement PlayOrder (course diagram)

Filled diamond = `Playlist` **owns** many `Song`s (composition). Plain arrow = `Player` **uses** the `PlayOrder` interface only. Dashed hollow-triangle arrows = `InOrder`/`Shuffled` **implement** it; a third rule is one more box.



Step 5: the code

```java
// File: code/lld/ch01/playlist/PlayOrder.java
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"

    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}
```

```java
// File: code/lld/ch01/playlist/Player.java
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;          // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;
    // ... constructor

    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}
```

Full runnable code: code/lld/ch01/playlist/ (see the Full Edition).

`Song` is a validating **record**; `Playlist` keeps its list `private`; `Shuffled` is **given** its order (not `Random`), so output is the same every run: **deterministic = testable**. `Player` never checks which mode is active.

What to notice

- **Shuffle (one line) became a type**: classes come from requirements.
- **One job per class**: `Song` doesn't know playlists exist.
- **Open to change**: "repeat one" is a new `PlayOrder` (Strategy + Open/Closed).
- **Announce deliberate gaps** (pause, volume, queue).

TL;DR

Bounded requirements, one-sentence classes, one diagram, code for the main flow. A varying rule becomes an interface.

1.4 The step-by-step answer framework

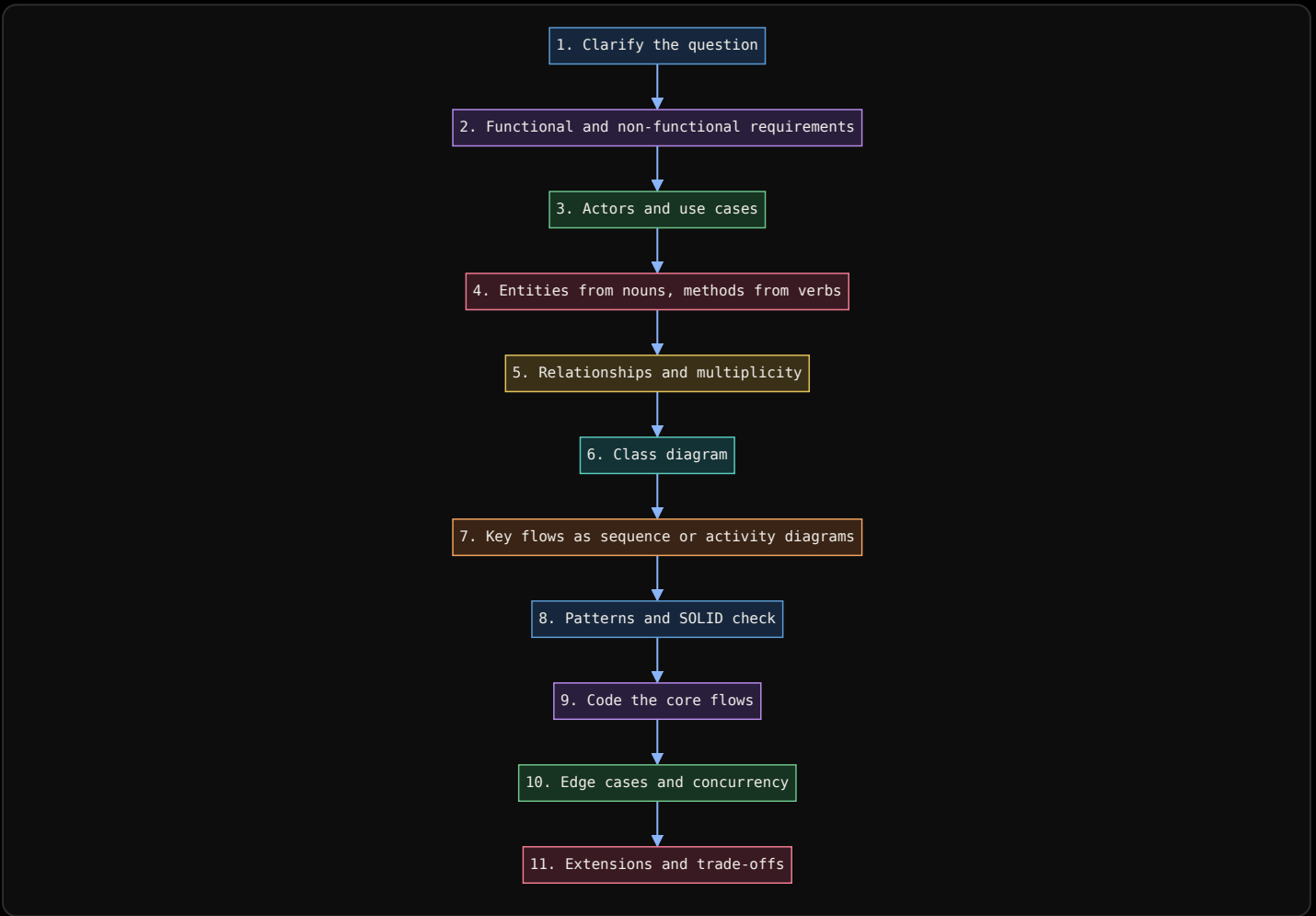
What it is

A checklist for every question, expanding the five steps into eleven.

The problem it solves

Stops freezing and premature coding; makes progress **visible** (which is scored).

How it works



Ask at step 8: varying rule (Strategy)? lifecycle (State)? many listeners (Observer)? At step 10: invalid input, full, two users on one seat?

Trade-offs

Do this	Instead of	Why
8–13 min on steps 1–3	Coding at minute 2	Unbounded scope costs more later
2–3 flows fully	12 stubs	Depth is judged
1–2 patterns with a signal	Six patterns	Over-engineering = poor judgement
Pass time in; `BigDecimal`/cents	Clock in logic; `double` money	Testable; `double` can't hold 0.10 exactly

When not to use it (rigidly)

Follow the interviewer if they steer. In machine coding, steps 5–7 are often mental; **the code is the diagram**.

TL;DR

Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases → extensions. Announce each step.

1.5 From one line to a list of requirements

What it is

A **requirement** is one testable capability. **Functional** = behaviour ("customer takes a ticket"). **Non-functional** = quality (thread safety, extensibility, in-memory).

The problem it solves

Without a list the design has **no edges**, you **can't defend omissions** ("reservations were out of scope; we agreed"), and you **can't tell when you're done**.

How it works: the four questions

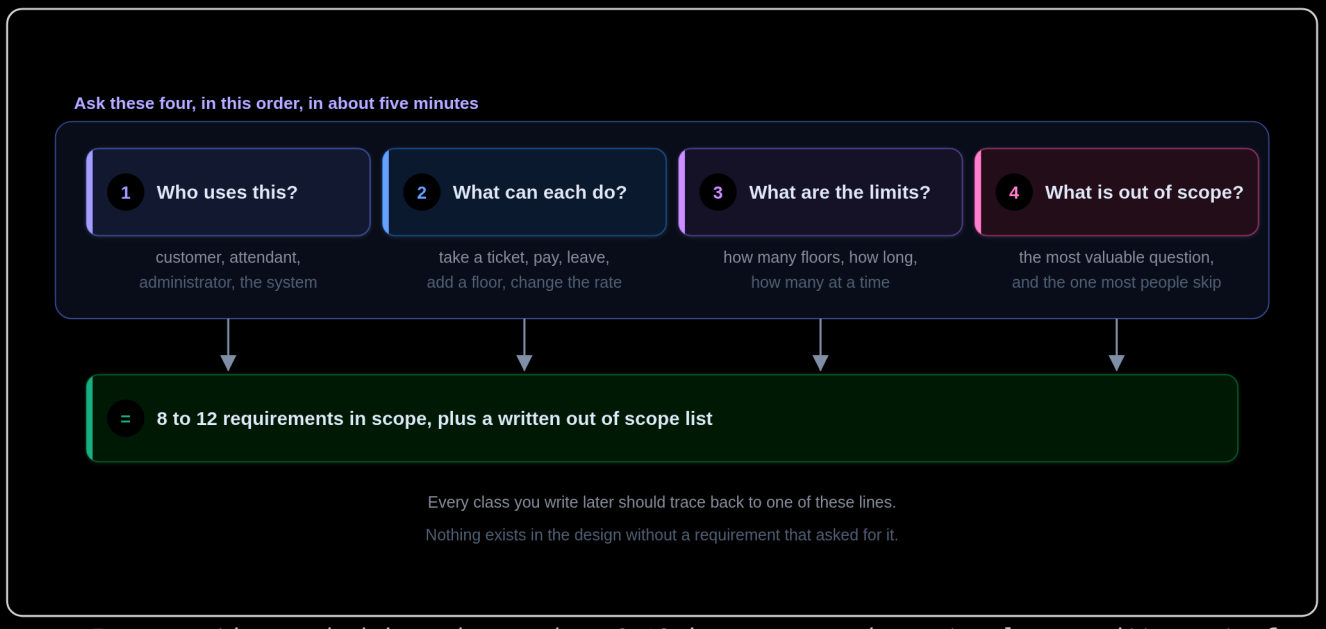


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

1. **Who uses it?** (actors) 2. **What can each do?** (use cases; most requirements) 3. **What are the limits?** (→ constants + validation) 4. **What is out of scope?** (the only question that makes the problem

smaller).

Ask early: payments in scope? one site or many? users/permissions modelled? Each answer adds or removes 2–3 classes. If told "your choice": **choose, say it, write it down**.

What good looks like: the parking lot in ten lines

In scope: multiple floors of spots; spot types (compact, large, handicapped, motorbike, electric); vehicle types; ticket at entrance; pay at exit/portal/attendant; cash or card; hourly rate by stay length; free-spot display per floor; refuse entry when full; admins add floors/spots/panels and change rates. **Out of scope:** season passes, plate recognition, pre-booking, refunds, multi-site reports. **If no line asked for it, it's not in the design.**

Make each requirement checkable

Put a number on it ("at most **five** books on loan" → constant → one check) and **name the actor** ("**a member** can reserve a copy on loan" → tells you the owning class).

Use cases

Customer: take ticket, park, pay, exit. Attendant: collect cash. Admin: add floor/spot/panel, change rate. System: assign spot, update boards, refuse when full. A list is enough in 45 minutes.

Common mistakes

Saying yes to every idea (declining politely is a **strong** move); forgetting non-functional requirements; vague lines without number or actor.

In the interview

> **30-second answer:** "I ask who uses it, what each actor does, the limits, and what's out of scope. I write 8–12 numbered requirements plus an out-of-scope list, confirm them, and every class must trace back to a line."

- *"Why not just start designing?"* — No boundary means endless growth and indefensible omissions.
- *"'Your choice'?"* — I pick the simpler option and write it as a visible assumption.

Self-check

- **Q:** Which question shrinks the problem? **A:** Out of scope.
- **Q:** Make "users can book some seats" checkable. **A:** "A customer can hold at most 10 seats per show; unpaid holds release after 10 minutes."

TL;DR

Who, what, limits, out of scope → 8–12 checkable lines plus out-of-scope.

1.6 Finding the classes

What it is

Collect candidates, cut non-classes, give each survivor one job.

How it works

1. **Nouns** → candidate classes (deliberately too many). 2. **Verbs** → methods; a verb with no home = a missing class. 3. **Cut with five tests:**



Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	A **value**?	Hour, plate, cash/card option	Field or **enum**
2	**Another name**?	Customer/driver	Merge
3	**Changing state**?	Spot free/occupied	Class
4	Anything **calls** it?	Nobody asks an `Hour`	Value
5	**Behaviour** or only **data** differs?	Compact vs large spot (data); cash vs card (behaviour)	**Data → field. Behaviour → class.**

Test 5 cuts the most: one `ParkingSpot` with a `type` field, but separate cash and card classes. Smell: subclasses that only pass a constant to the parent.

4. **One sentence per class**, e.g. `ParkingFloor` owns a level's spots and finds a free one; `ParkingSpot` tracks its occupant; `ParkingTicket` records vehicle, spot, entry time; `ParkingRate` turns duration into price; `Payment` collects money; `ParkingDisplayBoard` shows free spots. `ParkingLot` (issues tickets **and** tracks fullness) has two jobs: **name the tension aloud**.
5. **Method goes where its data is** ("find a free spot" → `ParkingFloor`). Needing three classes' data = misplaced (GRASP Information Expert; *Feature Envy*).

Common mistakes

One subclass per value; missing the **record-keeping** class (ticket/loan/booking) and the **rule** class (rate/strategy); "Manager" god classes.

Self-check

- **Q:** Car, van, truck: subclasses or enum? **A:** Enum/field unless behaviour truly differs.
- **Q:** Class with fields but no verbs? **A:** Probably a value (record/field).

TL;DR

Nouns → candidates, verbs → methods. **Different data = field; different behaviour = class.** Method where its data is.

1.7 Connecting the classes (preview)

For each pair: **uses** briefly (dependency), **holds** (association), **contains** parts that live alone (aggregation), **owns** parts that die with it (composition), or **is-a** (inheritance/realization); then **multiplicity** (1, 0..1, 1..*, *). Chapter 3 details all. Top error: inheritance where composition fits. A car is not an engine, so `Car` **has an** `Engine`.

1.8 Keeping the design honest with SOLID

What it is

The five SOLID rules (Chapter 4) used as **five quick checks** on your diagram (~5 minutes).



Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: run the five checks in this order

Check	Failure signal	Parking-lot example
S One reason to change	Sentence needs "and"	Lot that assigns spots **and** prints receipts → split
O Extend without editing	`switch` over a type	Weekend rate = new class, not new case
L Child works wherever parent does	Child throws / does nothing	`ReadOnlySpot` whose `occupy` throws
I No unused methods	Empty bodies	Split an interface half its implementers can't honour
D Depend on interfaces	High-level class does `new` of low-level	Lot depends on `Payment`, not `new CreditCardTransaction()`

The two that earn the most credit

S improves the class list; **O** is what gets probed ("trucks charged differently?" → "add a class").

When not to use it

No single-implementer interfaces "because SOLID" (unless a test double needs one); no splitting past what a reader can follow.

A five-minute pass over the parking lot

S: lot has two jobs (split or say it). O: pricing behind `ParkingRate`, passes. L: five behaviour-less spot subclasses aren't a Liskov violation, but they fail "different data → field": collapse. I, D: pass. **Two findings is good; five = rework; zero = you didn't look.**

TL;DR

Check S-O-L-I-D before the interviewer does; S and O matter most; don't over-apply.

1.9 The design patterns that come up in this round

What it is

A **pattern** is a named solution to a recurring problem (GoF has 23; Chapters 8–11). Six recur here. **A misapplied pattern hurts more than an unknown one**: learn the **signal**.

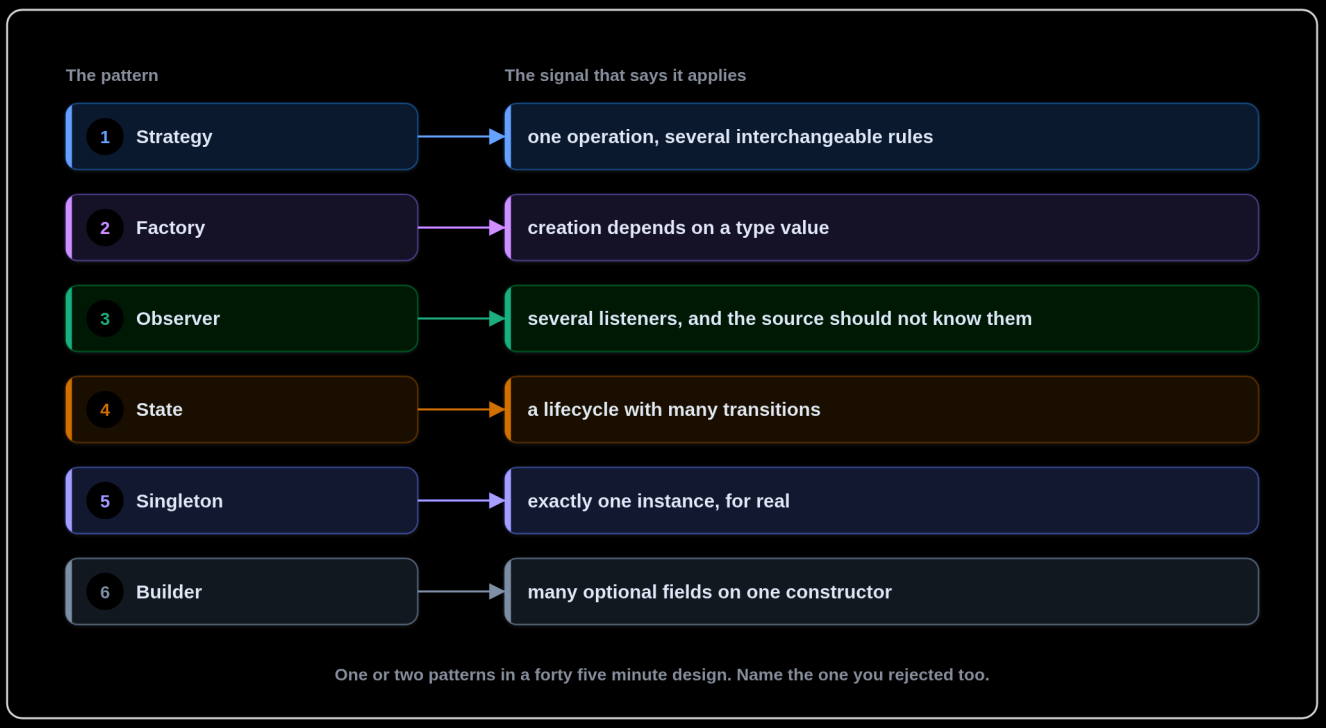


Figure — The six patterns most used in this round and the signal for each (course diagram)

What to say about each one

Pattern	Signal	What to say
Strategy	Several swappable rules for one operation	`ParkingRate` → `HourlyRate`, `WeekendRate`; a new rate is a new class, not a `switch` case
Factory	Object to build depends on a type value	Registering a new type happens in **one place**; a factory method is usually enough
Observer	One change, many listeners the source shouldn't know	The spot is the subject; boards register. One listener → direct call
State	Behaviour depends on state, fixed transitions	Illegal moves become impossible; for 3–4 states, **enum + guarded method**
Singleton	Exactly one instance	Must be **thread-safe**; **global state** hurts tests; often inject one instance instead
Builder	Many optional fields (~6+ constructor args)	Sets fields by name; a readability fix

Choosing patterns without overusing them

Skip each when its signal is absent: one rule (Strategy), one concrete class (Factory), one listener (Observer), three states and one guard (State), just wanting global access (Singleton), three required fields (Builder). **One or two patterns** per design; best is a comparison: "Rate gets Strategy; ticket doesn't get State, one guard covers three statuses."

Real-world examples

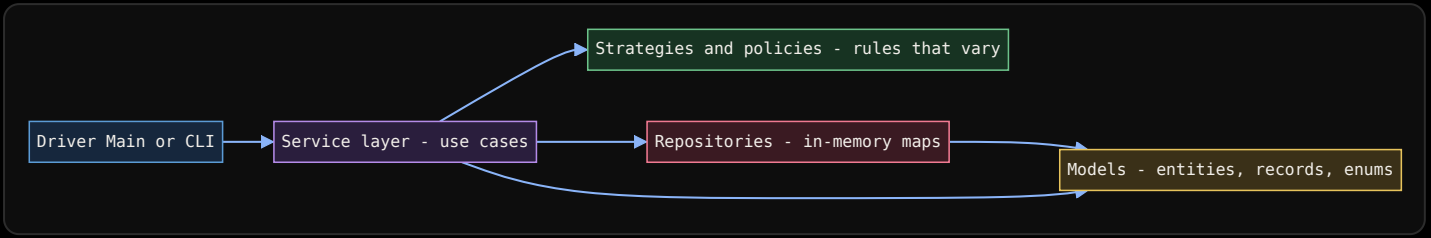
`Comparator` (Strategy); `List.of`, `Executors.newFixedThreadPool` (Factory); `ActionListener`, Spring events (Observer); order lifecycle, TCP (State); `Runtime.getRuntime()` (Singleton; Spring beans are singleton **per container**); `StringBuilder`, `HttpRequest.newBuilder()` (Builder).

TL;DR

Know six; use one or two where the signal appears; name the rejected one.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



Models (validated entities, records, enums) → **repositories** (in-memory maps behind an interface) → **strategies** (rules that vary) → **services** (use cases: the **service interface**) → **driver** (`Main` demo).

Habits: pass time in; money as `BigDecimal` (from strings) or cents, never `double`; fail loudly (`IllegalArgumentException`, `IllegalStateException`, clear custom exceptions); `Optional` for normal "not found"; `private final` fields; domain names.

Edge cases and concurrency checklist

Input (null, negative, duplicate/unknown IDs) · **Capacity** (full lot, empty playlist) · **State** (pay twice, exit without entry) · **Time** (exit before entry, round part hours) · **Concurrency** (two threads, same seat: `synchronized`, `ReentrantLock`, `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** thing that keeps the invariant; Chapters 12–13).

TL;DR

Models → repositories → strategies → services → driver; inject time, exact money, validate, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

Problem & clarifying questions

Assumed answers: single lot, one or more floors; motorbike/car/truck; MOTORBIKE/COMPACT/LARGE spots (smaller vehicles may use bigger spots); payment out of scope (we compute the fee); tiered hourly pricing, swappable; several panels act concurrently.

Requirements

1. Floors with MOTORBIKE, COMPACT, LARGE spots. 2. Motorbike → MOTORBIKE/COMPACT; car → COMPACT/LARGE; truck → LARGE; smallest first. 3. Entry assigns a spot and issues a ticket. 4. No fit → refused. 5.

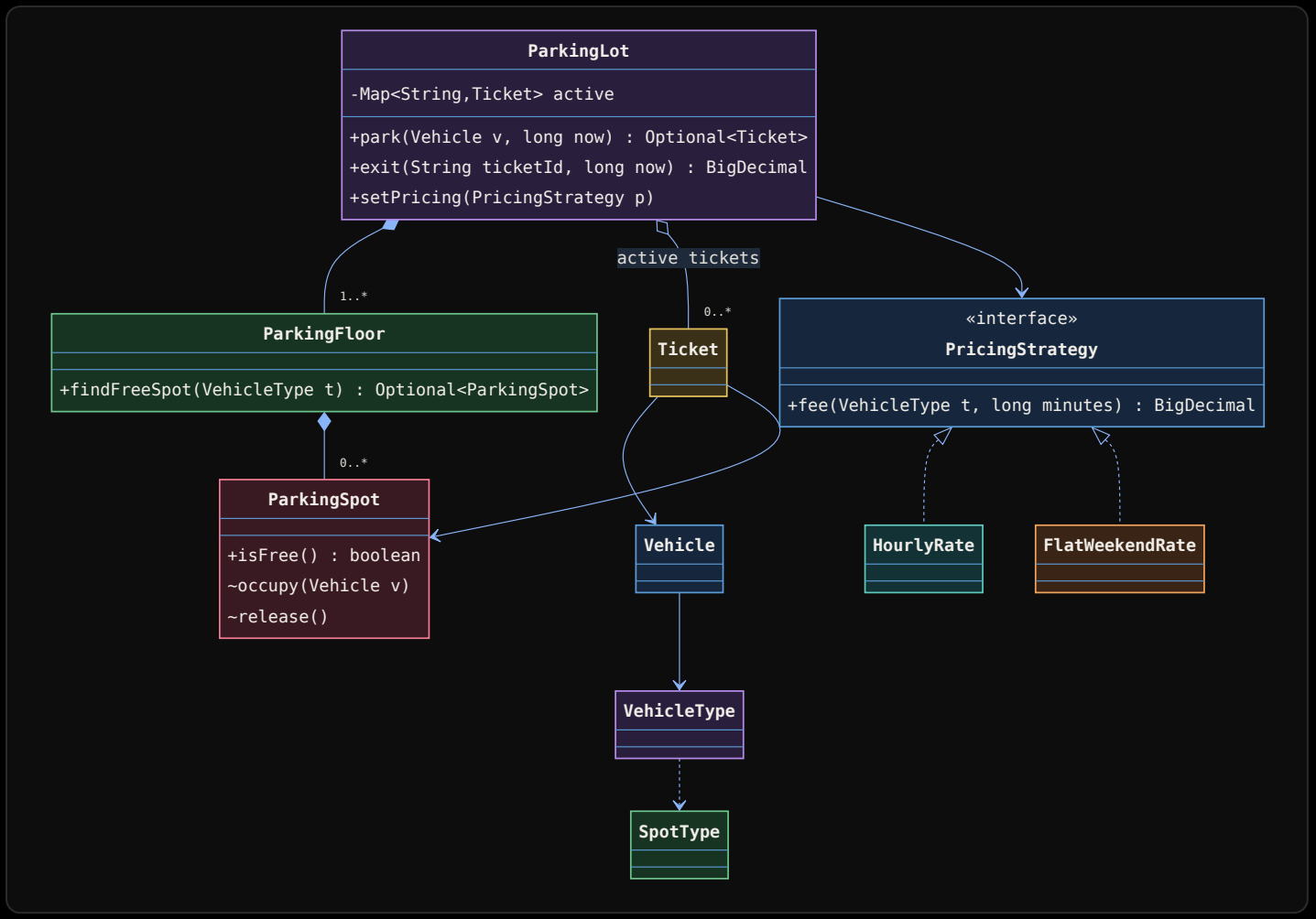
Exit closes ticket, frees spot, returns fee. 6. $4.00 first hour, $3.50 hours 2–3, $2.50 after; round up; minimum 1 h; trucks double. 7. Pricing swappable at run time (flat $10 weekend). 8. Free-spot count per type.

Non-functional: in-memory, thread-safe, extensible, time passed in. **Out of scope:** payment collection, reservations, boards, plate recognition, multiple lots.

Use cases · Core entities

Driver: enter/exit. Admin: add spots, change pricing. System: find spot, refuse, compute fee. Types: `SpotType`, `VehicleType` (enums; vehicle knows which spots fit), `Vehicle` and `Ticket` (records; the ticket is the "connection object"), `ParkingSpot`, `ParkingFloor`, `PricingStrategy` + `HourlyRate` / `FlatWeekendRate`, and `ParkingLot` (the service). Spot sizes are an enum: **different data → field**.

Class diagram



Key flows

`park`: lot asks each floor `findFreeSpot(type)`; if found, `occupy` it and return a `Ticket`, else return empty (refuse). `exit`: remove the ticket, `release()` the spot, return `pricing.fee(type, minutes)`.

Design decisions & patterns used

Decision	Why	Rejected
Strategy pricing	Weekend rule required; open/closed	`switch` in the lot
Enums for types	Data only differs	Subclass per type
`Ticket` record	Fixed after issue (only *shallowly* immutable)	Mutable ticket
`BigDecimal`	`3.50 + 3.50` is exactly `7.00`	`double`
`synchronized` lot methods	Two panels never share a spot	Per-floor locks (extension)
No Singleton, no State	Injectable/testable; only active/closed	`getInstance()`; state classes

Java implementation

```java
// File: code/lld/ch01/parkinglot/HourlyRate.java
public final class HourlyRate implements PricingStrategy {
    // ... FIRST_HOUR = 4.00, HOURS_2_TO_3 = 3.50, LATER_HOURS = 2.50
    @Override
    public BigDecimal fee(VehicleType type, long minutesParked) {
        long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
        BigDecimal total = BigDecimal.ZERO;
        for (long h = 1; h <= hours; h++) {
            total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
        }
        return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingLot.java
public final class ParkingLot {
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    // ... floors, nextTicket, constructor

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();              // lot full for this vehicle type -> refuse entry
    }

    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }
    // ... freeSpots(type) and setPricing(p), both synchronized
}
```

Full runnable code: code/lld/ch01/parkinglot/ (see the Full Edition).

Output (excerpt): the third car is refused; `T1 exits after 150 min, pays $11.00` (4.00 + 3.50 + 3.50); the truck's 50 min → 1 h × 2 = `$8.00`; after `setPricing(new FlatWeekendRate())` the exit pays `$10.00`; a second exit of T1 prints `error: unknown or already closed ticket T1`.

Concurrency & edge cases

`synchronized park/exit` makes find-then-occupy **atomic** (no *check-then-act* race); `freeSpots` uses the same lock; `ParkingFloor` is only called under it. `exit` **validates before changing state**; `occupy` throws if taken (second defence) and is package-private.

Extensions & follow-up questions

- **EV charging:** `ELECTRIC` spot + `ELECTRIC_CAR`; if charging has behaviour, a `ChargingSpot` or composed component.
- **Nearest spot:** inject a `SpotSelectionStrategy`.
- **Display boards:** Observer on spot events.
- **Throughput:** per-floor/type locks, or a `ConcurrentLinkedQueue` of free spots with atomic `poll()`.
- **Cash/card:** `Payment` interface + implementations, collected by a `PaymentService` (DIP).

In the interview

~5 min requirements, 3 actors, 8 entities (enum vs subclass), 8 diagram, 15 coding `park`, `exit`, `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected), and per-floor locking.

Self-check

- **Q:** What race does `synchronized park` prevent? **A:** Two threads occupying the same free spot.
- **Q:** How did the weekend rate respect open/closed? **A:** A new class; nothing edited.

TL;DR

8 requirements, 10 types, one Strategy, one lock, an honest out-of-scope list.

1.12 Communication tips

Do: think out loud (name the step), write requirements/classes where visible, confirm before moving on, state assumptions, give trade-offs in pairs ("X over Y because..."), welcome hints, say what you leave out, watch the clock and announce the switch to code, use domain names (not `Manager` / `Helper`), and **when stuck, return to the requirements**. **Don't:** go silent, argue, talk `HashMap`s in minute two, or recite patterns without a reason.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

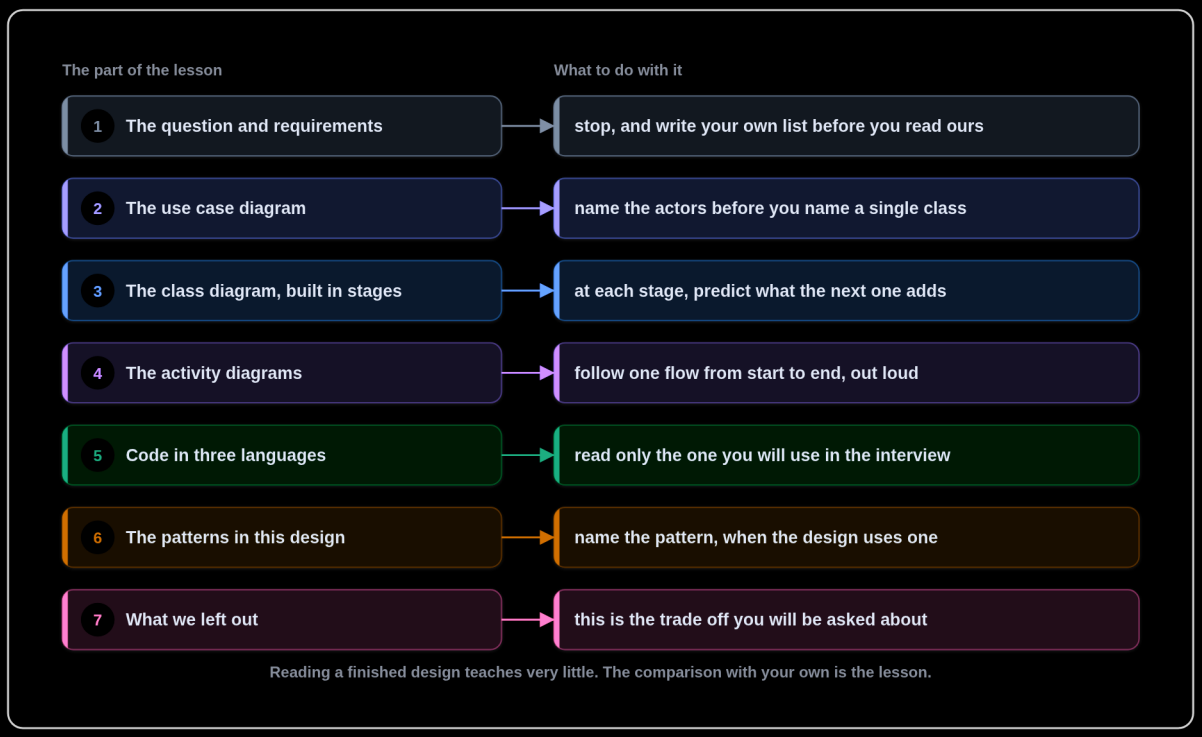


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

Reading builds little skill; **attempt, then compare**. Stop (1) after the question (write requirements), (2) after the requirements (write classes; most people are **two short**: the record class and the rule class), (3) at each diagram stage (predict). Also: argue with decisions in writing, read one language, **never skip extensions**, don't invent pattern names, switch to **timed 45-minute blank-page runs** after 6–7 case studies, run `bash tools/compile_all.sh`.

Map of this handbook

1 approach · 2 OOP · 3 relationships & UML · 4 SOLID · 5 design principles · 6 clean code · 7 refactoring · 8 creational · 9 structural · 10–11 behavioural (+MVC) · 12 concurrency concepts · 13 concurrency problems · 14 everyday objects · 15–16 booking & inventory · 17 games · 18–19 platforms · 20 components (LRU, rate limiter, Splitwise) · 21 machine coding · 22 cheat sheets.

Study plans

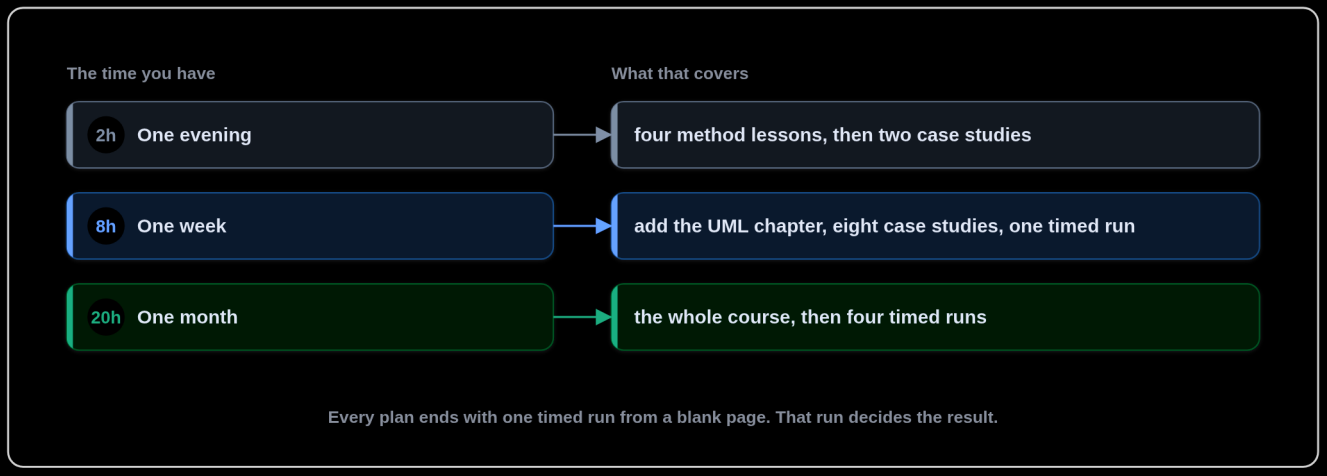


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

- **Tomorrow (~2 h):** §1.2, §1.5, §1.6, Ch 3 relationship guide + class notation, parking lot (30 min), vending machine (25 min, a **lifecycle**).
- **One week:** Ch 1 + OOP pillars; Ch 3 diagrams; parking lot + library; ATM + vending; movie tickets + hotel; Splitwise + Amazon; **day 7: redo parking lot blank-page in 45 min**.
- **One month (~20 h):** Ch 1–4 + 3 case studies; Ch 5, 8–11 + case studies 4–10; Ch 12–13 + 11–18 + a timed run; the rest, Ch 21, three timed runs, Ch 22.
- **Beginner (6 weeks):** Ch 1–2; 3–5; 6–8; 9–11 + two case studies; 12–13 + four incl. ticket booking; Ch 21 timed runs + Ch 22.

Only six case studies? Parking lot, library, vending machine, movie tickets (concurrency), Splitwise (money), Amazon (too big: choose). Add **LRU cache** if data-structure questions are likely. **Night before:** nothing new; re-read the time budget; recite **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Fix
Coding in minute two; no out-of-scope list; yes to every feature	8–13 min on requirements; write and agree scope
One subclass per value; god class	Data → field/enum; split on "and"
Missing record class; "Car extends Engine"	Connection with data = object; composition
Pattern stuffing; `switch` on type	1–2 patterns with signal; polymorphism
Equal depth everywhere; silent designing	2–3 flows fully; narrate
Ignoring concurrency; non-running code	Lock shared state; thin path running by halfway
`double` money; clock in logic	`BigDecimal`/cents; inject time

In the interview

30-second answer: "I clarify and write 8–12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns with one responsibility each, connect them in a class diagram, sketch the key flow, run a SOLID check and pick one or two patterns where the signal is clear. Then I code the core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

- *"Different from system design?"* — That's many machines and scale; LLD is classes and code inside one application.
- *"You won't finish; what matters?"* — Bounded scope, clean responsibilities, correct relationships, code matching the diagram, one defended trade-off.

Self-check

- **Q:** Five core steps? **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** Two classes beginners miss? **A:** The record-keeping class and the rule class.
- **Q:** Patterns in 45 minutes? **A:** One or two where the signal appears; name a rejected one.

TL;DR

- Design the classes of one app and code main flows; machine coding = same, running, ~90 min.
- A quarter of the time before naming a class; always an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- SOLID as quick checks; one or two patterns with a signal.
- Inject time, exact money, guard shared state; attempt first, then timed blank-page runs.

Chapter 14 — Case Studies: Everyday Objects (Study Edition)

> **Where this fits.** Chapters 1–13 gave you the tools (OOP, UML, SOLID, clean code, the GoF patterns, concurrency). Chapters 14–21 apply them to real interview questions. This chapter starts with **machines you have used yourself** — parking lot, vending machine, coffee machine, ATM, elevator, traffic light, parcel locker — so all your attention can go to the design. This is the study edition: every topic stays, the padding goes. The Full Edition (README.md) has the complete code, outputs and stage-by-stage course diagrams.

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System (Amazon Locker)	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

What these problems test

1. **State machines** — a fixed set of states plus transition rules: no dispensing before payment, no withdrawal before the PIN, never green on two crossing roads. The **State pattern** (Chapter 11) gives every button an answer in every state.
2. **Resource allocation** — spots, notes, milk, cars, locker doors handed out never twice, never below zero, fairly.
3. **Concurrency** — several gates, two ATMs on one account, a timer plus an emergency thread; the design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** The hardest part of each problem is **one method**: pick a spot, make change, deduct ingredients, pick the next stop. Find it early, name it out loud, spend your time there.

How each case study is laid out

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → Self-check → TL;DR. This is also a good order to speak in during an interview.

How to read the course diagrams

The course draws class diagrams in stages (earlier classes in grey, new ones in colour). This study edition keeps only the **finished** course diagram per case plus a mermaid diagram of our Java code.

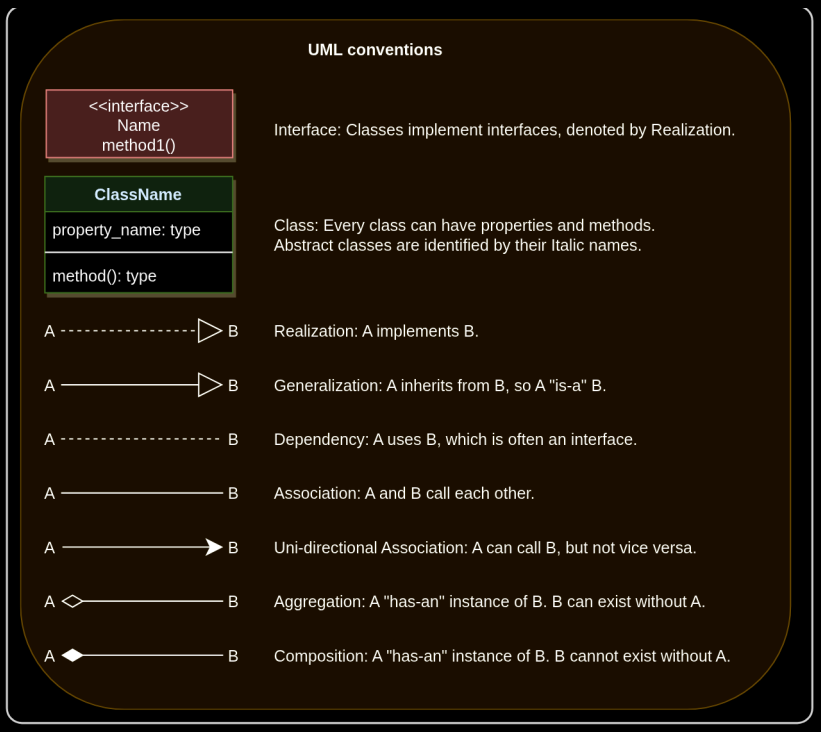


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Hollow triangle = inheritance ("is a"); **dashed line + hollow triangle** = implements an interface; **filled diamond** = composition (the part dies with the whole); **plain arrow** = association (refers to, does not own). See Chapter 3.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

Java 21, standard library only. **Code-style note:** some files hold several small package-private types (e.g. `Vehicles.java`); fine except for the `auxiliaryclass` lint — in production give each type its own file.

14.1 Design a Parking Lot

A **parking lot** (car park) with many floors, many spot sizes and several ways in and out. It is the **most-asked LLD question**. It looks easy, so the interviewer can watch how you **scope** it, how you model the **fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Question	Assumed answer
Scope?	One lot, many floors; several entrances and exits working at once.
Vehicles / spots?	Car, truck, van, motorcycle, EV / compact, large, handicapped, motorcycle, electric.
Payment?	Cash or card at the exit panel, an attendant, or a per-floor info portal.
Fee?	$4 first hour, $3.50 hours 2–3, $2.50 after; a started hour counts as full.
EV charging?	Electric spots have a pay-and-charge panel.
Reservations, passes, plate recognition?	Out of scope (extensions).

Requirements

Functional: (1) multiple floors; (2) multiple entries/exits; (3) ticket at entry, pay at exit; (4) pay at the exit panel or an attendant; (5) cash or card; (6) pay early at a per-floor **info portal** (not charged again); (7) **never exceed capacity** — "full" on the entrance panel and ground-floor board; (8) spot types; (9) **electric** spots with a pay-and-charge panel; (10) vehicle types; (11) a **display board per floor**; (12) tiered hourly fees.

Non-functional: correct under concurrency (no shared spot, no double payment); fast gate (O(log n) from a sorted free set); extensible; testable (injectable clock).

Use cases

Actors: Admin (floors, spots, attendants, rates), Customer, Parking attendant (a customer proxy who takes cash), System (messages, assigns/frees spots). **Top use cases: take ticket**, **scan ticket**, card/cash payment, admin edits, set rate.

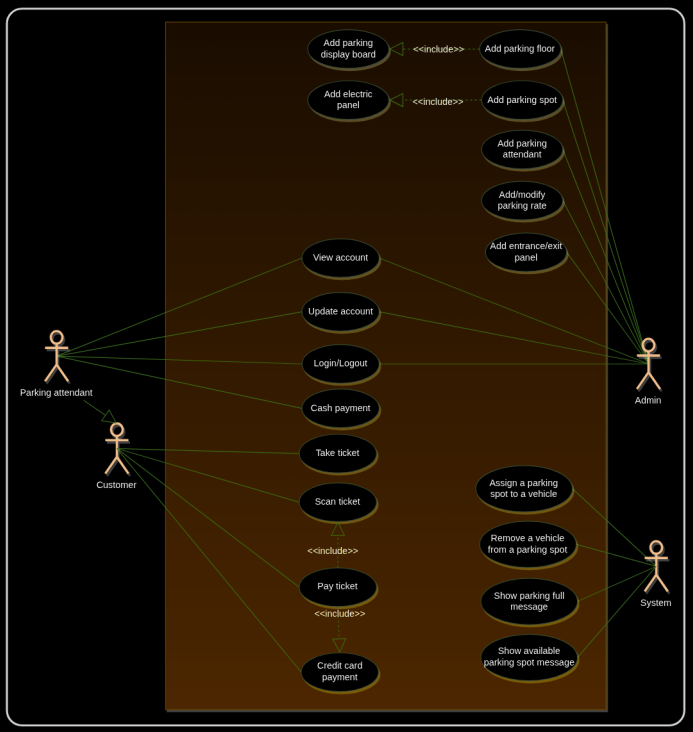


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

Core entities

Class	Responsibility
`VehicleType` (enum)	Kinds of vehicle and, for each, the spot types it fits, **best fit first**.
`SpotType` (enum)	Motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit; knows its preferred spot types.
`ParkingSpot`	One spot: id, floor, type, occupant (atomic).
`ChargingPanel`	Pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Free spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot.
`PricingStrategy`	*How much* a stay costs.
`Ticket`	Vehicle, the **actual spot**, entry time, status ACTIVE → PAID → EXITED.
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	Devices customers touch; **no rules**, they call the lot.

The course also has `Account` with `Admin` and `ParkingAttendant` subclasses; they carry no parking logic, so the code omits them.

Class diagram

The course grows the design in **five stages**, each leaving something that runs (a good interview habit: if time runs out, what you drew already works):

Stage	Adds	Key decision
1	Vehicle and spot types	Spot holds **0..1** vehicle; the **fit rule lives on the vehicle** (truck/van → large; EV → electric, compact, large).
2	Floors, boards	Floor owns spots and board; board stores **counts**, not a stale "sample free spot"; one map keyed by type.
3	Ticket, rate, payment	**Ticket stores the exact spot**; abstract `Payment` with cash/card (LSP holds).
4	`ParkingLot`	Owns floors and rate; course makes it a **Singleton** and serialises assignment.
5	Panels, portal, electric panel	Panels are **thin**, forwarding to the lot; electric panel owned by its spot.

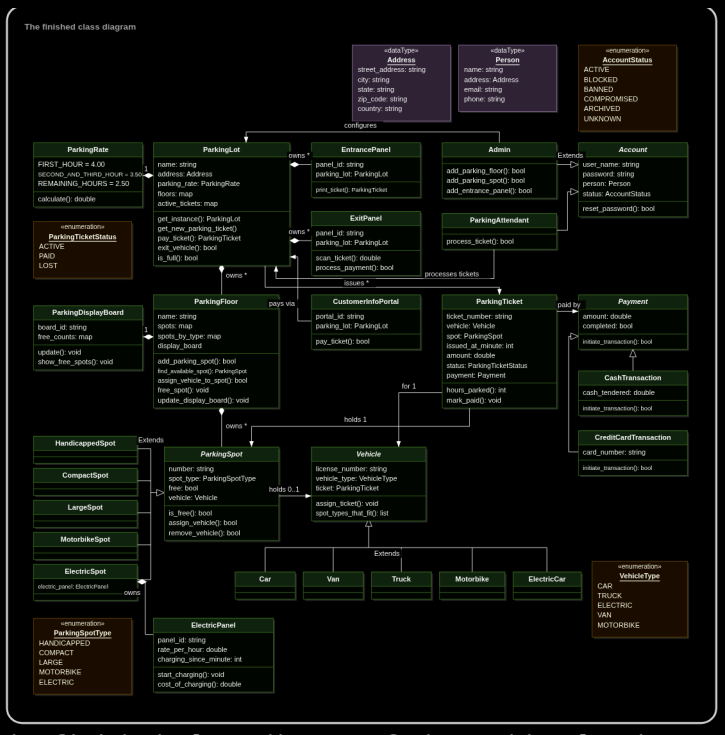
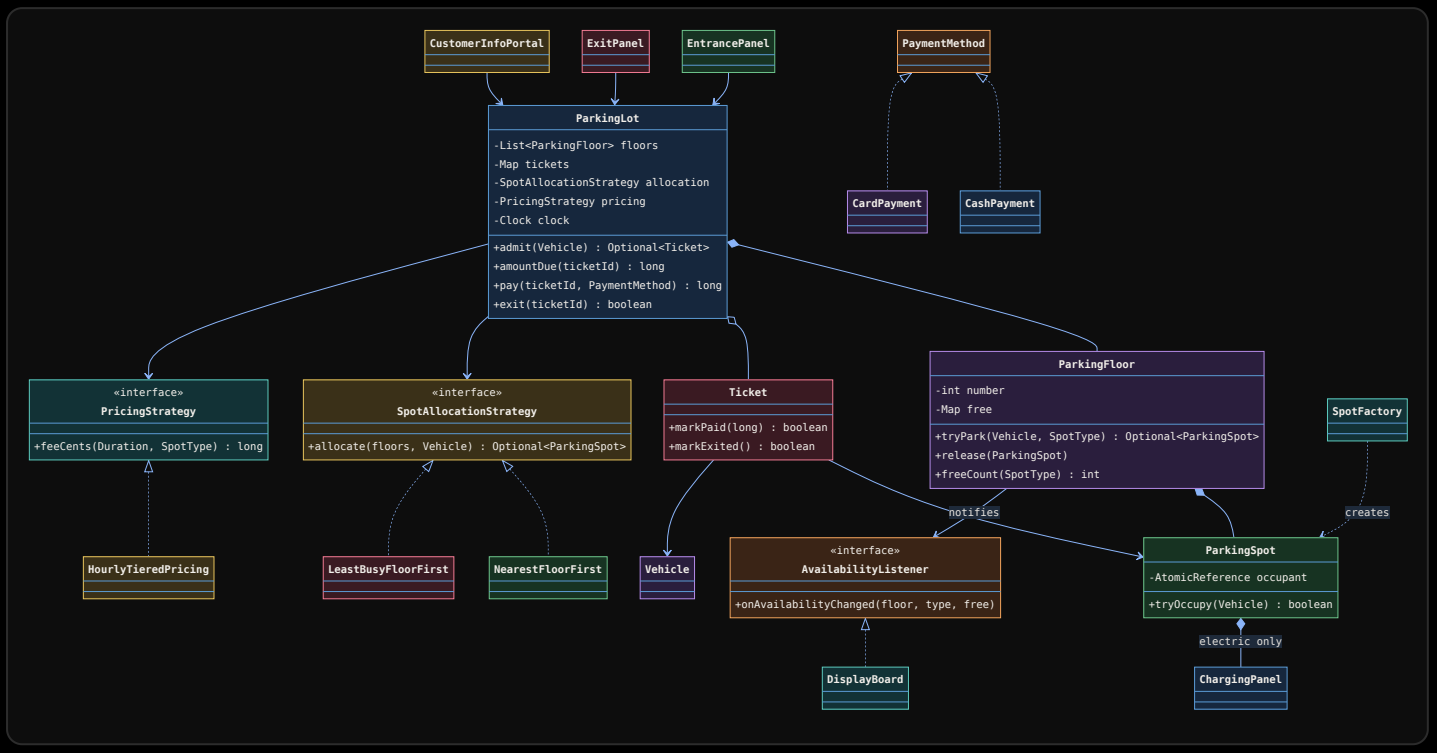


Figure — The finished class diagram of the parking lot (course diagram)

Our Java code keeps the same shape with three changes interviewers push for: spot types are an **enum** (not five empty subclasses), pricing and spot-finding are **Strategies**, and the display board is an **Observer**.



Key flows

- **Entry:** `EntrancePanel` → `lot.admit(vehicle)` → strategy tries each preferred spot type on each floor → `floor.tryPark` does `pollFirst()` on the free set (**atomic**) and notifies the board → ticket printed, or "LOT FULL".
- **Exit:** `ExitPanel` → `lot.pay(ticketId, card)` **locks the ticket**; if ACTIVE, price it and charge, then mark PAID; if already PAID at the portal, charge 0 → `lot.exit` moves PAID → EXITED and releases the spot → barrier opens.
- **Ticket life cycle:** `ACTIVE → PAID → EXITED`.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Fit rule on `VehicleType`	Information Expert, OCP	New vehicle type = one enum constant	`switch(vehicleType)` in the lot
Spot type is an **enum field**	Composition over inheritance	The 5 course subclasses had no behaviour	`CompactSpot`, `LargeSpot`…
`SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy, VIP… swap freely	`if (mode == NEAREST)`
`PricingStrategy`	**Strategy**	"What if pricing changes?"; admin swaps at run time	Tiers hard-coded in `Ticket`
`SpotFactory` attaches `ChargingPanel`	**Factory**	One place knows "electric ⇒ has a panel"	Every caller remembers
Boards subscribe to floors	**Observer**	Boards, apps, bay lights update without the floor knowing them	Lot pushing into boards
Ticket holds the **spot**	—	Exit frees the exact spot	Counting occupied spots
Lot is **not** a Singleton	Dependency injection	A Singleton is a hidden global; tests can't build two lots	`ParkingLot.getInstance()`
`Clock` injected	Testability	Demo jumps hours instantly	`Instant.now()` inside

> **Honest note on "Strategy".** A cash/card subclass pair is just an abstract base class; Strategy means an object **holds** a swappable behaviour (`setPricing(...)`). The course admits the lot does two jobs; spot-finding moved to strategy and floors, and tickets could move to a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are **cents** (`long`) to avoid floating-point rounding.

```
enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    // ...
}

class ParkingFloor {
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    // ...
    /** pollFirst() is atomic: two threads can never receive the same spot. Lowest id = nearest. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }
    // ...
}

final class ParkingLot {
    // ...
    Optional<Ticket> admit(Vehicle v) {
        String placeholder = "pending";
        if (ticketByPlate.putIfAbsent(v.plate(), placeholder) != null) {
            throw new IllegalStateException(v.plate() + " is already inside");
        }
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) {
            ticketByPlate.remove(v.plate());
            return Optional.empty();
        }
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }
    // ...
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);              // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
    // ... exit(): markExited() (PAID -> EXITED), then floor.release(spot)
}
```

Full runnable code: code/lld/ch14/parkinglot/ (see the Full Edition).

How it works: `Vehicle.preferredSpots()` puts `HANDICAPPED` first for permit holders only. `ParkingSpot.tryOccupy` (`compareAndSet(null, v)`) is a second safety net. `NearestFloorFirst` and `LeastBusyFloorFirst` are interchangeable strategies.

Output (excerpt):

```
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false
== 20 cars race for the remaining CAR spots ==
cars admitted: 5, distinct spots: 5
```

4 h 50 min rounds to 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**; 20 racing threads get exactly 5 distinct spots.

Concurrency & edge cases

Situation	Risk	Handling
Two gates, last spot	Double booking	Atomic `pollFirst()` + CAS `tryOccupy`
Portal and exit pay the same ticket	Charged twice	`synchronized (ticket)`; a PAID ticket costs 0
Same plate enters twice	Two tickets	`ticketByPlate.putIfAbsent`
Card declined	Marked paid anyway	`method.pay` throws **before** `markPaid`
Leaves without paying	Fee lost	`exit` only from PAID; barrier stays closed
Pays at portal, stays 3 more hours	Free parking	**Grace period** (e.g. 15 min); add `paidAt`, re-price after
Board shows an old number	Out-of-order notifications	OK for display; board can **pull** `freeCount` on render
Lost ticket / 0-minute stay	No entry time / $0	Daily maximum or plate lookup / minimum one hour
Lock scope	Global lock serialises gates	Per-spot CAS and per-ticket lock only

`ConcurrentSkipListSet.size()` is not constant-time and only an estimate under change — fine for a board, never for deciding a spot is free (use `pollFirst()`).

Extensions & follow-up questions

1. **Pricing changes** (weekend, per spot type, surge) → new `PricingStrategy` (or a decorator), `setPricing`. Nothing else changes.
2. **Reservations** → `Reservation(plate, spotType, from, to)` + reserved set per floor; hold with a timeout (e.g. 30 min).
3. **Monthly passes** → pricing returning 0 for pass plates, or a `PassHolderPricing` decorator.
4. **Plate recognition** → the camera is another entrance panel calling `admit`.
5. **Database** → tables `floor`, `spot`, `ticket`, `payment`; claim with `UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL` (the DB equivalent of CAS). Show **cardinalities** (floor 1..* spots, spot 0..1 vehicle, ticket 1 spot).
6. **Many lots in a city** → a `ParkingNetwork` routing to lots — another reason for no Singleton.
7. **EV idle fees** → a charging session on `ChargingPanel` with a per-minute fee after charging completes.

Real-world example

Parking guidance systems (per-bay sensors, free counts per level) are our `DisplayBoard` observers; airports use **plate recognition**; SpotHero/ParkMobile pre-booking = reservations; Tesla **idle fees** = the EV extension.

In the interview

45 minutes: 0–5 questions/requirements → 5–10 use cases → 10–25 stages 1–4 ("fit rule on the vehicle", ticket holds the spot) → 25–38 code `admit`, `tryPark`, `pay`, `exit` → 38–45 concurrency and extensions.

> **30-second answer:** "Vehicles and spot types are enums with the fit rule on the vehicle; floors keep a concurrent sorted set of free spots per type; the lot admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates never get the same spot, and payment locks only the one ticket."

- *"Why not a Singleton?"* — It is a hidden global that makes testing two lots impossible; I create one and inject it. If pressed, a lazy holder-class Singleton is thread-safe.
- *"How do you make finding a spot fast?"* — Keep only free spots, indexed by type, in a sorted set: O(log n), no scan.

Self-check

1. **Q:** Why must the ticket store the spot? **A:** So the exit frees exactly that spot; a count can't say which.
2. **Q:** What makes assignment thread-safe? **A:** `pollFirst()` gives a spot to exactly one caller; `compareAndSet` double-checks.
3. **Q:** Where does a weekend price go? **A:** A new `PricingStrategy` via `setPricing`.
4. **Q:** Paid at the portal — what does the exit charge? **A:** Nothing: `amountDue` is 0 for PAID; `exit` moves PAID → EXITED.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference; status ACTIVE → PAID → EXITED.
- **Strategy** (pricing, spot-finding), **Factory** (electric spots), **Observer** (boards); inject the lot, no Singleton.
- Atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine

A **vending machine** is an unattended shop: key in a slot code, pay, take the item from the tray. The interviewer skips the domain and checks one thing: **which actions are legal at which moment**. Accepting a coin with nothing chosen, or two items for one payment, are bugs no tidy code elsewhere can excuse.

Problem & clarifying questions

Question	Assumed answer
Select or pay first?	Select a slot code first, then pay.
Which money?	Fixed set: 5¢, 10¢, 25¢, $1, $5. Anything else is returned.
Card / contactless?	Out of scope (extension).
Change?	Yes. If **exact** change is impossible, refuse the sale and return everything.
Cancel?	Any time before dispensing, full refund.
Concurrency?	One customer at a time (one physical machine).
Restocking?	An operator restocks, loads coins, collects takings.

Requirements

Functional (the course's twelve): (1) many **slots**, each with a code, a **single** product and a **count**; (2) a product has a name and a **price in cents**; (3) **choose, then pay**; (4) only known coins/notes, others returned; (5) the **display** says what to do after every action; (6) when paid enough, release **item and change**; (7) if **exact change is impossible**, refuse and return every cent; (8) **cancel** with full refund until the item drops; (9) an empty slot **cannot be chosen**; (10) one payment = **exactly one** item; (11) an **operator** refills, adds change, removes takings; (12) item and cash counts are **exactly right** after every sale, cancel and refusal.

Non-functional: no illegal transitions (every button answered in every state); integer cents only; extensible (new payment device or state without editing every class).

Use cases

Actors: Customer, Operator, System. **Use cases:** select · insert money · dispense · collect · cancel · restock · load change / collect takings. "Dispense" is a **System** use case — the customer only causes it.

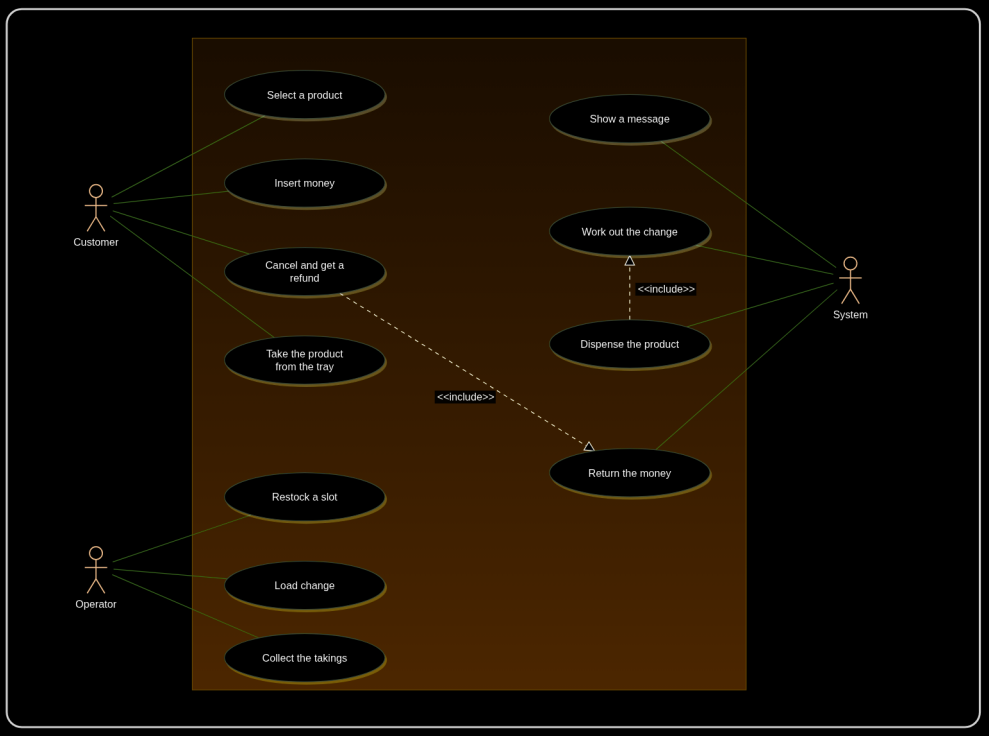


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents — *what* is sold.
`Slot`	One column: code, one product, **count** left.
`Inventory`	All slots by code.
`Denomination` (enum)	Accepted coins/notes and their cents.
`CashBox`	Counts pieces by value; plans **exact** change or reports it can't.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations; each answers every button and names the next state (course calls `HasMoneyState` `PaidState`).
`Display`, `Keypad`, `CashSlot`, `Dispenser`, `Operator`	Course hardware/actor classes; in our code folded into `VendingMachine.show(...)` and its public/operator methods.
`VendingMachine`	Parts, current state, selection, balance; forwards each button to the state.

Why is the count on the slot, not the product? `Product` answers "what is it?"; `Slot` is the physical column. The same product can sit in two columns (even at different prices), and emptying one must not affect the other.

Class diagram

Course stages: (1) `Product`, `Slot` (association — a product can be in many slots), `Inventory` (composition); (2) `Coin`, `Note`, `CashBox` — **a piece goes into the box on insert**, so a refund is "pay exact change equal to the balance"; the planner returns an exact plan or "impossible"; (3) **the heart: one class per state**, each button a method on one interface — a state that forgets a button does not

compile; think of a **grid** of buttons × states where every cell needs an answer; (4) `Display` and `VendingMachine` — no `if` on state; (5) `Keypad`, `CashSlot`, `Dispenser`, `Operator` point **to** the machine; `take` finishes the sale; operator actions stay **off** the state interface.

The course draws four states; our code adds `OutOfStockState` so an empty machine refuses customer buttons until restocked. Cancel and "no exact change" end identically, so there is a **single** refund method.

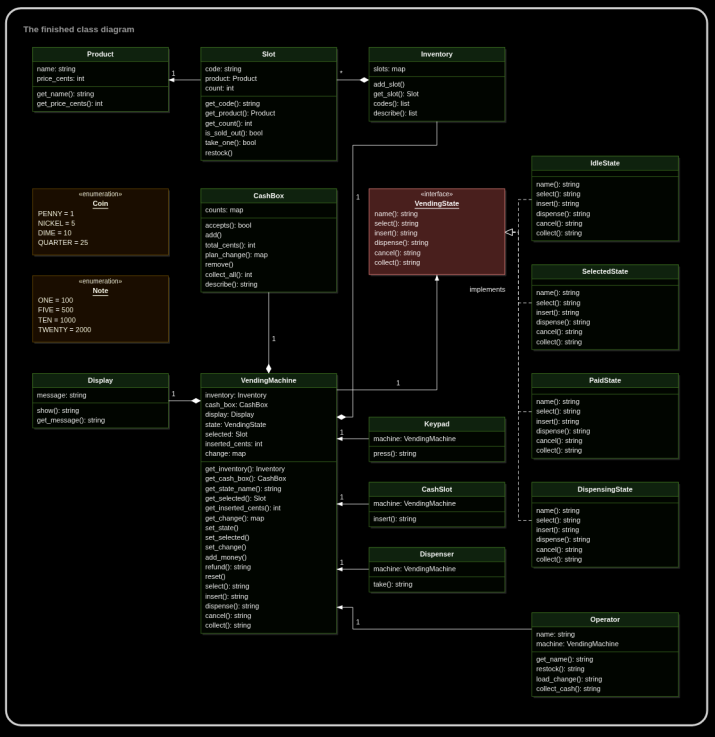
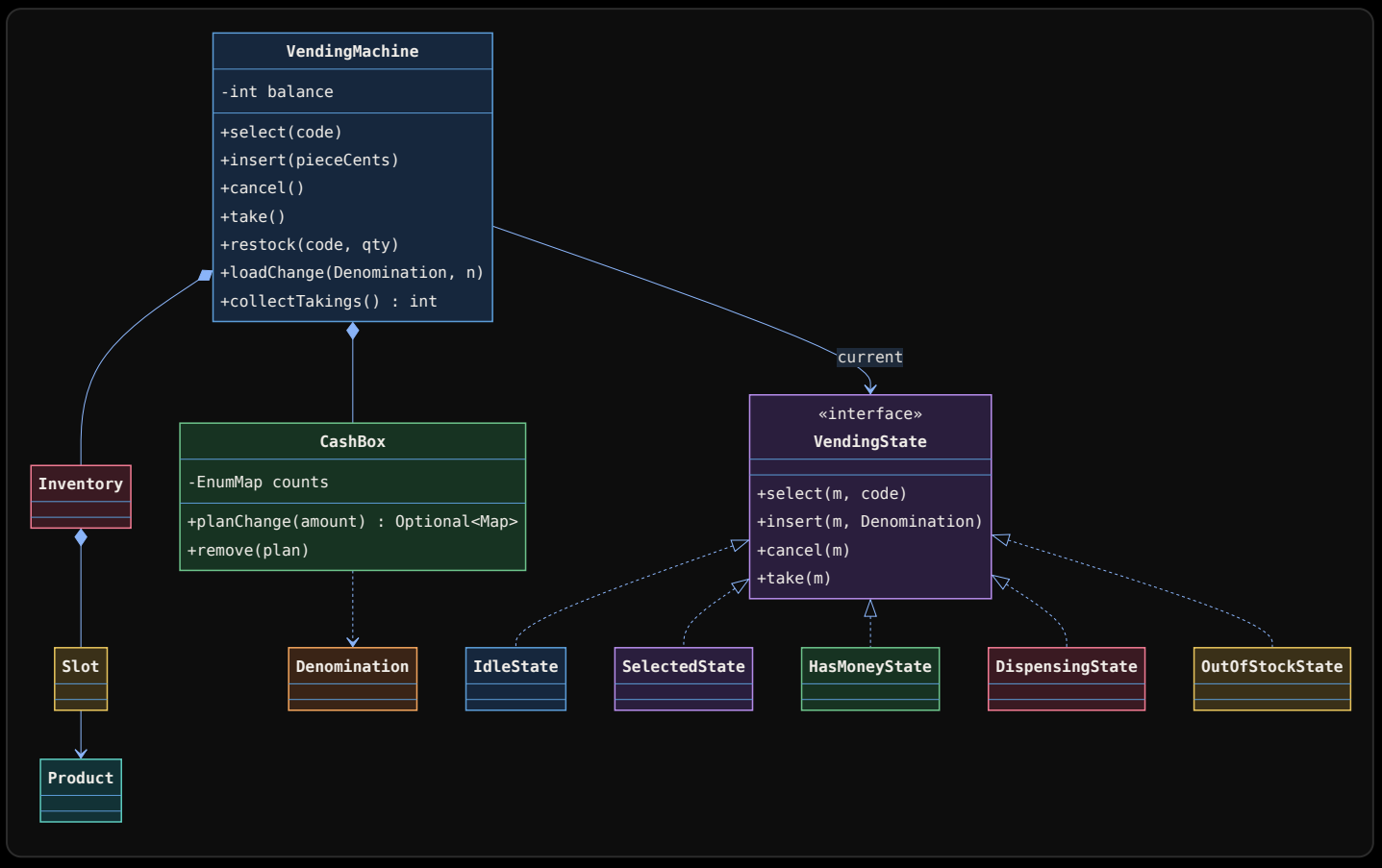
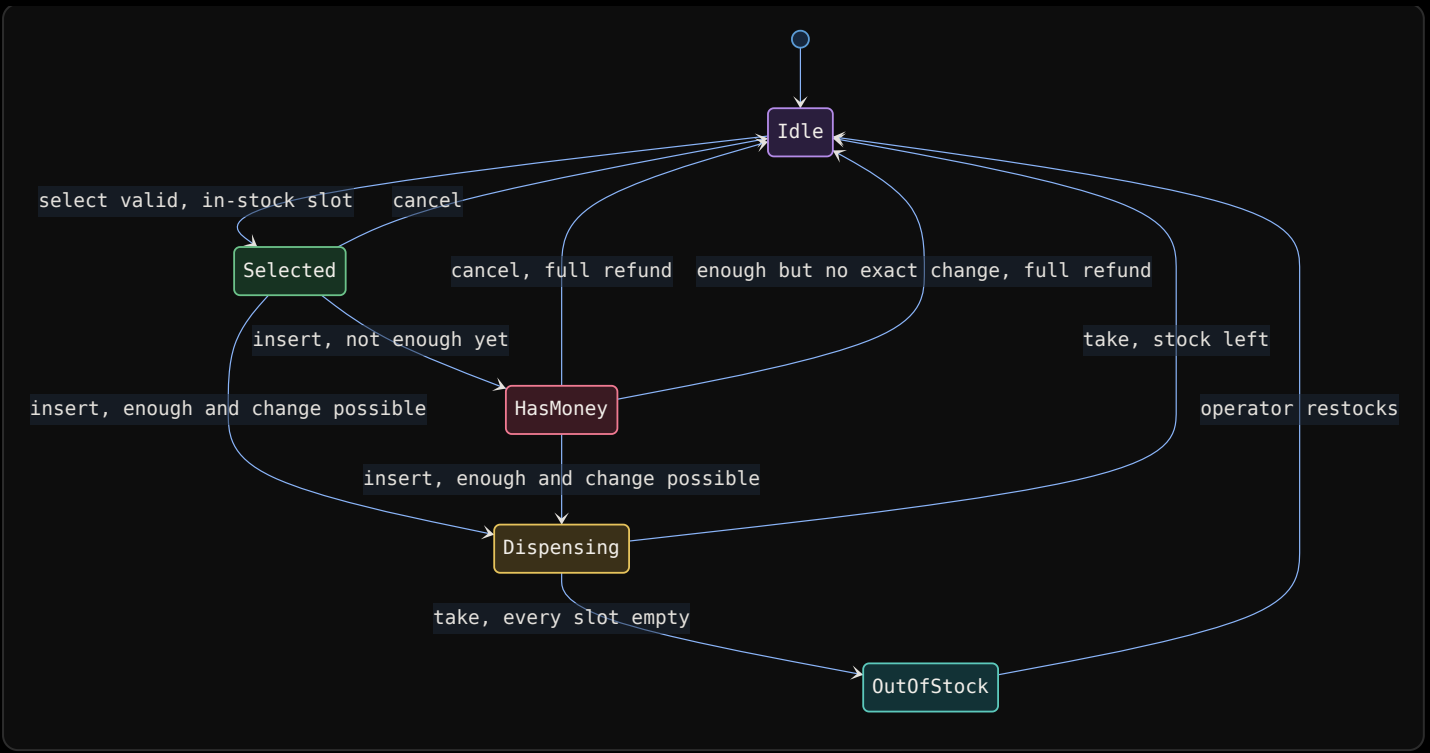


Figure — The finished class diagram of the vending machine (course diagram)



Key flows

The state machine is the design — draw it before coding.



State table (every cell answered):

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough: insert → `acceptPiece` (into box, balance += value) → `checkPaid` → `planChange(balance − price)`; if a plan exists, remove it, `dispenseOne`, go to Dispensing; otherwise refund all and go Idle.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation	**State**	Every cell answered; compiler enforces completeness	Flags + `if/else` (double-dispense bugs)
States are stateless singletons (`INSTANCE`)	Flyweight-style sharing	No data, so one instance each	`new IdleState()` each transition
Operator actions off the state interface	Interface Segregation	Restock is valid in every state	`restock` on `VendingState`
Money counted by value in an `EnumMap`	—	Which physical quarter comes back doesn't matter	List of coin objects
Inserted money straight into the box	—	Refund = make change for the balance	Separate escrow pile
Exact change by dynamic programming	Correctness	Greedy fails when short of one coin	Greedy only
Refuse if exact change impossible	Req. 7	Never short-change	"Close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first) and says so: correct for standard coin sets when well stocked, but it can fail when the box is short. The fix is **dynamic programming** — "coin change with limited coins". Example: 30¢ from {1 quarter, 3 dimes}: greedy takes the quarter and is stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`.

```java
/** State pattern: every button is on the interface, so every state must answer every button. */
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

/** Some money is in, but not enough yet. */
final class HasMoneyState implements VendingState {
    static final HasMoneyState INSTANCE = new HasMoneyState();

    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
    @Override public String name() { return "HAS_MONEY"; }
}
// ... IdleState, SelectedState, DispensingState, OutOfStockState

final class VendingMachine {
    // ...
    /** Called after every accepted piece in SELECTED / HAS_MONEY. */
    void checkPaid() {
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) {                              // refuse the sale rather than short-change
            refundAll("Cannot make exact change.");
            return;
        }
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    /** One refund path for "cancel" and for "cannot make change". */
    void refundAll(String reason) {
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }
    // ...
}
```

Full runnable code: code/lld/ch14/vendingmachine/ (see the Full Edition).

How it works: unknown pieces are returned before reaching a state. `CashBox.planChange` is bounded coin-change DP (fewest pieces); `greedyPlan` is only for comparison. `refundAll` cannot fail — the inserted pieces are in the box. Public methods are `synchronized`.

Output (excerpt):

```
> insert 500c   (SELECTED)
  [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]
```

Concurrency & edge cases

Edge case	Handling
Money before selecting	`IdleState.insert` returns it
Unknown coin / foreign note	Returned before reaching the state
Empty slot selected	Refused; stays Idle
Change choice after paying	"Cancel first" (avoids price confusion)
Overpayment	Change planned **before** dispensing; refuse if impossible
Cancel after dispensing	"Too late"
Two products for one payment	Impossible: balance reset to 0, Dispensing accepts no money
Last item sold	`take` → `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods
Machines sharing one stock / power failure	Slot count needs a lock or atomic decrement / non-volatile transaction log (out of scope)

Extensions & follow-up questions

1. **Contactless card** → a new input device calling the same machine (e.g. `payByCard(amount)` setting the balance); states unchanged.
2. **Maintenance state** → a new `MaintenanceState` class; existing states untouched (OCP).
3. **Different price per slot** → move price from `Product` to `Slot`.
4. **Report empty slots** → `Inventory` notifies an observer; a telemetry client sends it.
5. **Walk-away timeout** → a timer calls `cancel()` after ~60 s through the same state machine.
6. **What did you leave out?** Card payment, network reporting, refrigeration, discounts — say they are scope decisions.

Real-world example

Cashless readers (**Nayax**, **Cantaloupe ePort**) plug in as another payment device; coin mechanisms light **"exact change only"** when a tube runs low — our planner saying it cannot make change.

In the interview

45 minutes: 5 questions → 5 entities → **10 draw the state diagram and table** → 15 code the interface, two or three states and `checkPaid`/`refundAll` → 10 edge cases.

> **30-second answer:** "The question is the order of events, so I use the State pattern: Idle, Selected, HasMoney, Dispensing, OutOfStock, all implementing an interface listing every button. The machine forwards to its state. Money goes into the box on insert, so a refund is 'make exact change for the balance'; if exact change is impossible the sale is refused and everything returned. Integer cents."

- *"Why not an enum with a switch?"* — Every new state means editing every switch; a State class is forced by the compiler to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets; use bounded coin-change DP.

Self-check

1. **Q:** What makes the design complete? **A:** Every state implements every button, so every (state, button) cell has an answer.
2. **Q:** Why is `restock` not on `VendingState`? **A:** Not a customer action, valid in every state (ISP).
3. **Q:** A greedy failure? **A:** 30¢ with one quarter and three dimes.

4. **Q:** How is "two items for one payment" prevented? **A:** Balance reset to 0 and Dispensing returns any coins until `take`.

TL;DR

- **State pattern is the design**: five states, one interface listing every button, no `if` on state.
- Count on the **slot**; **integer cents**; money into the **box immediately**.
- Change **exact or refuse**; greedy can fail, **DP** is correct.
- One refund path; operator actions off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee machine** makes drinks (espresso, americano, latte, cappuccino, mocha, hot chocolate) from **shared ingredients** (beans, water, milk, chocolate, sugar). Unlike 14.2, a drink **consumes several ingredients at once**, and many drinks draw on the same milk. The interviewer wants: (1) **recipes as data** — a new drink is a data change; (2) **atomic, all-or-nothing deduction** — a latte takes beans, water *and* milk together or nothing, never negative even with ten concurrent orders; (3) **low-stock alerts** via Observer before an ingredient runs out.

Problem & clarifying questions

Question	Assumed answer
Drinks?	Six, loaded from a menu file.
Customisation?	Extra espresso shot (+50¢), sugar in grams.
Payment?	Amount in cents; change returned (coins covered in 14.2).
Concurrent orders?	Yes — several touch screens.
Refills?	Operator, alerted at a threshold.
Alert frequency?	**Once per crossing** of the threshold, re-armed after refill.

Requirements

Functional: (1) drinks with a **price** and a **recipe**; (2) **menu** with prices and availability; (3) selection and **payment**, **change**, full refund if the drink can't be made; (4) track inventory, deduct exactly the recipe; (5) **low-stock notifications**; (6) **refills**; (7) **customisation** without changing the menu.

Non-functional: thread-safe (no over-draw, no half recipe); data-driven; observable.

Use cases

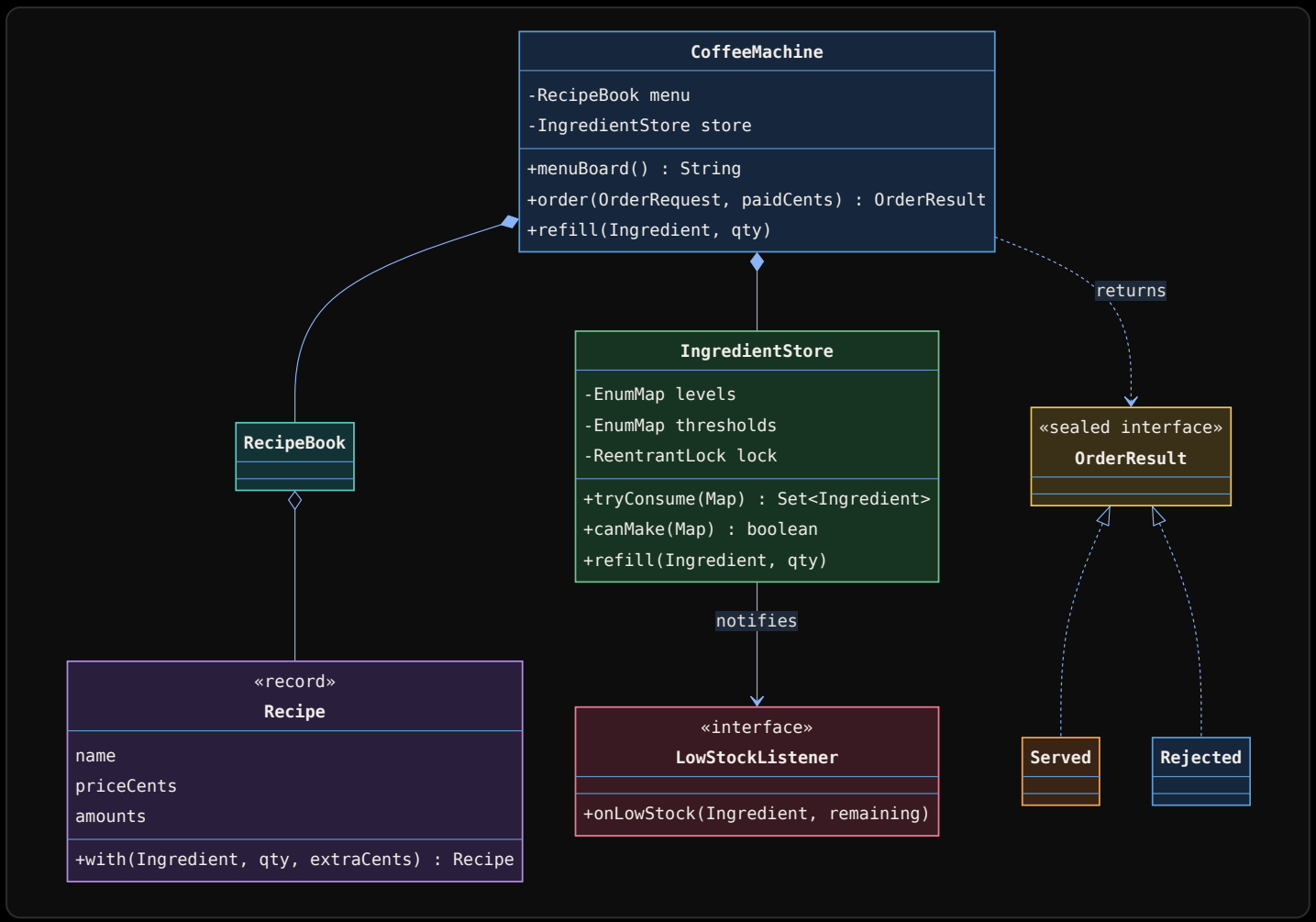
Actors: Customer (view menu, order, pay, take drink/change), Operator (refill, receive alerts), System (check stock, deduct, brew, alert). **Use cases:** view menu · order (with customisation) · pay / change / refund · refill · receive low-stock alert.

Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with unit (g/ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient,Integer>`. Immutable; `with(...)` returns a customised copy.
`RecipeBook`	Parses menu text into recipes; lookup by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts.
`LowStockListener`	Observer called once per threshold crossing.
`OrderRequest` (record)	Drink + extra shots + sugar grams.
`OrderResult` (sealed)	`Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade: menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement (*awesome-low-level-design*): `Coffee` → `Recipe`; `Ingredient` with `synchronized` updates → enum + levels in `IngredientStore`; `Payment` → `paidCents` and `OrderResult`; Singleton → injected machine. We lock the **whole recipe**, not each ingredient (per-ingredient locks can't make a multi-ingredient deduction all-or-nothing).

Class diagram



Key flows

Ordering: find recipe → apply customisation (`recipe.with(...)`) → if underpaid, `Rejected` with full refund → `tryConsume(amounts)`: lock, check **every** ingredient; if any shortage, nothing is deducted and the

order is `Rejected`; else deduct all, note thresholds crossed, **unlock, then** notify listeners → `Served` with change.

Order life cycle: `Selected → Reserved (ingredients deducted atomically) → Brewing → Served`, or `Selected → Rejected` (unknown drink, underpaid, or short) with full refund.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** from menu text	Data-driven, OCP	New drink = new line	`Latte extends Coffee` (class explosion)
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Stops two orders both seeing "enough milk" (check-then-act)	Separate check and deduct; per-ingredient locks
All-or-nothing	Transaction semantics	No half-made recipe	Deduct one by one and roll back
`LowStockListener`	**Observer**	Operator app, dashboard, telemetry subscribe	Store calling an SMS class
Alert **once per crossing**	—	No alert storm	Alert on every order below threshold
Listeners **outside** the lock	Concurrency hygiene	Slow listener can't block orders; no deadlock on callback	Notifying inside the lock
Customisation returns a **new** `Recipe`	Immutability	Menu recipe never corrupted	Mutating it; a Decorator chain (heavier)
Sealed `OrderResult`	Algebraic data type (Java 17+/21 switch)	Callers must handle both; exhaustive `switch`	`null` or exceptions for normal refusals
No Singleton	DI	Tests build fresh machines (demo builds two)	`getInstance()`

> **Decorator or data?** *Head First Design Patterns* models add-ons as Decorators (`new Whip(new Mocha(new DarkRoast()))`). Good when add-ons have **behaviour**. Here they only change **quantities and price**, so `recipe.with(...)` is simpler. Say both and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`.

```java
/** A recipe is DATA: a name, a price and the amount of each ingredient. */
record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {
    // ...
    /** Customisation returns a NEW recipe; the menu entry is never modified. */
    Recipe with(Ingredient extra, int qty, int extraCents) {
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}

final class IngredientStore {
    // ...
    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;              // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i)) {    // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
                }
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);   // call listeners OUTSIDE the lock
        return shortages;
    }
    // ...
}
```

Full runnable code: code/lld/ch14/coffeemachine/ (see the Full Edition).

How it works: the menu is a Java text block (`Latte, 400, COFFEE_BEANS=18, WATER=40, MILK=180`) parsed by `RecipeBook.parse`. `Recipe`'s compact constructor stores an unmodifiable `EnumMap`. `refill` clears the `alerted` flag when the level goes back above the threshold. `CoffeeMachine.order` validates drink → customises → checks payment → reserves ingredients.

Output (excerpt):

```
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Concurrency & edge cases

Situation	Risk	Handling
Ten orders, milk for four	Negative stock (check-then-act)	Check + deduct under one lock
One ingredient short	Beans taken, milk missing	Shortages found before any deduction
Slow listener (SMS)	Orders wait on the network	Listeners after `unlock()`
Many orders below threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving
Brew fails after deduction	Pays, gets nothing	Extension: refund, optionally refill the amounts (compensation)
Menu says available, order fails	Display/order race	Menu is advisory; `tryConsume` is the truth

One lock is right: the critical section is tiny and the machine brews one drink at a time. For a cloud inventory shared by many machines, use optimistic concurrency (version / CAS on a snapshot) or a DB transaction.

Extensions & follow-up questions

1. **New drink** → one menu line. 2. **Sizes** → `recipe.scaled(size)`. 3. **Several brewers** → a `BlockingQueue` consumed by N `Brewer` threads, one store. 4. **Remote monitoring** → another `LowStockListener`. 5. **Different brewing steps** → Template Method or `Step` objects (Command). 6. **Cash** → reuse 14.2's `CashBox`.

Real-world example

The Thermoplan Mastrena (Starbucks) stores **recipes as configuration**; Starbucks and Microsoft (2019, Azure Sphere) push recipes and collect telemetry from the cloud; connected kiosks report stock remotely — the low-stock Observer.

In the interview

40 minutes: 5 questions → 5 entities → 5 "recipes are data" and the menu format → 15 code `tryConsume` and `order` → 10 concurrency and extensions.

> **30-second answer:** "Recipes are data — ingredient-to-amount maps loaded from a menu, so a new drink is a new row. The store checks and deducts **every** ingredient of a recipe under one lock: all-or-nothing, never negative. Crossing a threshold is noted and observers are notified once, after the lock is released. Orders return a sealed Served/Rejected result; rejected orders are refunded in full."

- *"Why not an `AtomicInteger` per ingredient?"* — Each is safe alone, but a recipe updates several at once; without a common lock you get partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

Self-check

1. **Q:** All-or-nothing for `tryConsume`? **A:** Every ingredient deducted, or none (shortages detected first).
2. **Q:** Which race does the lock prevent? **A:** Check-then-act: two threads see enough milk and both deduct.
3. **Q:** How is an alert storm prevented? **A:** An `EnumSet` of alerted ingredients, re-armed by refill.
4. **Q:** When use Decorator for add-ons? **A:** When they carry behaviour, not just quantities.

TL;DR

- **Recipes are data**; customisation makes a new immutable recipe.
- **One lock, check-then-deduct, all-or-nothing**.
- **Observer** alerts, **once per crossing**, **outside** the lock.
- **Sealed** `OrderResult` ; full refund on rejection.

14.4 Design an ATM

An **ATM** is a cashier-less machine where a bank's customers do everyday banking outside branch hours. The interviewer checks that you keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate first**, and model the **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Accounts / identification?	Checking and savings (daily cap); card + PIN, checked by the bank on every transaction.
Deposits available at once?	Checks **not** — they wait. Keep **total** and **available** balances.
Notes?	$100, $50, $20, $10 cassettes.
Wrong PIN?	3 wrong attempts block the card and the ATM keeps it (course leaves this out; we add it plus a journal).
Customers at a time?	One; the ATM must not shut down mid-session.

Requirements

Parts: card reader, keypad, screen, cash dispenser, deposit slot (cash/check), printer, network link to the bank.

Functional: (1) card + PIN authenticated **by the bank** first; (2) five transactions: balance inquiry, cash deposit, check deposit, withdraw, transfer; (3) the ATM **keeps the card** all session; cancel ejects it; (4) a **journal** of transactions and hardware faults; (5) a **check is held** until verified (the course holds all deposits; we credit counted cash at once); (6) **bank manager** reports.

Actors' operations: Operator — on/off key switch, refill cash, paper, ink, empty deposits. Customer — the transactions. Bank manager — total deposits/withdrawals reports, print them, check remaining cash.

Non-functional: security (no clear PINs on the ATM; limited attempts); consistency (debit and dispense together or compensate); concurrency (many ATMs, one account, no overdraft, no deadlock); auditability.

Use cases

Only Customer use cases need the PIN (Operator uses a key; Manager reads bank data). The course's activity diagrams (authenticate, withdraw, deposit check, transfer) all end by ejecting the card — **every** exit path, including declined retries, must return it.

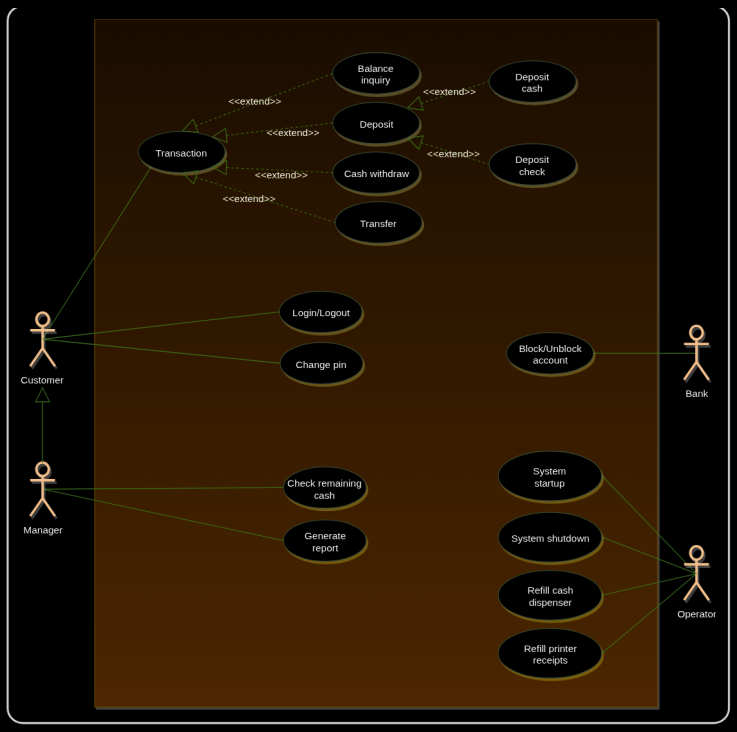


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

Core entities

Class	Responsibility
`Card` (record)	Number and holder (course also has `Customer` with one card).
`Account` (abstract)	Id, **total** and **available** balance, a lock, withdrawal-rule hook.
`CheckingAccount`, `SavingsAccount`	Savings adds a **daily cap** — a behaviour difference, hence subclasses.
`BankService` (interface)	Authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	Stand-in bank: PIN hashes, failed attempts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)`, `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** $100 → $50 → $20 → $10.
`CashDispenser`	Plans (dry run), then dispenses; can jam (fault injection).
`Transaction` (sealed) + 5 records	**Command**: one `execute(ctx)` each.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated, plus OutOfService.
`Atm`	Id; owns parts, session, journal; forwards buttons to state. Course hardware (`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot`) is folded in.

Class diagram

Course stages: (1) **money and card** — two balances (a check raises *total* now, *available* after clearing); savings overrides one method (behaviour ⇒ inheritance); (2) **the parts** — none knows what a transaction is; whether $85 can be made from loaded notes is the **dispenser's** job; the reader **keeps the card** all session; `DepositSlot` (the opening) ≠ the deposit *transaction*; (3) **five transactions**, one `execute` each; the course's `Withdraw` checks **denominations → account → dispenser** and debits only after notes are counted; (4) **the machine** owns its parts and holds the session (one customer at a time); the `Bank` owns its ATMs.

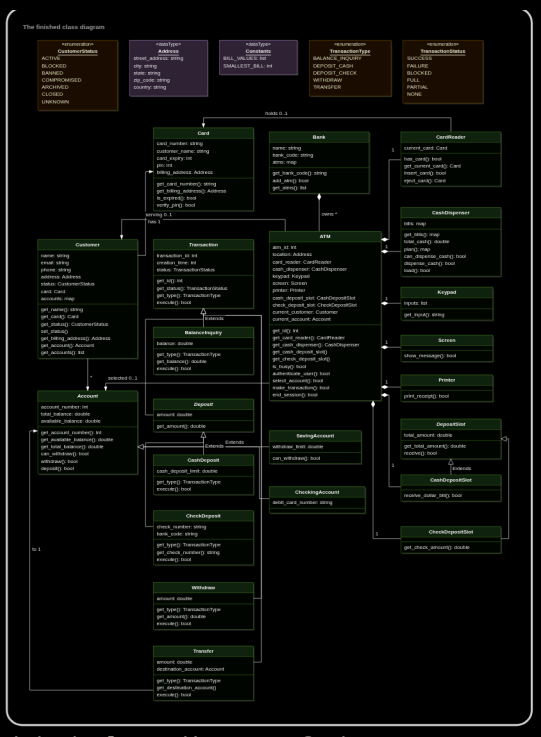
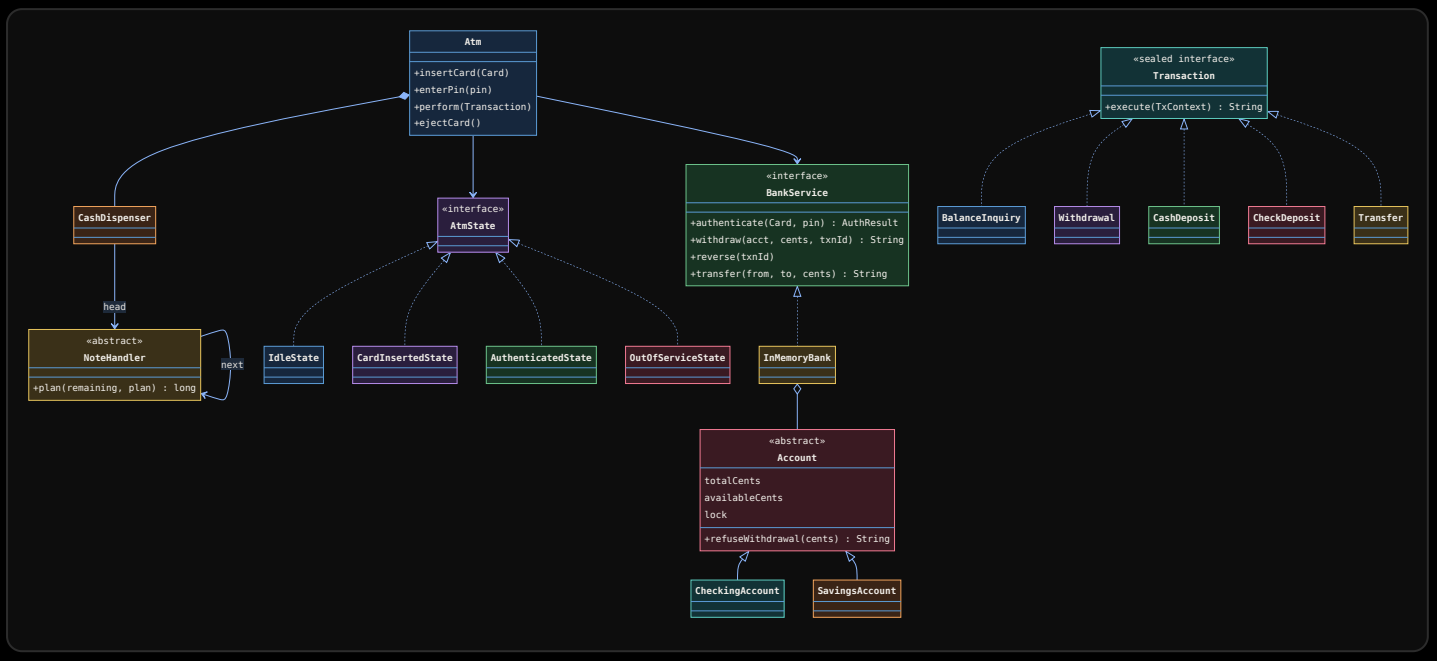


Figure — The finished class diagram of the ATM system (course diagram)



Key flows

Session states: Idle → CardInserted → Authenticated (PIN OK) → Idle (finish/cancel, card ejected). **3rd wrong PIN → Idle, card retained.** Session ends with no cash → OutOfService (card out first); key switch only when Idle; refill → Idle. **Balance inquiry:** the ATM creates a transaction, which asks the account; the result goes to the screen.

Withdrawal with compensation (our code): multiple of $10? → `plan(280)` dry run on the note chain (2×100, 1×50, 1×20, 1×10) → bank `withdraw` (lock account, check rule, debit) → `dispense(plan)` → on **jam**, `reverse(txnId)` and report "account NOT charged" → journal the result.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session as state classes	**State**	Each button means something different per state	Booleans + nested `if`s
Dispenser as chain of note handlers	**Chain of Responsibility**	Each takes what it can, passes the rest; a $5 cassette is one new handler	Hard-coded note list
Plan first, then dispense	—	Dry run touches nothing	Removing notes while searching
Each transaction = one class	**Command**	Rules in one place; can be journaled, queued, replayed	`switch (type)` in the ATM
`Transaction`, `AuthResult` sealed	Sealed types	Exhaustive `switch`	`instanceof` chains
ATM → `BankService` interface	Dependency Inversion	In-memory, ISO 8583, or a test fake	ATM changes balances
Total vs available	Domain modelling	Check holds correct	Single balance
Savings subclass	Inheritance for **behaviour**	Overrides one hook	`if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** counting notes (safer for the customer) and admits a crash in between leaves the bank short. Our code debits then dispenses, with a **compensating** `reverse(txnId)` on a jam. Real systems make the pair atomic (**hold/authorise, dispense, confirm**) and **reconcile** from the journal. This is the "power fails halfway" follow-up.

> **Cash deposits.** Many banks make machine-counted cash available immediately and hold checks. `depositCash` credits both balances, `depositCheck` only the total — a one-line policy change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`).

```
abstract class NoteHandler {
    // ...
    /** Dry run: fills 'plan', returns the amount that nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }
    // ...
}

record Withdrawal(String account, long cents) implements Transaction {
    static final long SMALLEST_NOTE = 10_00;

    public String execute(TxContext c) {
        if (cents <= 0 || cents % SMALLEST_NOTE != 0) return "Withdrawal refused: amount must be a multiple of
$10";
        var plan = c.dispenser().plan(cents);                                  // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());          // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                 // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                        // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }
    // ...
}

final class InMemoryBank implements BankService {
    // ...
    /** Locks both accounts in a fixed (id) order, so two opposite transfers cannot deadlock. */
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b;
        Account second = first == a ? b : a;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents;
                a.totalCents -= cents;
                b.availableCents += cents;
                b.totalCents += cents;
                return null;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch14/atm/ (see the Full Edition).

How it works: the bank stores a SHA-256 hash of card+PIN and counts failures; the 3rd returns `Blocked` and the ATM **retains** the card (demo only — real banks verify an encrypted PIN block in an HSM). `withdraw` checks and debits under the account's `ReentrantLock`. `finishSession` picks Idle or OutOfService only after the card is out.

Output (excerpt):

```
  [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
  [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
  [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
== Two ATMs, one account, same instant: $300 each from $500 ==
  cash handed out $300.00, JOINT left $200.00
== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
  successful=2000, A+B=$2000.00
```

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs, one account	Overdraft (check-then-act)	Check + debit under the **account lock**
A→B and B→A at once	**Deadlock**	Lock in a global order (account id)
Jam after debit	Charged, no cash	`reverse(txnId)` + journal
Amount not makeable	Partial dispense	Dry-run `plan` before touching the bank
Greedy chain misses a combination	$60 with one $50, three $20, no $10	OK for standard cassettes; DP planner (14.2) if asked
Many wrong PINs	Brute force	3 tries, block **at the bank**, retain card
Walks away / unlinked account	Misuse	Session timeout (extension) / `sessionAccounts` check
Cash runs out	Card trapped	Finish session, then `OutOfService`; refill → Idle
Key switch mid-session	Shuts down while serving	Refused unless idle

Extensions & follow-up questions

1. **Can't make the amount** → plan fails, refused before the bank is called.
2. **Power fails mid-withdrawal** → hold at the bank, dispense, confirm; on restart read the journal and confirm or reverse; bank reconciles.
3. **Daily limit for all accounts** → in `Account` or a policy object; ATM unchanged.
4. **Other banks' cards** → a routing `BankService` (Proxy/Adapter to the card network).
5. **Cardless** → a new state path `enterOneTimeCode`; transactions unchanged.
6. **Manager reports** → aggregate the journal; cash left = `dispenser.total()`. (Receipts / PIN change → more `Transaction` records.)

Real-world example

ATMs (NCR, Diebold Nixdorf) drive devices through **CEN/XFS** interfaces, talk **ISO 8583**, and use **ISO 9564** PIN blocks verified in HSMs; after a dispense error they reverse and reconcile from the journal.

In the interview

45 minutes: 5 parts, actors, transactions → 5 total vs available → 10 state diagram → 15 code `Withdrawal.execute` and the chain → 10 concurrency and follow-ups.

> **30-second answer:** "Parts behind interfaces, the session as a State machine, five transactions as Commands with one `execute`. The ATM never touches balances; it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans notes through a Chain of Responsibility, debits atomically at the bank, dispenses, and reverses on hardware failure. The bank locks per account; transfers lock in id order to avoid deadlock."

- *"Where is the PIN checked?"* — At the bank (really an HSM, from an encrypted PIN block).
- *"Two ATMs, joint account?"* — Check and debit together under the account's lock (or one DB transaction).

Self-check

1. **Q:** Why two balances? **A:** A check raises the total at once but must not be spendable until it clears.
2. **Q:** Safe order of debit and dispense? **A:** Neither alone; use hold + confirm (or debit + compensating reverse) and reconcile from the journal.
3. **Q:** How are transfer deadlocks avoided? **A:** Always lock both accounts in the same global (id) order.
4. **Q:** What happens after the 3rd wrong PIN? **A:** The bank blocks the card (every ATM refuses it) and this ATM retains it.

TL;DR

- **State** (session), **Command** (transactions), **Chain of Responsibility** (cassettes), `BankService` interface.
- **Total vs available** balance.
- Withdrawal = **plan → atomic debit → dispense → reverse on failure**, journaled.
- **Per-account lock**, transfers in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

A building's cars are run as a **group** by a shared controller. Two decisions carry the interview: **which car answers a hall call** (dispatching) and **in what order one car visits its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
Floors and cars?	Floors 0 … N-1, several cars serving all.
Buttons?	Hall UP/DOWN per floor (no UP on top, no DOWN on ground); car panel per floor plus door open/close.
Displays?	In car: floor, direction, load. Per floor: one display per car.
Scheduling?	Serve every stop ahead before turning — **LOOK**.
Dispatching?	Nearest suitable car; swappable.
Capacity?	Overloaded car doesn't move and says so.
Maintenance, emergencies?	Out-of-service cars; fire alarm recalls all to the lobby.
Real time?	Simulated: one `step()` = one action (testable without a clock).

Requirements

Functional: (1) many floors and cars; (2–3) hall and car panels; (4–5) car and floor displays; (6) a hall call **picks one car**; (7) a car button **adds a stop**; (8) **serve stops ahead** before turning; (9) **never move with doors open**; (10) **max load** — overloaded car stays and says so; (11) idle car waits where it finished; (12) **out of service** cars get no riders; (13) *(ours)* **fire recall**.

Non-functional: efficient (no useless trips to the shaft end); no starvation; safety overrides scheduling; thread-safe; deterministic stepping.

Use cases

Actors: Rider (call, ride, choose destination), Maintenance engineer (out of / back in service), System (choose car, move, doors, displays). **Use cases:** call · choose destination · dispatch · move one floor · open/close doors · report overload · take out of service. Dispatch and move belong to the System; the rider only triggers them.

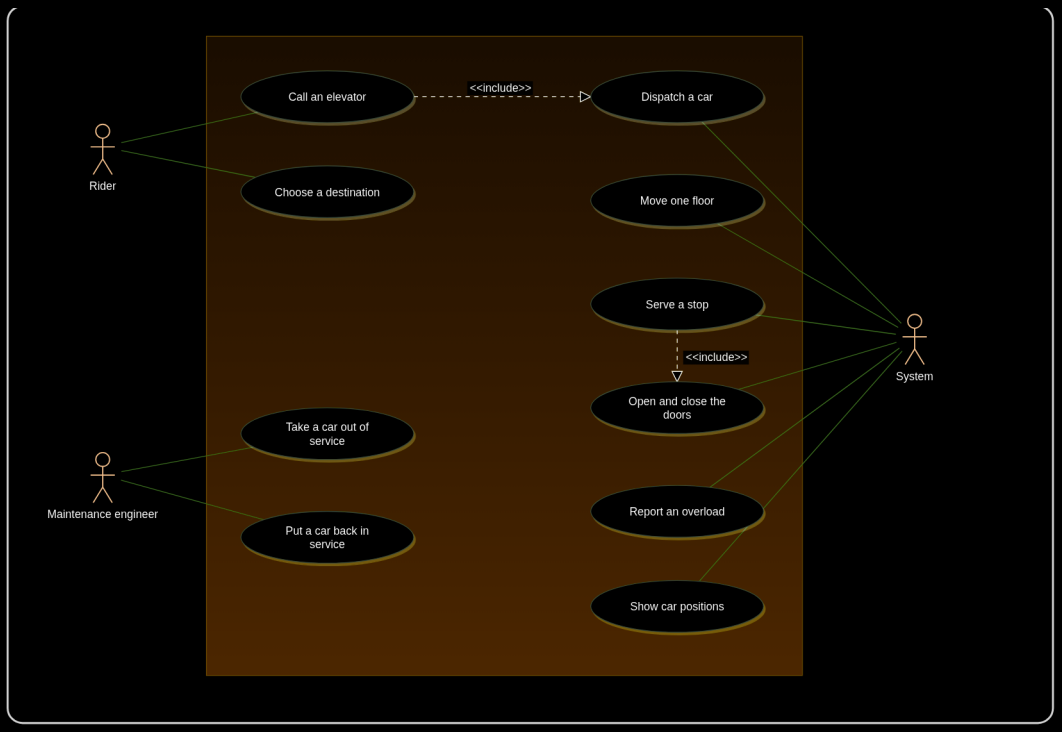


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	Floor **plus** direction on leaving — "5 going up" and "5 going down" are two stops.
`StopList`	Stops in **two sorted sets** (up, down); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	Floor, direction, mode, load, door, stops; `step()` does one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What a step did (MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING).
`DispatchStrategy` / `NearestCarStrategy`	*Which car* answers a hall call.
`Dispatcher`	Cars + one strategy; queues calls no car can take.
`Building`	Hall lights, cars, dispatcher; ticks cars; clears lights on arrival; fire alarm.

The course also models `Button`, `HallPanel`, `ElevatorPanel`, `Display` and `Floor` one-to-one; our code folds them — say which you chose.

Class diagram

Course stages: (1) `Button` family (shared light/reset behaviour) and `Door`; (2) panels, displays, `Floor` — all **composition**; (3) **most important:** `Stop` **and** `StopList` — two sets, not one list; the next stop is reachable **without reversing**, and with nothing ahead the car heads to the **farthest** request; `Elevator` **moves**, `StopList` **chooses**; (4) `Elevator` — `step` does **one** of: close doors, refuse (overload), move one floor, stop and open; it never touches a `Floor` or another car; (5) `DispatchStrategy`, `Dispatcher`, `Building` — the only class knowing cars and floors, so it clears hall lights.

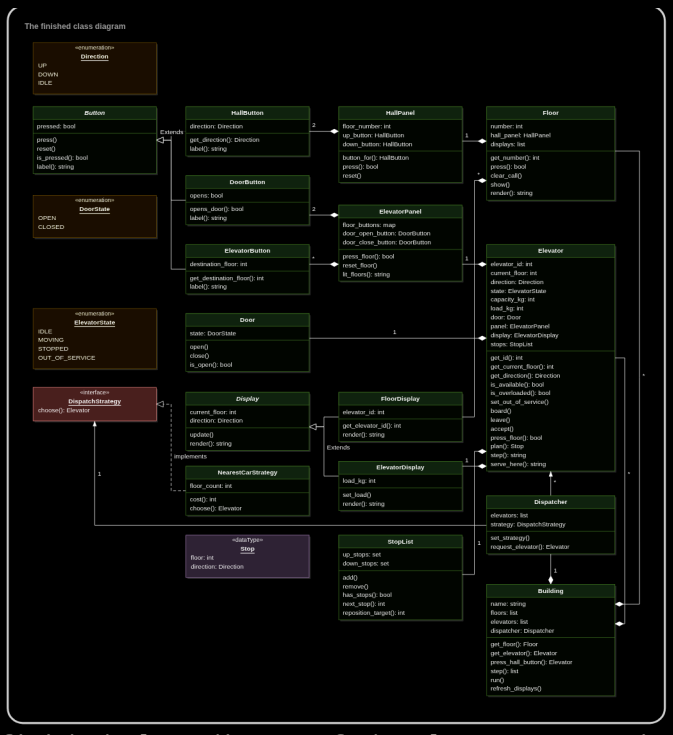
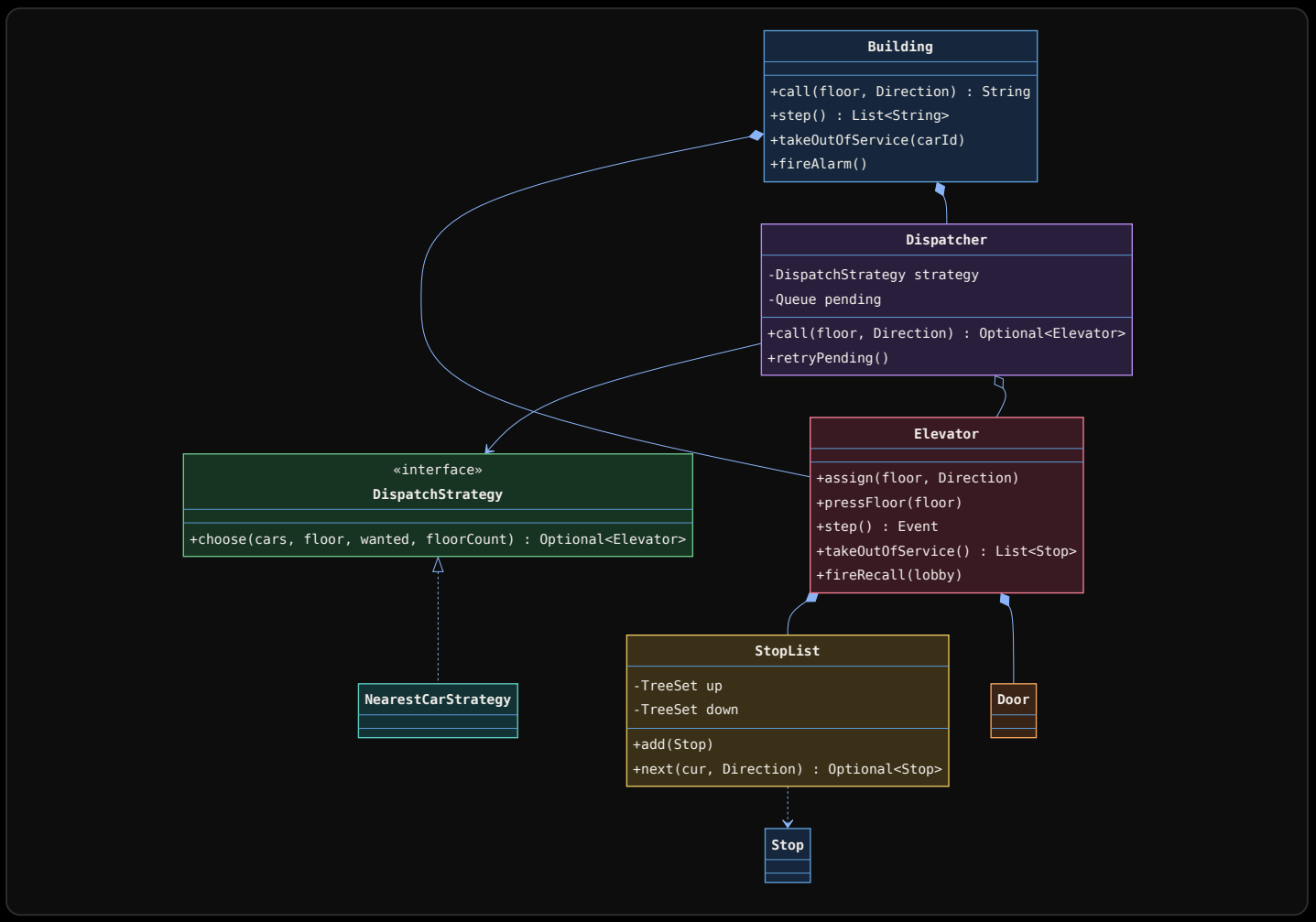


Figure — The finished class diagram of the elevator system (course diagram)



Key flows

Scheduling algorithms compared (borrowed from disk-arm scheduling — same shape: a head moving along a line of requests):

Algorithm	Rule	Good	Bad
FCFS	Arrival order	Simple	Zig-zags; very inefficient
SSTF	Nearest stop first	Short trips	**Starvation** of far floors
SCAN	Sweep to the **end of the shaft**, reverse	No starvation	Wasted travel to the ends
LOOK	Sweep only **to the last request**, reverse	No starvation, no waste	Slightly more bookkeeping
C-SCAN / C-LOOK	One direction; jump back to start	Uniform waits	Empty return trips

Our `StopList.next` (LOOK, direction-aware), going UP: (1) nearest up-stop at or above (`up.ceiling(cur)`); (2) else, if someone *above* wants DOWN, go to the **highest** such floor, then turn; (3) else turn: nearest down-stop at or below (`down.floor(cur)`); (4) else go down to the **lowest** up-stop and serve upward. DOWN mirrors it; an IDLE car picks the side of the nearest stop.

Car states: Idle → Moving → DoorsOpen (overload: stay) → Moving/Idle; Idle ⇄ Maintenance; any → FireRecall. **Hall call:** light button → strategy picks a car → `assign` → `step()` each tick → on ARRIVED, clear the light.

Dispatch cost (`NearestCarStrategy`, from the course): idle car = distance; car already coming this way **in the same direction** = distance; any other car = distance **+ two building heights**. Out-of-service cars are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`DispatchStrategy`	**Strategy**	Office rush vs quiet hotel; the *requirement* to swap justifies the interface	`if (officeMode)`
Sweep rule in `StopList`	SRP	Car drives, stop list decides	Next-stop logic in `step()`
Two sets (up/down)	Data modelling	"5 up" ≠ "5 down"	One list
`step()` = one action	Deterministic simulation	Tests = calls + expected events, no clock	Thread per car with `sleep`
Modes as an enum	Lightweight State	Only 3 modes; switch to State classes if modes grow	State classes from day one
Re-dispatch when a car leaves service	Robustness	Accepted calls never lost	Silently clearing stops
Queue calls when no car is free	—	"Not now" ≠ "rejected"	Dropping the call

Course's honest weak points: (1) **no threads** — a real controller locks the stop list (ours is `synchronized`); (2) the dispatcher **decides once**; (3) `Elevator` nearly does two jobs, hence `StopList`. Only **one** pattern (Strategy); out of scope: emergency calls, fire service, express cars, door sensors.

Java implementation

Package `lld.ch14.elevator`.

```java
final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();
    // ...
    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }

    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                          // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then
turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                        // turn around now
        return new Stop(up.first(), Direction.UP);                                // go down to lowest up-call
    }
    // ... goingDown is the mirror image
}

final class Elevator {
    // ...
    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();   // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing(); // parked at lobby, doors open
            if (loadKg > capacityKg) {
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            }
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) {
            direction = Direction.IDLE;
            return nothing();
        }
        Stop target = next.get();
        if (target.floor() == floor) {                      // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }
    // ...
}
```

Full runnable code: code/lld/ch14/elevator/ (see the Full Edition).

How it works: `TreeSet` `ceiling` / `floor` / `first` / `last` give O(log n) "nearest stop" answers. `takeOutOfService` returns the car's stops for re-dispatch; `Dispatcher.retryPending` runs every tick.

Output (excerpt):

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
re-dispatch hall 4 DOWN -> car2
hall 2 UP -> queued (no car available)
```

Concurrency & edge cases

Situation	Handling
Buttons pressed while the controller steps	`synchronized` car and dispatcher methods; concurrent set of hall lights
Same hall button twice	`litHallButtons.add` false: "already lit", no second dispatch
Up and down on the same floor	Two `Stop`s in two sets
Car call for current floor, doors open	Ignored
Overload	Doors stay open, OVERLOAD shown, no movement
Car out of service with accepted calls	Re-dispatched or queued, never lost
No car available	Queued; retried every tick
Fire alarm	Stops dropped, lobby added, car buttons ignored, doors open at lobby
Starvation	LOOK serves every stop within one round trip
UP on the top floor	`IllegalArgumentException`

Extensions & follow-up questions

1. **Choosing a car** → `DispatchStrategy`: zoning, **up-peak** (idle cars to the lobby at 9 a.m.), energy saving.
2. **Simultaneous presses** → synchronized dispatcher + per-car lock (the course omits it).
3. **Testing** → presses, N steps, expected events; no sleeps.
4. **Destination dispatch** → a strategy taking (from, to), grouping nearby destinations.
5. **Re-evaluate / real time** → move unserved calls to a better car each tick; a `ScheduledExecutorService` calls `step()` every second.
6. **Emergency stop / more modes** → refactor modes into State classes.
7. **Express cars / door sensor** → served-floor sets the strategy checks; an obstruction makes `step()` reopen (same slot as overload).

Real-world example

Destination dispatch (Schindler PORT, Otis CompassPlus, KONE) = extension 4; controllers switch to **up-peak** mode; fire recall follows **Phase I emergency recall** (ASME A17.1).

In the interview

45 minutes: 5 requirements/scope → 5 entities → **15 stop list and LOOK** → 10 `step()` and dispatcher → 10 concurrency, testing, follow-ups.

> **30-second answer:** "Each car keeps stops in two sorted sets, up and down, and uses LOOK: serve stops ahead, turn at the last request, never run to the shaft end for nothing. `step()` does one action — close doors, refuse if overloaded, move one floor, or stop and open — so the system is testable without a clock. A dispatcher assigns hall calls via a `DispatchStrategy` (nearest suitable car) and queues calls when no car is free. Doors, load, maintenance and fire recall override scheduling."

- *"Why LOOK, not SSTF?"* — SSTF can starve a far floor; LOOK serves everything within one round trip.
- *"Where's the lock?"* — Per car (its monitor) and around dispatch; no global lock across cars.

Self-check

1. **Q:** SCAN vs LOOK? **A:** SCAN runs to the shaft end; LOOK reverses at the last request.
2. **Q:** Car at 4 going up; stops 6(up), 9(car), 2(down), 1(up) — order? **A:** 6, 9, 2, 1.

3. **Q:** Why is scheduling in `StopList`? **A:** SRP: the car drives, the stop list decides; each testable alone.
4. **Q:** Every car in maintenance — what happens to a call? **A:** Queued, retried every tick.
5. **Q:** When would you move modes to State classes? **A:** When many modes behave differently per action.

TL;DR

- **Two sorted sets** + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance, fire recall override scheduling.
- **Strategy** for dispatch, pending queue, re-dispatch when a car leaves service.
- Per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which **approaches** (roads entering: N, S, E, W) get green, for how long, and in what order. A **phase** is a set of approaches that may be green together because their paths do not cross (e.g. N and S straight). The controller cycles phases with **yellow** (warning) and **all-red** (clearance) between them. It tests a **timed state machine** with a **hard safety rule** — *two crossing flows are never green together* — plus an **emergency override** that interrupts the cycle *safely* (never cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
Roads?	Four-way (N, S, E, W); phases configurable.
Signals?	Red, yellow, green; flashing red in fault mode.
Timing?	Fixed, or adaptive from queue sensors.
Pedestrians, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Green as soon as *safely* possible, held until they pass.
Many intersections?	One controller each; central coordination is an extension.
Real time?	1 tick = 1 s; tick-driven so it can be simulated.

Requirements

Functional: (1) multiple roads; (2) red/yellow/green; (3) **configurable, traffic-adaptive** durations; (4) **safe transitions**: green → yellow → all-red → next green; (5) **emergency** priority; (6) scalable/extensible; (7) fault → **all flashing red** (all-way stop).

Non-functional: safety invariant (never conflicting greens, not even for an instant); deterministic 1 s ticks; thread safety (timer vs sensor and emergency threads); observability (listeners for light changes).

Use cases

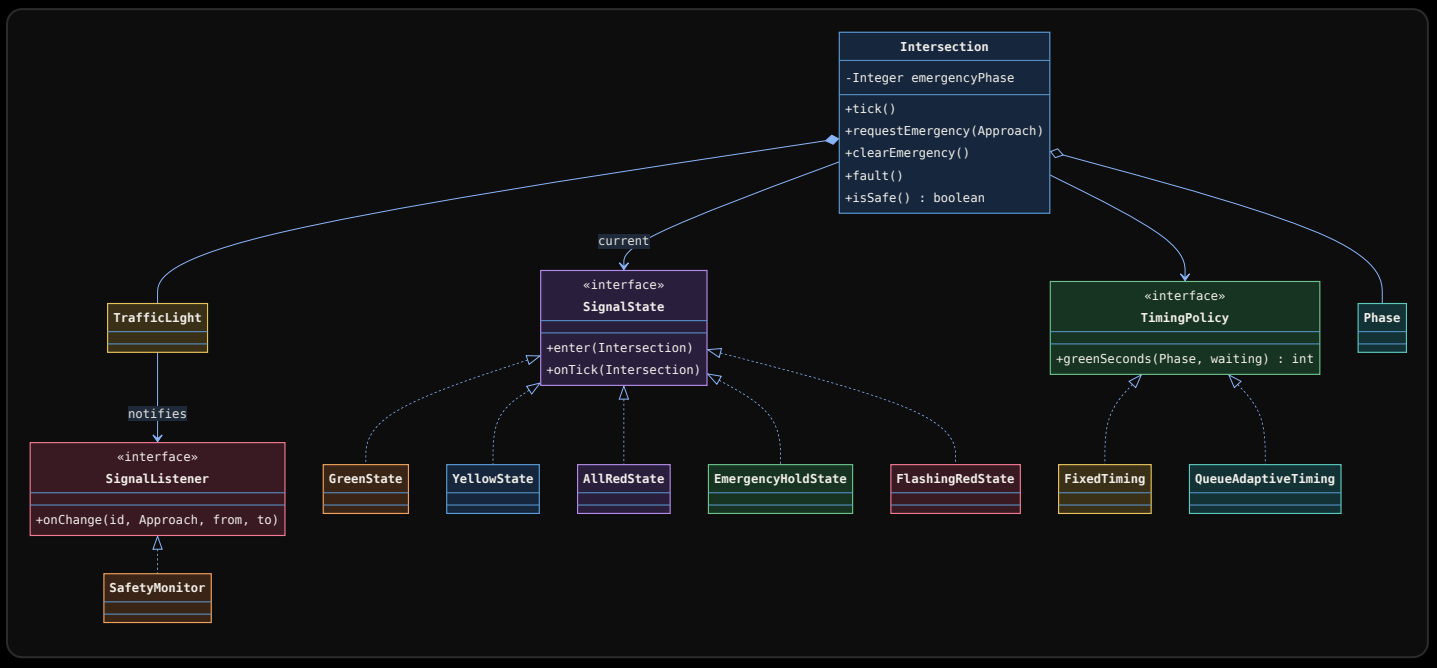
Actors → use cases: Timer → advance the cycle; Vehicle sensor → report waiting cars; Emergency vehicle → request/clear pre-emption; Traffic engineer → configure phases/timing, fault mode; Drivers see the lights.

Core entities

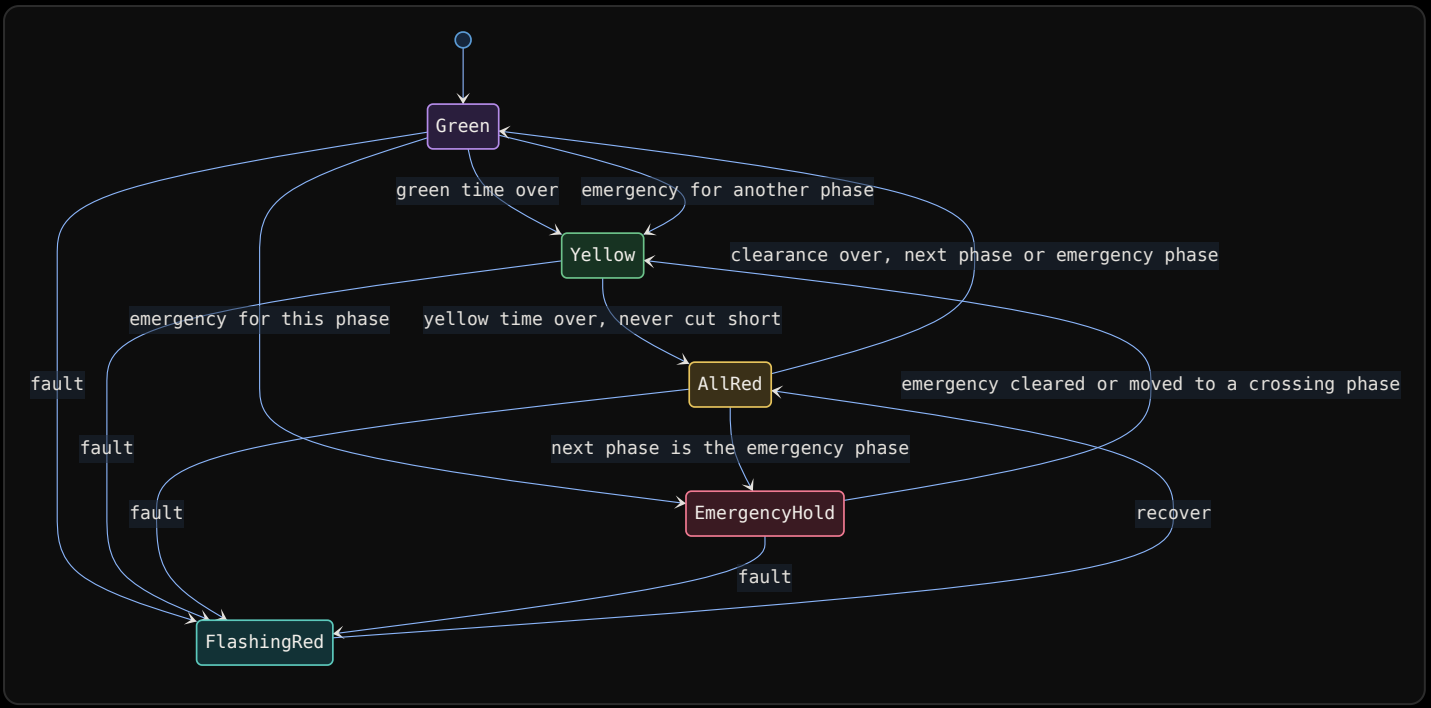
Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	Name + set of approaches green together.
`TrafficLight`	One head for one approach; notifies listeners.
`SignalListener`	**Observer** for light changes.
`SafetyMonitor`	Listener checking the invariant after every change to green.
`TimingPolicy`	**Strategy**: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: Green, Yellow, AllRed, EmergencyHold, FlashingRed.
`Intersection`	State context: phases, lights, timing, sensors, emergency flag; `tick()`.

Mapping to the common problem statement (*awesome-low-level-design*): `Road` → `Approach` + its `TrafficLight`; durations move into `TimingPolicy`; Singleton `TrafficController` → our `Intersection` (**not** a Singleton). We add what the reference leaves implicit: phases, all-red and the safety check.

Class diagram



Key flows



Emergency pre-emption: ambulance from WEST → `requestEmergency(WEST)` sets `emergencyPhase = East-West` → next tick N-S green ends **early** into yellow → yellow runs in full → all-red → `EmergencyHold(East-West)` → `clearEmergency()` → yellow, all-red, normal cycle.

One fixed cycle (green 5, yellow 2, all-red 1): N-S green 5 → N-S yellow 2 → all red 1 → E-W green 5 → E-W yellow 2 → all red 1 — **16 s**.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval a state class (`enter`, `onTick`)	**State**	Transitions and "yellow never cut short" live in one small class each	One `switch` with counters
States **carry data** (remaining seconds), new per transition	State with per-instance data	Unlike vending's stateless states, they hold a countdown	Shared singletons with counters on the context
Green time from `TimingPolicy`	**Strategy**	Fixed, adaptive, time-of-day swap at run time	Hard-coded in `GreenState`
Light changes published	**Observer**	Displays, logs, central monitoring, safety monitor	Calling each consumer
`showPhaseGreen`: others **red first**, then green	Safety by construction	Never an instant of conflicting greens	Arbitrary order
Separate `SafetyMonitor`	Defence in depth	Like hardware conflict monitors forcing flash	Trusting controller logic alone
Emergency is a **flag** states read	Safety	Only *shortens* green; yellow and all-red run in full	Jumping straight to emergency green
No Singleton controller	Rejected	Many intersections; a `TrafficControlCenter` can own them	`getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`.

```java
/** State pattern: each state knows what one second of time means for it, and which state is next. */
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }
    // ...
}

final class YellowState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {          // yellow is never cut short: that is unsafe
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));
    }
    // ...
}

final class Intersection {
    // ...
    synchronized void tick() { clock++; state.onTick(this); }
    // ...
    void setState(SignalState s) { state = s; s.enter(this); }
    // ...
    /** Turns red first, then green: never two crossing flows green, even for an instant. */
    void showPhaseGreen(int p) {
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
    // ...
    /** Safe = every approach that is green or yellow belongs to ONE phase. */
    synchronized boolean isSafe() {
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }
    // ...
}
```

Full runnable code: code/lld/ch14/trafficsignal/ (see the Full Edition).

How it works: `QueueAdaptiveTiming` gives `secondsPerCar × waiting cars`, clamped to [min, max]. `setState` calls `enter` at once (set lights, load timer). `AllRedState` picks the emergency phase if pending, else the next. `SafetyMonitor` checks `isSafe()` on every change to GREEN; a real `ScheduledExecutorService` run with 20 concurrent emergency requests shows zero violations.

Output (excerpt):

```
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
light changes seen by observer: 48, safety violations: 0
```

With 12 cars waiting E-W, the adaptive policy gives a 10 s green (the maximum).

Concurrency & edge cases

Situation	Handling
Emergency between ticks	Stored under the lock; next tick acts
Emergency during yellow	Yellow completes; all-red then picks the emergency phase
Emergency for the phase already green	Straight to `EmergencyHold` — no needless yellow
Two emergencies from crossing directions	Latest wins; the hold exits via yellow + all-red. Real systems queue by priority and arrival
Emergency never cleared	Maximum hold time
Timer and sensor threads overlap	`synchronized` methods; `CopyOnWriteArrayList` listeners
Adaptive timing starves a quiet road	Minimum green; every phase still gets its turn
Bug causes conflicting greens	`SafetyMonitor` detects; real controllers force flashing red
Timer drift	`scheduleAtFixedRate`; real deployments use the hardware clock

Extensions & follow-up questions

1. **Pedestrians** → WALK during the phase's green, flashing DON'T WALK before yellow, push-button requests.
2. **Left-turn arrows** → more phases; phases are data, state machine unchanged.
3. **Green wave** → a `TrafficControlCenter` sets cycle length and **offset** per intersection.
4. **Emergency corridor** → the centre pre-empts intersections along the route.
5. **Night mode** → time-of-day `TimingPolicy`, or flashing yellow/red states.
6. **Testing** → call `tick()` N times, assert snapshots.

Real-world example

SCATS and **SCOOT** adapt timing city-wide from sensors; **emergency pre-emption** emitters still time out yellow and red first; **conflict monitors / MMUs** force flash on conflicting greens — our `SafetyMonitor`.

In the interview

40 minutes: 5 requirements → 5 entities → 10 state machine → 15 code Green/Yellow/AllRed and `tick()` → 5 concurrency and follow-ups.

> **30-second answer:** "Phases are sets of approaches that can be green together; the controller is a timed State machine — Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` each second; each state counts down and names the next. Green time comes from a swappable timing Strategy. An emergency only shortens green: yellow and all-red always run in full, then the emergency phase is held until cleared. Lights publish to observers, including a safety monitor that checks no crossing flows are ever green together."

- *"Why not jump straight to emergency green?"* — Cars in the junction need yellow and all-red to clear it.
- *"Why not a Singleton?"* — One per intersection; a city has thousands.

Self-check

1. **Q:** What is a phase? **A:** Approaches that may be green together because their paths don't cross.
2. **Q:** Why all-red? **A:** To let vehicles still in the junction clear it.
3. **Q:** Emergency during yellow? **A:** Yellow finishes; all-red then picks the emergency phase.
4. **Q:** Why do these states hold data but vending states don't? **A:** Each interval has its own countdown.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing, **Observer** for lights, emergency as a flag.
- Tick-driven and synchronized: deterministic tests.

14.7 Design a Shipping Locker System (Amazon Locker)

Also asked as: "Design Amazon Locker" or "Design a parcel/pickup locker".

A **parcel locker** is a wall of numbered doors in a shop or lobby: a courier puts a parcel behind one door, the customer types a **pickup code** and that door opens (Amazon Hub Locker, InPost Paczkomat). It looks like the parking lot — hand out limited spaces of several sizes — plus three extras interviewers probe: **deadlines** (a reservation that times out, a parcel that expires), **a secret** (the code, hard to guess, never shown to the courier) and **a network** (station full → suggest another).

Problem & clarifying questions

Question	Assumed answer
Door sizes?	Small, medium, large; a parcel may be upgraded to a bigger free door.
When is the door chosen?	In advance: the carrier **reserves** it; the hold lasts 2 h, then the door is released.
How does the customer open it?	6-digit **one-time code** by SMS/app on deposit; the courier never sees it.
How long can a parcel stay?	3 days; then it **expires** and the courier takes it back.
Code guessing?	Random, single-use; 5 wrong codes in a row lock the keypad for 15 min.
Many stations?	Yes; offer the nearest one with space. Cash on delivery is an extension.
Returns?	Yes: a door is held 2 days, a single-use **drop-off code** opens it empty once, the courier collects later.

Requirements

Functional: (1) numbered doors of fixed sizes; (2) **reserve** the best-fitting free door, all-or-nothing, with a hold deadline; (3) **deposit** → one-time code **sent to the customer only**; (4) **pickup** opens one door once; (5) **expiry** → customer told, courier collects for return; (6) keypad lockout; (7) nearest station with space; (8) doors out of service; (9) **customer return** to a locker with a drop-off code.

Non-functional: never one door for two parcels under concurrency; codes unpredictable, single-use, stored as a hash; deadlines correct even if the timer is late; clock and randomness injected.

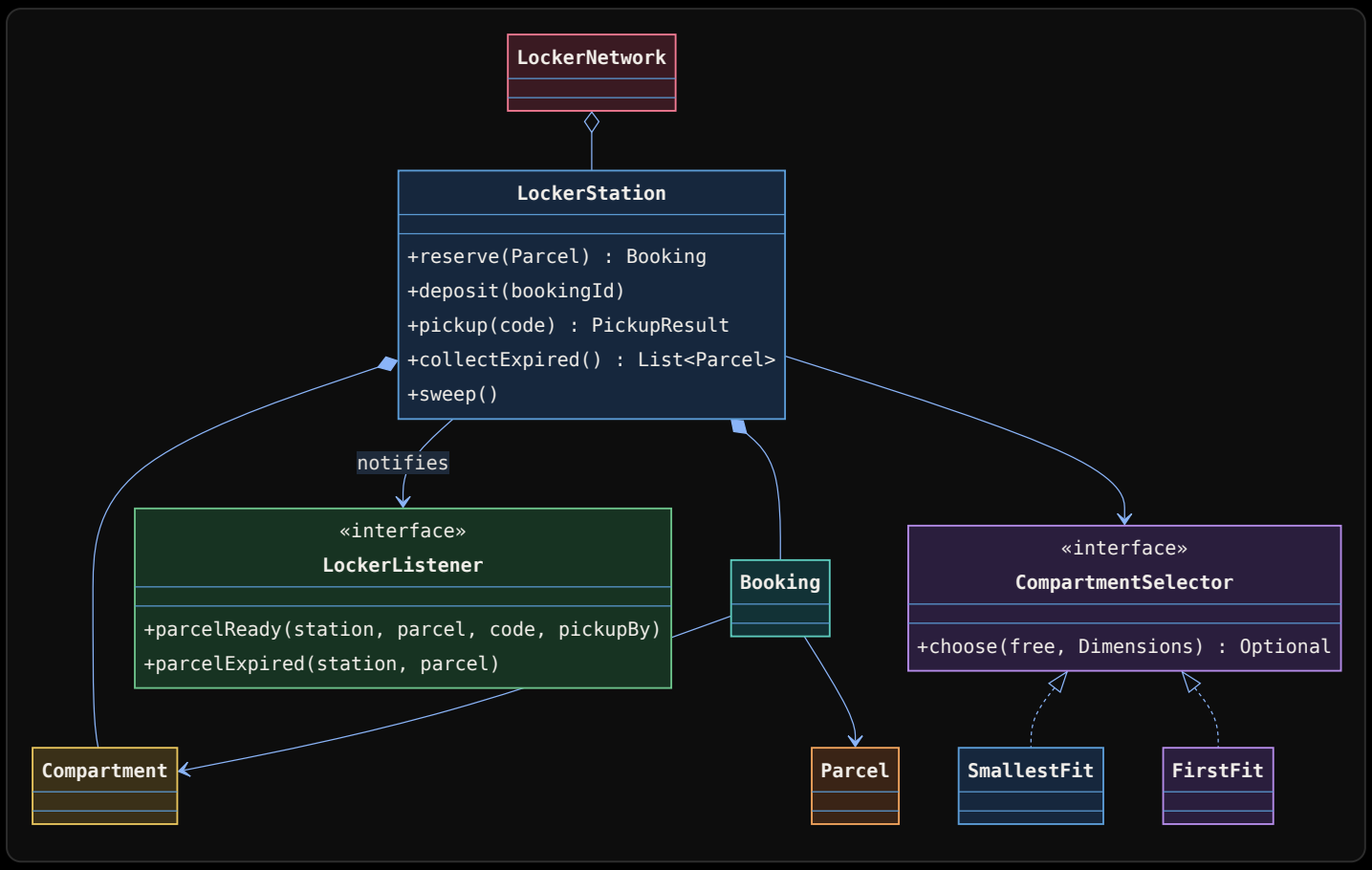
Use cases

Actors → use cases: Customer → find a locker near me, pick up with code, start a return, drop off with code; Carrier system → reserve; Delivery agent → deposit (includes: send code), collect expired parcels and returns; Locker admin → door out of service; Scheduler → sweep deadlines and notify. Keypad lockout *extends* pickup after 5 wrong codes. (Use-case and activity diagrams: Full Edition.)

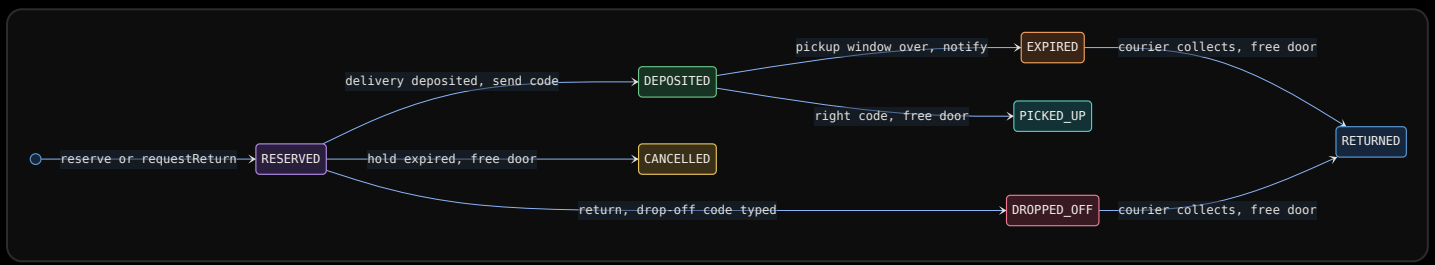
Core entities

Class	Responsibility
`Size` (enum), `Dimensions`, `Parcel` (records)	Door sizes with `fits()` (sort both boxes' sides, compare in order); parcel data.
`Compartment` + `CompartmentState`	One door: FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE; legal moves in `moveTo`.
`Booking` + `BookingKind` + `BookingStatus`	One parcel's stay, DELIVERY or CUSTOMER_RETURN (statuses below); deadlines and code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `DropOffDoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired`.
`CompartmentSelector`	**Strategy**: `SmallestFit`, `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with code), parcel expired.
`LockerStation`, `LockerNetwork`	Doors, bookings, code index, lockout under one lock; nearest station with space.

Class diagram



Key flows



Delivery: carrier `reserve(parcel)` → `SmallestFit` picks the smallest free door that fits → door RESERVED, hold 2 h → courier `deposit` → door OCCUPIED, random code, only its SHA-256 hash stored → listener sends the code to the customer **after the lock is released** → customer `pickup(code)` → hash matches → door FREE, code deleted. The door has its own small machine: FREE ↔ RESERVED → OCCUPIED → FREE, and FREE ↔ OUT_OF_SERVICE.

Station full / courier late: no door fits → ask the network for the nearest station with space (none → home delivery); courier misses the 2 h hold → booking CANCELLED, reserve again.

Keypad (pickup or return drop-off): locked? → show wait. Hash unknown? → count it; the 5th in a row locks for 15 min. Delivery expired? → "will be returned". Otherwise the booking's kind decides: delivery → door opens, PICKED_UP, door FREE; return → empty door opens, DROPPED_OFF, door OCCUPIED. Either way the code is deleted. The courier's visit (`collectExpired`, `collectReturns`) marks both kinds RETURNED and frees the doors.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice as `CompartmentSelector`	**Strategy**	Smallest-fit saves big doors; first-fit, eye-level-first swap in	`if (mode == …)` in `reserve`
Door state separate from booking status	Two enum state machines	Doors are reused; bookings keep deadlines and history	One field mixing both
Sealed `PickupResult`	Sealed result types	Every keypad outcome handled, compiler-checked	`null` / magic strings
Random, single-use, hashed codes	Security by design	A DB leak reveals no live code; courier never sees it	Plain codes; codes derived from the parcel id
Listeners called outside the lock	**Observer**	A slow SMS gateway can't block the keypad	SMS call inside `deposit`
Lazy expiry on every call + `sweep()` for messages	Check deadlines on read	Doors come back even if the scheduler is late	A reaper thread as the only mechanism
Injected `Clock`, `RandomGenerator`	DI	Deterministic tests	`Instant.now()`, `new Random()` inside
One lock per station	Monitor	Tens of doors, microsecond calls	Per-door CAS (needless here)

Java implementation

Package `lld.ch14.locker`.

```java
// File: lld/ch14/locker/LockerStation.java
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        // ... reject a parcel that already has an active booking
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " +
id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, now() + reservationHold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            // ... expireOverdue(); booking must be RESERVED
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            do { code = "%06d".formatted(random.nextInt(1_000_000)); }       // unique among active codes here
            while (bookingByCodeHash.containsKey(hash(code)));
            b.codeHash = hash(code);
            bookingByCodeHash.put(b.codeHash, b.id);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            // ... count the wrong code; the 5th in a row sets lockedUntil = t + 15 min and returns KeypadLocked
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                        // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }
    // ... expireOverdue(): RESERVED past hold -> CANCELLED (door FREE); DEPOSITED past window -> EXPIRED
```

Full runnable code: code/lld/ch14/locker/ (see the Full Edition).

How it works: `Size.fits` sorts the sides of box and door and compares in order (no need to try six rotations). `sweep()` (called by a scheduler) runs the same deadline check and sends "not collected" messages outside the lock. `LockerNetwork.nearestWithSpace` sorts stations by distance and returns the first within the radius that can fit the parcel now.

Output (excerpt):

```
P3 9x12x12cm    -> MG01-03(MEDIUM)
error: Parcel P1 already has a locker
SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
same code again -> WrongCode[attemptsLeft=4]  (a code works once)
guess 000004 -> KeypadLocked[secondsLeft=900]
booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}
customer types the code -> ParcelExpired[parcelId=P3]
station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)
reserved=20 rejected=30 distinct doors=20
customer types the drop-off code -> DropOffDoorOpened[compartmentId=RT04-02, parcelId=R1]
2 days later, never dropped off: booking RT04-B2 is CANCELLED, old code -> WrongCode[attemptsLeft=4]
```

The mug was upgraded to MEDIUM because both small doors were taken; `FirstFit` wasted the large door and had to refuse a large parcel; 50 concurrent reservations on 20 doors gave exactly 20 distinct doors.

Concurrency & edge cases

Situation	Handling
Two carriers, last door	`synchronized reserve`: choose + mark is one step
Courier late past the hold	Booking CANCELLED; `deposit` refuses; carrier re-reserves
Retried reserve for the same parcel	Rejected while a booking is active (idempotent by parcel id)
Duplicate random code	Regenerate until unique among live codes
Guessing	10^6 codes, lockout after 5 wrong, codes expire
Scheduler down	Lazy expiry still frees doors; only the SMS is late
`canFit` true but `reserve` fails	`nearestWithSpace` is a hint; reserve, then try the next station
Broken door	Only a FREE door can go OUT_OF_SERVICE (`moveTo`)

Extensions & follow-up questions

1. **Extend pickup window** → move `pickupBy` once, only while DEPOSITED.
2. **Cash on delivery** → a payment step before the door opens.
3. **Keep large doors for large parcels** → a selector that refuses upgrades when few large doors are left.
4. **Thousands of stations** → shard by station; geo index (geohash, R-tree) for the network search.
5. **Database** → `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check one row changed.

Real-world example

Amazon Hub Locker sends a pickup code and returns uncollected parcels after a limited number of days (our EXPIRED → RETURNED path); InPost's Paczkomat lockers in Poland open with a code or an app and are dense enough that "nearest locker with space" matters. In both, the system — not the courier — chooses the door.

In the interview

40 minutes: 5 questions → 5 entities → 8 the two state machines → 15 code `reserve` / `deposit` / `pickup` → 7 security, concurrency, follow-ups.

> **30-second answer:** "Doors come in a few sizes; a Strategy picks the smallest free one that fits, and choose-and-mark runs under the station lock so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked on every call, so doors come back even if the scheduler is late."

- *"Why a Booking separate from the Compartment?"* — The door is reused; the booking keeps each parcel's deadlines and history.
- *"Why hash the code?"* — Anyone reading the database or logs could otherwise open doors.

Self-check

1. **Q:** How does `fits` handle a box on its side? **A:** Sort both sets of sides and compare in order.
2. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every call.
3. **Q:** Why notify after the `synchronized` block? **A:** A slow SMS call must not hold the station lock.
4. **Q:** Which pattern swaps smallest-fit for first-fit? **A:** Strategy (`CompartmentSelector`).

TL;DR

- **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** per station.
- Two **state machines**: door and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED; returns RESERVED → DROPPED_OFF → RETURNED).
- Codes **random, single-use, hashed**, sent via an **Observer** outside the lock; lockout after 5 wrong.
- Deadlines enforced **lazily**; inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Stateless vs data-carrying states? **A:** Vending and ATM share stateless singletons; traffic-signal states hold countdowns, new per transition.
2. **Q:** The check-then-act race in parking, coffee, ATM? **A:** Two gates one spot (atomic `pollFirst`/CAS); two orders the last milk (one lock around check+deduct); two ATMs one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton? **A:** Parking lot, coffee machine, traffic controller — hidden global, untestable, and real deployments have many.
4. **Q:** Where does greedy fail? **A:** Change with limited coins (vending) and notes with limited cassettes (ATM); use bounded coin-change DP.

5. **Q:** Which designs are tick-driven and why? **A:** Elevator and traffic signal — deterministic, clock-free tests.
6. **Q:** Which case enforces deadlines lazily? **A:** The shipping locker — every call expires overdue holds/parcels first; the sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** per problem and spend your time there.
- **State** for machines whose buttons change meaning; **Strategy** for changing rules; **Observer** for boards, alerts, monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** scope, lock in a **fixed order**, call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, tick-driven logic over sleeps.
- Deadlines (holds, pickup windows): check on every call; secrets (pickup codes): hashed, single-use.

Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who want it at once. Chapter 16 continues. This is the study edition: every topic stays, the padding goes; the Full Edition (README.md) has the complete code and outputs.

Why booking problems are interview favourites

A booking problem looks like simple CRUD until two people want the same thing. Then five hard questions appear, and each problem here is a different mix:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free CAS (15.3)
Payments	What if payment fails? Or succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words used everywhere:

- **Race condition** — two threads both read "free" and both act; one result silently overwrites the other.
- **Check-then-act** — "if free, then take it". Safe only if check and act are one **atomic** step.
- **Hold** (lock, soft lock, reservation) — a temporary claim **with an expiry**, so an abandoned checkout does not block the item forever.

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a GA counter, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

In the concurrency demos, *which* thread wins changes per run; the *number* of winners never does. Several small package-private types share a file (e.g. `Model.java`); `-Xlint:all` flags this as `[auxiliaryclass]` — a style warning, not an error.

> **Money.** Every amount is a `BigDecimal` built from a string (`new BigDecimal("0.50")`), never a `double` (0.10 is not exact in binary, so sums drift). Compare with `compareTo` / `signum()`, not `equals` (which also compares scale), and name the `RoundingMode` at every rounding step.

> **Time trick.** Every service takes a `java.time.Clock`; demos pass a `SimClock` whose `advance(Duration)` moves time forward, so "10 days later" needs no `sleep`. Production passes `Clock.systemUTC()`.

15.1 Design a Library Management System

Problem & clarifying questions

A **Library Management System** (LMS) tracks every title and physical copy, who borrowed which copy and when it is due, members and cards, reservations and fines.

Question	Answer we assume
Copies?	Many per title — each a **book item** with barcode and rack.
Limits?	**5 books** per member, **10 days** per loan, then a fine per day.
Reserve a book that is out?	Yes, with a notification when a copy returns.
Fines?	Calculated and recorded; payment out of scope. E-books, inter-library loans out of scope.

Requirements

Functional: (1) search by **title, author, subject, publication date**; (2) each book has an ISBN; each copy a **rack number**; (3) a title has **many copies**; (4) answer **"who took this copy?"** and **"which books does this member have?"**; (5) **max 5 books** per member; (6) **max 10 days** per loan; (7) **fines** for late returns; (8) **reserve** unavailable books; (9) **notify** when a reserved book is available and when a book is overdue; (10) **barcodes** on items and cards.

Non-functional: correct under concurrency (no double-lent last copy, never over 5); index-based search; swappable fine rules and channels.

Use cases

Actors: Librarian (add/edit/remove books, items, members; issue, reserve, return for a member), **Member** (search, check out, reserve, renew, return), **System** (overdue, reserved-book-back, cancelled-reservation notifications). **Top use cases:** manage books/items; search; register/cancel account; **check out**; **reserve**; **renew**; **return**.

In the course's use-case diagram, `<<include>>` means *always* performs another (checking out always clears the member's reservation), and `<<extend>>` is an *optional variation* (search by author extends search catalog).

Core entities

Class	Responsibility
`Book` (record)	The **`title`**: ISBN, title, subject, publisher, date, authors. Immutable.
`BookItem`	One **`physical copy`**: barcode, format, rack, price, status (`AVAILABLE` , `RESERVED` , `LOANED` , `LOST`), who it is held for.
`Author` , `Rack` (records)	Author name; rack number and location.
`Account` (sealed, abstract) → `Member` , `Librarian`	People; an account **`owns`** its `LibraryCard` (composition); a member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: barcode, member id, created, **`due date`**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` / `Catalog`	Four searches; one index per search; knows every copy of every title.
`LendingService`	Rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	Cost of a late return (Strategy).
`Notifier` / `ConsoleNotifier`	Where messages go.
`Library`	Facade: owns catalog and service; only librarians add stock.

All course classes are kept; `BookReservation` becomes a per-title waitlist and `Notification` the `Notifier` interface.

Class diagram

The course builds five stages: (1) the "what happened" records — `BookReservation` , `BookLending` , `Fine` , each with barcode and member id (answers requirements 4, 7, 8); (2) **`Book`** **`vs`** **`BookItem`** — the split that carries the design (the course has `BookItem` *inherit* `Book` ; we use **`composition`**); (3) abstract `Account` with `Member` / `Librarian` — limits (req. 5, 6) belong to people; the card is **`composition`** (dies with the account); (4) `Search` interface implemented by `Catalog` ; (5) `Library` owns catalog, racks, items; abstract `Notification` per channel.

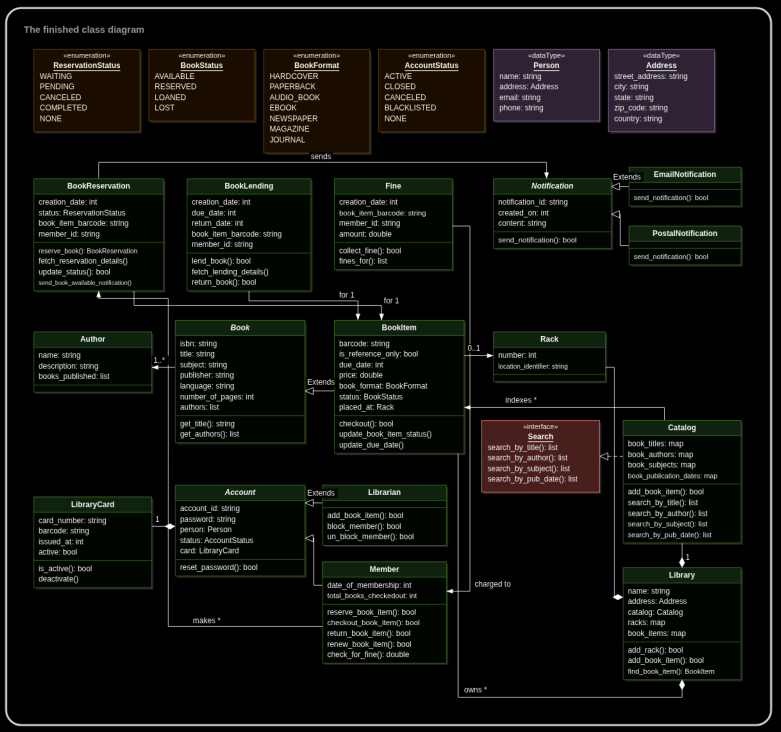
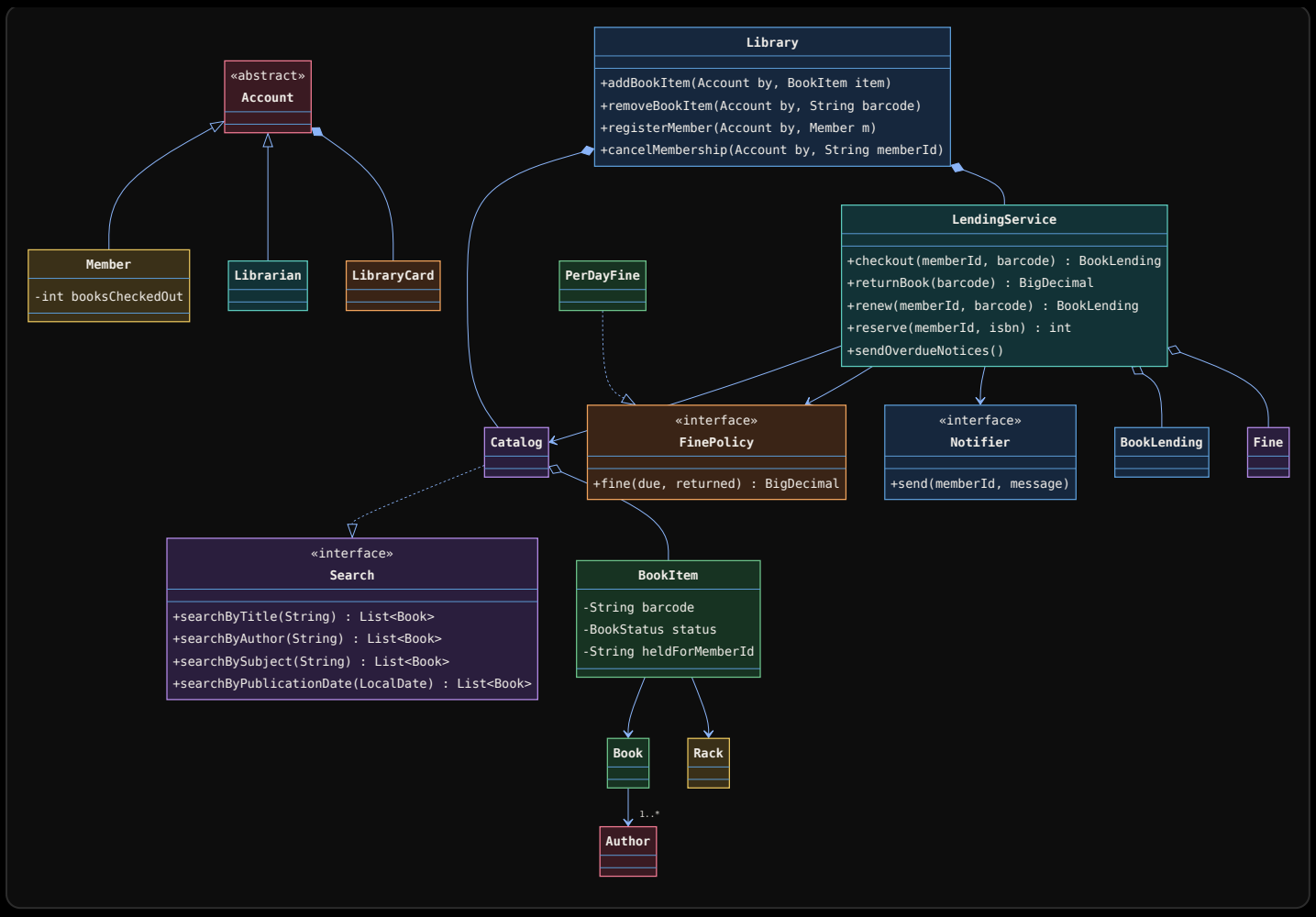


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)



Key flows

The course's three activity diagrams:

- **Check out:** scan book → can it be issued (not reserved for someone else)? → member under quota? → create the loan, mark "Loaned", increment the member's count, complete this member's reservation.
- **Return:** scan → late? charge fine → decrement count → reserved? notify the first waiting member and mark it held; else make it available.
- **Renew:** scan card and book → charge a fine if already late → someone reserved it? refuse; else new due date.

Copy state machine: `AVAILABLE → LOANED` (checkout); `LOANED → AVAILABLE` (return, nobody waiting) or `→ RESERVED` (return, waitlist non-empty); `RESERVED → LOANED` (checkout by the member it is held for); `LOANED → LOST`.

Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem`	Members borrow *copies*; search works on *titles*. Say it early.	One `Book` with a `copies` counter — can't say *which* copy is late or where.
Composition (`BookItem` has a `Book`)	Many copies share one immutable title; inheritance would duplicate title fields per copy; a title has no barcode, so "copy *is a* title" fails strict substitution.	`BookItem extends Book` (course) — valid for a small model; mention it.
Rules in `LendingService`, not `Member`	The course itself flags `Member` doing checkout/return/renew as two reasons to change (SRP).	Methods on `Member` (fine for a first sketch).
Reservation = FIFO waitlist per title	A member wants "a copy of Dune"; any returned copy goes to the first in line.	Per-copy reservation — wait on copy #3 while #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Caps, discounts, holidays change.	`if/else` in `returnBook`.
Observer-style `Notifier`	Lending announces; delivery is someone else's job (course's `Notification` subclasses = same idea).	Emailing from `LendingService`.
Facade (`Library`)	One entry point; role checks live there.	Exposing catalog and service to the UI.
One index per search	Every lookup is a map read; cost: a new search adds a map (a single `(field, value)` index would stay one structure).	Scanning all books.

Java implementation

Package `lld.ch15.library` (`Model`, `Catalog`, `Support`, `LendingService`, `Library`, `Main`).

```java
/**
 * The three core flows (check out, return, renew) plus reservations.
 * Lives apart from Member so a Member stays "a person", and the rules live in one place.
 *
 * Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
 */
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;
    // ...
    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE)
                    throw new LibraryException(memberId + " account is " + m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
                switch (item.status()) {
                    case AVAILABLE -> { /* ok */ }
                    case RESERVED -> {
                        if (!memberId.equals(item.heldForMemberId()))
                            throw new LibraryException(barcode + " is held for another member");
                    }
                    case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
                }
                item.markLoaned();
                m.incrementLoans();
                LocalDate today = today();
                BookLending loan = new BookLending(barcode, memberId, today, today.plusDays(MAX_LENDING_DAYS));
                activeLoans.put(barcode, loan);
                removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
                return loan;
            }
        }
    }
    // ... returnBook(): same lock order, close loan, chargeFineIfLate, handToNextInLine
    /** Caller holds the item's monitor. Give the copy to the first waiting member, or shelve it. */
    private void handToNextInLine(BookItem item) {
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }
    // ...
}

/** Strategy: how much a late return costs. Swappable without touching LendingService. */
interface FinePolicy {
    BigDecimal fine(LocalDate dueDate, LocalDate returnDate);
}
```

Full runnable code: code/lld/ch15/library/ (see the Full Edition).

Output (excerpt):

```
  REFUSED: M1 already has 5 books
== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  REFUSED: cannot renew B-DUNE-1: another member reserved this title
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  M4 REFUSED: B-DUNE-1 is held for another member
```

A sixth book is refused; one of eight racing members wins the only copy; renewal is refused while someone waits; a 3-day-late return is fined 3 × $0.50 and the copy is **held** for the first in line.

Librarian housekeeping: a copy on loan can't be removed, a member with a book out can't be cancelled, a cancelled member can't borrow.

Concurrency & edge cases

- **Last copy:** `synchronized (item)` makes "available? → mark loaned" atomic; the 8-thread race always has one winner.
- **Two librarians, one member:** quota check and increment under `synchronized (member)` — never 6 books.
- **Deadlock avoidance:** always **member first, then item** (fixed global order). `returnBook` reads the loan to find the member, then locks in that order and re-checks the loan.
- **Renew vs reserve:** renewal refused if the waitlist is non-empty — checked *before* charging a fine, so no double fine.
- **Known small window:** `reserve` checks "no free copy" then queues in two steps; a copy returned in between goes to the shelf and the *next* return reaches the member. Accepted, because taking the queue lock before copy locks would reverse the order returns use — the recipe for deadlock.
- **Edge cases:** unknown barcode/member, duplicate barcodes, inactive/blacklisted accounts, removing a loaned/held copy, cancelling with books out, reserving a title with a free copy ("just check it out"), joining a waitlist twice (keeps place), returning a book not out.
- **Across servers:** use a conditional update — `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check exactly one row changed.

Extensions & follow-up questions

- **"Two members reserve at the same moment?"** — The course calls it unguarded; ours appends to a FIFO deque under a lock, so they queue in a definite order.
- **"Where is the available-copies count?"** — Derive it (`availableCopies(isbn)` counts AVAILABLE copies); if slow, cache it and update under the same lock as the status.
- **"How is the fine paid?"** — Add a `PaymentService`; block checkouts while unpaid fines exceed a threshold.
- **"Hold expiry?"** — `heldUntil` on `BookItem` plus a daily job passing it on — the TTL idea of 15.2.
- **"E-books?"** — `EBOOK` items with no rack, a fixed number of concurrent licences.

Real-world example

Koha and **Evergreen** use this model (records + barcoded items, circulation rules, holds queue per title); **Libby** applies the waitlist to e-book licences.

In the interview

30–45 minutes: 3 min clarify (copies, 5 books, 10 days, fines, reservations) → 5 min use cases → 10 min entities, **starting with** **`Book`** / **`BookItem`** → 10 min code `checkout` / `returnBook` with lock ordering → 5 min reservation hand-over and fines → follow-ups.

> **30-second answer:** "The key split is `Book` (title) vs `BookItem` (a copy with barcode and rack). Members borrow copies; search works on titles via a `Catalog` with one index per search. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and hands returned copies to a FIFO waitlist per title. Checkout locks member then copy so the last copy can't be double-lent."

Self-check

- **Q:** Why a fixed lock order? **A:** Otherwise A holds member→waits item while B holds item→waits member: deadlock.
- **Q:** What stops a 6th book with two desks scanning? **A:** Check and increment inside `synchronized (member)`.
- **Q:** When can't a book be renewed? **A:** When another member is waiting for that title.
- **Q:** Which pattern changes fine rules without touching lending? **A:** Strategy (`FinePolicy`).

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)

Problem & clarifying questions

An online movie booking system (**BookMyShow**, **Fandango**, **AMC**) lets customers find movies, pick a show, choose seats on a map, pay and get a ticket. The classic booking question: a place hierarchy (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping one seat).

Question	Answer we assume
Venues?	Many cities; several halls per cinema; a hall runs **one show at a time**.
Seats?	Several per booking from a seat map; **held 5 minutes** while paying.
Payment, refunds?	Card or cash, optional **coupon**; refund up to 2 hours before the show.
Users?	Customers, admins, front-desk officers (walk-ins), guests (search only).

Requirements

Functional (course order): (1) list **cities**; (2) cinemas have **halls**, one show at a time; (3) movies have many **shows**; (4) **search** by title, language, genre, release date, city; (5) show **cinemas and shows** for a movie; (6) **book** a show; (7) **seating plan**, pick several seats; (8) distinguish **available** from booked; (9) **notify** on new movie, booking, cancellation; (10) pay by **card or cash**; (11) **no two customers reserve the same seat**; (12) **discount coupon**.

Non-functional: strong seat consistency; high read, bursty write; **hold expiry** (a gap the course names; we implement it); never pay for a seat you don't get.

Use cases

Actors: Admin (add/cancel movies and shows, block customers), FrontDeskOfficer (book/cancel for walk-ins), Customer (view, book, cancel), Guest (search only — must register), System (notifications). **Use cases:** search; create/modify/view booking; pay; apply coupon; assign seats; refund on cancellation within the allowed time. In the course diagram a **Customer is a specialised Guest** — enforced in code because `BookingService` takes a `Customer`.

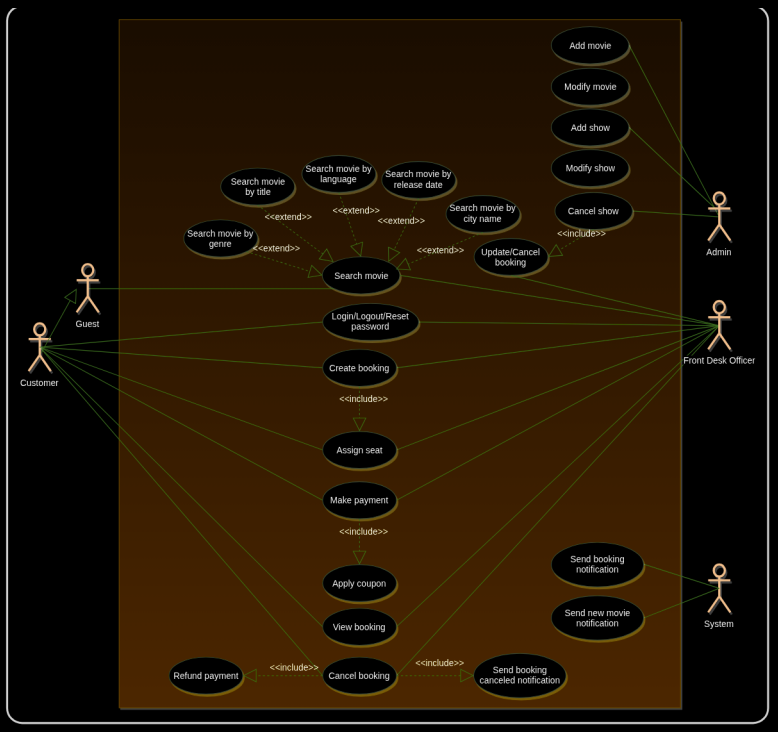


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Core entities

Class	Responsibility
`City`, `Cinema`, `CinemaHall`	Venue hierarchy; a cinema **owns** halls, refers to its city; a hall refuses an overlapping show.
`HallSeat` (record)	The bolted-down seat: id, `SeatType`. **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening; **owns one `ShowSeat` per hall seat**.
`ShowSeat`	A seat *for this show*: price, `booked`. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Expiring, all-or-nothing **holds**; commit; release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit; cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money; the gateway is the boundary to the provider.
`Coupon`	Percent off the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	Refund on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" comes from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`; `Notifier`	Actors (`Guest` can't book); messages.

Class diagram

Six course stages: (1) city, cinema, hall, physical seat; (2) movie, show, **show seat — where the design succeeds or fails**: one physical seat is free at 3 pm and sold at 9 pm, so you reserve a `ShowSeat` (which also allows per-show prices); (3) coupon, payment types, notification types — the coupon reduces **the booking's amount**, not the payment; (4) `Booking` and `BookingService` — one seat never goes to two customers; (5) search interface and catalog; (6) people — booking restricted to customers.

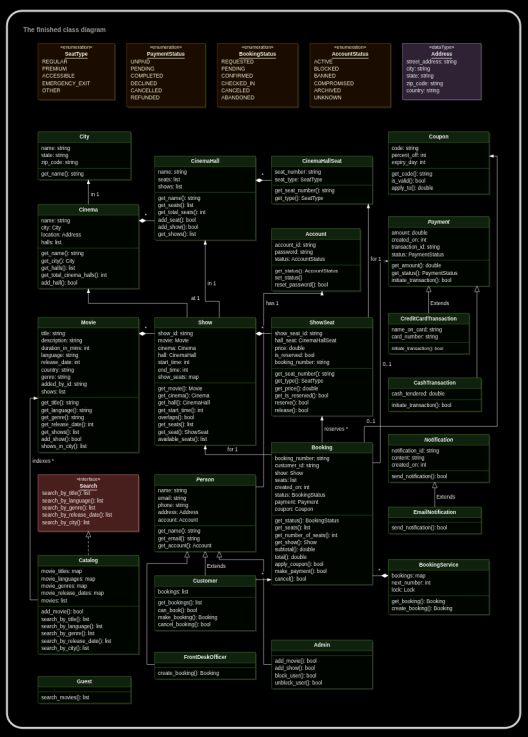
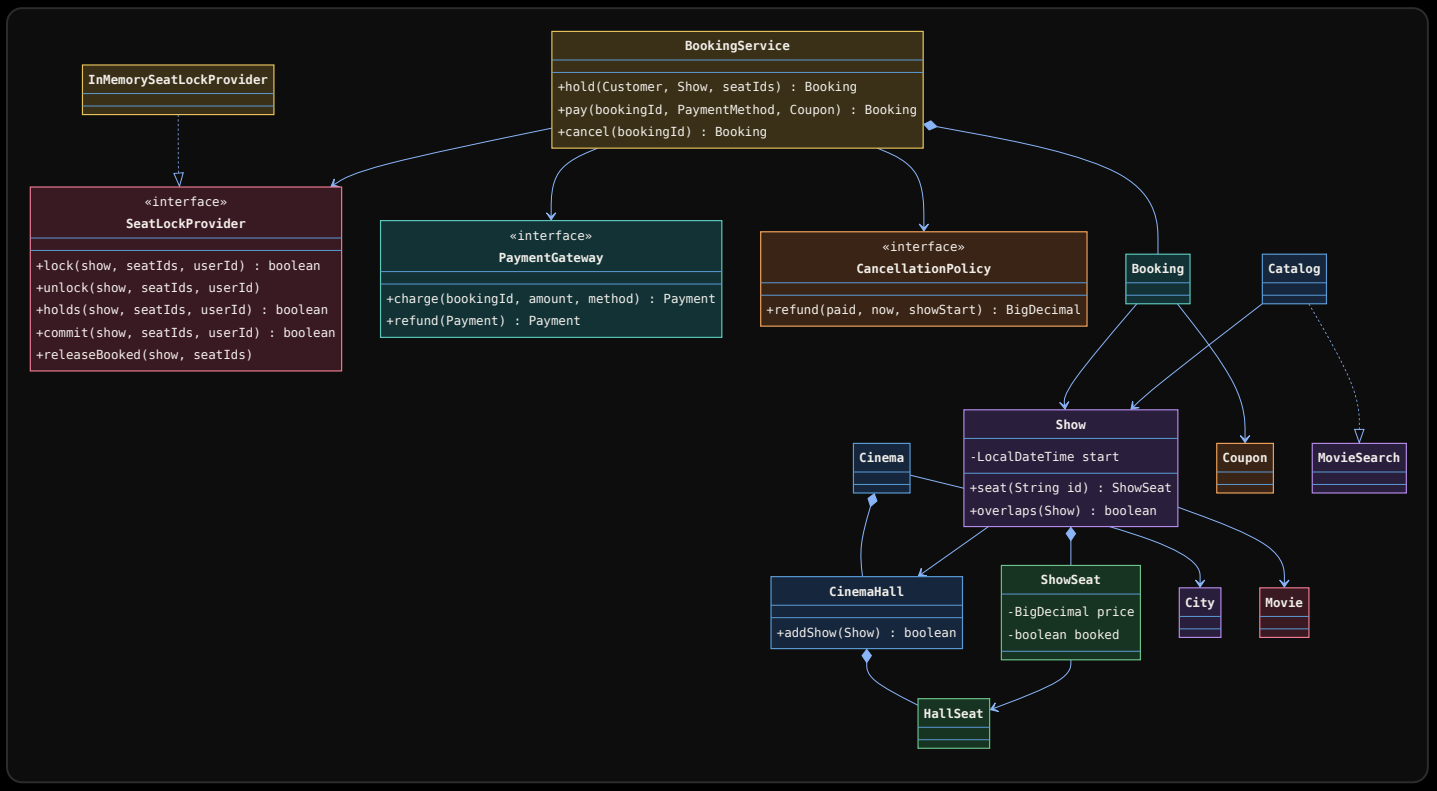


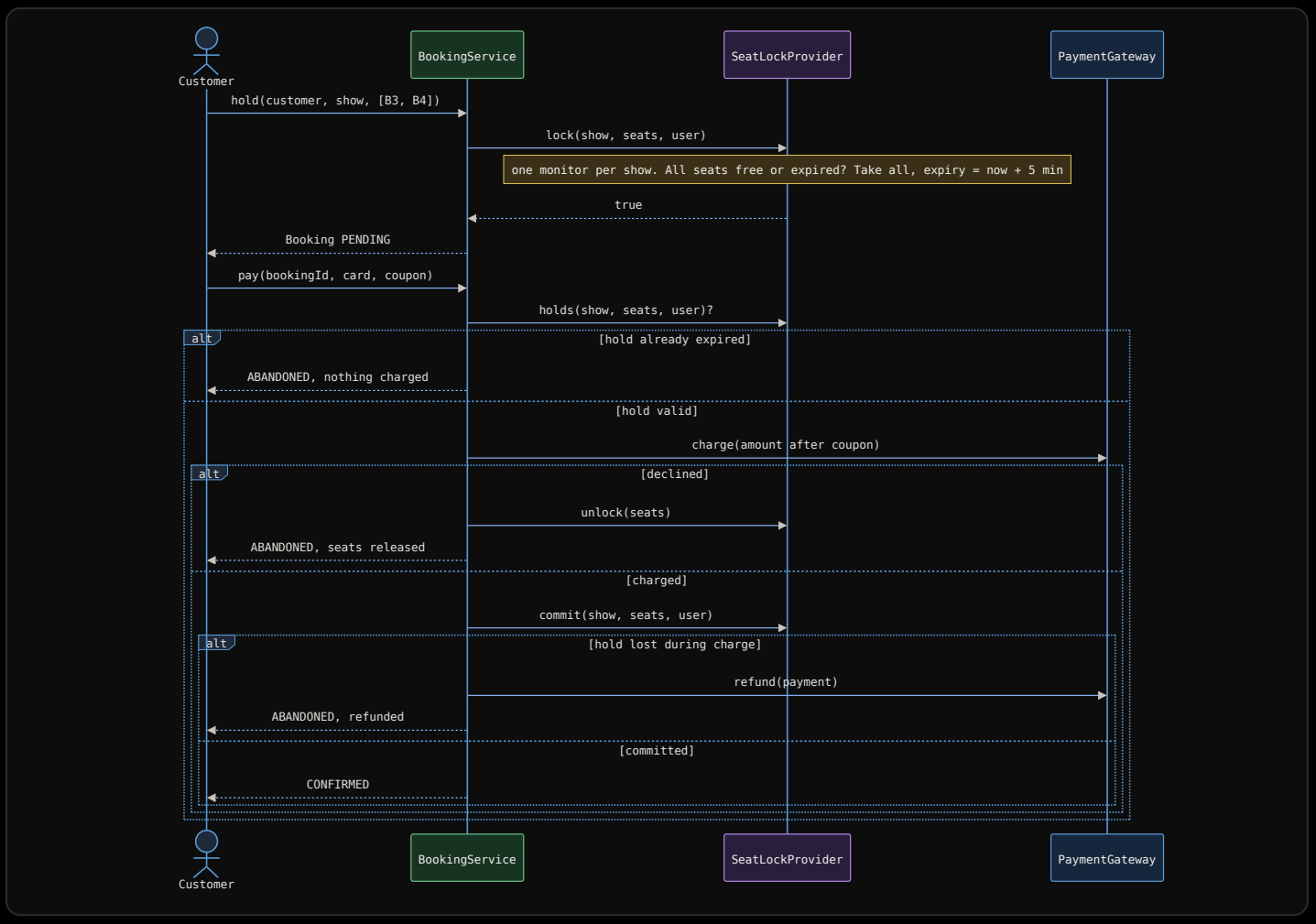
Figure — The finished movie booking class diagram (course diagram)

Our main difference from the course: a separate `SeatLockProvider` with **TTL holds** (the course's biggest self-named omission).



Key flows

The course's activity diagrams cover **make a booking** and **cancel a booking** (refund only within the allowed time). The heart is **hold → pay → commit**:



Why `commit` re-checks inside the lock: payment can take seconds; if the hold expires and someone else grabs the seat, a naive "mark booked" double-books. `commit` verifies "still held by me, not expired" **and** marks booked atomically. If that fails after money was taken, run a **compensating action** (refund) — a card charge can't be rolled back. This is the **Saga** idea: each step has an undo.

Booking lifecycle: `PENDING → CONFIRMED` (paid and committed) `→ CHECKED_IN` (scanned) or `→ CANCELED` (refund per policy); `PENDING → ABANDONED` (expired, declined, or lost and refunded) or `→ CANCELED`. On the seat map a seat is free (`.`), **held** (`L`) or booked (`X`); "held" lives only in the lock provider and disappears on expiry.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show	Same physical seat free in one show, taken in another; per-show prices.	Booking `HallSeat` — breaks with two shows a day.
`SeatLockProvider` with TTL holds	Abandoned checkouts free themselves; swap in **Redis** (`SET key value NX PX 300000`) or a DB `expires_at` column.	Holding until cancel (course's gap).
All-or-nothing lock	4 seats together, never 2: check all, then take all.	Locking one by one without rollback.
One monitor per show	Shows never block each other; within a show every decision is serialized.	One global lock (course) — correct but serializes the country.
Commit re-validates + compensation	Never pay for a seat you don't get.	Trusting the earlier check.
Idempotent `pay`	Double-click/retry can't charge twice: non-PENDING returns unchanged. Real systems add an **idempotency key**.	Charging on every call.
Strategy refunds, **Strategy-like** `PaymentMethod`	Rules and payment types vary independently.	`if (method == CARD)`.
Adapter `PaymentGateway`	Wraps Stripe/Razorpay; tests use `FakeGateway`.	Provider SDK in `BookingService`.
Coupon on the booking	Amount correct before payment.	Each payment type applying coupons.
Guest has no booking method	"Register to book" enforced by *types*: `hold` takes a `Customer`.	`Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only when a show is scheduled there.	A city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking` (`Venue`, `SeatLockProvider`, `Payments`, `Catalog`, `BookingService`, `Main`).

```
final class InMemorySeatLockProvider implements SeatLockProvider {
    // ...
    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }
    // ...
    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }
    // ...
}

final class BookingService {
    // ...
    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());          // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) + "
refunded");
                return b;
            }
            b.setStatus(BookingStatus.CONFIRMED);
            // ... notify CONFIRMED
            return b;
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch15/moviebooking/ (see the Full Edition).

Output (excerpt):

```
  add SH2 RRR 19:00 same hall -> false
== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  paying twice is harmless -> CONFIRMED
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  [notify NEHA] BK-6 payment DECLINED, seats released
```

The 10% coupon turns Rs.700 into Rs.630; an overlapping show in the same hall is refused; Meena's hold expired while the bank was charging and Kiran took the seat, so she is refunded.

Concurrency & edge cases

- **Atomic check-then-act:** `lock`, `holds`, `commit`, `unlock`, `releaseBooked` for one show all run `synchronized` on that show's lock table.
- **Lazy expiry:** no background thread; an expired `SeatLock` is treated as absent the next time anyone looks (Redis TTL does the same).
- **Payment-window race:** commit re-validation + refund.
- **Lock order:** booking monitor (serialises `pay`/`cancel` — no double charge, no pay-while-cancel) → show monitor, never reversed.
- **Across many servers**, from simplest: (1) **conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, check the row count; (2) **pessimistic** `SELECT ... FOR UPDATE`; (3) **serializable** transaction (course) — simplest to reason about, slowest under collision; (4) **optimistic** `version` column, retry on conflict (rare conflicts); (5) **unique constraint** on `(show_id, seat_id)` as the final net; (6) **Redis TTL** keys for holds, DB as truth for confirmed seats. The in-process lock only makes the common case quick; with two servers, correctness comes from the database.
- **Edge cases:** empty or duplicate seat ids (de-duplicated), unknown seat, blocked customer, overlapping show, cancelling an abandoned booking, cancelling too late (no refund).

Extensions & follow-up questions

- **Live seat map** → push seat events over WebSockets/SSE (Observer). **Extend the hold** → refresh `expiresAt` only if still held. **Dynamic pricing** → a `PricingStrategy`.
- **Paid but crashed before commit** → store PENDING with the gateway transaction id first; a reconciliation job (plus provider **webhooks**) commits or refunds.
- **Cancel a whole show** → cancel and fully refund every confirmed booking of that show.
- **Seat-gap rule** → a `SeatSelectionRule` chain (Chain of Responsibility) before locking; loyalty points → another `PaymentMethod`.

Real-world example

BookMyShow's payment-screen **countdown timer** is the hold's TTL; its **automatic refund** when the bank debits but booking fails is the compensation path.

In the interview

30–45 minutes: 4 min requirements (hold time, payment, refunds) → 4 min actors → 8 min entities, saying **why** `ShowSeat` → 12 min `SeatLockProvider` (`lock`, `commit`) and `BookingService.hold/pay/cancel` → 5 min payment failure and refund → multi-server concurrency.

> **30-second answer:** "City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat — that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, refunding if the hold was lost. In one process a per-show monitor prevents double booking; across servers a conditional update or unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups: *Why TTL?* Otherwise abandoned checkouts block seats forever. *Optimistic or pessimistic?* Pessimistic/conditional update for hot shows; optimistic when conflicts are rare.

Self-check

- **Q:** Why is `ShowSeat` separate from `HallSeat`? **A:** State and price are per show; the physical seat is shared by every show.
- **Q:** Payment succeeded but `commit` returned false? **A:** Refund (compensation), mark abandoned.
- **Q:** How are expired holds cleaned up? **A:** Lazily — any check treats an expired lock as absent.
- **Q:** What makes `pay` safe to call twice? **A:** It runs under the booking's monitor and returns immediately unless PENDING.

TL;DR

Book `ShowSeat`s. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM; conditional updates, unique constraints and Redis TTL across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

Concert ticketing (**Ticketmaster**, **Live Nation**, **SeatGeek**) has the same objects as 15.2 — we reuse them — but a different enemy: **extreme contention**. A stadium sells 50,000 tickets in minutes to a million people arriving at 10:00:00. That changes three things: (1) **fairness** — who tries first (a *virtual waiting room*); (2) **throughput** — locks that serialise everyone become the bottleneck (use *lock-free* CAS); (3) **abuse** — scalpers and bots (*per-user limits*) and fans who miss out (*waitlist*).

Question	Answer we assume
Seat types?	**VIP** and **PREMIUM** numbered seats; **GENERAL_ADMISSION** (GA) standing = a *count*.
Search?	By artist, venue, date, time (as 15.2).
Stampede?	**Virtual waiting room**: queue numbers, batches admitted in order.
Limit?	**4 tickets** per user per concert.
Hold time?	**10 minutes**.
Sold out?	Per-tier **waitlist**; released tickets offered to it first.
Notifications?	Confirmation; "a ticket freed up for you".

Requirements

Functional: show concerts with layout and availability; search; choose seats or a GA quantity and buy; **prevent double booking** under concurrency; **fair access**; secure payments; confirmations; **waitlist** for sold-out events.

Non-functional: extreme write contention; **no overselling** ever; FCFS fairness; per-user caps; self-expiring holds.

Use cases

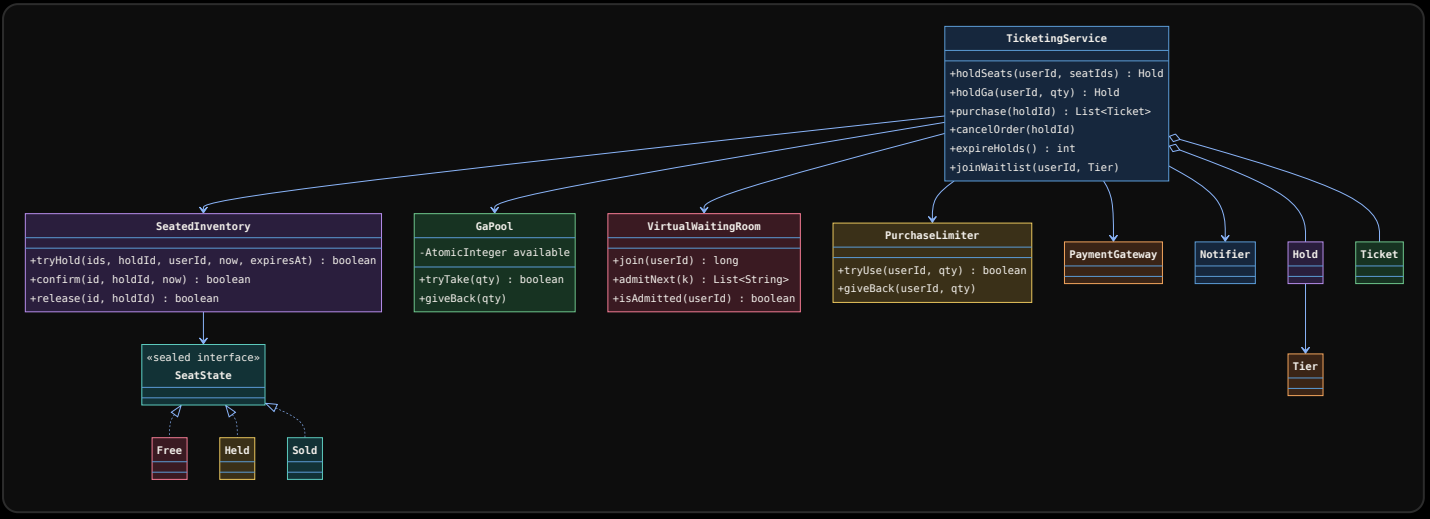
Actors: Fan (join queue, hold, buy, cancel, join waitlist), Organizer/Admin (create concert, tiers, prices, open sale), System (admit batches, expire holds, offer released tickets, notify), Payment provider. **Flow:** join waiting room → admitted → **hold** → **purchase** → tickets; cancel (refund); join waitlist; system **reaps** expired holds.

Core entities

Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated), `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` per seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger`; CAS take/give back.
`Hold` (record)	Seats or GA quantity, amount, expiry.
`Ticket` (record)	Tier and seat id or "GA".
`VirtualWaitingRoom`	Arrival-order numbers; admits FIFO batches.
`PurchaseLimiter`	Max N per user, atomic check-and-update.
`TicketingService`	hold → purchase → cancel; reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as 15.2).

Class diagram

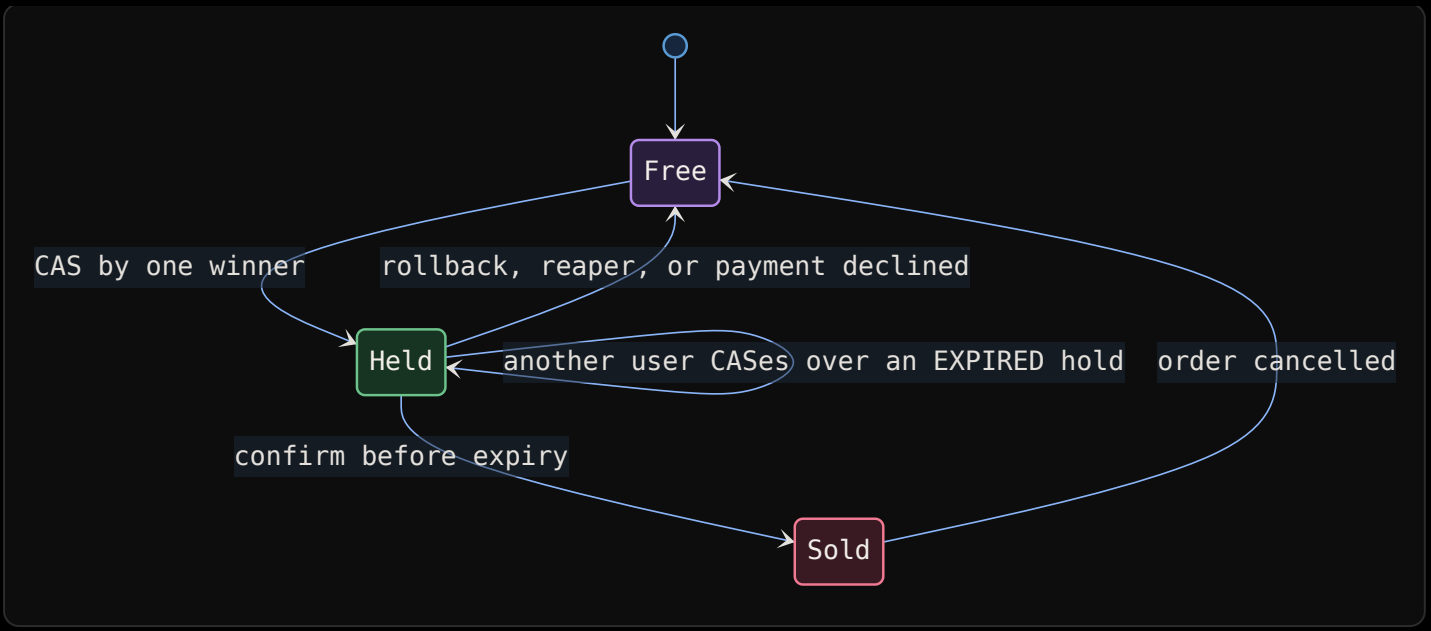
No course lesson here; the problem comes from the *awesome-low-level-design* set. Our design:



Key flows

Buying: fan joins the room (gets e.g. number 41,207; the system calls `admitNext(batch)` every few seconds) → `holdSeats`: is admitted? → `limiter.tryUse` → CAS each seat Free → Held in **sorted order**; if any CAS fails, roll back own Held seats and give back the limit (`SoldOutException`) → `purchase`: remove the hold from the active map (claims it once) → charge → CAS each seat Held → Sold → tickets.

Seat state machine — every arrow is one `compareAndSet`:



Released inventory (expired hold, cancellation) is **offered to the tier's waitlist first**: the next fan is admitted directly and given a fresh hold plus a notification; if unpaid, the reaper releases it to the next.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat	Different seats never wait; on one seat exactly one CAS wins, losers learn instantly; no thread blocks holding a lock, so no deadlock.	One monitor per concert (15.2) — at 50,000 buyers everyone queues on one lock.
Immutable sealed `SeatState`	CAS compares *references*; a fresh record per state means "what I read" is exactly "what I replace"; exhaustive pattern matching.	Mutable `status` / `heldBy` / `expiry` fields — readers see half-updated seats.
Sorted multi-seat hold + rollback	Release only seats *we* took; sorted order makes overlapping buyers contend on the same first seat, so one fails fast.	Nested locks — deadlock-prone.
GA as a counter	Standing places are interchangeable; CAS loop (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	40,000 `Seat` objects.
Virtual waiting room	Arrival order decides who may *try*; batches cap load; bots gain nothing. `join` is `synchronized` so number order = queue order.	"Whoever lands first" — rewards bots, melts the backend.
Per-user limit via `ConcurrentHashMap.compute`	Check-and-update for one key atomically.	`get` then `put` — a race.
TTL holds + reaper	Seats reclaimed lazily (CAS over expired `Held`); GA holds need `expireHolds()` . `holds.remove(id, h)` ensures reaper and `purchase` never both process a hold.	Holds that never expire.
Waitlist offers on release	Longest-waiting fan (FIFO), not fastest refresher.	Silent return to the pool.
Compensation on lost hold	As 15.2: release and refund.	Assuming the hold is valid.
No Singleton	The *awesome-lld* reference uses one; DI is testable and allows one service per concert.	Global Singleton.

Java implementation

Package `lld.ch15.concert` (`Model` , `Inventory` , `TicketingService` , `Main`). Demos use Java 21 **virtual threads** (`Executors.newVirtualThreadPerTaskExecutor()`), making 1,000 concurrent buyers cheap.

```java
/** A seat's whole life in one immutable value, swapped atomically with compare-and-set (CAS). */
sealed interface SeatState permits Free, Held, Sold {}
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

final class SeatedInventory {
    // ...
    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        if (ref == null) throw new IllegalArgumentException("no seat " + id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }
    // ...
}

/** General admission: no seat numbers, just a counter. A CAS loop means it can never go below zero. */
final class GaPool {
    // ...
    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    // ...
}

final class PurchaseLimiter {
    // ...
    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;            // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    // ...
}

final class TicketingService {
    // ...
    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);                    // atomic claim: a hold is purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        // ... expired / declined -> releaseInventory(h); then CAS each seat Held -> Sold, refund if lost
    }
    // ...
}
```

Full runnable code: code/lld/ch15/concert/ (see the Full Edition).

Output (excerpt):

```
REFUSED: eli is still in the waiting room
holds on VIP-1 = 1
sold=98 soldOut=902 gaLeft=0 (never negative)
REFUSED: ben would exceed the per-user ticket limit
[notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
```

40 concurrent CAS attempts on one seat → exactly one hold; 1,000 buyers for 98 GA places sell exactly 98; a cancellation goes straight to the waitlisted fan as a hold; money collected 85,900.00 = 98 × 800 + 3 × 2,500.

Concurrency & edge cases

- **Why CAS is correct:** `compareAndSet(expected, new)` replaces only if the value is still the object we read; otherwise re-read and decide. `tryHoldOne` retries only while the seat is still claimable.
- **ABA problem** (A → B → A fooling a CAS)? Every transition creates a *new* record and `AtomicReference` compares references, so a returned "A" is a different object — the CAS fails safely.
- **Hold claimed once:** `purchase` does `holds.remove(holdId)`, the reaper `holds.remove(id, h)` — only one wins.
- **Limit given back** on every failure path (sold out, declined, expired, lost seat).
- **Lost hold after payment:** `confirm` fails → release, give back limit, refund.
- **Fair queue scope:** fair within one process; at scale the waiting room is a separate service (often at the CDN edge) issuing signed, time-limited access tokens.
- **Across servers:** seat CAS → `UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())` or Redis `SET NX PX`; GA → Redis `DECRBY` in a Lua script, or `WHERE available >= ?`.

Extensions & follow-up questions

- **Best available seats** → a `SeatAllocator` strategy picks adjacent free seats, then the same CAS hold.
- **Verified fans / presale codes** → admission tokens for pre-registered fans; limiter keyed on verified identity.
- **Dynamic pricing** → a `PricingStrategy` using remaining inventory and time, frozen in the `Hold`.
- **Resale** → transfer ownership and re-issue a new barcode.
- **Waitlist for seated tiers** → same mechanism per released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue"), **Verified Fan** presales and per-account limits. The 2022 *Eras Tour* presale overwhelmed the system and the general sale was cancelled — admission control matters as much as seat locking.

In the interview

30–45 minutes: say "entities like movie booking; the hard part is contention and fairness" → tiers (why GA is a counter) → TTL holds; code the seat CAS and GA CAS loop → waiting room and atomic per-user limit → waitlist and reaper → scale-out (tokens, conditional updates/Redis).

> **30-second answer:** "Seated tiers are an `AtomicReference<SeatState>` per seat claimed by CAS — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop, never negative. Everyone enters through a virtual waiting room admitting in arrival order, with an atomic per-user limit. Holds expire after 10 minutes, a reaper returns them, and released tickets go to a FIFO waitlist first. A hold lost during payment is refunded."

Follow-ups: *Lock-free vs locks?* Lock-free wins under high contention with tiny critical sections; locks are simpler when many things change together. *Queue service fails?* Fail closed — pause admissions. *One person, 50 accounts?* Verified identity, card fingerprinting, limits on those.

Self-check

- **Q:** Why is GA a counter? **A:** Places are interchangeable; a CAS counter is simpler and can't oversell.

- **Q:** What does `compareAndSet` guarantee? **A:** Update only if the value is still the one you read.
- **Q:** Why must `join` be synchronized? **A:** So number and queue position are assigned together, in the same order.
- **Q:** How can't a hold be both purchased and reaped? **A:** Both remove it from `holds` first; only one removal succeeds.

TL;DR

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, FIFO virtual waiting room, atomic per-user limits, TTL holds with a reaper, waitlist first for released tickets, refunds when a hold is lost.

15.4 Design a Hotel Management System

Problem & clarifying questions

A **Hotel Management System** (property management system, PMS) knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Unlike a cinema, a room is sold **night by night** — that shapes the whole design.

Question	Answer we assume
Locations, types, search?	Many `HotelLocation`s; `STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`; search by type and **date range**.
Cancellation?	**Full refund if more than 24 h before check-in**; otherwise none (our assumption).
Extras?	Room service, kitchen, amenities on the bill.
Payment?	**Card, check or cash**.
Keys?	Electronic cards with barcode and key id; deactivated at check-out.
Housekeeping?	Logged per room, never overwritten.
Several rooms per booking?	No — one room (a known limitation).

Requirements

Functional (course): (1) book **room types**; (2) **search** and book any **available** room; (3) **who booked a room / which rooms a customer booked**; (4) **cancel**, full refund > 24 h before check-in; (5) **notify** before check-in and check-out; (6) **housekeeping log**; (7) **room service and food**; (8) **amenities**; (9) pay by **card, check or cash**.

Non-functional: never double-book a room-night; bill = sum of lines; actors limited to their jobs; refund rule in one place.

Use cases

Actors (course): Guest (search, book), Receptionist (rooms, bookings, check in/out), System (notifications), Manager (hires workers), Housekeeper (housekeeping records), Server (room-service records). **Top use cases:** manage rooms; search; register/cancel account; book; **check in**; **check out** (return keys); add room charge; update housekeeping.

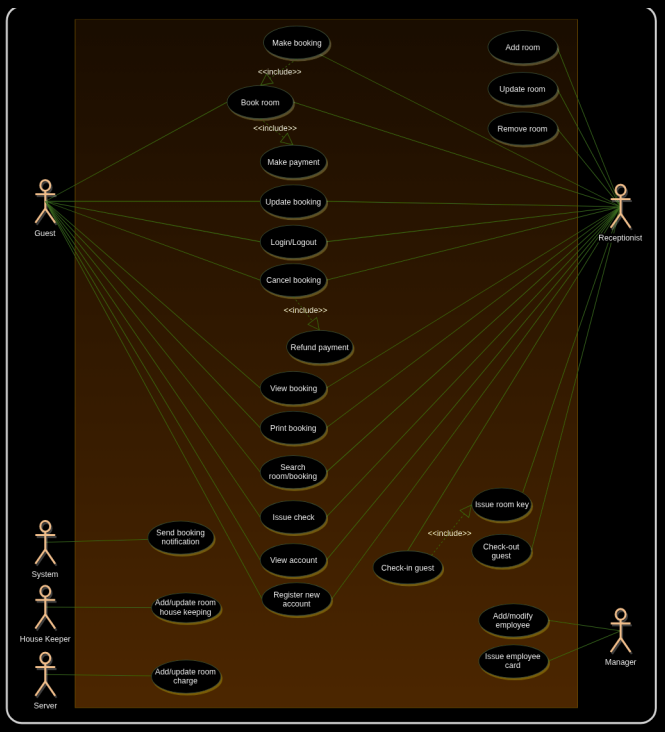


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns locations; searches each.
`HotelLocation` implements `Search`	Owns rooms; "which rooms of this type are free for these dates?".
`Room`	Number, style, price; the **set of booked nights**; owns keys and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus`	Type; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master, active.
`RoomHouseKeeping` (record)	When, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, dates, status, deposit; **owns its** `Invoice`; `refundDue()`.
`BookingStatus`	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	Lines; total computed.
`RoomCharge` → `Amenity`, `RoomService`, `KitchenService`	Writes itself onto the invoice (Template Method).
`Payment` (sealed) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three genuinely different behaviours.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; methods accept only the right type.
`FrontDesk`	Book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

Five course stages: (1) **invoice, payments, notifications** — the invoice stores **lines** and computes its total, so it can't drift from them (req. 7, 8); (2) **room, keys, housekeeping** — the key decision: **a status field is not availability** — `OCCUPIED` describes now, but guests ask about future date ranges, so each room stores its sold nights; (3) **hotel, location, search** — search sits on the **location** (a room can't list other free rooms; `Room.search()` is a common beginner mistake); (4) **booking and room charges**; (5) **people** — there is deliberately **no arrow guest ↔ booking**: the booking stores the guest's **id**, and queries filter by it.

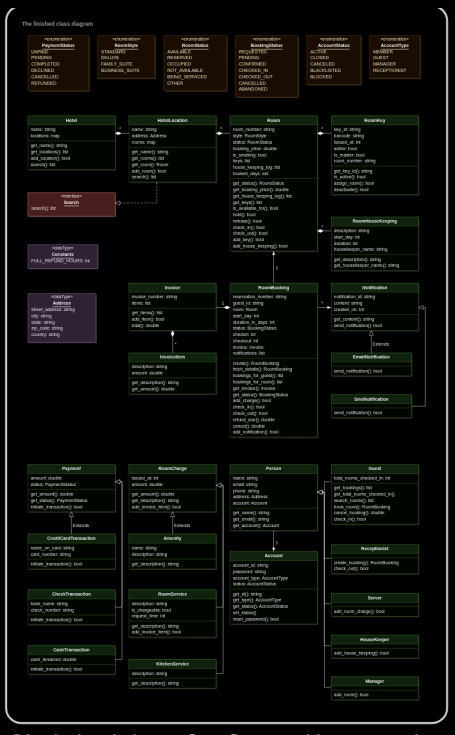
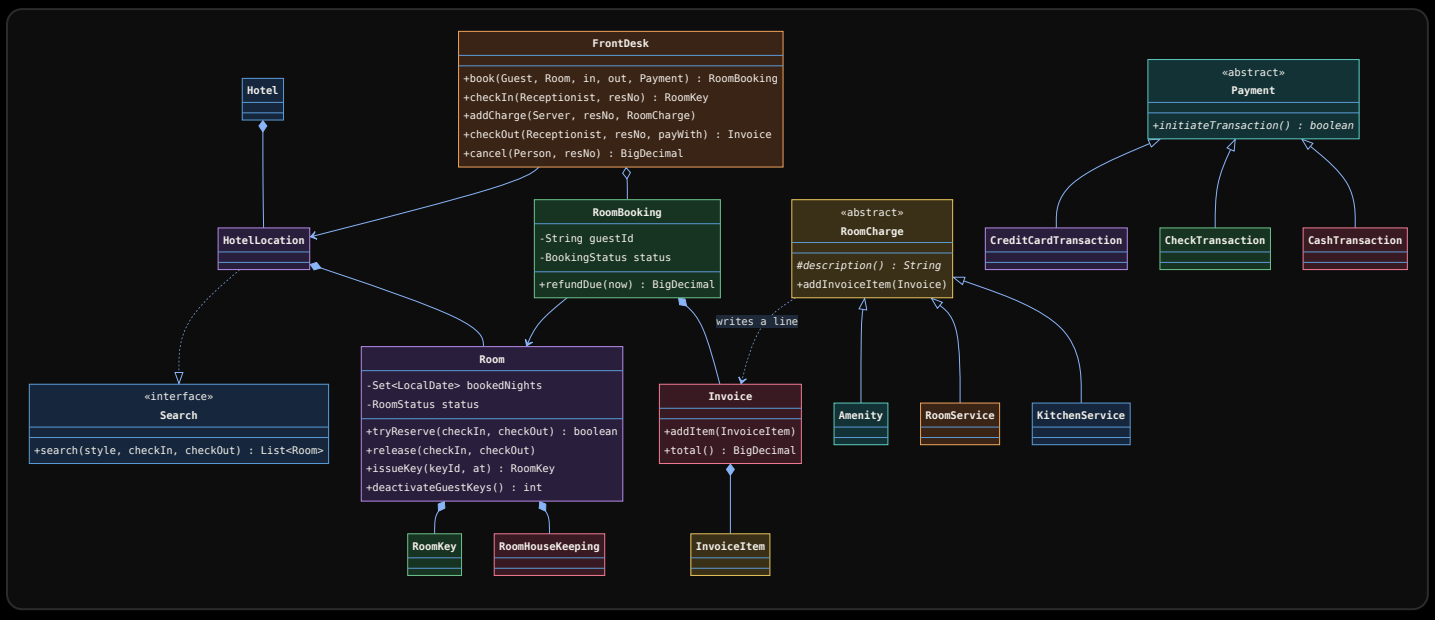


Figure — The finished hotel class diagram (course diagram)



Key flows

The course's activity diagrams: **make a booking** (search type and dates → none: tell customer; else find/register customer, book, pay, deliver details), **check in** (find booking; room not ready → wait; else issue key, status "Checked-in"), **cancel** (find booking; before the deadline → refund, else none; print/email).

A whole stay: guest `book(room 102, Jul 10–13, deposit)` → `tryReserve` under the room monitor (nights 10, 11, 12 taken) → CONFIRMED → receptionist `checkIn` (room OCCUPIED, key issued) → server `addCharge(KitchenService)` writes an invoice line → `checkOut(pay by card)` adds the nights line and subtracts the deposit, deactivates keys, room BEING_SERVICED → invoice total = sum of lines.

Booking lifecycle: `CONFIRMED → CHECKED_IN → CHECKED_OUT`, or `CONFIRMED → CANCELLED` (refund if > 24 h ahead). The room's own status cycles independently: `AVAILABLE → OCCUPIED → BEING_SERVICED → AVAILABLE` (after housekeeping).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights	Answers date ranges; nights are half-open `[checkIn, checkOut)`, so leaving and arriving on the 13th don't clash.	`RoomStatus` — only knows now.
`tryReserve` `synchronized` on the room	Check-then-act made atomic; rooms don't block each other.	Hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = lines, total computed	Total can't disagree with lines.	Stored running total.
Template Method `RoomCharge.addInvoiceItem`	`final` parent algorithm (describe → price → write line); subclasses supply only `description()`; new charge = new subclass (OCP). The course calls it its clearest Template Method; Strategy swaps a *whole* method instead.	`switch` on charge type in the invoice.
Three `Payment` subclasses	Card completes at once; **check stays `PENDING`**; cash declines if short. Each passes LSP.	One `Payment` with a `type` field.
Refund rule in one method (`refundDue`)	One place to edit; promote to a `CancellationPolicy` strategy (15.2) for several policies.	Scattered refund arithmetic.
Actor types in signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — compiler-enforced roles; a receptionist booking for a guest calls the *same* method.	Anyone calls anything.
Booking stores guest id	No two-way dependency; "rooms of this guest" is a query.	Bidirectional links.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo.	Phantom reservations.

Java implementation

Package `lld.ch15.hotel` (`Billing`, `Rooms`, `Stay`, `Main`).

```java
final class Room {
    // ...
    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d)) return
false;
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    // ...
}

abstract class RoomCharge {
    private final BigDecimal amount;
    RoomCharge(BigDecimal amount) { this.amount = amount; }
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // the template: final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}

final class RoomBooking {
    // ...
    /** The refund rule lives in exactly ONE method: full refund if cancelled more than 24h before check-in. */
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than 24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}

final class FrontDesk {
    // ...
    /** Guest or receptionist books: same call, two actors. The room's monitor makes it race-free. */
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." +
checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {                  // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        // ... store booking, notify
    }
    // ...
}
```

Full runnable code: code/lld/ch15/hotel/ (see the Full Edition).

Output (excerpt):

```
  successful bookings = 1 of 6
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00
  cash short: payment DECLINED, guest still checked in
    TOTAL                          $299.00
  key active after checkout? false, room 102 is BEING_SERVICED
```

Six concurrent overlapping bookings of room 101 → one wins; Ben's stay from the 13th fits after Asha's ending the 13th; Carl cancels 19 h ahead (no refund), Asha 8 days ahead (full); the bill is $5 + $24 + $30 + 3 × $120 − $120 deposit = **$299**, and a declined cash payment doesn't duplicate lines on retry.

Concurrency & edge cases

- **Double-booking a night:** `Room.tryReserve` is `synchronized` — check and add of every night are one step; the 6-thread race has one winner.
- **Search is advisory:** the room may be gone a millisecond later; `tryReserve` is the real check.
- **Booking transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor; lock order booking → room.
- **The 24-hour rule, exactly:** compare `Duration`s; `toHours() > 24` truncates minutes, so 24 h 30 min would wrongly lose its refund.
- **Retry-safe checkout:** nights and deposit lines are written only after payment succeeds; `addCharge` takes the same booking monitor, so no charge slips in after totalling.
- **Edge cases:** check-out not after check-in; checking in too early; charges when not in house; cancelling after check-in; short cash (declined); a check stays `PENDING`.
- **Chain-wide inventory:** per-room nights don't scale across channels; real systems use an **inventory table** keyed by `(room_type, date)` with `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`.

Extensions & follow-up questions

- **Two rooms in one booking** (course flags it) → a `Reservation` holding several `RoomStay`s; all-or-nothing across rooms by locking in room-number order, or one DB transaction.
- **Book by type, assign at check-in** → per-type-per-night counts for selling; pick the physical room at check-in.
- **Seasonal rates** → a `RatePlan` / `PricingStrategy`, one line per night.
- **Overbooking** → sell capacity × (1 + factor); "walk" extras to a partner hotel — a strategy on the availability check.
- **Booker vs occupants** (course names it missing) → `Occupant`s on the stay. **Lost key** → `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Oracle OPERA sells room types by date, assigns rooms at check-in, and posts charges to the folio; key cards (VingCard, dormakaba) expire at check-out; **Booking.com** shows per-rate policies like "free cancellation until 24 hours before arrival".

In the interview

30–45 minutes: clarify (types, date search, 24 h refund, charges, payments, keys, housekeeping) → actors → entities, saying early **"a status field is not availability"** → code `tryReserve`, `book`, `checkIn`, `checkOut` → Template Method charges, payment subclasses, refund rule → follow-ups (two rooms, chain inventory, overbooking).

> **30-second answer:** "A `Hotel` has `HotelLocation`s that own `Room`s and search by type and date range. Each room keeps its booked nights, and reserving nights is a synchronized check-then-act. A `RoomBooking` owns an `Invoice` of lines; room service, kitchen and amenities are `RoomCharge` subclasses adding a line via Template Method. Payment is card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

Self-check

- **Q:** Why isn't `RoomStatus` enough? **A:** It describes now; a booking asks about future nights.
- **Q:** Why half-open `[checkIn, checkOut)`? **A:** The checkout day's night isn't used, so the next guest can arrive that day.
- **Q:** Which pattern adds a charge type without touching `Invoice`? **A:** Template Method.
- **Q:** How is "only a server adds room service" guaranteed? **A:** `addCharge` takes a `Server`; other types don't compile.

TL;DR

Rooms are sold night by night: each room keeps booked nights and reserves them atomically. Search on the location. The bill is lines written by `RoomCharge` Template Methods; three payment subclasses; one 24-hour refund method. Keys issued at check-in, killed at check-out; housekeeping is append-only.

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with five more: car rental, restaurant, airline, course registration and a meeting scheduler. They are one problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Spot that skeleton and every new booking problem is quick to set up.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration	Meeting scheduler
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section	a room for a time range
The "calendar"	days	hourly time slots	the flight instance (route + date)	the term	continuous time (half-open intervals)
The atomic check-then-claim	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`	`Room.tryReserve`
When it is full	search another car	try the next table	another seat or flight	waitlist	next fitting room
Money	bill lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)	none

A **resource** is anything in limited supply. A **race condition** is a bug where the result depends on which thread runs first — here, two people both "win" one seat because both checked "free?" before either wrote "taken".

How to run: `cd chapters/16-case-booking-2/code && javac --release 21 -d /tmp/lld_16 $(find . -name '*.java') && java -cp /tmp/lld_16 lld.ch16.carrental.Main` (or `restaurant`, `airline`, `courseregistration`, `meetingscheduler`). Where threads race, the *counts* are always the same; *which* racer wins varies. One file sometimes holds several small package-private types (legal Java; production code would split them).

> **Money.** All prices use `BigDecimal`, never `double` — a `double` cannot store 0.1 exactly, so sums drift by fractions of a cent.

Design a Car Rental System

Problem & clarifying questions

A **car rental system** rents vehicles for hours to weeks from many branches; a car may be returned to a different branch.

Question	Assumed answer
Vehicle kinds?	Car, truck, SUV, van, motorcycle.
Branches?	Many, each with its own vehicles; drop-off branch may differ from pick-up.
Price?	Daily rate per vehicle type + add-ons + a late fee per day late.
Add-ons?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi), additional drivers.
Identification?	Unique barcode, scanned at pick-up and return.
History?	A vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Two members must never get the same car on overlapping days.

Requirements

Functional: (1) rent car/truck/SUV/van/motorcycle; (2) add each vehicle with a unique barcode and details incl. parking stall; (3) answer "who took this vehicle?" and "what has this member rented?"; (4) late fee after the due date; (5) search and reserve any available vehicle; (6) notify when pick-up is near, due date is near, and when overdue; (7) read barcodes; (8) cancel reservations; (9) vehicle log; (10) rental insurance; (11) extra equipment; (12) services.

Non-functional: no double booking for overlapping days; new add-ons or pricing rules without changing reservation code (Open/Closed); auditability — every charge is a bill line, every event a log entry.

Use cases

Actors: **Receptionist** (adds/edits vehicles and workers, can reserve for a customer), **Member** (searches, reserves, picks up, returns), **Worker** (handles returns, updates the vehicle log), **System** (reminders, overdue alerts, cancellations). Top use cases: add/edit vehicle; search by type and availability; register/cancel membership; reserve; check out; return; add equipment/services; update log. Reserving is shared by member and receptionist; the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Owns the branches and the reservation service.
`CarRentalLocation`	One branch with its own `VehicleInventory`.
`VehicleInventory`	Indexes by barcode, type and model; "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, type, parking stall, mileage, **the set of booked days**, its log.
`VehicleLogEntry`	One event (date, type, note).
`DateRange`	Half-open `[start, end)`; `end` is the due date.
`Reservation`	Member, vehicle, dates, pick-up/return branch, add-ons, drivers, bill, status; holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`), `AdditionalDriver`	Things that add a charge; a named extra driver (no charge).
`Bill`, `BillItem`	The one place that knows what a rental costs.
`PricingStrategy`, `LateFeePolicy`	Pluggable base-charge and late-fee rules.
`ReservationService`	Creates reservations atomically, answers "who rented what", runs reminders.
`Notifier`, `PaymentMethod`	Ports for messages and money.

Class diagram

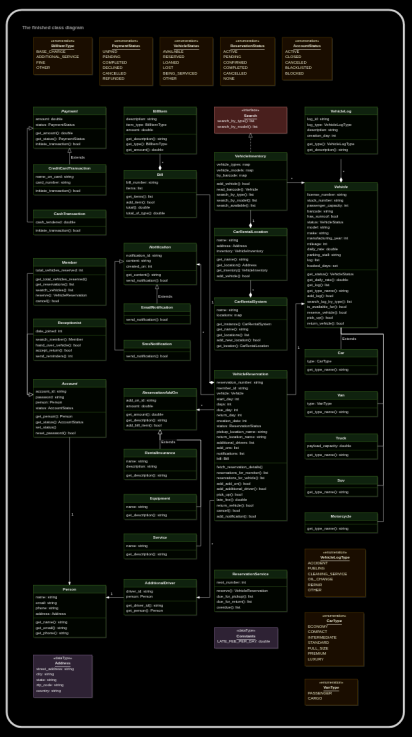
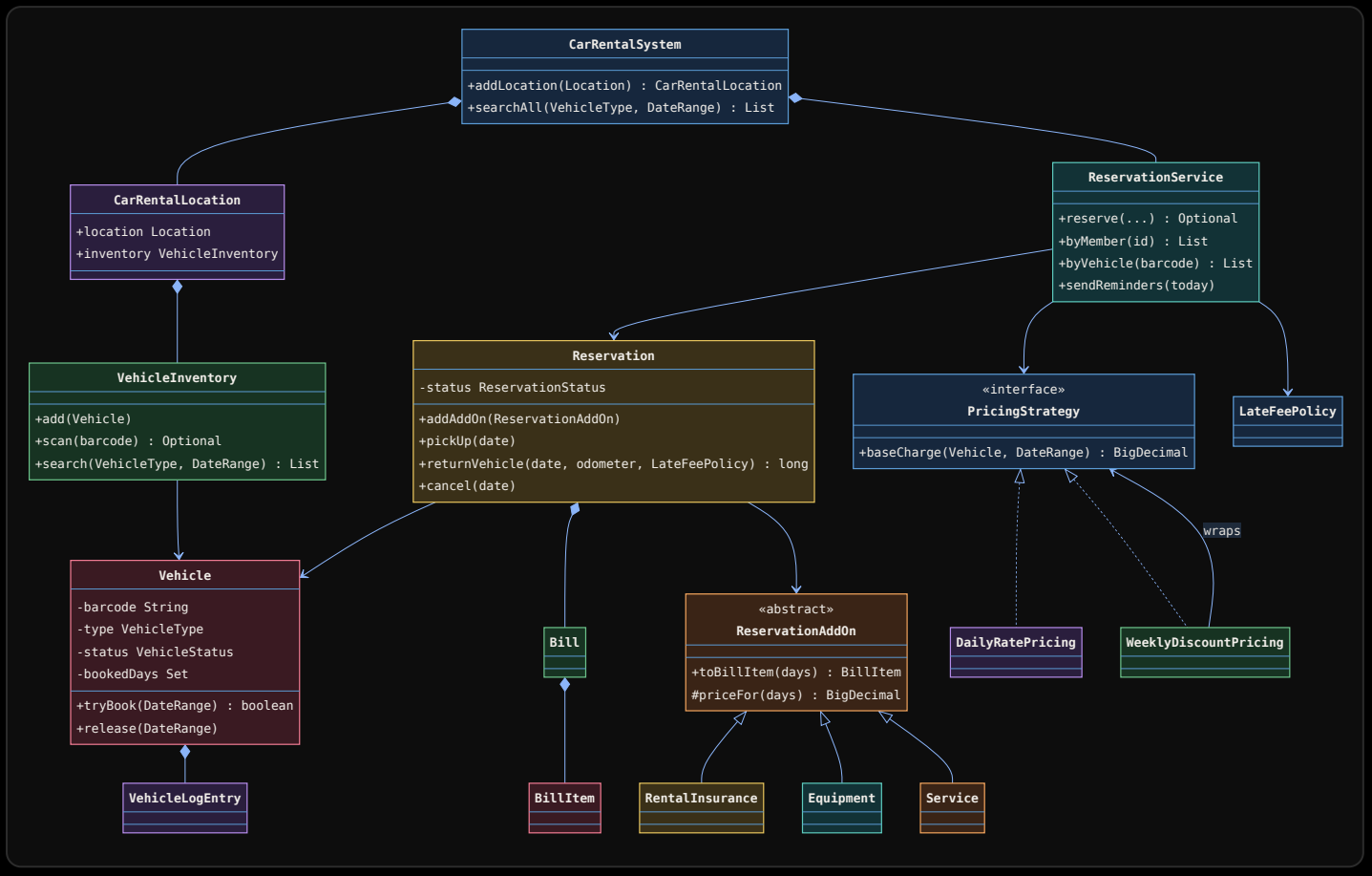


Figure — The finished class diagram of the car rental system (course diagram)

The system owns its branches and each branch owns its inventory (**composition** — the part dies with the whole); the inventory only *refers to* vehicles (**association**). A reservation owns its bill but refers to the vehicle. Our Java design differs in two ways: vehicle kinds are an enum (the five subclasses added no behaviour), and pricing is a strategy object.



Key flows

- **Pick up:** scan licence (or search by name) → check reservation → scan vehicle barcode → hand over keys; no reservation → reserve a free car on the spot.
- **Return:** worker scans the barcode → past due date adds a late fee (one line in `returnVehicle`) → check damage and fuel → settle the bill.
- **Reserve:** `ReservationService.reserve` calls `vehicle.tryBook(dates)`; if false the member picks another car; if true it prices via `PricingStrategy`, creates the reservation (bill gets a BASE line) and notifies.
- **Lifecycle (state machine):** `CONFIRMED →pickUp→ ACTIVE →returnVehicle→ COMPLETED`; `CONFIRMED →cancel→ CANCELLED` (days released on cancel and return).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = booked days per vehicle, not a status	Members ask about future dates; a car loaned now may be free next week, a car on the lot may be promised next week.	A `RESERVED` status (can't answer date questions; one reservation per car).
Keep `VehicleStatus` too	Staff need "where is it now" (lot, loaned, in service, lost).	Dropping it.
`tryBook` checks **and** books in one `synchronized` method	Closes the check-then-act race.	`isFree()` then `book()`.
Template Method on `ReservationAddOn`	Billing written once in `final toBillItem`; subclasses give `kind` and may override the `priceFor` hook.	Type enum + `switch`.
Strategy for pricing and late fee; **Decorator** `WeeklyDiscountPricing`	Swap/stack rules without touching the reservation.	Constants; a subclass per combination.
`Bill` is the only place charges live; methods `synchronized`, `settle` idempotent	One total; no line added mid-payment; a double-click pays once.	Per-add-on running totals.
`ReservationService` separate from `Reservation`	Creating/indexing vs one reservation's rules (SRP).	Static factory.
Actors call the rules, never compute them	Member calls the service; receptionist calls the reservation.	`Receptionist.computeLateFee()`.
Course's `CarRentalSystem` **Singleton** → we pass one in	Same "one per app", but testable (dependency injection).	`getInstance()`.

Java implementation

Package `lld.ch16.carrental`. Full runnable code: code/lld/ch16/carrental/ (see the Full Edition).

```java
// Vehicle.java — check AND book inside one lock
synchronized boolean isFreeFor(DateRange r) {
    return r.dates().noneMatch(bookedDays::contains);
}

synchronized boolean tryBook(DateRange r) {
    if (!isFreeFor(r)) return false;
    r.dates().forEach(bookedDays::add);
    return true;
}

synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }

// Billing.java — Template Method
abstract class ReservationAddOn {
    // ...
    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }

    protected abstract String kind();

    /** Hook: by default an add-on is charged per rental day. */
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); }
}

/** A service such as roadside assistance or wifi is a flat fee per rental. */
final class Service extends ReservationAddOn {
    Service(String name, BigDecimal flatFee) { super(name, flatFee); }
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }
}

// Pricing.java — Decorator over any pricing: 10% off rentals of 7 days or more.
final class WeeklyDiscountPricing implements PricingStrategy {
    private final PricingStrategy inner;
    WeeklyDiscountPricing(PricingStrategy inner) { this.inner = inner; }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}

// Reservation.java — the rules live here
synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
    require(ReservationStatus.ACTIVE, "return");
    long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
    if (daysLate > 0) {
        bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE,
                lateFee.fee(vehicle, daysLate)));
    }
    vehicle.setMileage(odometer);
    vehicle.setStatus(VehicleStatus.AVAILABLE);
    vehicle.release(period);
    // ... log RETURNED, status = COMPLETED
}

// RentalSystem.java — ReservationService
Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff, LocalDate
today) {
    if (!v.tryBook(period)) return Optional.empty();
    var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
    reservations.put(r.id(), r);
    v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
    notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
    return Optional.of(r);
}
```

Output excerpt:

```
3) 8 members race for VH-002 on the same dates
   successful reservations for VH-002: 1
6) Alice's bill
   Base: RAV4 x 3 days                      BASE_CHARGE   240.00
   Insurance: Collision damage waiver       ADD_ON        45.00
   Equipment: Child seat                    ADD_ON        24.00
   Service: Roadside assistance             ADD_ON        25.00
   Late fee: 2 day(s)                       LATE_FEE      80.00
   TOTAL 414.00
   paid: true, settle again (double click) -> true, no second charge
10) A 7-day Civic rental gets the weekly discount
   base charge 315.00 (7 x 50 = 350, minus 10%)
```

Eight threads race for one car and exactly one wins; the late fee is 2 × 40.00; a second `settle` does not charge again.

Concurrency & edge cases

- **Double booking** (a gap the course admits): `tryBook` is `synchronized` per vehicle, so check and mark happen as one step; per-vehicle locks let different cars book in parallel. In a database: unique key on `(vehicle_id, day)` or `SELECT … FOR UPDATE`.
- **Search is advisory** — `reserve` may still fail; show "no longer available".
- **Late return collides with the next rental** — keep a buffer day; reassign the next booking to another car of the same type.
- **Early return** releases the whole range; refunds are policy. **Cancel after pick-up**, **add-ons after the rental**, **invalid ranges**, **odometer going backwards** are all rejected.
- **Different return branch** — recorded, but the car stays in the pick-up inventory: a known gap (transfers).

Extensions & follow-up questions

- **One-way rentals?** On return at another branch move the vehicle between inventories, log a `TRANSFER`, add a one-way fee line.
- **Seasonal/dynamic pricing?** A `SeasonalPricing` strategy or another decorator; reservation unchanged.
- **Deposits?** `authorize` at pick-up, `capture` at return. **Mileage/fuel/damage?** `FINE` bill lines. **Reminders?** A daily scheduler calls `sendReminders(today)`. **Scale?** Partition by branch, `(vehicle_id, day)` unique key, search on a replica/index.

In the interview

> **30-second answer:** "Branches own inventories indexed by barcode, type and model. Availability is a question about *dates*, so each vehicle keeps its booked days and `tryBook` checks and books atomically. A reservation owns a bill; every charge is a line. Add-ons share a template-method base; pricing and late fees are strategies."

Plan (40 min): 0–5 clarify; 5–10 actors and entities; 10–20 class diagram ("dates, not status"); 20–35 code `tryBook`, `Reservation`, `reserve`; 35–40 concurrency, late collisions, transfers. Likely follow-ups: book next month while loaned now? (yes — dates); two clicks at once? (atomic `tryBook`); new add-on? (one subclass).

Self-check

1. **Q:** Why isn't a `RESERVED` status enough for "free next week?" **A:** Status describes *now*; only per-day bookings answer date questions.
2. **Q:** Pattern and hook in `toBillItem`? **A:** Template Method; `priceFor` is the hook (`Service` makes it flat).
3. **Q:** Why one `tryBook` not `isFree` + `book`? **A:** Atomic check-then-act under the vehicle lock.
4. **Q:** How is the late fee computed? **A:** Days from `period.end` to return × `LateFeePolicy` rate, as a `LATE_FEE` line.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.
- `tryBook` = atomic check-and-book per vehicle.
- One bill per reservation; late fee = days late × rate.
- Template Method (add-ons), Strategy + Decorator (pricing). Gaps: branch transfers, buffer between rentals.

Real-world example. Hertz or Enterprise sell a *car class* ("intermediate SUV or similar"), counting availability per type per day and binding a specific vehicle at the counter — the same design one level up.

Design a Restaurant Management System

Problem & clarifying questions

A **restaurant management system** runs tables, reservations, orders, the kitchen, and the bill.

Question	Assumed answer
One restaurant or chain?	Several branches, each with its own menu, tables, staff.
Menu?	Menu → sections → items with prices.
Orders?	Waiter creates an order per table and adds meals **per seat**; meal items point at menu items.
Reservations / walk-ins?	Search tables by time and party size and reserve; cancel; reminders; show tables free now.
Layouts?	Several seating arrangements per branch ("Weekday", "Banquet").
Payment?	Card, cheque or cash.
Kitchen?	Orders go to a chef queue, cooked in order; waiter told when ready.

Requirements

Functional: (1) several branches; (2) menu per branch; (3) sections with items; (4) waiter creates an order per table, meals per seat; (5) meal items ↔ menu items; (6) free tables for walk-ins; (7) table reservations; (8) search by date/time and reserve; (9) cancel; (10) reminder notification; (11) pay by card, cheque or cash; (12) several seating arrangements; (13, added) kitchen queue, waiter notified when ready.

Non-functional: no two parties on one table for overlapping time; prices live in one place (the menu item); easy to add a payment type or chef-assignment rule.

Use cases

Actors: **Receptionist** (tables, layouts, reservations, check-in), **Waiter** (orders), **Manager** (staff, menu), **Chef** (cooks), **Cashier** (check and payment), **System** (reminders). Top use cases: add/modify tables, search tables, place/update order, create/cancel reservation, check in, pay. Each role has a small separate set — each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns branches; closing it deactivates every employee account.
`Branch`	Owns menu, kitchen, seating charts, reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	Menu tree; **price only on `MenuItem`**.
`Table`, `TableSeat`	Capacity and seats; **current status** and **held time slots**.
`TableChart`	Named arrangement; refers to tables (a table can be in several charts).
`Reservation`	Customer, party size, time, table, status; `updatePeopleCount` refuses more than the table seats.
`ReservationService`	Best-fit search, atomic reserve, cancel, check-in, reminders, walk-ins.
`Order`, `Meal`, `MealItem`	One order per visit; one meal per seat; item × quantity; status machine + listeners.
`Kitchen`, `Chef`	Kitchen picks a chef by a pluggable rule; each chef has a FIFO queue.
`Employee` + roles	`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`.
`Check`	Subtotal, tax, total — computed from the order, never stored.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three behaviours.

Class diagram

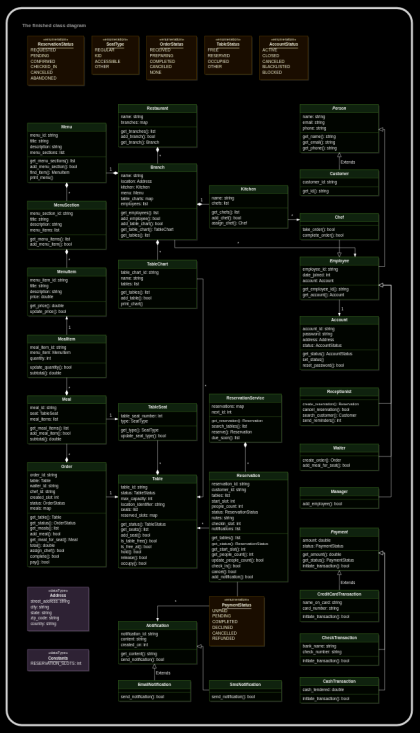
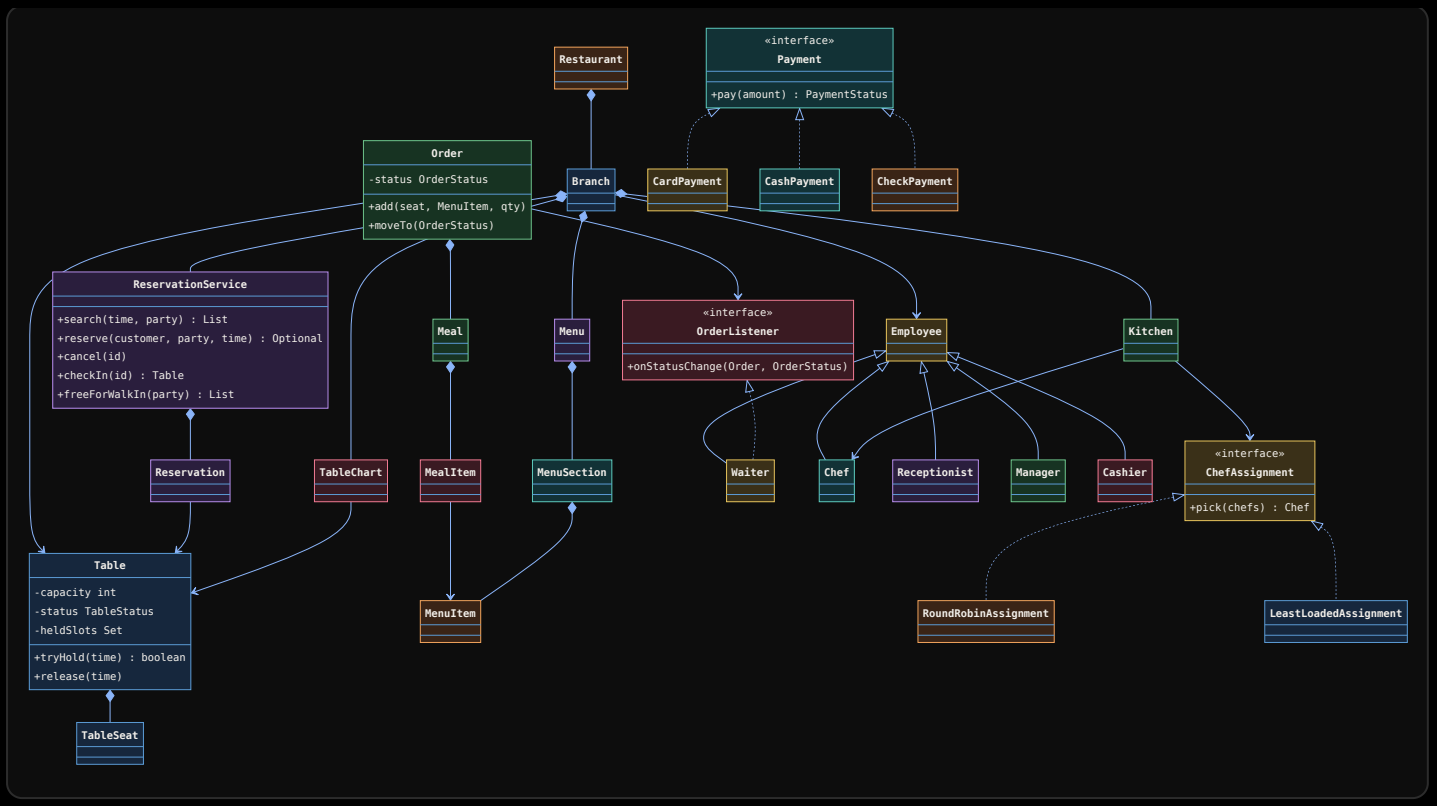


Figure — The finished class diagram of the restaurant management system (course diagram)

The branch is the hub: menu, kitchen and charts are composition; staff are associated (they can move branches). There is **no stored `Bill`** — the order already holds every priced line, so a stored total could disagree. Our `Check` is computed on demand.



Key flows

- **Place an order:** waiter opens an order for the table, adds a meal per seat, submits it to the kitchen.
- **Make a reservation:** search by time and party size → reserve and confirm, or "none available". **Cancel:** look up, cancel, release table, notify.
- **Order lifecycle:** `RECEIVED → PREPARING` (no more items) `→ READY` (waiter paged) `→ SERVED` (cashier settles); `RECEIVED → CANCELLED`.
- **Order to payment:** `Kitchen.submit` → `ChefAssignment.pick` → chef queue → `PREPARING` → `READY` → Observer pages the waiter → `Check.of(order)` → `payment.pay(total)` → `SERVED`, table `FREE`.

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Plate per guest; split bills are trivial.	Flat list of dishes.
Table keeps held slots as well as status	Status = now (walk-ins); slots = future bookings (a booking holds two hourly slots).	Status only.
Search on `ReservationService`, not `Table`	One table knows only itself (an old course version had an unworkable static `Table.search`).	Static search.
Best fit	Smallest table that fits keeps big tables for big parties.	First free.
Observer (`OrderListener`)	Waiter, screen, delivery app can all subscribe to "ready".	Chef calls waiter directly.
Strategy (`ChefAssignment`)	Course rotates chefs (fair, not load-aware); `LeastLoadedAssignment` is one class.	Hard-coded round-robin.
State machine in `Order.moveTo`	Illegal moves fail loudly.	Free `setStatus`.
`sealed` `Payment` with three classes	Card completes, cash may be short/gives change, cheque is pending.	One class + `if`s.
Check computed; `Customer` not an `Employee`	No second total; only staff have accounts.	Stored bill; one `Person` with a role flag.

Java implementation

Package `lld.ch16.restaurant`. Full runnable code: code/lld/ch16/restaurant/ (see the Full Edition).

```java
// Tables.java — Atomic check-then-hold of two hourly slots
synchronized boolean isFreeAt(LocalDateTime start) { return slots(start).stream().noneMatch(heldSlots::contains);
}

synchronized boolean tryHold(LocalDateTime start) {
    if (!isFreeAt(start)) return false;
    heldSlots.addAll(slots(start));
    return true;
}

// Reservations.java — Best fit first
List<Table> search(LocalDateTime time, int partySize) {
    return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
            .sorted(Comparator.comparingInt(Table::capacity)).toList();
}

Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
    for (Table t : search(time, partySize)) {
        if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
            var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
            reservations.put(r.id(), r);
            notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
            return Optional.of(r);
        }
    }
    return Optional.empty();
}

// Orders.java — state machine + Observer
void moveTo(OrderStatus next) {
    synchronized (this) {
        boolean ok = switch (status) {
            case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
            case PREPARING -> next == OrderStatus.READY;
            case READY -> next == OrderStatus.SERVED;
            case SERVED, CANCELLED -> false;
        };
        if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not allowed");
        status = next;
    }
    listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
}

// Kitchen.java — Strategy: which chef gets the next order
interface ChefAssignment { Chef pick(List<Chef> chefs); }

final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return
chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow(); }
}

// Staff.java — Cashier
PaymentStatus settle(Check check, Payment payment) {
    // Check the order state BEFORE taking money: a refused state change after payment would leave the guest
charged.
    if (check.order().status() != OrderStatus.READY)
        throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ", not
READY");
    var status = payment.pay(check.total());
    if (status != PaymentStatus.DECLINED) {
        check.order().moveTo(OrderStatus.SERVED);
        check.order().table().setStatus(TableStatus.FREE);
    }
    return status;
}
```

Output excerpt:

```
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   subtotal 43.00, tax 3.44, total 46.44
   O2 total 25.92 paid by check: PENDING
```

Concurrency & edge cases

- **Two receptionists, one table:** search is advisory; `tryHold` checks and holds both slots atomically; on failure try the next best fit.
- **Notify outside the lock:** calling unknown listener code while locked risks deadlock and long waits.
- **Reminder spam:** `markReminded` sends once per reservation.
- **Party larger than any table:** "no single table fits" (combining tables is an extension). **No-shows:** `NO_SHOW` status; a job frees tables 15 min after start.
- **Party grows** beyond the table → rejected. **Pay before ready** → `settle` checks `READY` before taking money. Unknown id → clear `IllegalArgumentException`; bad seat number rejected.
- **Cash short** → `DECLINED`, order stays `READY`. **Cheque** → `PENDING`.

Extensions & follow-up questions

- **Large party, two tables?** Reservation holds `List<Table>`; lock tables in a fixed order (by id) to avoid deadlock.
- **Split bill?** Already easy — per-seat `Meal.subtotal()`. **Tips?** Extra `Check` lines. **Walk-in waitlist?** FIFO per party size.
- **Kitchen stock?** Ingredient `Inventory`, recipes, mark items "86'd". **Real kitchen threads?** Each chef takes from a `BlockingQueue<Order>`. **Shifts?** A `Shift` entity; only on-shift chefs are picked.

In the interview

> **30-second answer:** "Branch owns a menu tree, kitchen, charts and reservation service. Tables keep status for walk-ins and held slots for bookings; the service searches best-fit and holds atomically. Orders have a meal per seat, a strict state machine and an observer for 'ready'. Chefs are assigned by a strategy. The check is computed; payment is card, cash or cheque strategies."

Plan: 0–5 clarify; 5–10 six actors; 10–20 diagram ("meal per seat", "status vs slots"); 20–35 code `tryHold`, `reserve`, `Order`, `Kitchen.submit`; 35–40 races, large parties, load balancing.

Self-check

1. **Q:** Why both status and held slots? **A:** Status = now (walk-ins); slots = future times (reservations).
2. **Q:** Why is search on the service? **A:** One table only knows itself.
3. **Q:** Which pattern pages the waiter, and why notify outside the lock? **A:** Observer; foreign code under a lock risks deadlock.
4. **Q:** Why is a cheque `PENDING`? **A:** It must clear at the bank.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); best-fit, atomic hold.
- Order → meal per seat; state machine; Observer for "ready"; Strategy for chefs.
- Check is computed; card/cash/cheque strategies.

Real-world example. POS systems like Toast send orders as tickets to kitchen display screens in order (FIFO + Observer); OpenTable models inventory by time slot and party size, like `heldSlots`.

Design an Airline Management System

Problem & clarifying questions

An **airline management system** schedules flights, reserves tickets, cancels flights, and manages staff and crew.

Question	Assumed answer
Search by?	Date, source and destination airport.
Trips?	An **itinerary** of several flights, several passengers (no accounts needed).
Seat classes/types?	Economy, economy plus, preferred economy, business, first; regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft/flights/schedules and cancels flights; everyone affected is notified.
Crew?	Pilots and crew assigned to flights, can view their schedule.
Payment / notifications?	Paid per itinerary; notify on booking changes and flight status changes.

Requirements

Functional: (1) search by date, source, destination; (2) reserve tickets, build itineraries; (3) view schedules and available seats; (4) several passengers per itinerary; (5) only the admin adds aircraft, flights, schedules and cancels flights (all notified); (6) customers cancel a reservation or itinerary; (7) assign pilots and crew; (8) take payment; (9) notify on booking create/modify and flight status change.

Non-functional: a seat on an instance is sold at most once; a multi-passenger booking gets all seats or none (**atomic**); fare rules change without touching booking code.

Use cases

Actors: **Admin**, **Front desk officer** (books for walk-ins), **Customer**, **Pilot/Crew** (view assignments), **System** (notifications). Top use cases: search; create/modify/view reservation; assign seats; pay; update schedule; assign crew. The officer's booking is the same operation as the customer's, triggered by another actor.

Core entities

The key is **four "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat	`Seat`	12A, extra legroom, economy plus
That seat on that instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147

Others: `Airport`; `Aircraft` (registration, model, seats); `WeeklySchedule` ("every Tuesday 10:00"); `Passenger` (a name on a ticket — **not** an account); `Customer` (books and pays); `Admin`, `FrontDeskOfficer` (admin-only methods need an `Admin` argument); `CrewMember`; `FlightReservation` (one leg: instance, passenger → seat, amount, status); `Itinerary` (legs, connection rule, the unit paid); `FareStrategy`; `Airline` (Facade, keeps the index instance → reservations).

Class diagram

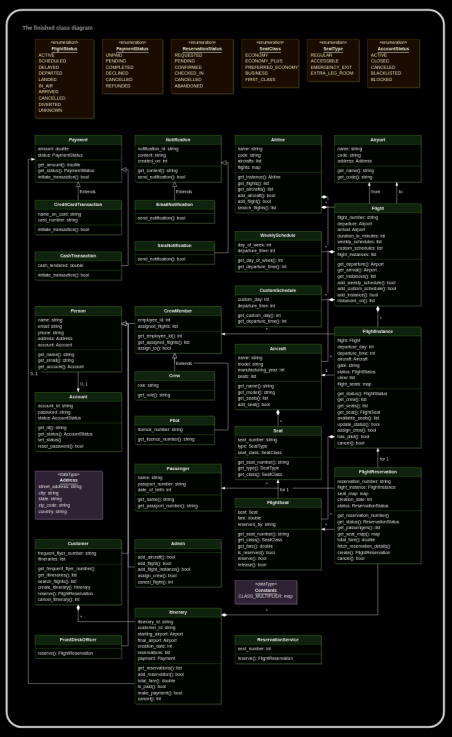
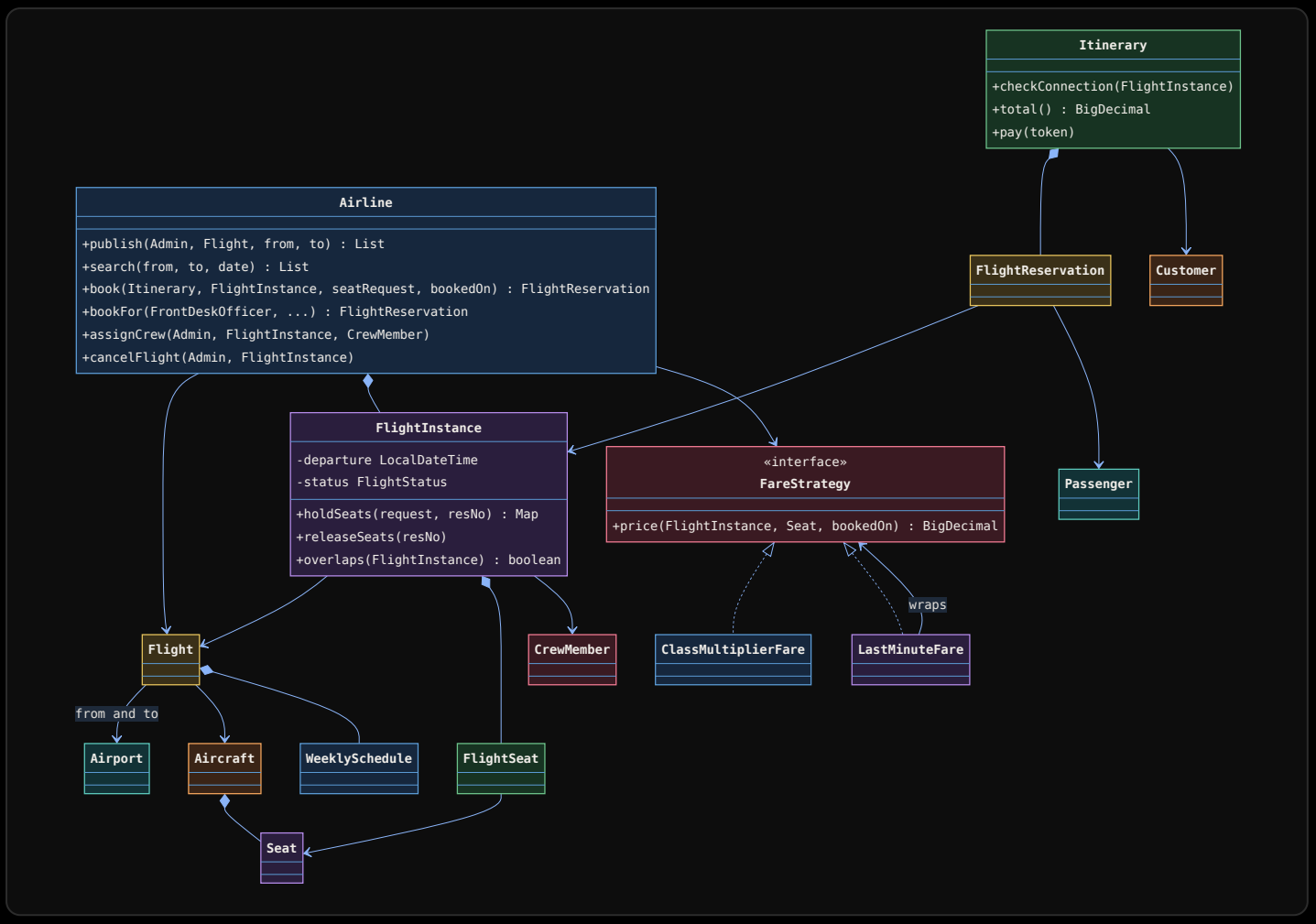


Figure — The finished class diagram of the airline management system (course diagram)

Ownership runs one way: schedules and instances belong to their flight, flight seats to their instance; airports, aircraft and crew are only referenced. `Passenger` sits outside the account tree — a passenger may never log in.



Key flows

- **Reserve:** search → choose seats and passengers → pay → confirmation. In code: `checkConnection` (same airport, ≥ 45 min after the previous leg lands) → `holdSeats` all-or-nothing → price each seat → add leg → index instance → reservation → notify.
- **Cancel reservation:** validate → cancel → release seats → notify.
- **Admin cancels a flight:** under the instance lock set `CANCELLED` and copy the index → cancel each reservation (releases seats) → notify each customer with the refund → notify every crew member.

Design decisions & patterns used

Decision	Why	Alternative rejected
Flight / FlightInstance / Seat / FlightSeat	12A is free Tuesday, taken Thursday: booking and fare belong to the seat *on an instance*.	`booked` flag on `Seat`.
`holdSeats` all-or-nothing, `synchronized` per instance	A family gets all seats or none; flights sell in parallel.	One-by-one holds; a global lock.
`Passenger` not an account	Customers book for others.	`Passenger extends Person`.
Itinerary is paid; **connection rule on `Itinerary`**	Course gap: no connecting-time check.	Pay per leg; no check.
Index instance → reservations	Course gap: cancelling notified crew but not passengers.	Scanning every itinerary.
Strategy + Decorator fares	Multipliers 1.0/1.3/1.5/2.5/4.0; `LastMinuteFare` +25% within 3 days.	Price on the seat.
Crew overlap check	No pilot on overlapping instances.	Trust the admin.
Facade `Airline`; admin ops take `Admin`	Compiler enforces "only the admin"; `bookFor` = `book` by another actor.	Rules in actors.
Course Singleton → construct and pass in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.airline`. Full runnable code: code/lld/ch16/airline/ (see the Full Edition).

```java
// FlightInstance.java — All-or-nothing: a family of three either gets all three seats or none.
synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
    if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
        throw new IllegalStateException(key() + " is " + status);
    for (var seatNo : request.values()) {
        var fs = seats.get(seatNo);
        if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
        if (!fs.isFree()) return Map.of();
    }
    if (request.values().stream().distinct().count() != request.size())
        throw new IllegalArgumentException("two passengers asked for the same seat");
    Map<Passenger, Seat> out = new LinkedHashMap<>();
    request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat()); });
    return out;
}

// Booking.java — Decorator on any fare: +25% when booked less than 3 days before departure.
public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
    var p = inner.price(fi, seat, bookedOn);
    long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
    return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
}

// Booking.java — Itinerary: the next leg must leave from where the last one landed, with time to change.
synchronized void checkConnection(FlightInstance next) {
    if (legs.isEmpty()) return;
    var last = legs.getLast().instance();
    if (!last.flight().to().equals(next.flight().from()))
        throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
    if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
        throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " + last.key()
+ " lands + 45 min");
}

// Airline.java
FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate bookedOn) {
    it.checkConnection(fi);
    String resNo = "PNR" + seq.incrementAndGet();
    FlightReservation r;
    synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
        var seatMap = fi.holdSeats(seatRequest, resNo);
        if (seatMap.isEmpty()) return null;
        var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
        r = new FlightReservation(resNo, fi, seatMap, amount);
        owner.put(r, it);
        byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
    }
    it.addLeg(r);
    // ... notify customer, return r
}

void cancelFlight(Admin by, FlightInstance fi) {
    List<FlightReservation> affected;
    synchronized (fi) {        // same lock as the hold+index step in book(): no booking can slip past the cancel
        fi.setStatus(FlightStatus.CANCELLED);
        affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
    }
    for (var r : affected) {
        if (r.cancel()) {
            // ... notify the itinerary's customer with the refund
        }
    }
    fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
}
```

Output excerpt:

```
  itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
  rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
  successful bookings of 10C: 1, free seats left: 3
  economy-plus fare 500 x 1.3 = 650.00, charged 812.50
  rejected: Capt. Kim already flies GA212@2026-10-08 at that time
   [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary IT-101
now totals 800.00
```

20 threads race for seat 10C and exactly one wins; 650 + 25% last-minute = 812.50; the cancellation reaches every passenger through the index.

Concurrency & edge cases

- **Selling one seat twice:** `holdSeats` checks every seat, then assigns all, under the instance lock.
- **Booking while the admin cancels:** hold + index and set-`CANCELLED` + copy use the same `synchronized (fi)` — every reservation is either in the copy (cancelled and notified) or refused.
- **Double payment:** `Itinerary.pay` is idempotent. **Idempotent cancel:** `FlightReservation.cancel()` returns `false` if already cancelled — no double refund.
- **Duplicate seat in one request**, **booking a cancelled/departed flight**, **crew on overlapping instances** — all rejected.
- **Time zones:** real systems use `ZonedDateTime`/`Instant` per airport. **Holds during payment:** 10–15 min `heldUntil` on `FlightSeat` plus a sweeper.

Extensions & follow-up questions

- **Overbooking?** Per-class `overbookLimit`; assign physical seats at check-in.
- **Check-in/boarding passes, baggage, change fees, codeshares (`MarketingFlight`)** — small new entities.
- **Delays?** `DELAYED` + new departure; notify through the index; re-check connections.
- **Scale?** One row per seat with `UPDATE … WHERE reservation IS NULL` or optimistic locking; cache search.

In the interview

> **30-second answer:** "`Flight` is the route, `FlightInstance` one departure; `Seat` is the physical chair, `FlightSeat` that chair on that departure, where booking and fare live. Seats are held all-or-nothing per instance. An itinerary groups legs, checks connections and is paid. Fares are a strategy with a last-minute decorator. An instance → reservations index lets a cancelled flight notify every passenger."

Plan: 0–5 clarify; 5–10 actors; 10–20 the four-way distinction and diagram; 20–35 code `holdSeats`, `book`, `checkConnection`, `cancelFlight`; 35–40 holds, overbooking, time zones. Expected follow-up: "How do passengers learn of a cancellation?" — "Through the instance → reservations index; without it you only reach the crew."

Self-check

1. **Q:** Where does "booked" live? **A:** On `FlightSeat` — per instance, not per aircraft.
2. **Q:** All seats or none? **A:** `holdSeats` checks all, then assigns all, in one synchronized block.
3. **Q:** `LastMinuteFare`? **A:** +25% within 3 days; a Decorator over any `FareStrategy`.
4. **Q:** Two course gaps closed? **A:** Passenger notification on cancel (index) and connection-time check (`Itinerary.checkConnection`).

TL;DR

- Flight ≠ FlightInstance; Seat ≠ FlightSeat.
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.
- Fares: Strategy + Decorator; instance → reservations index for cancellations.

Real-world example. Airlines separate a flight number from a leg on a date and control inventory per leg and booking class; a PNR groups passengers and segments — like `FlightReservation` / `Itinerary`. On cancellation they find every PNR on the leg, which is why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets students sign up for classes each term — a harsh concurrency problem: at 09:00 thousands click "register" for a few popular sections.

Terms: a **course** is a catalogue entry (CS201, 4 credits); a **section** is one offering with instructor, time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats) — students register for **sections** (like Flight vs FlightInstance); a **prerequisite** must be completed first; a **waitlist** is a FIFO queue for a full section; a **registration window** is when a student may register (seniors often first).

Question	Assumed answer
Courses or sections?	Sections.
Prerequisites?	Must be completed.
Capacity?	Per section; when full, a bounded waitlist.
Promotion?	Automatic on drop, FIFO, after re-checking the timetable.
Limits?	Credit limit; no time clashes; no two sections of one course.
Windows?	Open by class year; close together.
Concurrency?	Never exceed capacity; no double registration from two tabs.

Requirements

Functional: (1) register for sections and view them; (2) course has code, name, credits; section has instructor, time slot, limit; (3) search by code or name; (4) never exceed capacity, else waitlist (if room); (5) enforce prerequisites, credit limit, time conflicts, one section per course, window; (6) on drop, promote the waitlist head and notify.

Non-functional: safe concurrent registration; rules easy to add/remove ("majors only"); extensible (fees, holds, cross-listing).

Use cases

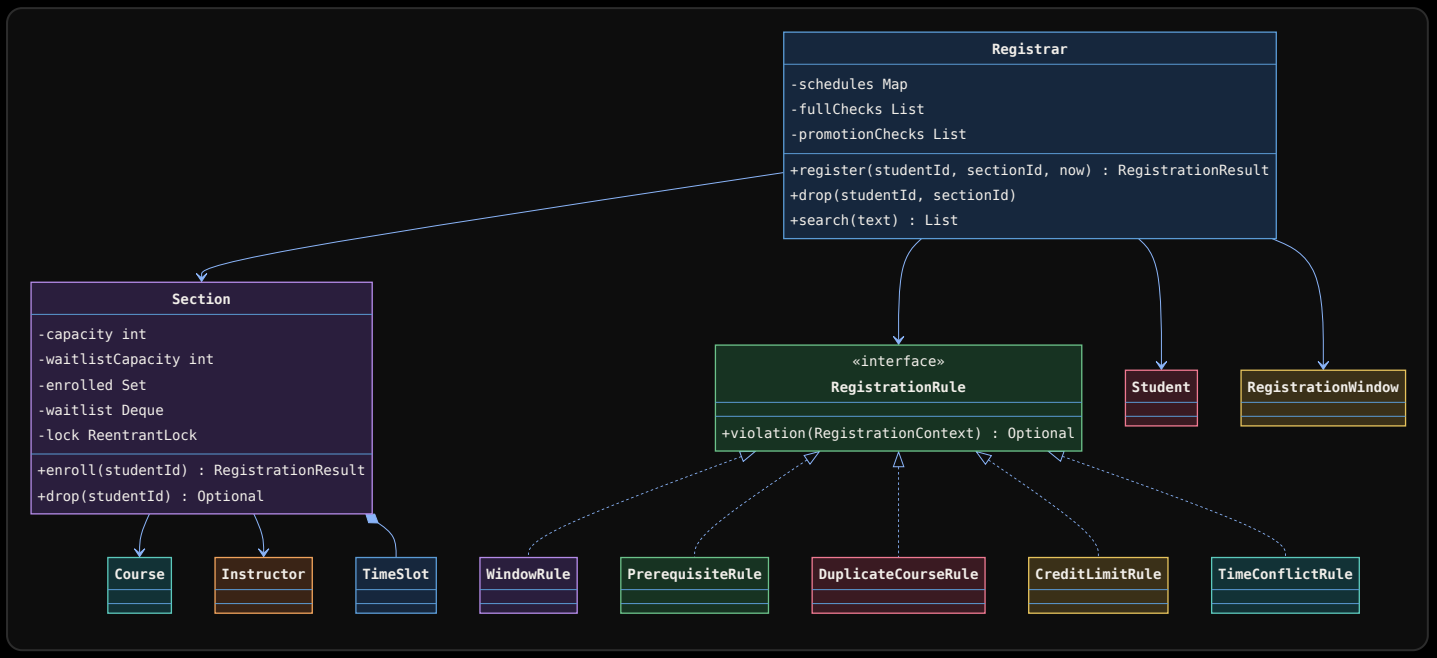
Student — search, register, drop, view schedule, join waitlist. **Registrar/Admin** — create courses/sections, set capacity and windows. **Instructor** — view roster. **System** — enforce rules, promote from waitlist, notify (a drop triggers promotion).

Core entities

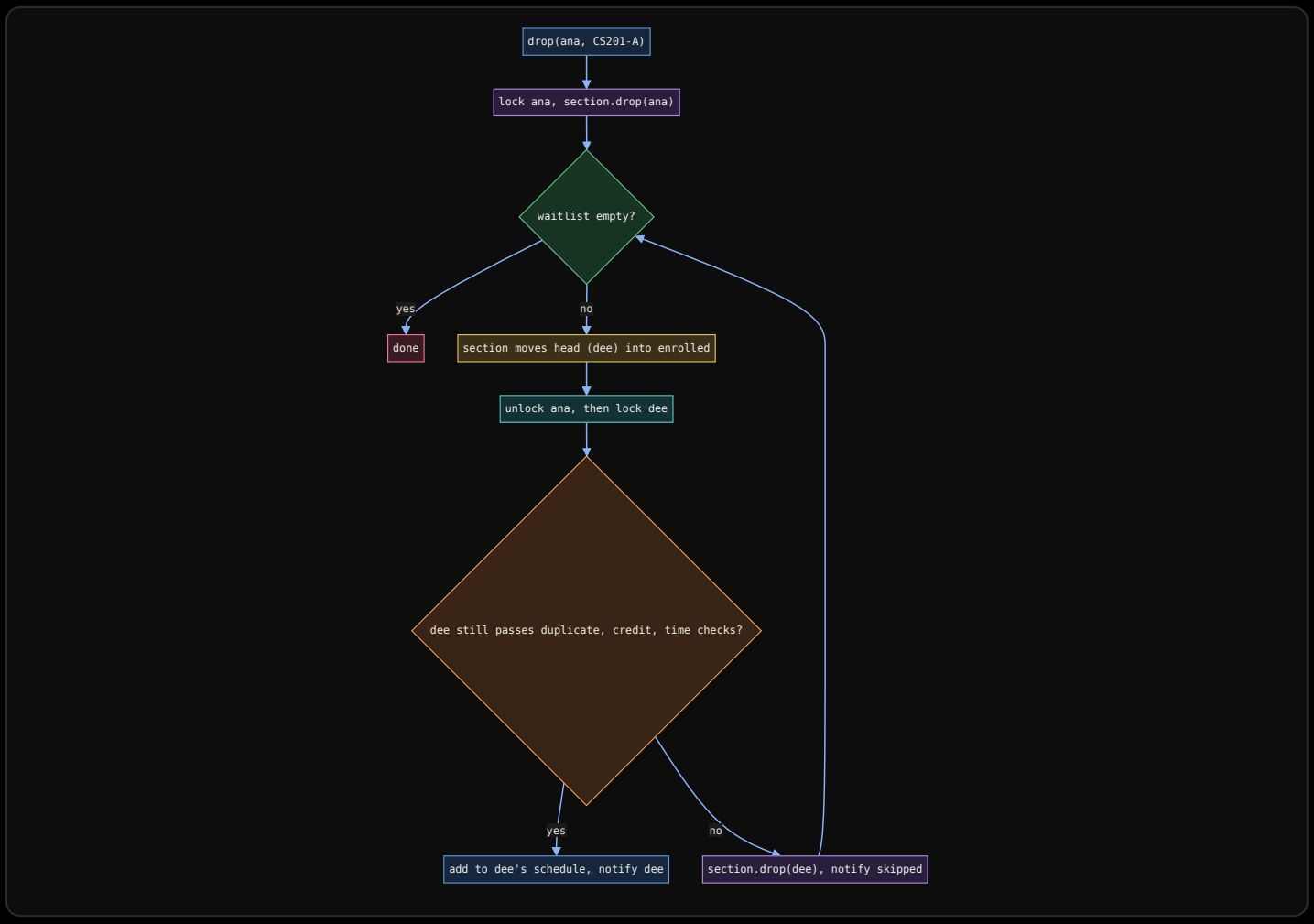
Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	Instructor, slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows overlap.
`Instructor`, `Student`	Who teaches; year, completed courses, credit limit.
`RegistrationWindow`	Opening per class year, common close.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns a violation reason.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED` / `WAITLISTED` / `REJECTED` + message.
`Registrar`	Runs the rule chain, takes the seat, drops and promotes, keeps schedules, defines the lock order.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set (register, view, search, capacity, concurrency, extensibility) — all covered, with instructor and capacity on `Section`.



Key flows



- **Register:** lock the student's schedule → run window, prerequisites, duplicate, credits, time conflict → first failure = `REJECTED` + reason; else `section.enroll` (section lock) → `ENROLLED` (add to schedule), `WAITLISTED at position n`, or `REJECTED` (both full).
- **Student status in a section:** `ENROLLED`, `WAITLISTED` (→ `ENROLLED` on promotion, or `SKIPPED` if the re-check fails), `REJECTED`; drop leaves.
- **Drop and promote:** as in the flowchart above.

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time, instructor belong to an offering.	Capacity on `Course`.
Seat taken only in `Section.enroll`, `ReentrantLock` per section	Atomic check-capacity-then-add; sections don't block each other.	`synchronized` registrar (the whole university in one line).
Per-student lock around rules + enroll	Credit/time checks read the schedule; two tabs would both pass without it.	Section lock only.
Lock order: student, then section	Deadlock needs opposite orders; promotion takes the next student's lock only *after* releasing the dropper's.	Promote while holding the dropper's lock.
Rules as objects (Specification / Chain of Responsibility)	A new rule = new class + list entry (Open/Closed); promotion reuses a subset.	One long `if` method.
Bounded FIFO `Deque` waitlist	Fair, bounded.	Unbounded list.
Re-check on promotion	Timetable may have changed while waiting.	Promote blindly.
`now` passed in	Testable windows.	`LocalDateTime.now()` in rules.
GitHub problem's **Singleton** → one `Registrar` passed in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`. Full runnable code: code/lld/ch16/courseregistration/ (see the Full Edition).

```
// Section.java — The only place a seat is taken: check-and-add in one critical section.
RegistrationResult enroll(String studentId) {
    lock.lock();
    try {
        if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
        if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " + id);
        if (enrolled.size() < capacity) {
            enrolled.add(studentId);
            return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/" +
capacity + ")");
        }
        if (waitlist.size() < waitlistCapacity) {
            waitlist.addLast(studentId);
            return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
        }
        return RegistrationResult.rejected(id + " and its waitlist are full");
    } finally {
        lock.unlock();
    }
}

// Rules.java
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }

final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

// Registrar.java — lock order is ALWAYS student-then-section
RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
    // ... look up student, section (reject if unknown)
    var schedule = schedules.get(studentId);
    synchronized (schedule) {                                   // 1st lock: the student
        var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
        var v = firstViolation(fullChecks, ctx);
        if (v.isPresent()) return RegistrationResult.rejected(v.get());
        var result = section.enroll(studentId);                 // 2nd lock: the section
        if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
        return result;
    }
}

void drop(String studentId, String sectionId) {
    // ...
    synchronized (schedule) {
        promoted = section.drop(studentId);
        schedule.remove(section);
    }
    notifier.accept(studentId + " dropped " + sectionId);
    // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
    while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
}
```

`finishPromotion` locks the promoted student, re-runs `promotionChecks` (duplicate, credit, time), and either adds the section to their schedule or drops them and returns the next promotion.

Output excerpt:

```
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
    [notify] dee promoted from waitlist into CS201-A
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

The 09:00 rush counts are exact every run; the double-click succeeds exactly once.

Concurrency & edge cases

- **Overselling** — section lock around "size < capacity → add". **Double-click / two tabs** — per-student lock; the second request hits `DuplicateCourseRule` or "already enrolled".
- **Deadlock** — fixed order student → section; never two student locks at once.
- **Waitlisted in one section, enrolled in another of the same course** — caught by the promotion re-check, which skips to the next student.
- **Window closes while waitlisted** — promotion doesn't re-check the window; add `WindowRule` to `promotionChecks` if that is the policy.
- Dropping a section you're not in → `IllegalArgumentException`; windows are half-open (`opens ≤ now < closes`).
- **Beyond one JVM:** `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` (atomic conditional update) or optimistic locking; a queue in front of hot sections.

Extensions & follow-up questions

- **Co-requisites / "prereq OR permission"?** Composite expression tree (`And`, `Or`, `CourseCompleted`, `Permission`) in `PrerequisiteRule`.
- **Holds?** A `HoldRule`. **Priority waitlist?** `PriorityQueue` by (year, time joined). **Major seat buckets?** Split capacity in `Section.enroll`.
- **Lecture + lab?** Lock both sections by id order, check both, enrol both. **Notifications/fees?** Swap the `Consumer<String>` notifier; a `Bill` per student per term.

In the interview

> **30-second answer:** "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist and takes seats only inside its own lock. The registrar runs a chain of rule objects under the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the waitlist head is promoted after a re-check."

Plan: 0–5 clarify; 5–12 entities and diagram; 12–30 code `Section.enroll/drop`, the rule interface and two rules, `register`; 30–40 concurrency, lock order, DB conditional update.

Self-check

1. **Q:** Which two locks does `register` take, in what order? **A:** Student's schedule, then the section — always.
2. **Q:** Why not `synchronized` on the whole registrar? **A:** Correct but serialises the whole university.
3. **Q:** Why re-check on promotion? **A:** The timetable may have changed while waiting.
4. **Q:** Capacity with many servers? **A:** Atomic conditional update (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (capacity, slot, waitlist).
- Rules are small objects run as a chain.
- Seats taken only in `Section.enroll`; student lock first, section lock second.

- Drop → promote head → re-check → notify. At scale: DB conditional update.

Real-world example. Student-information systems such as Ellucian Banner run priority windows by class standing, check prerequisites and clashes, and keep section waitlists; registration-day slowdowns are the 09:00 "thundering herd" this design handles.

Design a Meeting Scheduler

Problem & clarifying questions

A **meeting scheduler** books meeting rooms: the organizer picks a time and invitees, the system finds a room that fits and is free, sends invitations and collects answers. Two twists: the calendar is **continuous time** (10:15–10:40), not slots; and **people** are booked too, but a person in two meetings is only a warning, while a room in two meetings is impossible.

Terms: **organizer** / **invitee**; **RSVP** = an invitee's answer (accepted, tentative, declined; **pending** before answering); a **half-open interval** `[start, end)` includes start, excludes end, so 10:00–11:00 and 11:00–12:00 do not overlap — two intervals overlap iff `a.start < b.end && b.start < a.end`; a **hard conflict** blocks (room taken), a **soft conflict** warns (person busy); a **recurring** meeting is a **series** of **occurrences**.

Question	Assumed answer
Who picks the room?	The organizer names one, or the system picks one that fits.
Fits?	Capacity ≥ organizer + invitees, and has the required equipment.
Invitee double-booked?	Allowed, reported as a warning.
Reschedule / cancel?	Organizer only; reschedule resets every RSVP to pending.
Notifications?	Invite, RSVP (to organizer), reschedule, cancel; channel pluggable.
Recurring?	Extension: daily/weekly × N, all in one room or not at all.
Concurrency?	A room must never hold two overlapping meetings.

Requirements

Functional: (1) add rooms (capacity, equipment); (2) book a meeting, in a named room or any fitting room; (3) reject room overlaps, allow back-to-back; (4) warn on busy people; (5) invite, RSVP, notify the organizer; (6) reschedule and cancel with notifications; (7) agenda and free-room search; (8) extension: recurring series.

Non-functional: no double-booking under concurrency; different rooms never share a lock; new room-selection policy or notification channel = new class.

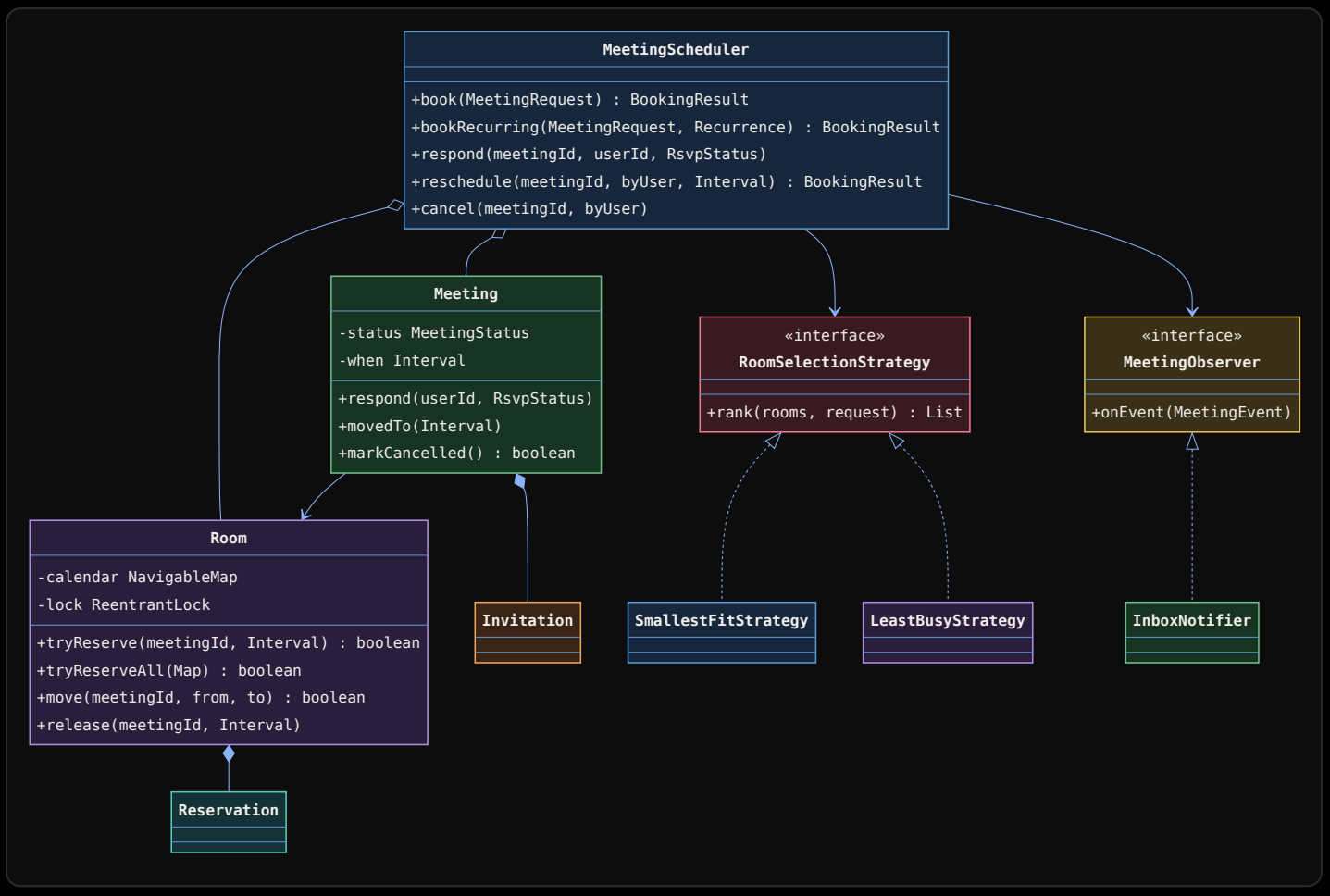
Use cases

Organizer — book (includes *pick room and detect conflicts* and *notify*; extended by *book recurring*), reschedule, cancel, search rooms. **Invitee** — RSVP, view agenda. **Facilities admin** — add rooms. Picking and notifying are included use cases, not a "System" actor.

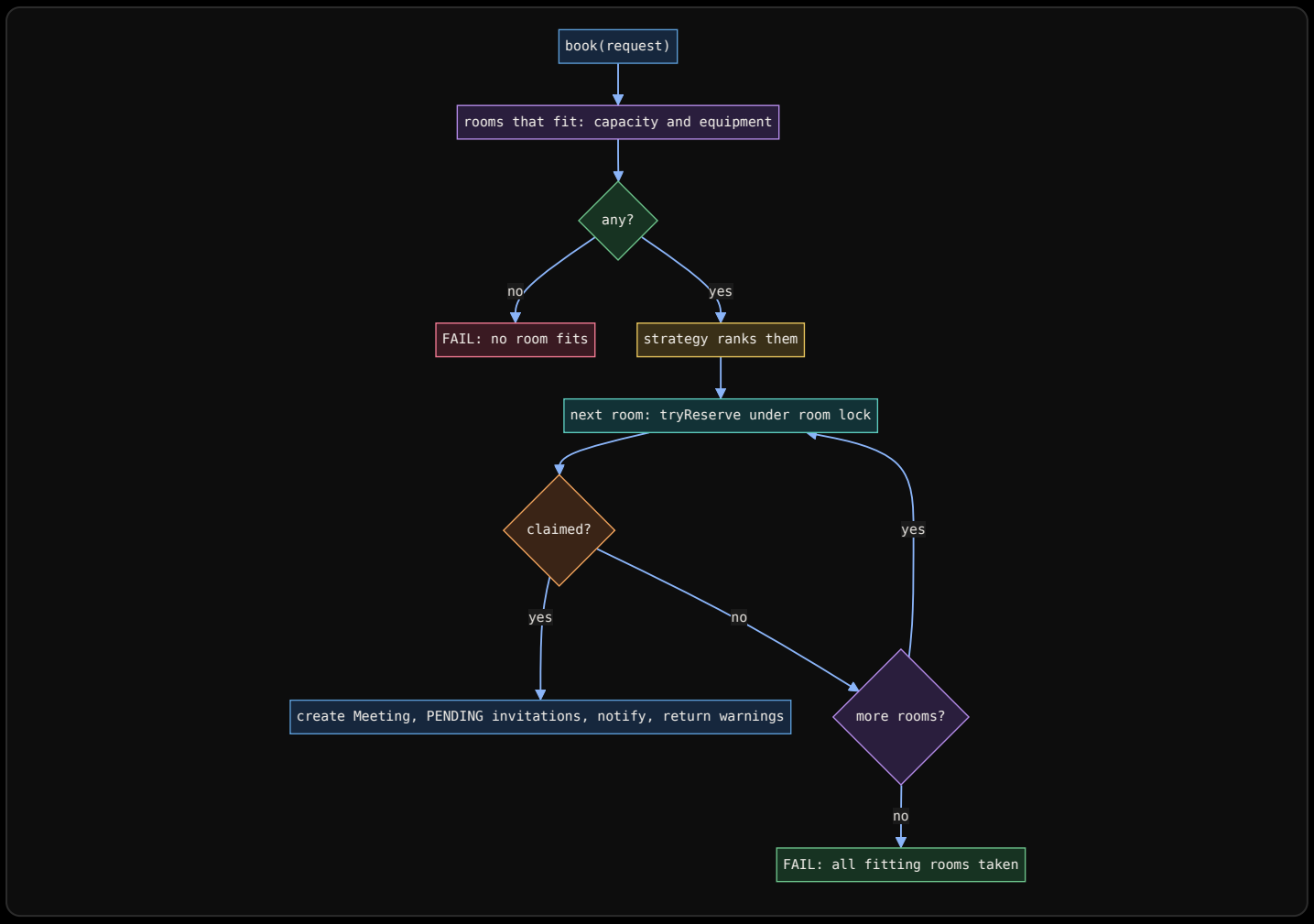
Core entities

Class	Responsibility
`Interval`	Half-open `[start, end)`; `overlaps`.
`Room`	Capacity, equipment, **its own calendar**. The only place a time range is claimed, under the room's lock.
`Meeting`, `Invitation`	Organizer, room, interval, `SCHEDULED` / `CANCELLED`; one `RsvpStatus` per invitee.
`RoomSelectionStrategy` (`SmallestFitStrategy`, `LeastBusyStrategy`)	Orders the rooms that fit.
`MeetingObserver` (`InboxNotifier`, `EventCounter`)	Receives invited / RSVP / rescheduled / cancelled events.
`Recurrence`	Extension: frequency × count → occurrences.
`MeetingScheduler`	Book, book series, RSVP, reschedule, cancel, agenda, soft-conflict warnings; lock order.

Class diagram



Key flows



- **Reschedule:** lock meeting → organizer and still scheduled? → `room.move(old, new)` under the room lock (old slot restored if new is taken) → new time, all RSVPs `PENDING` → unlock → notify.
- **Cancel:** lock meeting → organizer? → `markCancelled` (second cancel is a no-op) → `room.release` → unlock → notify.
- **Invitation states:** `PENDING` → `ACCEPTED` / `TENTATIVE` / `DECLINED` (changeable); any → `PENDING` on reschedule; all end when the meeting is cancelled.

Design decisions & patterns used

Notifications are **Observer** (§11.1) and room ranking is **Strategy** (§11.3), as in earlier cases. What is specific here:

Decision	Why	Alternative rejected
Room owns its calendar and its lock	The no-overlap invariant is per room; rooms never block each other.	One global list under one lock.
`TreeMap` keyed by start; clash = `lowerEntry(end)`	Stored bookings are disjoint, so only the one with the latest start before the new end can overlap. O(log n).	Scanning a list.
Half-open intervals	Back-to-back is legal with no special case.	Closed intervals.
Rank, then *try* rooms in order	The ranking is a stale snapshot; each room's atomic claim decides.	"Find a free room, then book it" (check-then-act race).
Room clash blocks, people clash warns	People double-book and choose by RSVP; declined meetings don't count as busy.	Blocking on a busy invitee.
Reschedule = atomic `Room.move`, RSVPs reset	Slot never lost; old answers were for the old time.	Cancel + re-book (slot can be stolen; confusing notices).
Notify after unlocking	Slow e-mail must not block bookings.	Publishing inside the lock.

Java implementation

Package `lld.ch16.meetingscheduler`. Full runnable code: code/lld/ch16/meetingscheduler/ (see the Full Edition).

```java
// Room.java — the only place a time range is claimed
private Optional<Reservation> clash(Interval iv) {
    var e = calendar.lowerEntry(iv.end());           // latest booking starting before iv.end
    return e != null && e.getValue().interval().end().isAfter(iv.start()) ? Optional.of(e.getValue()) :
Optional.empty();
}

boolean tryReserve(String meetingId, Interval iv) {
    lock.lock();
    try {
        if (clash(iv).isPresent()) return false;
        calendar.put(iv.start(), new Reservation(meetingId, iv));
        return true;
    } finally {
        lock.unlock();
    }
}
// tryReserveAll: check every occurrence, then insert every one, under one lock (recurring series).
// move: remove old, check new, insert new or restore old, under one lock.

// MeetingScheduler.java
BookingResult book(MeetingRequest r) {
    var rooms = candidates(r);   // the named room if it fits, else strategy.rank(every room that fits)
    if (rooms.isEmpty()) return BookingResult.failure("no room fits " + r.headcount() + " people with " +
r.needs());
    String id = "M" + ids.incrementAndGet();
    for (var room : rooms) {
        if (!room.tryReserve(id, r.when())) continue;          // lost this room: try the next one
        var m = new Meeting(id, null, r.title(), r.organizer(), room, r.when(), r.invitees());
        meetings.put(id, m);
        for (var u : r.invitees()) publish(EventType.INVITED, m, u, "");
        return BookingResult.success(List.of(m), "booked " + m, busyPeople(m));
    }
    // ... failure message
}

BookingResult reschedule(String meetingId, String byUserId, Interval to) {
    // ...
    synchronized (m) {                                          // 1st lock: the meeting
        requireOrganizer(m, byUserId);
        if (!m.isScheduled()) return BookingResult.failure(meetingId + " is cancelled");
        from = m.when();
        if (!m.room().move(meetingId, from, to))                // 2nd lock: the room
            return BookingResult.failure(m.room().name() + " is not free at " + to);
        m.movedTo(to);                                          // new time, every RSVP back to PENDING
    }
    // notify invitees after unlocking
}
```

Output excerpt:

```
2) Conflicts in a named room (half-open intervals)
   FAIL R2 is taken by M1 Mon 11-02 10:00-11:00
   OK   booked M3 'Back-to-back' Mon 11-02 11:00-11:30 in Maple
   OK   booked M5 'Hiring sync' Mon 11-02 10:30-11:00 in Huddle warnings=[eve is busy in M1, dee is busy in M1]
   M1 [ben=PENDING, cy=PENDING, dee=PENDING, eve=PENDING]
   ben cancels -> ben is not the organizer of M1
   FAIL no single fitting room is free for all 3 occurrences
   OK   booked series S17 in Maple
9) Race: 20 organizers want Oak at Tue 10:00 at the same instant
   winners=1, Oak bookings at that time=1
10) Race: 20 organizers, any room, Wed 10:00 - 4 rooms exist, so exactly 4 win
   per room {Atrium=1, Huddle=1, Maple=1, Oak=1}, rejected=16
```

Overlap fails with the blocker named; back-to-back succeeds; a busy invitee only warns; reschedule resets RSVPs; a series that cannot fit in one room fails whole, and with automatic choice moves to the next room. Race counts are exact every run.

Concurrency & edge cases

Per-resource lock with check-and-claim inside, as for show seats (§15.2) and hotel rooms (§15.4). New here:

- **Overlap, not equal keys:** 10:00–11:00 and 10:30–11:15 have different start keys, so `putIfAbsent` can't catch them; the `lowerEntry` check and the insert must share one lock.
- **Auto-choice race:** all threads rank the same room first; losers fail the claim and move on.
- **Lock order meeting → room; never two rooms** → no deadlock. `book` needs only the room lock (the meeting isn't visible yet). Cancel vs reschedule and RSVP vs reschedule serialise on the meeting.
- **Series:** checked and inserted under one room lock; nothing slips in between.
- **Soft-conflict warnings** read without a global lock — advisory only.
- **Time zones:** store `Instant`, expand series in the organizer's zone.
- **Many servers:** PostgreSQL exclusion constraint `EXCLUDE USING gist (room_id WITH =, slot WITH &&)` (needs `btree_gist`), or `SELECT … FOR UPDATE` on the room row.

Extensions & follow-up questions

- **Common free time?** Merge everyone's busy intervals (sort, merge), walk the gaps in working hours.
- **Virtual meetings?** A `Location` interface: `Room` (has a calendar) or `VideoLink`.
- **Edit one occurrence?** Store rule + exceptions (iCalendar RFC 5545) instead of eager occurrences.
- **Reminders?** A scheduled job publishing `REMINDER` to the same observers.

In the interview

> **30-second answer:** "Each room owns a sorted calendar of half-open intervals; booking is check-then-insert under the room's lock, and one `lowerEntry(end)` lookup finds any clash. A strategy ranks fitting rooms and the scheduler tries to claim them in order. Room clashes block, people clashes warn. Reschedule is an atomic move that resets RSVPs; observers notify after unlocking; series are all-or-nothing in one room."

Plan: 0–5 clarify; 5–12 entities and diagram; 12–28 `Interval`, `Room.tryReserve`, `book`; 28–35 RSVP, reschedule, cancel; 35–45 concurrency, series, DB constraint, common free time.

Self-check

1. **Q:** Why does one `lowerEntry(end)` lookup find any clash? **A:** Bookings are disjoint and sorted; if the latest one starting before the new end ends by the new start, all earlier ones do.
2. **Q:** Why not `putIfAbsent`? **A:** Overlapping meetings have different keys; ranges need check + insert under one lock.
3. **Q:** Strategy said Huddle was free but Maple was booked — bug? **A:** No; another thread claimed Huddle first.
4. **Q:** Many servers? **A:** Database exclusion constraint or a row lock on the room.

TL;DR

> - Room owns a `TreeMap` calendar of half-open intervals; claim under the room lock; `lowerEntry(end)` finds clashes.
> - Strategy ranks, scheduler tries each room's atomic claim.
> - Room clash blocks; people clash warns.
> - Reschedule = atomic move + RSVP reset; cancel idempotent; notify after unlock. Lock order meeting → room.

Real-world example. Outlook/Exchange and Google Calendar treat rooms as resources with their own calendars that decline conflicting bookings, while people may be double-booked; recurring meetings are

a rule plus exceptions (iCalendar). For a rooms-free version of the same interval logic, see the Event Calendar in §21.3.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is about **dates**, not status	`Vehicle.tryBook(range)`	Template Method, Strategy + Decorator, state machine
Restaurant	**Meal per seat**; table = status + slots	`Table.tryHold(time)`	Observer, Strategy (chefs, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator, Facade, cancellation index
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` inside a student lock	Specification / Chain of rules, lock ordering
Meeting scheduler	**Room owns a calendar of half-open intervals**; room clash blocks, people clash warns	`Room.tryReserve(interval)`	Strategy, Observer, meeting → room lock order

Find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.

Chapter 17 — Case Studies: Games & Rules — Study Edition

> **Where this fits.** Chapters 14–16 modelled the real world. In games the "world" is a set of exact **rules** — every rule must live *somewhere*, and the interviewer will ask where. That is why games are a favourite interview topic.

What games test in an LLD interview

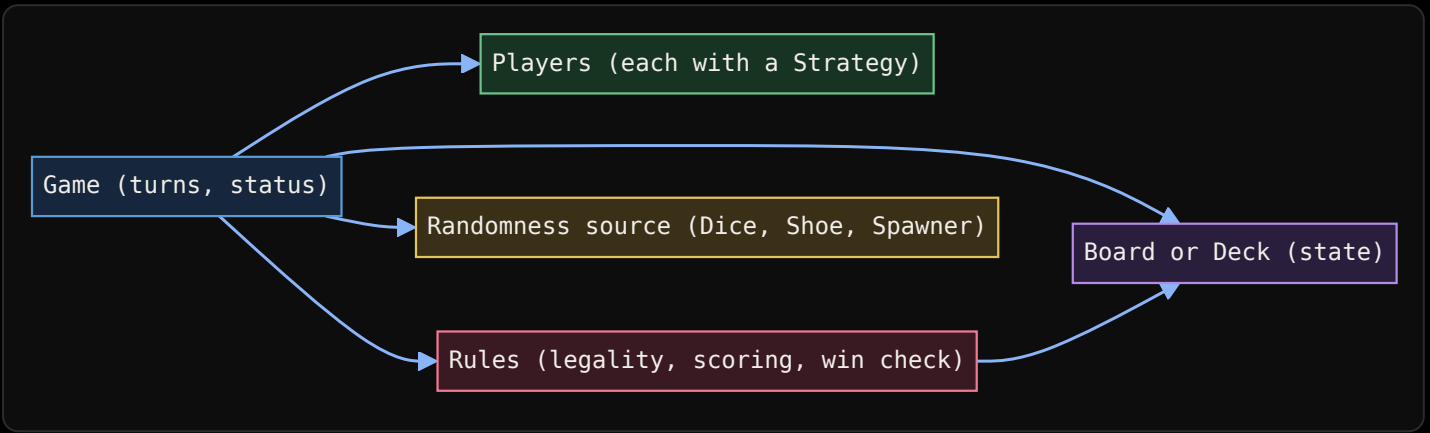
Skill	What it means	Where you see it
Rule encapsulation	Each rule lives in one obvious class.	`BlackjackScoring`; chess `Rules`; 2048 `LineMerger`.
Turn management	Whose turn, what may they do, what happens at the end of a turn.	Turn index (tic-tac-toe), player queue (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	One method call, many behaviours (*polymorphism*).	Chess `Piece.targets(...)`.
Pluggable strategies	Varying behaviour behind an interface (**Strategy**, Ch. 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing	Randomness is *injected*: `new Random(seed)` or a scripted source (stacked shoe, loaded dice).	Every `Main` prints the same output every run.
State and status	Games move through states; illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums.

> **Key idea: seeded randomness.** `java.util.Random` is *pseudo-random*: the same **seed** always gives the same sequence, and `Collections.shuffle(list, rnd)` is deterministic for a given `Random`. Rule: **never call `new Random()` or `Math.random()` inside the game — accept a `Random` (or a `Dice` / `Spawner`) from outside.** Production passes an unseeded (or `SecureRandom`) source; tests pass a seed. This is plain Dependency Injection (Ch. 5).

Problems: 17.1 Blackjack and a deck of cards · 17.2 Chess · 17.3 Tic-Tac-Toe · 17.4 Snake and Ladder · 17.5 2048 · 17.6 Jigsaw Puzzle.

How to run: `cd chapters/17-case-games/code && javac --release 21 -d /tmp/lld_17 $(find . -name '*.java') && java -cp /tmp/lld_17 lld.ch17.blackjack.Main` (or `chess`, `tictactoe`, `snakeladder`, `game2048`, `jigsaw`). All randomness is seeded or scripted, so output is exact.

A reusable skeleton for any board/card game — name these five parts early:



Part	Question it answers	Typical class names
State	What does the table look like now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? Which state?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

---

Design Blackjack and a Deck of Cards

Problem & clarifying questions

Blackjack: every player plays only against the **dealer** (the casino employee playing for the house); everyone tries to get close to **21** without going over (**busting** loses at once). Often asked as "design a deck of cards, then blackjack" — the first half checks the deck stays **generic**, the second checks rule placement.

Question	Assumed answer
Decks?	A **shoe** of several 52-card decks (casinos use 6–8); one deck = shoe of 1.
Players?	One or more, each against the dealer.
Options?	Hit, stand, double down, split (once), insurance. No surrender/side bets.
Dealer rule?	Hit ≤ 16, stand ≥ 17; "hit soft 17" pluggable.
Payouts?	Win 1:1, blackjack 3:2, push returns the bet, insurance 2:1.
Money?	Whole **chips** (`int`), even ante. Never `double`.
UI, accounts?	Out of scope (the course's `Account` / `Person` would attach to `Player`).

Requirements

Background rules: 13 ranks × 4 suits; two cards each; the dealer shows the **up card** and hides the **hole card**. J/Q/K = 10, **ace = 1 or 11**. **Soft** total = an ace counted as 11; **hard** = all aces as 1 (A+5 = soft 16 / hard 6). **Blackjack** (natural) = two cards totalling 21.

Functional:

1. Place an ante to play.
2. Deal two cards each (dealer: one up, one hole).
3. Dealer shows an **ace** → offer **insurance** (half the ante, pays **2:1** on dealer blackjack); the dealer then **peeks**; dealer blackjack ends the round (insurance wins, main bets lose, player blackjack pushes).
4. Below 21 the player chooses **hit**, **stand**, **double down** (double the bet, exactly one card, stop) or **split** (equal ranks only, second bet = ante, two hands, one card each, no re-split).
5. Over 21 → immediate loss.
6. Dealer plays by the fixed rule.
7. Dealer bust → every live hand wins; else higher total wins; equal = **push**. Blackjack pays **3:2**, other wins 1:1.
8. **Even money**: blackjack vs a dealer ace may take 1:1 at once (= insurance + blackjack; explained, not built).

Non-functional: generic `Card` / `Deck`; deterministic tests; pluggable dealer and player rules; whole-chip `int` money — deliberately not `BigDecimal`: an even ante keeps every payout whole (3:2 on 10 = 15). If fractions appear, use `BigDecimal` + `RoundingMode`.

Use cases

Player — bets; accepts/declines insurance, split, double, even money; hits or stands (log in / join game belong to a lobby service). **Dealer** — deals, plays by the fixed rule, collects and pays. Use cases: create hands, place bet, player plays, dealer plays, insurance / even money / examine hole card (ace up only), split, resolution; dealing and payout are `<<include>>`d in every round.

Core entities

Class	Responsibility
`Suit`, `Rank`	Fixed values; `Rank` stores *face value* 1–13 (ace 1, K 13).
`Card` (record)	Rank + suit; knows **nothing** about blackjack.
`Deck<T>`	Generic shuffleable stack; `standard52()` factory.
`Shoe`	Several decks; deals; reshuffles at the **cut card**; `Shoe.stacked(...)` for tests.
`BlackjackScoring`	Blackjack value of a rank (K = 10, ace = 1 hard) — kept off `Card`.
`Hand`	Cards + **its own bet**; hard/soft/best total, bust, blackjack, split, double.
`Action`, `PlayerStrategy`	HIT/STAND/DOUBLE/SPLIT; `BasicStrategy`, `MimicDealerStrategy` (a UI human would be a third).
`DealerPolicy`	`standOnAll17()`, `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	No class here: a `Hand` in `BlackjackGame` + the `DealerPolicy` (the course has `Dealer` beside `Player` under `BasePlayer`).
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

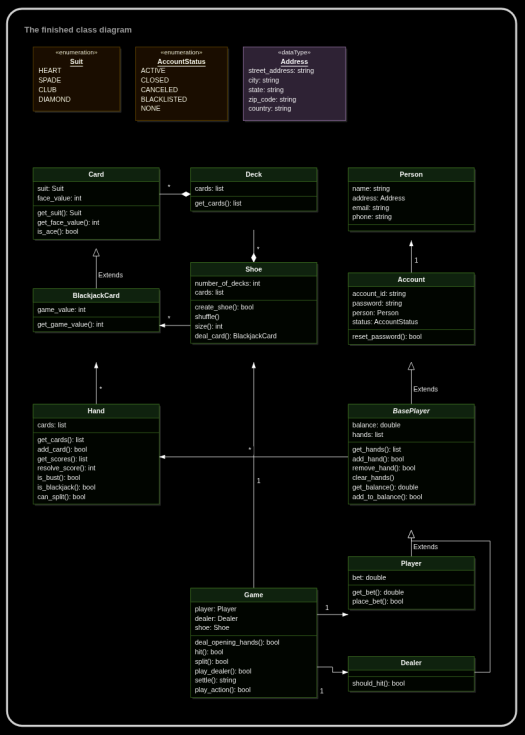
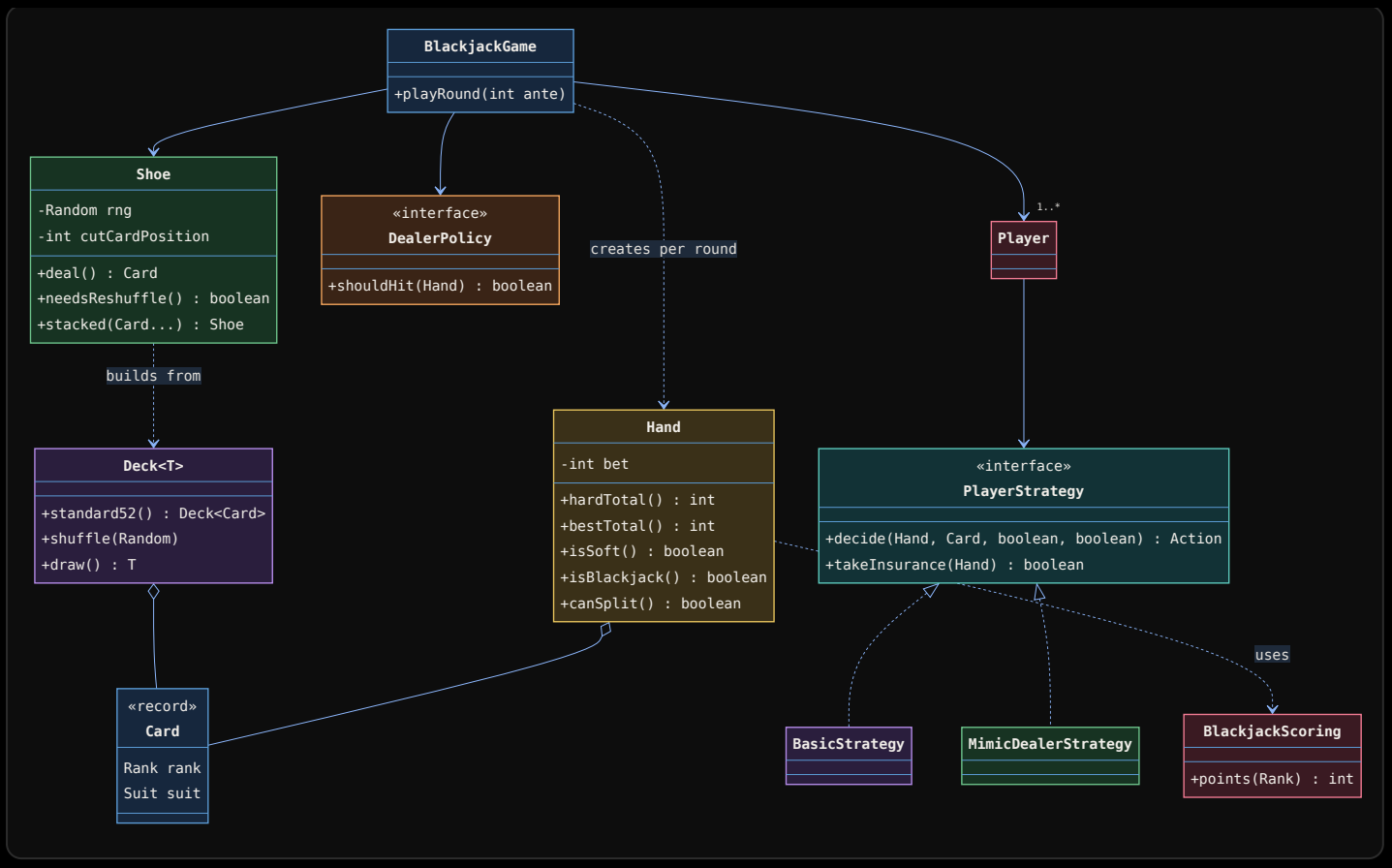


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

In the course, `BlackjackCard extends Card` adds the *game value* (K: face 13, worth 10); a `Shoe` is composed of `Deck`s; `Player` and `Dealer` share `BasePlayer` (balance, `*` hands because of splits). We keep the core idea — **the game value must not live on the generic card** — using a scoring function, and make player and dealer rules strategies:



Key flows

One round: debit antes → deal two each → if up card is an ace, offer insurance → if up card is ace or ten and dealer has blackjack, settle and stop → else each hand plays (`decide` until stand, bust, 21 or double) → dealer draws while `shouldHit` (skipped if all hands bust) → settle. States: `Betting → Dealing → (Insurance) → Peek → PlayerTurns → DealerTurn → Settled`.

Scoring a hand (the ace problem). At most one ace can ever count as 11 (two would be 22). So: (1) sum all cards with aces as 1 = **hard total**; (2) if there is an ace and `hard + 10 ≤ 21`, the hand is **soft** and best = `hard + 10`; (3) else best = hard. O(n), no search, and the soft flag comes free (needed for "hit soft 17").

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record**.	Poker, bridge, solitaire reuse it; K = 10 is a rule of this game.	`getValue()` returning 10 for K on `Card`.
Value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records can't be extended; one card type in the shoe.	The course's subclass — also valid; say both.
Generic `Deck<T>`.	Uno/tarot reuse shuffle/draw.	Deck hard-wired to `Card`.
Strategy for `PlayerStrategy`, `DealerPolicy`.	UI, bots, scripts plug in; S17 vs H17 is one line (Open/Closed).	`if (isHuman)`, `if (casino == X)`.
Factory methods (`standard52`, `stacked`, `standOnAll17`); injected `Random`.	Named constructors; exact tests.	Flag constructors; hidden randomness.
`hardTotal` / `bestTotal` / `isSoft`.	The course returns every possible total; hard + soft flag carries the same information.	List of totals to filter.
Hand stores its **own bet**.	Split and double change bets per hand.	One bet per player.
Game **validates** the strategy's choice.	A buggy client can't double without money.	Trusting the client.

Honest weakness. `playRound` owns the whole sequence *and* the payouts; each new option adds a branch. A larger design uses a **State** machine (one class per round state, like the Ch. 14 vending machine) and a `PayoutTable`. Say it out loud.

Java implementation

Package `lld.ch17.blackjack`. Full runnable code: code/lld/ch17/blackjack/ (see the Full Edition).

```java
/** A game-agnostic playing card: just a rank and a suit. Immutable, so it is safe to share. */
public record Card(Rank rank, Suit suit) {
    @Override public String toString() { return rank.symbol() + suit.letter(); }
}

// BlackjackScoring.java — Points with an ace counted as 1 (the "hard" value).
public static int points(Rank rank) {
    return switch (rank) {
        case ACE -> 1;
        case JACK, QUEEN, KING -> 10;
        default -> rank.faceValue();
    };
}

// Hand.java
public int hardTotal() {
    int sum = 0;
    for (Card c : cards) sum += BlackjackScoring.points(c.rank());
    return sum;
}
public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }
public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }
public boolean isBust() { return bestTotal() > 21; }
public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }

// DealerPolicy.java
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);

    /** Classic rule: draw on 16 or less, stand on every 17 ("S17"). */
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }

    /** Variant: also draw on a soft 17 such as A+6 ("H17"), which favours the house. */
    static DealerPolicy hitSoft17() { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
}

// BlackjackGame.java — validate the strategy's choice, then settle
Action a = p.strategy().decide(hand, up, canDouble, canSplit);
if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit)) {
    throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
}
// ...
private void settle(Player p, Hand h, Hand dealer) {
    String who = "  " + p.name() + " " + h + " vs dealer " + dealer.bestTotal() + ": ";
    if (h.isBust()) { log.accept(who + "LOSE " + h.bet()); return; }
    if (h.isBlackjack()) {                                  // dealer blackjack was handled by the peek
        p.credit(h.bet() + h.bet() * 3 / 2);
        log.accept(who + "BLACKJACK pays 3:2 (+" + h.bet() * 3 / 2 + ")");
        return;
    }
    int mine = h.bestTotal(), theirs = dealer.bestTotal();
    if (dealer.isBust() || mine > theirs) { p.credit(h.bet() * 2); log.accept(who + "WIN +" + h.bet()); }
    else if (mine == theirs) { p.credit(h.bet()); log.accept(who + "PUSH"); }
    else log.accept(who + "LOSE " + h.bet());
}
```

Output excerpt:

```
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
== 2. Stacked shoe: split 8s, then double both hands ==
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Balances: Alice=140
== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100
```

A stacked shoe forces exact scenarios; Bob's insurance hedges his loss to break even.

Concurrency & edge cases

- **One table = one thread** (or actor); tables share only the wallet, which needs atomic debit/credit — debit *before* dealing.
- **Odd antes** (3:2 on 5 = 7.5) → we require even antes; casinos round down or use smaller chips — decide and say it.
- **Split aces**: 21 after a split is *not* blackjack (`fromSplit`). **Shoe exhaustion**: cut card reshuffles between rounds; a stacked shoe throws instead.
- **Can't afford double/split** → `canDouble=false`, and the action is rejected if chosen. **Peek** only on ace or ten.

Extensions & follow-up questions

- **Even money?** Same as insurance with a blackjack (always nets +1× ante); add `takeEvenMoney(hand)` and settle 1:1 before the peek.
- **Surrender?** `Action.SURRENDER` as first decision; refund half.
- **Testing the dealer rule?** Stacked shoe, deal A+6, assert `hitSoft17()` draws and `standOnAll17()` stands.
- **Uno deck?** `Deck<UnoCard>`. **Security?** Cut card limits counting; never send the hole card early.

Real-world example

Online casino tables split a certified **RNG/shuffle service** (regulators such as the UK Gambling Commission require independent testing), a **game engine** enforcing table rules (S17/H17, 3:2 or 6:5), and a **wallet** debiting atomically — rule variants are configuration.

In the interview

> **30-second answer:** "A generic `Card` and `Deck<T>`. A `Shoe` combines decks with an injected `Random`, plus a stacked constructor for tests. The blackjack value lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, soft if one ace can be 11. Player decisions and the dealer rule are strategies. `BlackjackGame` runs bets, deal, insurance and peek, player turns, dealer turn, and settles 3:2, 1:1 or push."

Likely follow-ups: *Why not `int value` on `Card`?* — value is game-specific. *Best hand value?* — hard + 10 if an ace fits. *Testable?* — seeded `Random`, stacked shoe.

Self-check

1. **Q:** Why does `Hand` store the bet? **A:** After a split each hand has its own bet; a double changes only that hand.
2. **Q:** A+A+9? **A:** Hard 11; one ace as 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only on an ace or 10-value up card.
4. **Q:** Is 21 after a split a blackjack? **A:** No — only the original two cards.

TL;DR

- Keep `Card` / `Deck` generic; the value rule lives in the game.
- Hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player and dealer; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer → settle (3:2, 1:1, push).

Design Chess

Problem & clarifying questions

Chess: two players, 8×8 board, 16 pieces each (**8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**); goal is **checkmate**. We design an **online** two-player game.

Question	Assumed answer
Full rules?	Yes: all moves, check, checkmate, stalemate, castling, en passant, promotion.
Sides?	Random (`Game.withRandomSides` , injected `Random`); **white first**; alternate.
Take-backs?	Not in rated games (course); `undo()` for analysis and tests.
Endings?	Checkmate, stalemate (draw), resignation, forfeit.
Move log?	Yes.
Timers, rating, AI?	Out of scope.

Requirements

Functional:

1. Two players, a colour each, white first, alternating.
2. Move only your **own** piece; reject out-of-turn moves, impossible moves, and moves leaving your king in check.
3. All special moves:
   - **Castling** — king moves two squares toward a rook, rook jumps over; only if neither moved, squares between empty, king not in check and not passing through or landing on an attacked square.
   - **En passant** — a pawn that advanced two squares beside an enemy pawn may be captured as if it moved one, only on the very next move.
   - **Promotion** — a pawn on the last rank becomes Q, R, B or N.
4. Detect **check**, **checkmate** (in check, no legal move — attacker wins) and **stalemate** (not in check, no legal move — draw).
5. **Resignation** and **forfeit** (losing without choosing, e.g. leaving or timeout).
6. **Log** every move.

Non-functional: new piece types without touching existing ones (Open/Closed); correctness before speed; safe when two player connections call concurrently.

Use cases

Player — create/join games, **make moves**, resign/forfeit, manage account. **Admin** — add, modify, block members. "Make move" is extended per piece type — our `Piece` polymorphism. The move activity `Game.move` follows: valid for the piece? → own king safe? → capture → promote → castling moves the rook → checkmate/stalemate ends the game.

Core entities

Class	Responsibility
`Color`	WHITE/BLACK, `opposite()`.
`Position` (record)	Square a–h/1–8; validates; parses "e4" (course: *Box*).
`Piece` (abstract, sealed)	Colour + `targets(board, from)`; `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	One "slide until blocked" loop; subclasses list directions.
`Board`	8×8 grid; `copy()`, `kingOf`, `isAttacked`, `render`; no turns or history.
`Move` + `MoveType`	from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	Four permissions and how a move changes them.
`Rules`	Stateless: pseudo-legal, legal (king-safety filter), special moves, `apply`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, log, undo (Command + Memento), resign, forfeit, random sides. Thread-safe.
`Player`, `GameStatus`, `IllegalMoveException`	Supporting types.

Class diagram

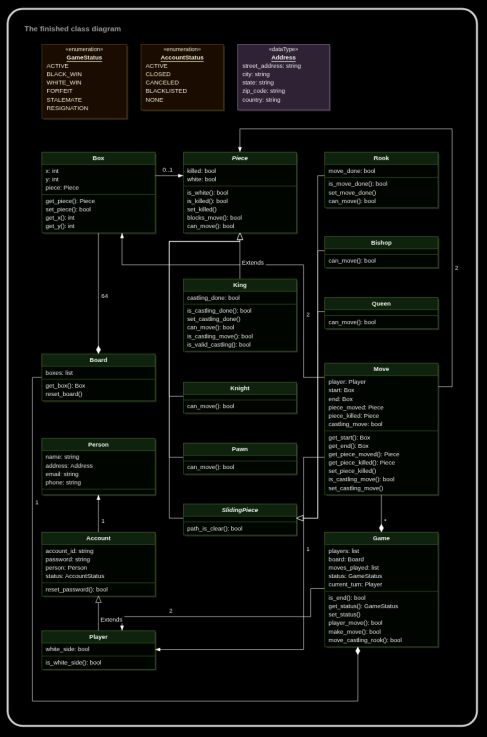
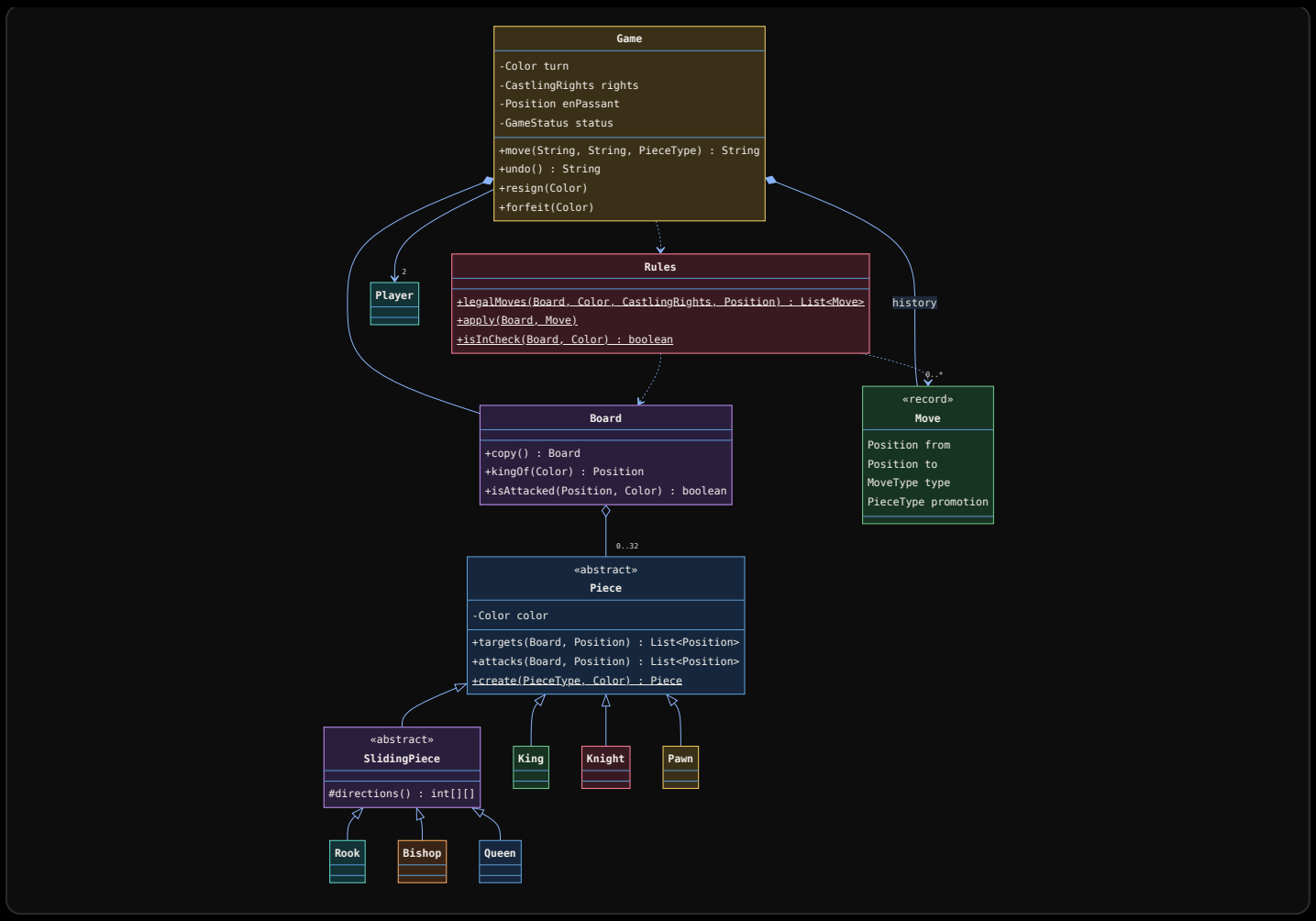


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

In the course, `Board` is **composed** of 64 `Box`es; a box refers to a piece by **association** (a captured piece isn't destroyed, so the log is replayable). `SlidingPiece` writes the "path is clear" check once; `Move` records piece moved and captured; course `GameStatus`: ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE, RESIGNATION. We fix two limits the course admits — history rules (castling, en passant) had nowhere to live, and checkmate/stalemate weren't detected — by adding `Rules` and a `MoveType` on `Move`.



Key flows

Making a move: check status, piece and colour → `Rules.legalMoves` → match or throw `IllegalMoveException` → save a `Snapshot` (memento) → `Rules.apply` → update rights, en passant, turn → check the opponent → notation (`Qh5xf7#`). **Status:** `ACTIVE` → `CHECKMATE` / `STALEMATE` / `RESIGNATION` / `FORFEIT` (undo can reactivate).

Detecting check, checkmate, stalemate:

1. **Pseudo-legal moves** — the pieces' movement rules + castling, en passant, promotions.
2. **Legal moves** — apply each to a *copy* of the board, keep it only if the mover's king is **not attacked**. This one filter handles pins, moving into check and ignoring check.
3. **King attacked?** — every enemy piece's `attacks(...)`; pawns attack diagonally even onto empty squares.
4. After a move, if the **opponent** has **zero legal moves**: in check = **checkmate**, else **stalemate**.

This one-ply search is all the *rules* need (deeper search is an *engine's* job). Cost: ~200 pseudo-legal moves × an attack scan over ≤ 16 pieces — trivial; engines use bitboards.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces, template-like `SlidingPiece`.	Each rule in its own class; path loop written once.	Giant `switch (pieceType)`.
Sealed hierarchy.	Closed set; exhaustive `switch`.	Open hierarchy — fine for variants.
Immutable pieces; `Board.copy()` copies references.	Cheap, safe trial moves and snapshots.	Mutable `hasMoved` flags undo must restore.
History rules in `Rules` / `Game`.	Castling and en passant depend on earlier moves.	`King` reading the game log.
`Move` carries a **`MoveType`**.	Castling moves two pieces; en passant captures off-square.	Special cases in `Game` (course's `move_castling_rook`).
Command + Memento undo.	Restoring a snapshot is always correct for special moves.	Hand-written inverse per move type (bug source).
Factory method `Piece.create`.	Set-up and promotion.	Scattered `new`.
`synchronized` `Game`.	A move and a resignation can't interleave.	No locking.

Memento vs incremental undo. A snapshot is 64 references (~0.5 KB) — nothing for a human game. An engine searching millions of positions stores only what changed (captured piece, old rights, old en passant) and reverses it. Know both.

Java implementation

Package `lld.ch17.chess`. Full runnable code: code/lld/ch17/chess/ (see the Full Edition).

```java
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    // ...
    /** Squares this piece may move to by its ordinary rules (captures included, own-king safety NOT checked). */
    public abstract List<Position> targets(Board board, Position from);

    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }
    // ...
}

public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    // ...
    protected abstract int[][] directions();

    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) {
                    out.add(p);
                } else {
                    if (occupant.color() != color()) out.add(p);   // capture, then stop
                    break;
                }
                f += d[0];
                r += d[1];
            }
        }
        return out;
    }
}

// Rules.java
public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
    List<Move> legal = new ArrayList<>();
    for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {
        Board trial = board.copy();
        apply(trial, m);
        if (!isInCheck(trial, side)) legal.add(m);
    }
    return legal;
}

public static void apply(Board board, Move m) {
    Piece piece = board.get(m.from());
    board.set(m.from(), null);
    board.set(m.to(), piece);
    switch (m.type()) {
        case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
        case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
        case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
        case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
        case NORMAL -> { }
    }
}

// Game.java — inside play(): end-of-game detection
boolean check = Rules.isInCheck(board, turn);
boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
if (noMoves) {
    status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE;
    winner = check ? mover.color() : null;
}

public synchronized String undo() {
    if (history.isEmpty()) throw new IllegalStateException("nothing to undo");
    HistoryEntry last = history.pop();
```

```
    Snapshot s = last.before();
    board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
    status = s.status(); winner = s.winner();
    return last.notation();
}
```

`Pawn` overrides `attacks` to return both forward diagonals; `Rules.addCastling` checks rights, empty squares and attacked squares; `CastlingRights.after` clears rights when the king or a rook moves or a corner rook is captured.

Output excerpt:

```
e7-e5 rejected: it is WHITE's turn
a1-a4 rejected: ROOK on a1 cannot move to a4
h5-f7 played as Qh5xf7#
Status: CHECKMATE, winner WHITE
undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves
c7-d8 played as c7xd8=Q+
b2-a1 rejected: b2-a1 would leave the BLACK king in check
Status: STALEMATE, black in check? false, black legal moves: 0
e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
e8-g8 rejected: cannot castle e8-g8: the king is in check
```

Validation, Scholar's mate and undo, a checking promotion, Sam Loyd's 10-move stalemate, and castling refusals with reasons (`Rules.whyCannotCastle`).

Concurrency & edge cases

- **Two players, one game:** `synchronized` public methods; each game has its own monitor, so games don't block each other.
- **Out of turn / game over / empty square** rejected before rule work. **Pins, discovered checks** handled by the copy-and-test filter.
- **Castling through check** tested for the king's crossing and landing squares (the queenside rook's crossing square b1/b8 may be attacked — legal). **Rights** lost when king or rook moves or the rook is **captured** on its corner.
- **En passant expires** after one move. **Promotion** defaults to a queen; underpromotion supported.
- **Not implemented, but name them:** threefold repetition, 50-move rule, insufficient material, clocks, draw offers.

Extensions & follow-up questions

- **Threefold repetition?** `Map<PositionKey, Integer>` keyed on (board, side, rights, en passant); engines use Zobrist hashing.
- **50-move rule?** Half-move clock, reset on pawn moves/captures. **Clocks?** A `Clock` per player (Observer on flag-fall).
- **AI?** A `MoveStrategy` over `legalMoves()`; minimax + alpha-beta. **Chess960/new piece?** New `Piece` subclass + start factory.
- **Spectators/UI?** Observer `GameListener.onMove`; the course's `GameController`/`GameView` is MVC (Ch. 11).

Real-world example

Lichess and Chess.com treat the **server** as the source of truth. Games export as PGN (a move log) and positions as FEN (board, side to move, castling rights, en passant — our `Snapshot` fields); takebacks are a server-side undo.

In the interview

30-second answer: "`Board` is an 8×8 grid of immutable `Piece`s. Each piece overrides `targets`; rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game`, and filters out moves that leave your king attacked. If the opponent then has no legal move, it's checkmate if in check, else stalemate. `Game` handles turns, status, the log, and undo via Command plus Memento."

Likely follow-ups: *Checkmate?* — no legal moves + in check. *Where does castling live?* — `Rules`; `MoveType.CASTLE_*` tells `apply` to move the rook. *Why is Box→Piece not composition?* — a captured piece isn't destroyed. *Undo with promotion?* — the memento holds the old board with the pawn.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** It moves straight but attacks diagonally, even onto empty squares.
2. **Q:** Checkmate vs stalemate? **A:** Both = no legal move; in check = loss, not in check = draw.
3. **Q:** Three castling conditions? **A:** King and rook unmoved; squares between empty; king not in, through, or into check.
4. **Q:** Why immutable pieces? **A:** Board copies and snapshots can share them safely.

TL;DR

- Polymorphic `targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add specials → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History rules live in `Game` / `Rules`. Undo = Command + Memento. `synchronized` game.

Design Tic-Tac-Toe

Problem & clarifying questions

Players take turns placing X or O; first to fill a **row, column or diagonal** wins; a full board with no winner is a **draw**. Interviewers generalise to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3.
Win?	A full line of N marks: row, column, or one of the two main diagonals.
Players?	Two by default; more allowed, each with its own mark.
Human or computer?	Both, via a pluggable **move strategy** (human, random, rule-based, scripted).
Invalid moves?	Rejected; same player tries again.

Requirements

Functional: (1) N×N board and 2+ players with unique marks; (2) turns, current player picks an empty cell; (3) validate inside and empty; (4) detect **win** or **draw** after each move; (5) announce result and show board.

Non-functional: win check **O(1)** per move (not $O(N^2)$); new player types without touching `Game`; seeded random bots and scripted players; games share no state.

Use cases

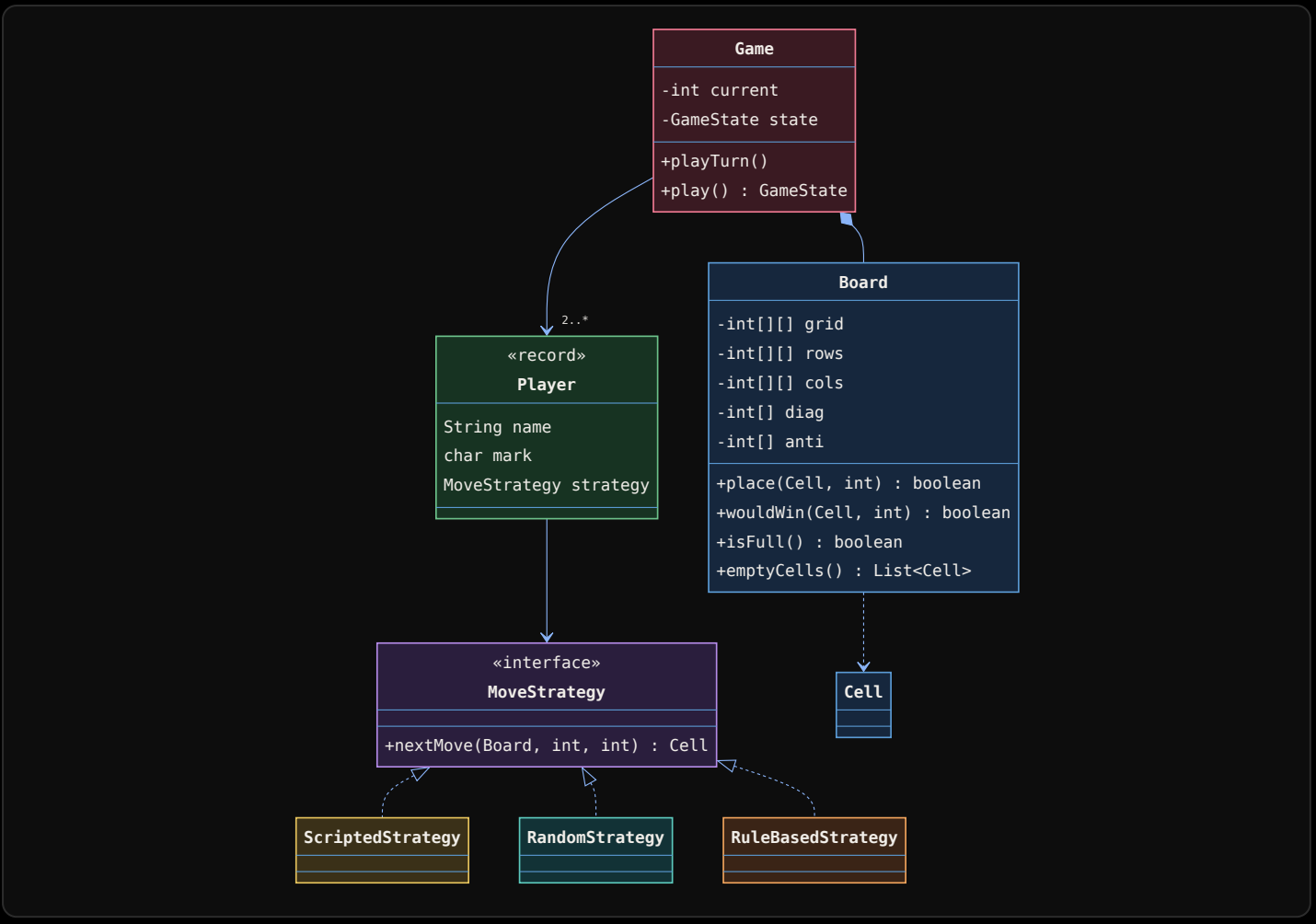
Player (human or bot): start game (choose N and players), make move, view board, see result. The `Game` validates moves, switches turns, declares the result.

Core entities

Class	Responsibility
`Cell` (record)	Row + column.
`Board`	Grid + **per-player line counters**; `place` returns "won?" in O(1); `wouldWin`, `isFull`, `emptyCells`, `render`.
`MoveStrategy`	`ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record), `GameState`	Name, mark, strategy; IN_PROGRESS, WON, DRAW.
`Game`	Turn order, validation, state, winner.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (3×3, X/O turns, line wins, full board draws, validate, announce) — covered and generalised to N×N.



Key flows

O(1) win detection. A move at `(r, c)` can only complete **row r, column c, and maybe the two diagonals**. Keep `rows[p][r]`, `cols[p][c]`, `diag[p]` (`r == c`) and `anti[p]` (`r + c == N-1`); increment the ones the cell belongs to; if any reaches **N**, player `p` wins (only `p`'s marks bump `p`'s counters). Four increments: **O(1) time, O(P·N) memory**.

> **The 2-player trick:** one counter per line, +1 for X and −1 for O, `|count| == N` wins — half the memory but only for two players.

The same counters give **"would this move win?"**: a line already holding `N-1` of `p`'s marks. The rule-based bot uses it to win, else block.

One turn: `nextMove` → `place` → occupied/off-board throws, `Game` logs and the same player goes again; else `WON`, `DRAW` if full, or next player. States: `IN_PROGRESS → WON | DRAW`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for moves.	Human, random, rule-based, network, minimax plug in.	`if (player.isComputer())`.
Counters inside `Board`.	O(1), next to the data.	Scanning — O(N^2), duplicated for "would win".
`place` returns "won?".	A win can only appear right after a move.	A `checkWinner()` callers forget.
Board throws, Game catches.	Board guards its invariant; game sets policy (retry).	Silently ignoring bad moves.
`current = (current+1) % size`.	Any number of players.	X/O toggle.

Java implementation

Package `lld.ch17.tictactoe`. Full runnable code: code/lld/ch17/tictactoe/ (see the Full Edition).

```java
// Board.java — Places a mark and returns true if this move completed a line. O(1).
boolean place(Cell c, int player) {
    if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
    if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
    grid[c.row()][c.col()] = player;
    filled++;
    int r = c.row(), k = c.col();
    boolean win = ++rows[player][r] == n;
    win |= ++cols[player][k] == n;
    if (r == k) win |= ++diag[player] == n;
    if (r + k == n - 1) win |= ++anti[player] == n;
    return win;
}

public boolean wouldWin(Cell c, int player) {
    int r = c.row(), k = c.col();
    return rows[player][r] == n - 1 || cols[player][k] == n - 1
            || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
}

public boolean isFull() { return filled == n * n; }

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything. */
public final class RuleBasedStrategy implements MoveStrategy {
    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        for (Cell c : empty) if (board.wouldWin(c, me)) return c;
        for (int other = 0; other < playerCount; other++) {
            if (other == me) continue;
            for (Cell c : empty) if (board.wouldWin(c, other)) return c;
        }
        // ... centre, then corners, then empty.get(0)
    }
}

// Game.java
public void playTurn() {
    if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
    Player p = players.get(current);
    Cell cell = p.strategy().nextMove(board, current, players.size());
    boolean won;
    try {
        won = board.place(cell, current);
    } catch (IllegalArgumentException e) {
        log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
        return;
    }
    log.accept("  " + p.name() + " (" + p.mark() + ") -> " + cell);
    if (won) { state = GameState.WON; winner = p; }
    else if (board.isFull()) state = GameState.DRAW;
    else current = (current + 1) % players.size();
}
```

Output excerpt:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  ...
    X O O
    . X .
    . . X
  Result: Xena wins
== 3. 4x4 rule-based vs rule-based ==
  Result: draw
```

Concurrency & edge cases

- **Many games:** each `Game` owns its `Board`; keep them in a `ConcurrentHashMap<gameId, Game>`, one move at a time per game.
- **Two clicks at once:** `synchronized` `makeMove(player, cell)` that checks the caller is the current player.
- Move after game over → `IllegalStateException`; off-board/occupied → retry; scripted strategy out of moves → `null`, fail fast; N ≥ 3.
- **Centre of an odd board** is on both diagonals → two independent `if`s.

Extensions & follow-up questions

- **K in a row on N×N (Gomoku)?** Counters fail; walk out from the new cell in 4 directions counting own marks — O(K).
- **Unbeatable AI?** Minimax (3×3 has ≤ 9! orders), alpha-beta or memoisation for bigger boards — another `MoveStrategy`.
- **Undo?** Move stack; clear the cell and **decrement** counters. **Online?** Server holds the `Game`, broadcasts state (Observer). **Replay?** Move list + `ScriptedStrategy`.

Real-world example

Google Search's built-in tic-tac-toe offers difficulty levels — different players behind the same rules, the Strategy split here.

In the interview

> **30-second answer:** "`Board` is N×N plus per-player counters for each row, column and both diagonals. A move bumps at most four; if one hits N the player wins — O(1). `Game` holds a player list and turn index, validates moves, and ends on a win or full board. Each `Player` has a `MoveStrategy` — human, seeded random, or rule-based that wins if it can and blocks otherwise."

Time plan: spend 10 minutes on the counter idea — draw it.

Self-check

1. **Q:** Why check only four lines? **A:** A cell is in one row, one column and at most two diagonals.
2. **Q:** Anti-diagonal test? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters, not +1/−1? **A:** Work for 3+ players and give "would win" for anyone.
4. **Q:** Draw detection? **A:** No win and filled cells = N².

TL;DR

> - Per-player row/column/diagonal counters → O(1) win; `N-1` → O(1) "would win".
> - `Game` = players + turn index + state; invalid moves retried.
> - Players via `MoveStrategy`; K-in-a-row needs a directional walk instead.

Design Snake and Ladder

Problem & clarifying questions

A race on a numbered board (usually 1–100): roll, move forward; a **ladder foot** takes you up, a **snake head** down; first to the last square wins. No decisions — the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board?	Configurable (default 100, given snakes/ladders); validated.
Dice?	Configurable count and faces (default one d6).
Players?	Two or more, starting off the board (square 0).
Overshoot?	Configurable: **exact finish** or **bounce back**.
Rolling a six?	Roll again; three sixes cancels the turn (on/off).
End?	**First to finish** or **rank everyone**.
Several games?	Yes, independent sessions.

Requirements

Functional: (1) numbered board (~100) with predefined snakes and ladders; (2) several players with one token, off the board; (3) roll and move by the total; (4) snake head → tail, ladder foot → top; (5) overshoot rule; (6) optional bonus on a six with the three-sixes penalty; (7) continue until the win strategy says over; (8) many independent sessions.

Non-functional: deterministic dice (seeded or loaded); pluggable rules without editing the loop; invalid boards rejected **at construction**.

Use cases

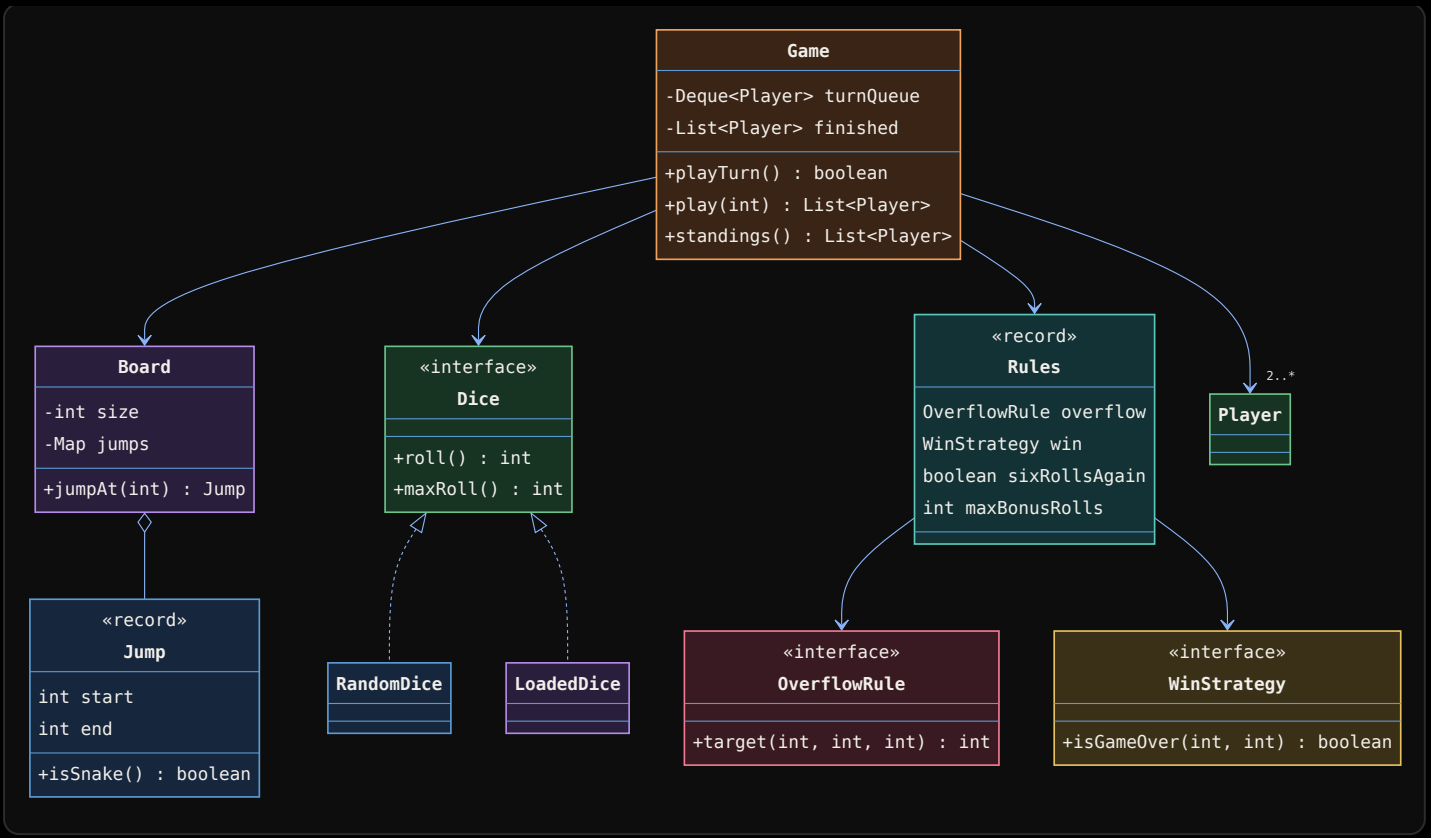
Player (rolls, or the system rolls) and **Game admin/host** (configures board, dice, rules; starts). Use cases: configure board, add players, start, take turn (roll → move → snake/ladder → maybe bonus), declare winner/standings.

Core entities

Class	Responsibility
`Jump` (record)	Snake or ladder: `start → end`; one class, `isSnake()` = `end < start`.
`Board`	Size + `Map<start, Jump>` (O(1)); validates config.
`Dice`	`roll()`, `maxRoll()`; `RandomDice` (seeded), `LoadedDice` (scripted).
`OverflowRule`	`exactFinish()` or `bounceBack()`.
`WinStrategy`	`firstToFinish()` or `rankAll()`.
`Rules` (record)	A variant: overflow, win, six-rolls-again, max bonus rolls.
`Player`, `Game`	Name + position; turn queue, `playTurn`, `play(maxTurns)`, standings, event log.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (board, snakes and ladders, tokens, turns, win at the last cell, many concurrent sessions — its reference uses a singleton `GameManager`, a thread per session). All covered.



Key flows

One turn: take the next player from the queue → roll → a max roll with bonus enabled that is the 3rd in a row voids the turn (back to the start square) → else `OverflowRule` gives the target → apply a jump starting there → bonus roll if earned and not finished → on the last square add to `finished`, else re-queue → stop when `WinStrategy` says so.

Board validation (once, in the constructor): (1) no jump to itself; (2) no jump starting on square 1 or the last square; (3) no two jumps with the same start; (4) no jump ending on another jump's start — forbids chains and therefore infinite loops.

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` for snakes and ladders.	Same behaviour; direction is data.	Duplicate `Snake` / `Ladder` classes.
`Map<Integer, Jump>`.	O(1); fine for sparse/huge boards.	Array of 101 (also fine).
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary; lambdas keep them tiny.	Flags and `if`s in the loop.
`Rules` record.	One object = one variant ("classic", "kids").	Many `Game` constructor params.
Queue of players.	Finished players just aren't re-queued.	Index arithmetic skipping finishers.
`Consumer<String>` event log.	Engine doesn't print; full design uses **Observer** (`GameListener`).	`System.out` in the engine.
Validate in the constructor; `play(maxTurns)` guard.	Fail fast; a bad rule set can't hang a thread.	Mid-game loop detection; `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`. Full runnable code: code/lld/ch17/snakeladder/ (see the Full Edition).

```java
// Board.java — validate once, up front
public Board(int size, Collection<Jump> all) {
    if (size < 10) throw new IllegalArgumentException("board too small");
    this.size = size;
    for (Jump j : all) {
        if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
        if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on
first/last square: " + j);
        if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
        if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at " +
j.start());
    }
    for (Jump j : jumps.values()) {   // no chains: an end square may not be another jump's start (also rules out
loops)
        if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another jump's
start");
    }
}

// OverflowRule.java
static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }
static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll - size) :
from + roll; }

// WinStrategy.java
static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }
static WinStrategy rankAll() { return (finished, total) -> finished >= total - 1; }

// Rules.java
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(),
true, 3); }
}

// Game.java
public boolean playTurn() {
    if (isOver()) throw new IllegalStateException("game is over");
    Player p = turnQueue.poll();
    takeTurn(p);
    if (p.position() == board.size()) {
        finished.add(p);
        log.accept("  *** " + p.name() + " finishes in place " + finished.size() + " ***");
    } else {
        turnQueue.offer(p);
    }
    return isOver();
}

private void takeTurn(Player p) {
    int start = p.position();
    int bonus = 0;
    // ...
    while (true) {
        int roll = dice.roll();
        // ...
        boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
        if (max && ++bonus == rules.maxBonusRolls()) {
            p.moveTo(start);                    // too many sixes in a row: the whole turn is cancelled
            // ...
            break;
        }
        move(p, roll, line);
        if (!max || p.position() == board.size()) break;
        // ...
    }
    // ...
}
```

`move` asks the overflow rule for the target, then looks up `board.jumpAt(target)` and moves to its end.

Output excerpt:

```
  Ann @0: rolls 4 -> 4 (ladder to 14)
  Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
  Ann @14: rolls 3 -> 17 (snake to 7)
  Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back to
31
  rejected: jump Jump[start=10, end=30] ends on another jump's start
  winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
  same as running each session alone? yes
```

Eight seeded sessions on eight threads match their solo results — sessions share no mutable state.

Concurrency & edge cases

- **Many sessions:** each `Game` owns players, queue and **its own** `Random` (a shared one is contended and breaks per-game reproducibility).
- Rolling after the end → `IllegalStateException`. A finished player is never re-queued. A bonus roll that finishes stops immediately.
- **Three sixes** voids the turn and returns the token to the turn's start (even past a ladder). **Bounce-back onto a snake** is applied.
- **Endless games** impossible with a valid board and fair dice, but `maxTurns` guards custom rules.

Extensions & follow-up questions

- **Special cells?** Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`).
- **Two dice?** `new RandomDice(2, 6, rng)`; 2d6 isn't uniform (7 most likely). **Roll a 6 to enter?** An `EntryRule`. **Pause/resume?** Serialize positions, queue, dice state.
- **UI/spectators?** Observer events (`PlayerMoved`, `SnakeBitten`, …).

Real-world example

Apps such as Ludo King (with a Snakes & Ladders mode) isolate each match, roll dice where players can't tamper (the server), and push moves as events.

In the interview

30-second answer: "`Board` holds size and a map from square to `Jump` — snake or ladder, same behaviour — validated on construction. `Dice` is a strategy: seeded for play, loaded for tests. `Rules` bundles overflow (exact or bounce), win strategy (first or rank all) and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies overflow, then any jump, and re-queues the player unless they finished."

Self-check

1. **Q:** Why is one `Jump` class enough? **A:** Both mean "land on start, go to end"; direction is data.
2. **Q:** How is an infinite loop prevented? **A:** No jump may end on another jump's start.
3. **Q:** Testing "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)`; assert the position is unchanged.
4. **Q:** Why a queue? **A:** Finishers simply aren't put back.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win and bonus rules are strategies; seeded or loaded dice.
- Queue-based turns with a `maxTurns` guard; events to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048: single-player 4×4 puzzle. Each move slides all tiles one way; two **equal** tiles that collide **merge** into their sum, added to the **score**. After each move that changes the board a new tile (usually 2, sometimes 4) appears in a random empty cell. Win on a 2048 tile; **over** when no move changes the board.

No course lesson; the common machine-coding statement adds: start with **two 2s**; with **three** equal tiles the pair **nearest the wall** merges, with **four** the first two and last two; input **0 = left, 1 = right, 2 = up, 3 = down**.

Question	Assumed answer
Size?	N (default 4).
New tile?	One per successful move: 2 (p = 0.9) else 4, uniform empty cell; start with two 2s.
Merge rule?	Each tile merges **at most once per move**, from the wall you move toward.
No-change move?	Not a turn: no tile, no score.
Win?	Target tile (2048, configurable); may continue.
Over?	No empty cell **and** no equal horizontal/vertical neighbours.

Requirements

Functional: (1) N×N with two 2s; (2) four directions: slide, merge once, score; (3) spawn a 2 or 4 only if the board changed; (4) detect WON, allow continuing; (5) detect LOST; (6) accept 0/1/2/3 (`Direction.fromKey`).

Non-functional: deterministic spawns via a seeded `Random`; merge logic written **once**, not four times; pluggable spawner (e.g. an "evil" one for hard cases).

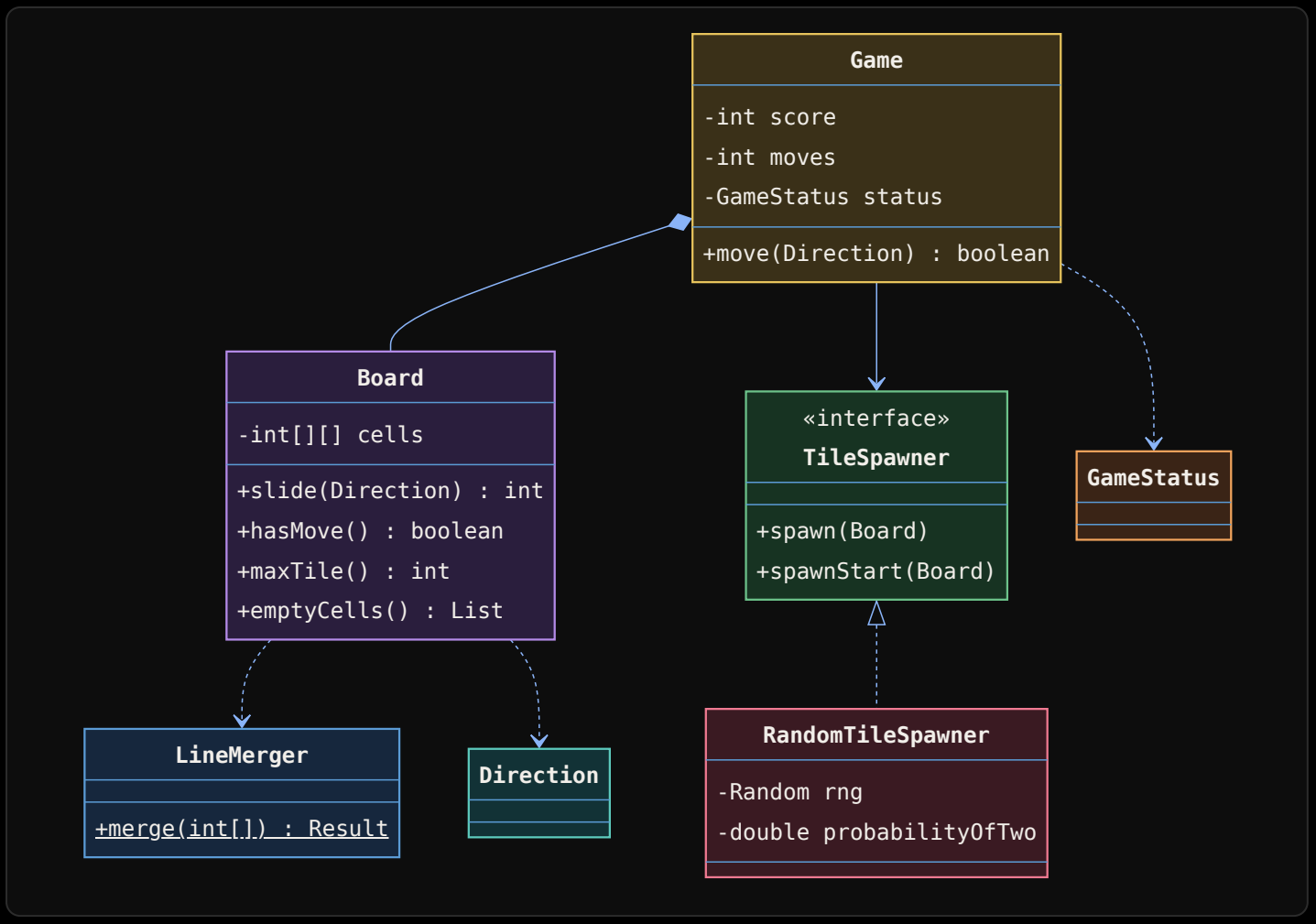
Use cases

Player: start new game, make a move (includes slide and merge, spawn tile, update score), see win, see game over, restart.

Core entities

Class	Responsibility
`Direction`	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)`.
`LineMerger`	Pure function: slide and merge **one line toward index 0**; new line + points.
`Board`	N×N values; maps directions to lines; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` / `RandomTileSpawner`	Seeded, 90% twos; `spawnStart` places the two starting 2s.
`GameStatus`, `Game`	PLAYING, WON, LOST; score, moves, status; slide + spawn + status.

Class diagram



Key flows

Merging one line toward index 0: walk from the wall, skip zeros; keep one **pending** tile; if the next equals it, write the double, add points, clear pending (so it **can't merge again**); if different, write pending and make the new tile pending; finally write any pending tile. Examples (left): `[2,2,2,2] → [4,4,0,0]` (not one 8), `[2,2,2,0] → [4,2,0,0]` (pair nearest the wall), `[2,2,4,0] → [4,4,0,0]` (new 4 doesn't re-merge), `[4,0,4,8] → [8,8,0,0]`.

One function for four directions — `coords(direction, i, j)` maps "line i, position j from the wall" to a cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

A move: `slide` merges every line; nothing changed → return `false`, no spawn; else add points, spawn, update status. States: `PLAYING → WON` (target first reached) `→ PLAYING` (keep going); `PLAYING/WON → LOST` (no move left).

Game-over check: a move exists if any cell is empty or two neighbours are equal; comparing each cell with its right and lower neighbour is enough. $O(N^2)$.

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction mapping.	Tricky rule written and tested once (DRY).	Four copies of nested loops — a bug farm.
`LineMerger` is **pure**.	Trivial unit tests.	Merge mixed with board mutation.
Strategy `TileSpawner`.	Seeded tests, evil spawner, variants.	`Math.random()` in `Board`.
No change = no turn.	Matches the real game; no spawn on a no-op.	Always spawning — unfair.
WON set once; keep playing.	Like the original game.	Ending at 2048.

Java implementation

Package `lld.ch17.game2048`. Full runnable code: code/lld/ch17/game2048/ (see the Full Edition).

```java
// LineMerger.java
public static Result merge(int[] in) {
    int n = in.length;
    int[] out = new int[n];
    int write = 0, points = 0;
    int pending = 0;                        // tile waiting for a possible merge partner
    for (int v : in) {
        if (v == 0) continue;
        if (pending == 0) {
            pending = v;
        } else if (pending == v) {
            out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
            points += v * 2;
            pending = 0;
        } else {
            out[write++] = pending;
            pending = v;
        }
    }
    if (pending != 0) out[write] = pending;
    return new Result(out, points);
}

// Board.java — returns the points earned, or -1 if nothing moved
int slide(Direction d) {
    boolean changed = false;
    int points = 0;
    for (int i = 0; i < n; i++) {
        int[] line = new int[n];
        for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
        LineMerger.Result res = LineMerger.merge(line);
        for (int j = 0; j < n; j++) {
            if (read(d, i, j) != res.line()[j]) changed = true;
            write(d, i, j, res.line()[j]);
        }
        points += res.points();
    }
    return changed ? points : -1;
}

private int[] coords(Direction d, int i, int j) {
    return switch (d) {
        case LEFT -> new int[]{i, j};
        case RIGHT -> new int[]{i, n - 1 - j};
        case UP -> new int[]{j, i};
        case DOWN -> new int[]{n - 1 - j, i};
    };
}

public boolean hasMove() {
    for (int r = 0; r < n; r++)
        for (int c = 0; c < n; c++) {
            if (cells[r][c] == 0) return true;
            if (c + 1 < n && cells[r][c] == cells[r][c + 1]) return true;
            if (r + 1 < n && cells[r][c] == cells[r + 1][c]) return true;
        }
    return false;
}

// Game.java
public boolean move(Direction d) {
    if (status == GameStatus.LOST) throw new IllegalStateException("game over");
    int points = board.slide(d);
    if (points < 0) return false;          // nothing moved: no spawn, no turn
    score += points;
    moves++;
    spawner.spawn(board);                  // at least one cell is empty, because something moved or merged
    updateStatus();
    return true;
}
```

Output excerpt:

```
[2, 2, 2, 2] -> [4, 4, 0, 0]  +8
[2, 2, 2, 0] -> [4, 2, 0, 0]  +4
[2, 2, 4, 0] -> [4, 4, 0, 0]  +4
move LEFT changed board? false, moves=0
checkerboard, no empty cell, no equal neighbours -> LOST
reached 128 after 85 moves (status WON), score 852
game over after 232 moves, score 2584, max tile 256
```

A seeded corner-strategy game reaches the 128 target and keeps playing until the board locks.

Concurrency & edge cases

- Single-player, single-threaded; if a UI thread and an input thread both call `move`, make it `synchronized` (or post to the UI event queue).
- **Key repeat:** a no-op move returns `false` — harmless.
- **Spawn needs an empty cell:** guaranteed after a change (tiles move into gaps or merge, freeing a cell); the spawner still checks.
- **Integer overflow:** unreachable on 4×4 (max tile 2^17 = 131072). **Merge order:** `[2,2,2]` left → `[4,2,0]`.

Extensions & follow-up questions

- **Undo?** Memento: push board + score before each successful move.
- **Sizes/targets?** Already parameters (`new Game(5, 4096, spawner)`).
- **AI?** Expectimax (max over moves, average over spawns) as a move-choosing strategy.
- **Animations?** `LineMerger` also returns tile origins for the view (Observer, MVC). **Leaderboard?** A separate service.

Real-world example

2048 (Gabriele Cirulli, 2014, open source) separates a grid model, a game manager and an animating view; new tiles are 2 at 90%, 4 at 10%.

In the interview

> **30-second answer:** "One pure function slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merges. The board maps each direction to lines in the right order, reusing the function four times. After a move that changes the board, a seeded `TileSpawner` adds a 2 (90%) or 4. Won when the target appears; over when no empty cell and no equal neighbours."

Time plan: give `LineMerger` + mapping 15 minutes; write its tests first.

Self-check

1. **Q:** `[4,4,8,8]` moving left? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why no spawn on a no-op? **A:** It isn't a turn; spawning would punish harmless key presses.
3. **Q:** Avoiding four copies of slide code? **A:** Map directions to lines, one merge function.
4. **Q:** Repeatable tests? **A:** Seeded spawner or `Board.of(...)`.

TL;DR

> - One pure `LineMerger` (merge once, from the wall) + direction→line mapping.
> - Spawn only after a real change; 2 (90%) or 4; seeded spawner.
> - WON once at the target; LOST with no empty cell and no equal neighbours.

Design a Jigsaw Puzzle

Problem & clarifying questions

A picture cut into roughly square pieces, each with **four sides**: **flat** (an edge side — belongs on the outer frame), **tab** (a knob that sticks out) or **blank** (a hole a tab slots into). Two sides **interlock** when one is a tab, the other a blank, with the same **cut profile**. Solved = every cell filled and every shared side interlocks. The interview tests: the matching rule in one place, and the search algorithm behind a swappable interface.

Question	Assumed answer
Shape?	`rows × cols` rectangle, at least 2 × 2; one piece per cell.
How do sides fit?	Each tab/blank has a **shape code**; TAB fits BLANK with the same code.
Rotation?	Quarter turns only; no flipping.
Picture matching?	No — shapes only (extension).
Who places?	Players (co-op allowed) and an automatic **solver**, which can finish a half-built board.
Look-alike pieces?	Generator supports unique codes or a few reused codes (forces search).

Requirements

Functional: (1) piece = four sides; kind = **corner** (2 flat) / **border** (1) / **interior** (0); (2) side matching; (3) place with rotation, refused **with a reason** if off board, cell taken, piece used, non-flat side facing out, flat side facing in, or a side not fitting a placed neighbour; (4) remove; (5) **validate** the box before play; (6) **solve** with a pluggable algorithm, from empty or partial; (7) progress and SOLVED.

Non-functional: one home for placement rules (players and solvers agree); new solvers without touching `Board` (Open/Closed); seeded generator; safe co-op.

Use cases

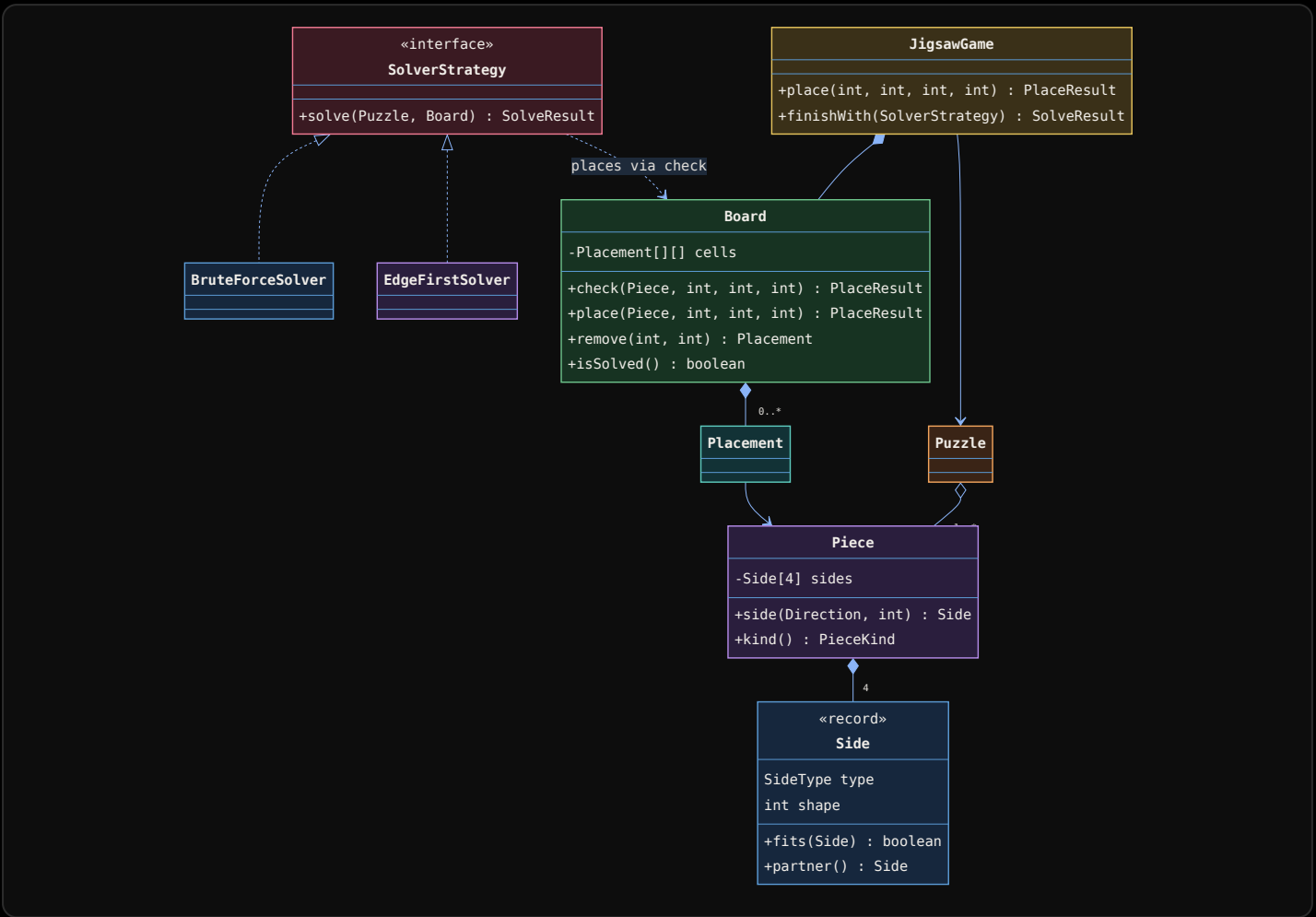
Player: start a puzzle (includes *validate the box*), place a piece (includes *choose a rotation*), remove a piece, ask the solver to finish (includes *place a piece*: the solver is part of the system, not an actor), see progress and SOLVED.

Core entities

Class	Responsibility
`SideType`, `Side` (record)	FLAT/TAB/BLANK + shape code; `fits`, `partner()` (the one side that fits).
`Direction`	TOP, RIGHT, BOTTOM, LEFT clockwise; neighbour offset; `opposite()`.
`Piece`, `PieceKind`	Immutable id + 4 sides at rotation 0; `side(d, rotation)`, `kind()`.
`Board` + `Placement(piece, rotation)`	Grid; **every placement rule** in `check`; `place`, `remove`, `verify`, `isSolved`.
`PlaceResult`	OK or the reason a placement was refused.
`Puzzle`, `PuzzleGenerator`, `PuzzleValidator`	The box; seeded cutting + shuffling + turning; cheap sanity checks.
`SolverStrategy` → `BruteForceSolver`, `EdgeFirstSolver`	Swappable algorithms; `SolveResult` (attempts, backtracks).
`JigsawGame`, `GameStatus`	Validates on start; `place` / `remove` / `finishWith(solver)`; `synchronized`.

Rotation lives in `Placement`, not `Piece`: it is a fact about where a piece lies; immutable pieces mean trying a rotation needs no undo.

Class diagram



Key flows

Rotation: sides stored clockwise `[TOP, RIGHT, BOTTOM, LEFT]`; one clockwise turn moves TOP to RIGHT, so after `r` turns the side facing `d` is `sides[(d − r) mod 4]` (`Math.floorMod`, since `-1 % 4` is `-1` in Java).

`Board.check`: bounds → cell free → piece unused → for each direction: outside the picture ⇒ side must be FLAT; inside ⇒ not FLAT, and must `fit` a placed neighbour's opposite side. O(1). Corners land only in corners, in the one right rotation, with no special-case code.

Edge-first solve (the human way): validate → sort by kind → index every non-flat side (`Side → (piece, side position)`) → corner at top-left → walk the frame clockwise → fill the inside row by row. Each next cell has a placed neighbour; it needs that neighbour's facing side's **partner**, so the index returns candidates in O(1) and rotation = `(d − i) mod 4`. Dead end ⇒ **backtrack** (undo the latest placement, try the next candidate). An attempt budget bounds the search.

Design decisions & patterns used

Decision	Why	Alternative rejected
All rules in `Board.check`.	Player and solver moves judged identically.	Each solver re-checking fit.
Strategy `SolverStrategy` (§11.3).	Add solvers without touching board/game.	`switch (algorithm)` in the game.
`Side` is a value record.	Usable as a map key — enables the index.	Identity-compared mutable sides.
`PlaceResult` reason, not exception.	Wrong guesses are normal play; UI can explain.	Exceptions for routine refusals.
Validate first; budget every search.	Bad boxes fail fast; solvers always return.	Discovering a missing piece by failing to solve.

Java implementation

Package `lld.ch17.jigsaw`. Full runnable code: code/lld/ch17/jigsaw/ (see the Full Edition).

```java
// Side.java
public boolean fits(Side other) {
    return !isFlat() && !other.isFlat() && other.type == type.complement() && other.shape == shape;
}
public Side partner() { return new Side(type.complement(), shape); }

// Piece.java
public Side side(Direction d, int rotation) { return sides[Math.floorMod(d.ordinal() - rotation, 4)]; }

// Board.java
public PlaceResult check(Piece piece, int rotation, int r, int c) {
    if (!inside(r, c)) return PlaceResult.OUT_OF_BOUNDS;
    if (cells[r][c] != null) return PlaceResult.CELL_OCCUPIED;
    if (placedIds.contains(piece.id())) return PlaceResult.ALREADY_PLACED;
    for (Direction d : Direction.values()) {
        Side mine = piece.side(d, rotation);
        int nr = r + d.dRow, nc = c + d.dCol;
        if (!inside(nr, nc)) {
            if (!mine.isFlat()) return PlaceResult.EDGE_MUST_BE_FLAT;
            continue;
        }
        if (mine.isFlat()) return PlaceResult.FLAT_FACES_INSIDE;
        Placement neighbour = cells[nr][nc];
        if (neighbour != null && !mine.fits(neighbour.side(d.opposite()))) return PlaceResult.SIDES_DO_NOT_FIT;
    }
    return PlaceResult.OK;
}

// EdgeFirstSolver.java — candidates for an empty cell
Side needed = board.at(nr, nc).side(d.opposite()).partner();      // what must face d from this cell
for (Hit h : index.getOrDefault(needed, List.of()))
    if (!board.isPlaced(h.piece().id()) && h.piece().kind() == wanted)
        out.add(new Board.Placement(h.piece(), Math.floorMod(d.ordinal() - h.sideIndex(), 4)));
```

Output excerpt:

```
  piece 12 (interior) at corner (0,0)    EDGE_MUST_BE_FLAT
  piece 3 at (0,0), wrong turn           FLAT_FACES_INSIDE
  piece 3 at (0,0), right turn           OK
  solver finishes the rest: EdgeFirst+Index    solved=true  attempts=        9 backtracks=        0
  one side mis-cut: [shape 8: 2 tabs but 0 blanks]
  10 x 10, shapeVariety 0 (every joint unique)
    BruteForce         solved=true  attempts=   11,060 backtracks=        0  verify=true
    EdgeFirst+Index    solved=true  attempts=      101 backtracks=        0  verify=true
  6 x 6, shapeVariety 2 (look-alike pieces)
    BruteForce         solved=true  attempts=2,099,470 backtracks=   46,642  verify=true
    EdgeFirst+Index    solved=true  attempts=  501,976 backtracks=   72,683  verify=true
```

Unique shapes: edge-first ≈ one attempt per piece; brute force grows with the square of the piece count. Look-alikes force both to search.

Concurrency & edge cases

- **Co-op:** `synchronized` game methods; a second player racing for the same cell or piece gets CELL_OCCUPIED / ALREADY_PLACED. One lock per game.
- **Solver holds the lock** for the whole solve; for big puzzles solve on a copy and offer hints.
- **Bad box** (missing/extra piece, 3 flat sides, opposite flats, tabs ≠ blanks per shape) — refused before play. Validation is **necessary, not sufficient**.
- **Look-alikes:** any layout passing `verify` counts as solved. **Failed solve** leaves the board as it was.

Extensions & follow-up questions

- **Picture matching?** Edge colour signatures on `Side` + a new strategy that ranks candidates.
- **Faster on look-alikes?** Next cell = the one with the fewest candidates; check both neighbours when listing candidates.
- **Undo / live UI?** Command stack of placements (§10.2); `PiecePlaced` events (Observer, §11.1).

Real-world example

Shape-only jigsaw solving with look-alike pieces is NP-complete in general; practical solvers add picture-content scores between edges to edge-first, greedy placement.

In the interview

> **30-second answer:** "Immutable `Piece` with four clockwise `Side`s; a side is FLAT/TAB/BLANK + shape code, TAB fits BLANK with the same code. Rotation lives in the placement: facing side = `sides[(d − r) mod 4]`. `Board.check` holds every rule and both players and solvers use it. Solving is a Strategy — brute force, or edge-first with a side→pieces index (≈ n attempts). Validate the box first; synchronized game for co-op."

Time plan: 10 min on `Side.fits`, `Piece.side(d, r)`, `Board.check`; 10–15 min on the solver interface and the index.

Self-check

1. **Q:** `[F, T1, B2, F]` turned clockwise twice — what faces TOP? **A:** `sides[2] = B2`.
2. **Q:** Border (non-corner) pieces in 5 × 7? **A:** `2·3 + 2·5 = 16`.
3. **Q:** How does a corner piece end up only in corners? **A:** `check` needs outward sides flat and inward sides not flat.
4. **Q:** Why ≈ one attempt per piece for edge-first? **A:** The index returns the pieces having the partner of a placed neighbour's side — with unique shapes, exactly one.

TL;DR

> - `Side` = type + shape code; TAB↔BLANK with equal codes. Rotation in `Placement`; `sides[(d − r) mod 4]`.
> - `Board.check` = the one home of every rule; returns a reason.
> - **Strategy** solvers: brute force vs edge-first + `Side → pieces` index; budget the search.
> - Validate first; `synchronized` for co-op.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot = move strategy)	None needed	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted	O(1) win with counters
Snake & Ladder	`Board` + `Rules`	`Dice`, `OverflowRule`, `WinStrategy`	`RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of`	One merge function for 4 directions
Jigsaw	`Board.check`	`SolverStrategy`	Seeded `PuzzleGenerator`	Side matching + rotation formula; edge-first with side index

Patterns used: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), template-style base (`SlidingPiece`), Command + Memento (chess undo), State (status enums; full State pattern suggested for blackjack rounds), Observer (event-log hooks; suggested for UIs).

Three habits interviewers look for: (1) **put every rule in exactly one place** and say where; (2) **inject randomness** so the design is testable; (3) **model states explicitly** and reject actions invalid in the current state.

Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition

> **Where this fits.** Chapters 14–17 designed *things*, *bookings* and *games*. This chapter and the next design **platforms**: websites where many people create content, react to each other and build up a history. The objects are people, posts, votes, relationships and feeds. This is the study edition: every topic of the full chapter, without the padding.

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The toolkit reused in every social design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base, Strategy (reputation), Observer (badges), privilege table
18.2	Cricinfo (+ Udaan "Cricket Match Dashboard")	A real sport's rules; live scorecards; stats from balls	Template Method (stat queries), Observer (live feeds), "derive, don't store"
18.3	LinkedIn	Connections and degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS, Strategy (feed ranking)
18.4	Facebook	Friends vs followers, privacy lists, pages, groups, feed, suggestions	One visibility policy for feed + search, sealed `Audience`, pull feed, suggestions

Running the code: `cd chapters/18-case-platforms-1/code && javac --release 21 -d /tmp/lld_18 $(find . -name '*.java') && java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main` (or `cricinfo`, `cricketdashboard`, `linkedin`, `facebook`). Java 21, standard library only. A file may hold several small package-private types (e.g. `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag`, `Bounty`) to keep a design readable in one place.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a site where *users produce the content* — Stack Overflow, Cricinfo, LinkedIn, Facebook. There is no single "machine" object; instead there is a **rich domain model**: many small classes (profile, post, vote, tag, badge, invitation, group…) each with a little data and a little behaviour. Analogy: a vending machine is a toaster; a platform is a town — people, buildings, entry rules, a notice board (feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers use platform problems to test six skills at once:

Skill tested	What it means	Where you practise it
Rich domain model	Nouns → classes with clear jobs; knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Guest, member, moderator, admin, commentator may do different things	Stack Overflow privileges; Facebook admin blocks members
Feeds	A personalised, ranked list of recent content from people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user something happened, through **channels** (in-app, email, push)	Observer + Strategy in every design
Search	Finding things by words, using an **inverted index** (word → items containing it)	Question, member and privacy-aware post search
Reputation / trust	Numbers summarising past behaviour that unlock abilities	Reputation and badges; endorsements; page ratings

How it works — the reusable toolkit

1. **A facade in front** (Chapter 9): one class offers `ask`, `vote`, `post`, `invite` and hides the rest; it checks "is this account active / allowed?" before anything changes.
2. **Shared content behaviour goes in a base class** (`Post`, `SocialContent`): voting and liking written once.
3. **Store *who*, not *how many*.** Likes and votes are a **set of member ids**: blocks double votes, allows "unlike", and the count is `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you keep (follower count, batter's runs, page rating), compute it. A stored copy will one day disagree. If too slow, keep a **cache** you can always rebuild.
5. **An operation that changes two objects lives in a service** (a class whose job is to perform an operation). Accepting a connection changes *both* members, so neither member owns it.
6. **Notifications through Observer + Strategy.** The domain publishes events; observers react; the delivery channel is a Strategy, so a new channel is a new class, not a new `if`.
7. **Search is its own class** — a `SearchIndex`, not a method on the entity. Every key maps to a *collection*, even for one match, so the second "Sharma" never breaks it.
8. **Privacy and permissions are asked in one place.** Write `canSee(viewer, post)` once and call it from both feed and search.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	No double votes, reversible	Memory grows; real systems use a votes table + cached count
Derive stats from raw events	Never inconsistent	Slow at scale; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; overdone, entities become "anemic"

When not to use it

For a tiny problem ("just a like button"), a set on the post is enough — no services, no observers.

Common mistakes

- Search as a static `Question.search()`, so the entity must know every other question.
- `likeCount` as a constructor argument — a post can be created "with 1,000 likes".

- Forgetting that privacy also applies inside search and the feed.
- `acceptInvitation` on the sender (or receiver), updating only one side.

Real-world examples

Kafka was built at LinkedIn to move activity events between systems (industrial Observer). **Facebook's TAO** serves the social graph at huge scale. **Stack Overflow** publishes its reputation-to-privilege table — exactly the `Privilege` enum in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

- *Why a set instead of a counter?* — A number does not record *who* liked, so it can neither block a repeat nor undo one; the set gives the count for free.
- *Where does notification code go?* — The domain raises an event; a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing it.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; a service updates both together so the halves cannot drift.
- **Q:** Two notification channels and the pattern that makes adding a third easy? **A:** Email and push; Strategy (`NotificationChannel`).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow

Stack Overflow is a Q&A community for programmers. Members **upvote/downvote** questions and answers, edit posts wiki-style, and earn **reputation** (points) and **badges** (named awards). Higher reputation unlocks **privileges**. An upvote on an answer is worth +10; the course says +5 for a question upvote, but since November 2019 it is +10 as well, and the code uses today's value.

Problem & clarifying questions

Question to ask	Answer we assume
Who uses it?	Guests, members, moderators, admins, and the system (notifications, badges)
What can a guest do?	Search and read only
What can be voted on?	Questions, answers, comments (comments: upvote only)
Reputation per vote?	Upvote +10 (question or answer); downvote −2 to the author, −1 to whoever downvotes an *answer* (downvoting a question is free); accepted answer +15 (+2 to the asker)
Privileges?	Vote up 15, comment everywhere 50, set bounty 75, vote down 125, create tag 1,500, edit 2,000, close votes 3,000, delete votes 10,000
Closing?	3 close votes from privileged members, or one moderator. Moderators also reopen, delete, undelete
Bounty?	50–500 points staked to attract answers, runs 7 days (+24 h grace on the real site), goes to the answer the sponsor picks. Deducted at once, **never refunded**; if unawarded, the site may auto-award half to a qualifying new answer
Search?	By keyword and tag, plus "most used tags"
Scale?	Read-heavy; votes must never double-count

Requirements

Functional (the course's thirteen): (1) guests search and view, must join to post or vote; (2) members ask; (3) answer **open** questions; (4) comment on questions and answers; (5) upvote questions, answers and comments; (6) **flag** posts for moderators; (7) add a **bounty** to their own question; (8) earn **badges**; (9) vote to **close**, and a moderator can close or reopen alone; (10) add **tags**; (11) vote to **delete** off-topic or low-quality questions; (12) moderators close and undelete; (13) report the **most used tags**. Added from the real site: downvotes, accepting an answer, reputation privileges, blocking accounts.

Non-functional: one vote per member per post, reversible reputation; search and top tags must not scan every question; concurrent votes each counted once; no voting on your own post and blocked accounts can do nothing; reputation, badge and channel rules change without touching voting code.

Use cases

Actors: **Guest** (search, view), **Member** (ask, answer, comment, vote, flag, bounty, tag, delete/undelete own posts), **Moderator** (close, reopen, delete, undelete *any* question), **Admin** (block/unblock), **System** (notifications, badges). In the course's use-case diagram, "Add question" `<<include>>`s "Add tag" and a bounty is an `<<extend>>`. The activity diagram for asking checks, per tag, "does it exist?" and if not "does the member have enough reputation to create it?". Our `ask()` follows the same order.

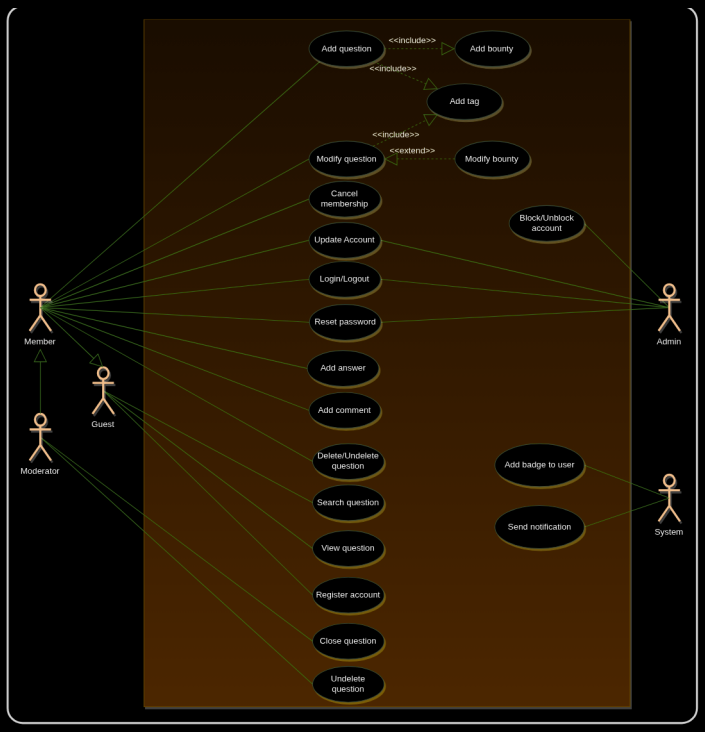


Figure — Use cases per actor: Guest, Member, Moderator, Admin, System (course diagram)

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, atomic reputation, badges, status; `hasPrivilege(p)`
`Privilege` (enum)	Reputation needed per action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status, closing remark, close/delete voters, bounty, views
`Answer` / `Comment`	Answer belongs to one question and can be *accepted*; comment belongs to a question or answer (vote/flag counts derived from sets)
`Tag`	Name + description; counts its own uses
`Bounty`	Staked reputation, sponsor, expiry, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: points per event
`ReputationListener` / `BadgeAwarder`	Observer: awards badges on reputation change
`Notifier` + `NotificationChannel` / `EmailChannel`	Pluggable delivery
`Search` / `QuestionIndex`	Inverted index by word and tag
`StackOverflow`	Facade: status + privilege checks, delegation, reputation

Class diagram

The course builds the diagram in five stages:

1. **`Badge`**, **`Tag`**, **`Notification`** (reqs 8, 10, 13). `Tag` counts its own uses, so "top tags" is a quick read. `EmailNotification` extends abstract `Notification` (passes the substitution test).
2. **`Photo`**, **`Bounty`** (req 7). The bounty owns the staked amount. The course says it can go back to the sponsor; on the real site it is **not** refundable, so our code deducts it at once and never returns

it.

3. **The posts** — the heart. Voting and flagging work the *same* on three classes, so that lives in abstract `Post`; what they *contain* differs, so there are three subclasses, not one class with a "kind" field. `Post` remembers **who** voted. Filled diamonds (composition): deleting a question deletes its answers, comments, photos, bounty. `Tag` is a plain association — it outlives any question.
4. **Search** — `Search` interface + `QuestionIndex`, not a static method on `Question`.
5. **Accounts** — `Guest` ← `Member` ← `Moderator` / `Admin`. Requirement 1 is enforced by *shape*: a guest has no `createQuestion` method. `Member` owns its badges.

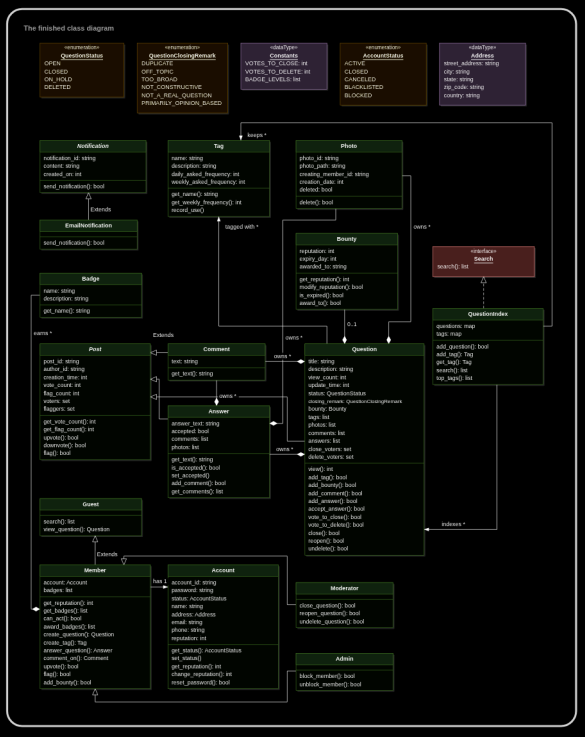
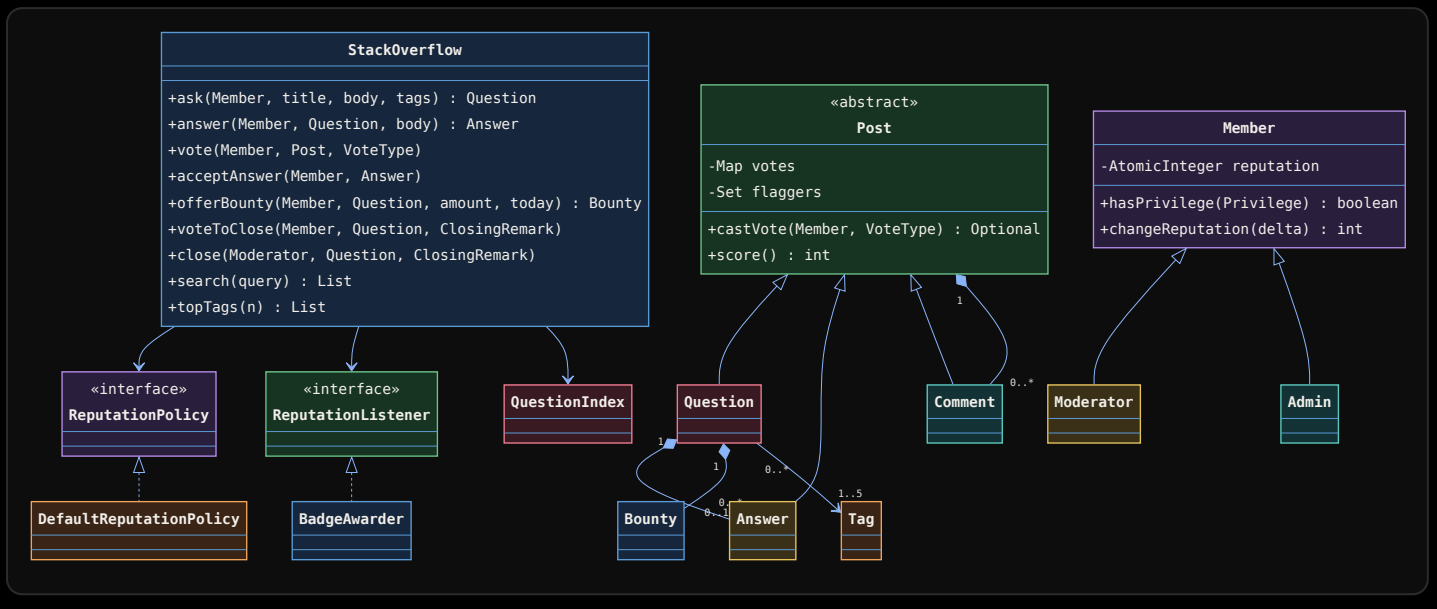


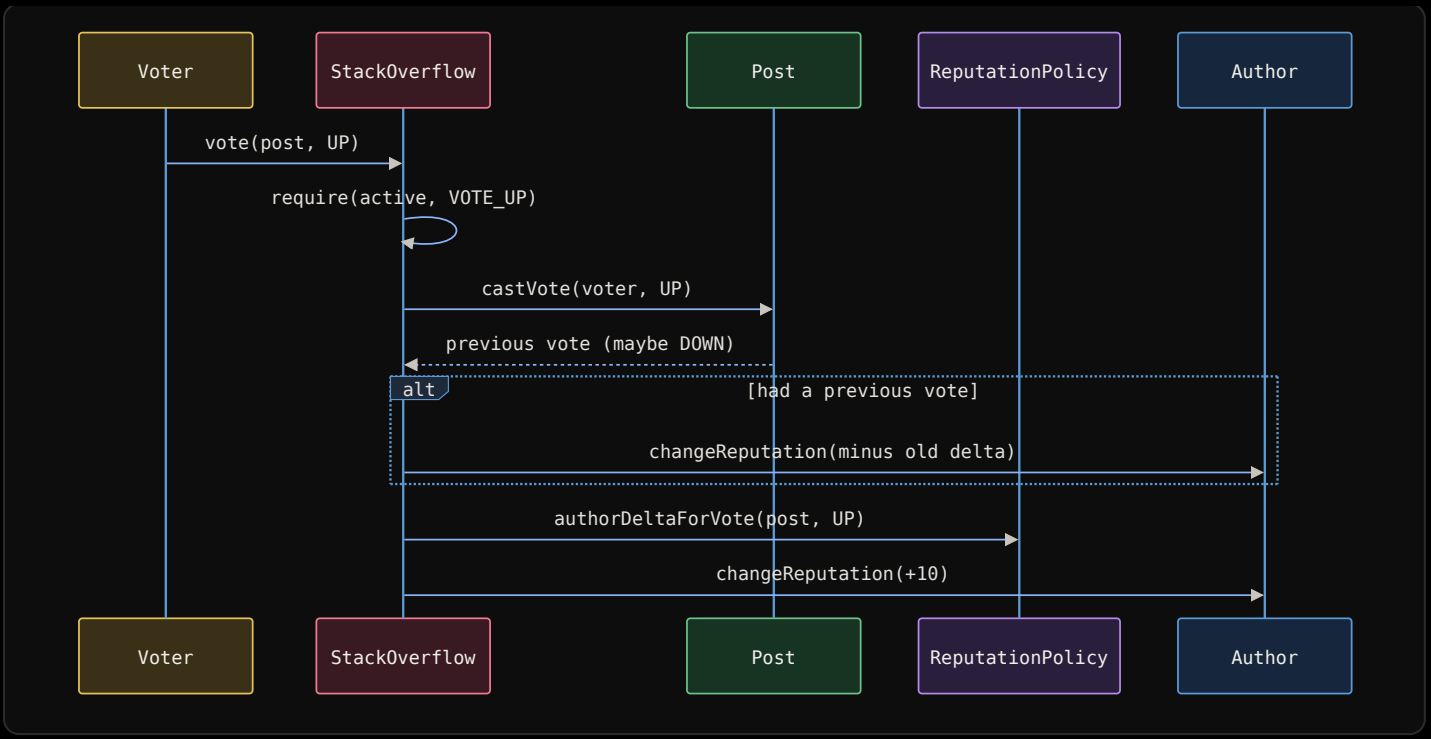
Figure — The finished Stack Overflow class diagram (course diagram)

Our Java design keeps this structure and adds a reputation **Strategy**, a badge **Observer** and a data-driven **privilege table**. A guest is simply "no `Member` object": every write needs a `Member` argument. We also add the downvote cost the course leaves out; edit history and duplicate links are extensions; chat is out of scope.



Key flows

Voting (with a changed vote). The facade checks privilege, asks the post to record the vote; the post returns the voter's *previous* vote so the facade can reverse its reputation effect before applying the new one; listeners (badges) are told.



Question lifecycle: OPEN → CLOSED (3rd close vote or moderator) → OPEN (moderator reopen); CLOSED → DELETED (3rd delete vote or moderator); OPEN → DELETED (moderator); DELETED → CLOSED (undelete). No new answers while closed.

Bounty: `offerBounty` checks `SET_BOUNTY` and the 50–500 range, attaches it to the open question and deducts the amount immediately; `awardBounty` (sponsor only, not their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base for question/answer/comment	Inheritance, `sealed`	Voting, flagging, commenting identical	One class + `type` field → `if (type == …)` everywhere
Votes as `Map<voterId, VoteType>`	Store who, not how many	Blocks double votes; allows reversal	`int voteCount`
`ReputationPolicy`	**Strategy**	Scoring rules change (they did in 2019)	Hard-coded numbers in `vote()`
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications react without the facade knowing	Calling `awardBadges()` inside `vote()`
`Privilege` enum with thresholds	Table-driven	One place for rules	`if (rep >= 125)` scattered
Moderator-only methods take `Moderator`	Type-level permission	Cannot call `close(…)` without a moderator	Forgettable runtime role check
`QuestionIndex` separate	SRP	Swap for Elasticsearch behind `Search`	Static `Question.search()`
`Tag` counts its uses	Aggregate at write time	Top tags = sort a small collection	Scan all questions
`StackOverflow` facade	**Facade**	One entry that validates, delegates, logs	UI calling entities directly
Pattern-matching `switch` over sealed `Post`	Exhaustive dispatch	Compiler forces every post kind	`instanceof` chains

Java implementation

Package `lld.ch18.stackoverflow` (`Types.java`, `Accounts.java`, `Posts.java`, `Reputation.java`, `QuestionIndex.java`, `StackOverflow.java`, `Main.java`). The core: vote storage, the policy, the badge observer and the facade's `vote`.

```
// File: lld/ch18/stackoverflow/Posts.java (excerpt)
abstract sealed class Post permits Question, Answer, Comment {
    // ...
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();
    // ...
    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on this
post");
        return Optional.ofNullable(previous);
    }

    synchronized int score() {
        return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum();
    }
    // ...
}

// File: lld/ch18/stackoverflow/Reputation.java (excerpt)
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                        // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) {
        return post instanceof Answer && type == VoteType.DOWN ? -1 : 0;
    }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}

final class BadgeAwarder implements ReputationListener {
    // ... levels: 15 Contributor, 50 Helpful, 125 Trusted
    @Override public void onReputationChanged(Member m, int oldValue, int newValue, String reason) {
        for (Level level : levels) {
            if (oldValue < level.reputation() && newValue >= level.reputation() && m.addBadge(level.badge())) {
                notifier.send(m, "You earned the '" + level.badge().name() + "' badge");
            }
        }
    }
}

// File: lld/ch18/stackoverflow/StackOverflow.java (excerpt)
    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can only
be upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        log(voter.name() + " voted " + type + " on " + post.label() + " -> score " + post.score());
        previous.ifPresent(p -> {                                   // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }
```

Full runnable code: code/lld/ch18/stackoverflow/ (see the Full Edition).

Output (excerpt) — a changed vote reverses the old effect first:

```
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
```

Key points: `votes.put` both records the vote and returns the previous one (voting the same way twice throws). The sealed `Post` makes the policy's `switch` exhaustive. `Member.changeReputation` uses `AtomicInteger.updateAndGet` with a floor of 1. `BadgeAwarder` fires only when a threshold is *crossed*, and `Set.add` stops duplicate badges. Alice (200 rep) cannot create the new tag `hashmap`. The third close vote closes the question; a moderator reopens it.

Concurrency & edge cases

- **Simultaneous votes:** `castVote`, `score`, `flag` are `synchronized` per post; reputation is `AtomicInteger`, so both changes apply.
- **Vote and reputation not in one lock:** a reader may briefly see the vote before the reputation — acceptable. In a database: one transaction plus a **unique constraint** on `(post_id, voter_id)`.
- **Reputation floor of 1** makes reversal not perfectly symmetric (same quirk as the real site); keep a ledger of events if exactness matters.
- **Accepting a different answer:** `Question.accept` returns the previous one so its +15 is removed; the asker's +2 is given once.
- **Self-actions** (vote own post, award own bounty) rejected; **closed** questions take no answers, **deleted** ones are hidden from search; **blocked** members fail `requireActive`; **bounty** award refused after expiry.

Extensions & follow-up questions

- **Stop double voting** — the per-post voter map; in a DB, a unique key on `(post_id, voter_id)`.
- **Edit history** — immutable `Revision`s appended per edit (close to **Memento**).
- **Daily reputation cap (200/day)** — a `CappedReputationPolicy` **decorator**; no voting code changes.
- **Duplicate link** — `duplicateOf` on `Question` set when closed as `DUPLICATE`.
- **Review queues** — flagged posts go to a `ReviewQueue` where reviewers vote keep/delete.
- **Hot questions** — ranking Strategy over recent votes/answers/views with time decay.
- **Tag frequency per day/week** — counters per time bucket rolled over by a scheduled job.

Real-world example

The numbers are real stackoverflow.com rules (privileges page: 15, 50, 75, 125, 1,500, 2,000, 3,000, 10,000). The 2019 change of question upvotes from +5 to +10 touches only the `ReputationPolicy`; lowering close votes from 5 to 3 after a 2019 experiment is one constant in our code.

In the interview

Minutes	What to do
0–5	Actors and reputation rules; requirements
5–10	Use cases; "guest has no posting method"
10–20	Class diagram: `Post` + 3 subclasses, `Member` hierarchy, `Tag`, `Bounty`, index, facade
20–30	`vote()` and `acceptAnswer()` with reversal; question states
30–40	Code `Post.castVote`, the policy, facade `vote`
40–45	Concurrency and extensions

> **30-second answer:** "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation rules are a Strategy; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter. Search is a separate inverted index, and tags count their own uses for 'top tags'."

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support changes; derive it from the voter map.
- *Is reputation awarded inside `upvote`?* — No; the facade asks the policy, so posts know nothing about scoring (the course's suggested improvement).
- *Scale search?* — Keep `Search`, replace `QuestionIndex` with a search engine fed by events.

Self-check

- **Q:** Which requirement is enforced by class shape? **A:** Guests cannot post or vote — no `Member` object, and every write needs one.
- **Q:** Why is `Post` abstract and sealed? **A:** A bare post is never created; sealed makes switches over post kinds complete.
- **Q:** Erin changes a downvote on an answer to an upvote — reputation? **A:** Old effect reversed (author +2, Erin +1), new applied (author +10).
- **Q:** How do comments differ in voting? **A:** Upvote only, and they earn no reputation.

TL;DR

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo

Cricinfo (ESPNcricinfo) is a cricket site with **live ball-by-ball commentary**, a database of historic matches, and news. This design is graded on whether you model the *rules*, not just the nouns. Cricket primer:

Term	Meaning
Delivery / ball	One ball bowled by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** or **no ball** gives a penalty run and is re-bowled — it does not count toward the six
Striker / non-striker	The striker faces; they change ends on an odd number of runs (1, 3, 5) and after every over
Runs	Off the bat (batter's) or **extras** — wides, no balls, **byes**, **leg byes** (the team's, not the batter's)
Wicket	Bowled, caught, LBW, stumped, hit wicket, run out, obstructing the field. **A run out is not the bowler's**
Innings	One team's turn to bat; ends all out (10 wickets), overs used up, or target reached
Formats	**T20** (20 overs/side), **ODI** (50 overs/side), **Test** (no limit, two innings per side)
Squad / playing eleven	A team names a squad per tournament, then picks exactly **eleven** from it per match

Problem & clarifying questions

Question	Answer we assume
Formats?	T20, ODI, Test
Who enters data?	**Admin** (scorer) records each ball; **Commentator** adds text
Real rules?	Yes: six legal balls, extras, wicket credit, strike rotation, bowler limits
Live updates?	Push to subscribers after every ball
Stats?	Per player, match, tournament, plus global ("highest wicket taker ever")
Tournaments?	Yes, with squads and points table
Out of scope?	Duckworth–Lewis–Stern, follow-on and declarations in Tests, powerplays, free hits, leg/off-side wides, partnerships, net run rate, news (our code adds a simple `declare()`)

Requirements

Functional (the course's eight): (1) track every team and its matches; (2) live ball-by-ball commentary; (3) follow international rules; (4) a squad per tournament; (5) a playing eleven per match from the squad; (6) stats for players, matches, tournaments; (7) global stat questions ("most wickets ever", "most Test hundreds"); (8) ODI, Test and T20.

Non-functional: stats exactly match the ball record (a correction fixes every derived number); live updates within seconds to millions; two scorers must not interleave balls; a new stat, format or subscriber is a new class.

Use cases

Admin adds/modifies teams, players, tournaments, matches, stadiums, umpires, referees and records balls (some stats system-generated); **Commentator** adds commentary. In the use-case diagram "Add match" is *extended* by the three formats and "Add ball" *includes* its runs and wicket. Activity "record a ball": choose ball type (normal, wide, no ball, wicket) → wicket details if any → a run record per run type → commentary → save.

Core entities

Class	Responsibility
`Person` (record)	Name, country; roles *hold* a person
`Player`; `MatchOfficial` → `Umpire`, `Referee`, `Commentator`	Roles; officials share "assign to match"
`Team` ⊇ `TournamentSquad` ⊇ `Playing11`	Everyone ever picked ⊇ one tournament ⊇ exactly eleven, in batting order
`Tournament`	Owns squads, lists matches, points table
`BallOutcome` (record)	Bat runs, extra type/runs, boundary?, wicket type, who was out, commentary; validates the laws
`Ball`, `Wicket` (records) + `WicketType`	Recorded delivery; `creditedToBowler()`
`Over`	Six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, over end, new batter, bowler limits, innings end
`Match` (abstract) → `T20`, `ODI`, `Test`	Formats override only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	Outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Live subscribers
`PlayerStats`, `StatsService`, `StatQuery` → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Derived stats; Template Method queries

Class diagram

The course builds it in six stages:

1. **People** — `Player` *holds* a `Person` (composition over inheritance: one human can play, then commentate). Officials share assignment in `MatchOfficial`.
2. **Teams, squads, eleven** — three nested sets. `Playing11` keeps its squad so it rejects outsiders and a twelfth pick: requirement 5 lives in code.
3. **The delivery** — three hidden rules: an over is six *legal* balls (two wides → eight deliveries); a **run out is not the bowler's wicket**; byes and leg byes are the team's. `Ball` owns runs, wicket, commentary; it only *refers* to bowler and batter.
4. **The innings** — owns the overs, so it alone decides when an over ends. `score` uses runs/wickets and overs.balls notation.
5. **Match and formats** — only two numbers differ (overs per innings, innings per match), so `Match` is abstract and each format overrides two short methods. Test returns 0 overs ("no limit") and four innings total.
6. **Stats** — **worked out from the balls**, never stored beside them (a stored counter drifts after a correction). Fifties/hundreds counted per innings (`countInnings`) beside `countBall`. Each global question is a small `StatQuery` subclass.

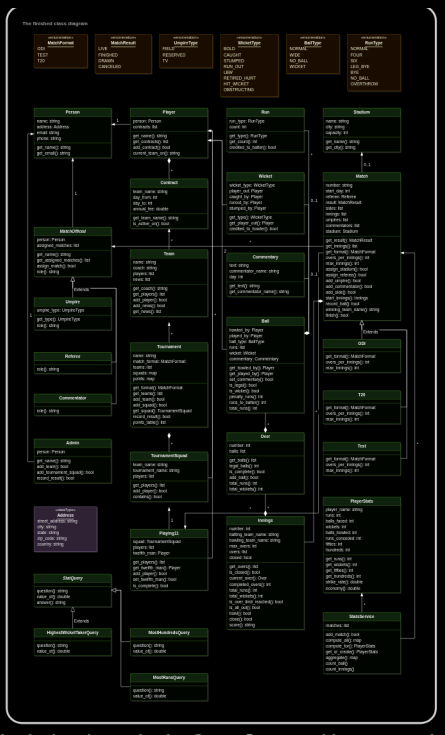
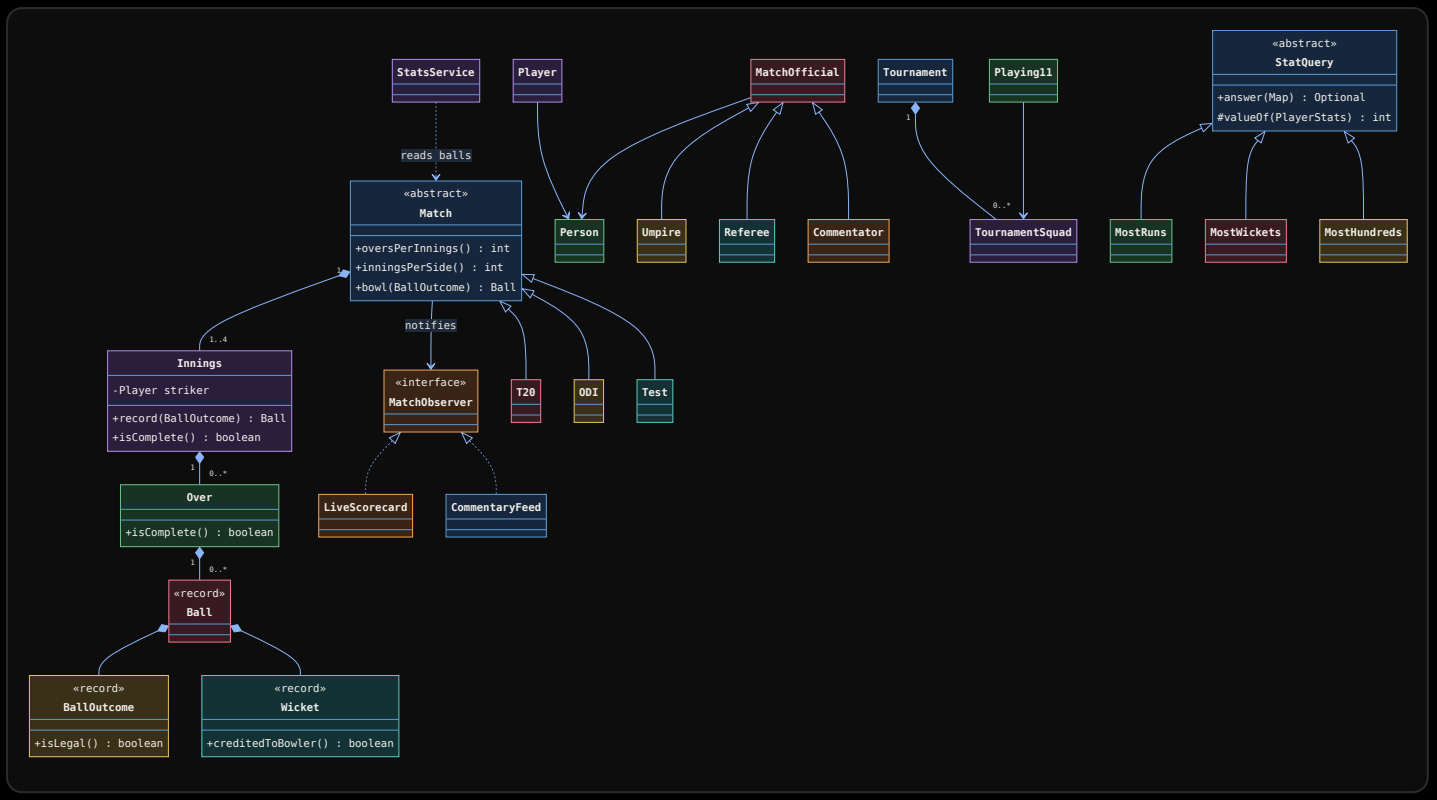


Figure — The finished Cricinfo class diagram (course diagram)

Our Java design adds what the course flags as missing: **strike rotation**, bowler rules (not two overs in a row; 4 overs per bowler in T20, 10 in ODI), which dismissals are legal off a wide/no ball, result calculation, and **Observers**. A `BallOutcome` record replaces separate `Run` objects.



Key flows

Recording a ball: scorer → `Match.bowl(outcome)` → `Innings.record`: build the `Ball` with current bowler and striker, add to the over, add runs; wicket → new batter; odd runs run → swap ends; over complete → swap ends. The match then notifies `onBall`, and `onOverEnd` / `onInningsEnd` if finished; after the last innings or target reached, `finish()` computes the `Result` and calls `onMatchEnd`.

Innings end (`Innings.isComplete`): all out (wickets = players − 1) → final innings and runs ≥ target → over limit reached and last over complete → declared.

Match status: SCHEDULED → LIVE → FINISHED; LIVE → DRAWN (Test out of time); SCHEDULED → CANCELED (washed out).

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Roles hold a `Person`	Composition over inheritance	One human, many roles	`Player extends Person`
`Match` abstract; formats override two numbers	Template-style base (not a named pattern here)	Only what differs is overridden	Three copies of `Match`
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	New statistic = one tiny subclass (OCP)	`switch (questionType)`
Stats derived from balls	Single source of truth	Corrections fix every number	Stored counters that drift
`Innings` owns over rule and strike	Information Expert	Class with the data decides	UI deciding when an over ends
`BallOutcome` validates in constructor	Fail fast, immutable	Bowled off a no ball never enters the record	Checking later in stats
`MatchObserver` (default methods)	**Observer**	Feeds subscribe without the match knowing	Match printing itself
`Result` sealed records	Algebraic data type	Exhaustive `switch`	Strings like "Team A won"
`Match.bowl` `synchronized`	Monitor lock per match	Scorers cannot interleave	Unsynchronised updates

Java implementation

Package `lld.ch18.cricinfo` (`People`, `Teams`, `Delivery`, `Innings`, `Match`, `Observers`, `Stats`, `Main`). The demo shows both eleven refusals, a scripted first over (a wide and a no ball make **eight deliveries**; a bowled that counts for the bowler; a run out that does not), then a seeded simulation of a T20 and an ODI.

```java
// File: lld/ch18/cricinfo/Delivery.java (excerpt)
enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;

    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}
// ... record BallOutcome: isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }
// ...
/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    // ...
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    // ...
}

// File: lld/ch18/cricinfo/Innings.java (excerpt)
    Ball record(BallOutcome o) {
        Over over = currentOver();
        if (over == null || over.isComplete()) throw new IllegalStateException("start a new over first");
        if (isComplete()) throw new IllegalStateException("innings is complete");

        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the next
number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();

        if (o.runsRun() % 2 == 1) swapEnds();             // odd runs: batters crossed
        if (wicket != null) {
            wickets++;
            if (!allOut()) {
                Player newBatter = batting.battingOrder().get(nextBatter++);
                if (wicket.batter().equals(striker)) striker = newBatter; else nonStriker = newBatter;
            }
        }
        if (over.isComplete()) swapEnds();                 // ends change after every over
        return ball;
    }

// File: lld/ch18/cricinfo/Stats.java (excerpt)
abstract class StatQuery {
    record Leader(Player player, int value) {}

    abstract String question();
    abstract int valueOf(PlayerStats s);

    final Optional<Leader> answer(Map<Player, PlayerStats> stats) {
        Leader best = null;
        for (PlayerStats s : stats.values()) {
            int v = valueOf(s);
            if (v > 0 && (best == null || v > best.value()
                    || (v == best.value() && s.player.name().compareTo(best.player().name()) < 0))) {
                best = new Leader(s.player, v);
            }
        }
        return Optional.ofNullable(best);
    }
}

final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
```

```
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}
```

Full runnable code: code/lld/ch18/cricinfo/ (see the Full Edition).

Output (excerpt) — the first over has eight lines but ends at `0.6`:

```
0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
0.5  Tarun to Arun, no run  [Ravi: defended]
0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
REJECTED: Tarun cannot bowl two overs in a row
```

Key points: ball numbers are "completed overs . ball"; an extra carries the number of the ball it replaces. Chetan is run out after one completed run: `record` swaps ends for the single first, then replaces the dismissed batter, so Dev arrives at the right end; Tarun's figures show only **1** wicket. Five bowlers rotate, so each bowls exactly 4 overs (`maxOversPerBowler() = 20 / 5`). The chase target is set automatically (`first-innings total + 1`). The T20 ends "Lions won by 23 runs"; the ODI "Lions won by 5 wickets". Global queries are one-line subclasses; "Most runs in ODIs" reuses the service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks for a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (Indian B2B e-commerce) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

- **Input:** players per team, number of overs, each team's batting order, then each over's balls: runs `0`–`6`, `W`, `Wd` or `Nb`.
- **After every over:** each batter's runs, 4s, 6s, balls (with `*` on the two at the crease), then total/wickets and overs.
- **Rules:** strike changes on 1s and 3s and at over end; wides and no balls add one run and are re-bowled; innings ends all out, overs done, or target passed.
- **At the end:** winner and margin (runs or wickets). **Good to have:** bowler figures, extras, strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough
`BallOutcome`	`BallEvent` sealed interface (`Runs`, `Wide`, `NoBall`, `Wicket`) with `parse(token)` factory	Only four token kinds
`Innings`	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data
Scorecard from balls	`Batter` counters + `ScorecardPrinter`	Printer separate so another view can replace it
`Match` + `Result`	`MatchRunner`	One format, two innings

The lesson: keep the *same responsibilities in the same places* (innings owns strike and the over rule; parsing, scoring and printing separate), but use the simplest data that satisfies the input.

Code

Package `lld.ch18.cricketdashboard` (`Model.java`, `Main.java`; `Main` feeds the sample input as a text block).

```java
// File: lld/ch18/cricketdashboard/Model.java (excerpt)
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}
    // ... static BallEvent parse(String token): "W", "Wd", "Nb..", or 0-6
    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}
// ...
    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());              // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;     // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();       // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }

    boolean isOver() {
        return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target);
    }
```

Full runnable code: code/lld/ch18/cricketdashboard/ (see the Full Edition).

```
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
...
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Our output matches the published sample except where the sample is inconsistent: after team 2's first over it prints `12/1`, but the over has **two** `W`s, so `12/2` is correct; and it later shows P6 with 2 balls after 3. Pointing this out calmly is a plus.
- Team 2's second over stops after four balls: 5 players → all out at 4 wickets ("Overs: 1.4").
- The over loop counts **legal** balls, so a wide makes an over seven tokens long.
- Extras and strike rate are printed; bowler figures would need bowler names in the input (add a `Bowler` class).

Concurrency & edge cases

- **Two scorers:** `bowl`, `startOver`, `startInnings` are `synchronized` per match; different matches never block each other.
- **Many readers:** observers in a `CopyOnWriteArrayList`. At web scale, publish each ball to a broker and push via WebSockets/SSE; the scorecard is a cached view rebuilt from balls.
- **Rejected sequences:** over while one is in progress, same bowler twice in a row, bowler over quota, bowling after the innings, bowler from batting side, twelfth player, non-squad player.
- **Impossible dismissals:** bowled/caught/LBW/stumped/hit wicket off a no ball; bowled/caught/LBW off a wide.
- **Wicket on the last ball:** new batter comes in, then ends change. **Tie** → `Result.Tie`. **Tests:** 0 = no over limit; `declare()` and `declareDraw()`.

Extensions & follow-up questions

- **Who is on strike?** — `Innings` tracks it (odd runs and over end swap).
- **Stats too slow** — a per-player running total updated by an Observer, rebuilt from balls after corrections; balls stay the truth.
- **Duckworth–Lewis–Stern** — a `TargetCalculator` Strategy (default: opponent total + 1).
- **Free hit** — innings remembers "next legal ball is a free hit"; only run-out-type dismissals allowed.
- **Partnerships / fall of wickets, net run rate** — derive from balls and innings totals.
- **Push notifications** — a new `MatchObserver` firing on wickets and 50/100 milestones.

Real-world example

ESPNcricinfo (founded 1993, bought by ESPN in 2007) is the reference for cricket stats; its **Statsguru** answers requirement-7 questions across all internationals. Live pages update ball by ball with commentary like our `CommentaryFeed`, for millions of phones — hence push-based Observers, brokers and caches.

In the interview

Minutes	What to do
0–5	Formats, actors; say early **"an over ends after six *legal* balls"**
5–10	Team ⊇ squad ⊇ eleven; people hold a `Person`
10–20	Match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball flow, strike rotation, **no bowler credit for a run out**, byes not the batter's
30–40	Derived stats; Template Method queries; Observers
40–45	One scorer lock per match, broadcast to readers; extensions

30-second answer: "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. The innings owns the overs, so only it decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and innings per match. Stats are derived from balls so corrections propagate, each stat question is a Template Method subclass, and live pages subscribe as observers."

- *Why not store `wickets` on `Player`?* — It drifts from the ball record and cannot be filtered by format; derive it.
- *Add "most catches"?* — Fielder on `Wicket`, count it in `PlayerStats`, `MostCatches extends StatQuery`.
- *Is `Match`/formats a Template Method?* — No; only `StatQuery` is a named pattern here — formats are plain abstract-class specialisation.

Self-check

- **Q:** A wide and a no ball in an over — how many deliveries? **A:** Eight.

- **Q:** Who gets a run-out wicket? **A:** The fielding side; not in the bowler's figures.
- **Q:** Are leg byes the batter's runs? **A:** No — team extras, not charged to the bowler either.
- **Q:** What distinguishes `T20`, `ODI`, `Test`? **A:** `oversPerInnings()` (20, 50, 0) and `inningsPerSide()` (1, 1, 2).
- **Q:** Correct team 2 score after over 1 in the Udaan sample? **A:** 12/2.

TL;DR

Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball`. Run outs are not the bowler's; byes are not the batter's. Stats derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn

LinkedIn is a networking site for **working lives**: profiles with skills, jobs and schooling, and a home **feed** of updates from your network. It is like Facebook with career features: **connections** (mutual), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question	Answer we assume
Is a connection mutual?	Yes: an **invitation** is accepted or rejected. Following is one-sided
Degrees?	1st, 2nd, 3rd; beyond that "out of network"
Messaging?	Only connections (non-connections need paid *InMail*, out of scope)
Endorsements?	Only 1st-degree connections, once per skill
Recommendations?	*Requested* from a connection; only that person writes it
Profile stats?	Views, connections, search appearances
Companies/jobs?	Members create pages; page admins post jobs; apply once per job
Feed?	Posts from connections and followed members, ranked (course omits it; we add one)

Requirements

Functional (the course's twelve): (1) profile with details, jobs, schooling, skills, accomplishments; (2) search members, companies (and jobs); (3) send/accept/reject connection requests; (4) request recommendations; (5) profile stats; (6) posts to connections; (7) comment on posts, like/share posts **and comments**; (8) messages; (9) notifications for new messages, invitations, comments on your posts; (10) company pages and job postings; (11) create and join groups; (12) follow members or companies.

Non-functional: a connection always appears on **both** members; fast reads (feed, search, degrees); blocked members' content disappears and no spamming strangers; new channels and ranking rules plug in.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search and connect, follow/unfollow, message connections, post/like/comment, notify. Activity diagrams: *add experience* (enter details, optional media, verify, save or ask for missing data) and *send message* (recipients, body, media, verify, deliver, email + push to each recipient).

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers
`Recommendation`	Born empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	Counters bumped by whoever witnesses the event
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments and may be shares
`Message` (record)	Not social content — nobody likes a message (text only in our code)
`ConnectionInvitation` + `InvitationStatus`	PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; BFS degree; mutuals; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

Six course stages:

1. **Profile** — each part is a small class because other code uses it. `Recommendation` is created *without* text (requirement 4 is about asking). The profile never bumps its own `ProfileStats` — whoever sees the event does (an object cannot count events it never sees). All composition.
2. **Posts, comments, messages** — *comments get likes too*, so both extend `SocialContent` holding author, text and **liker ids**. `Message` stays apart.
3. **Invitations and notifications** — an invitation **changes state over time**, so it is its own object that can remember "no". Notification kind is a **field**: **subclass when behaviour differs; field when only data differs.**
4. **Companies, jobs, groups** — a page owns its postings; followers are ids; a group's creator can never leave.
5. **Actors and `ConnectionService`** — accepting edits **two** members, so it belongs to neither; one service keeps connections **symmetric**. Following is one-sided: just a set on the follower.
6. **Search** — the course's real bug: the index stored *one* member per name and broke on the second Sharma. Rule: **an index value is always a collection.** Keys are lowercase words.

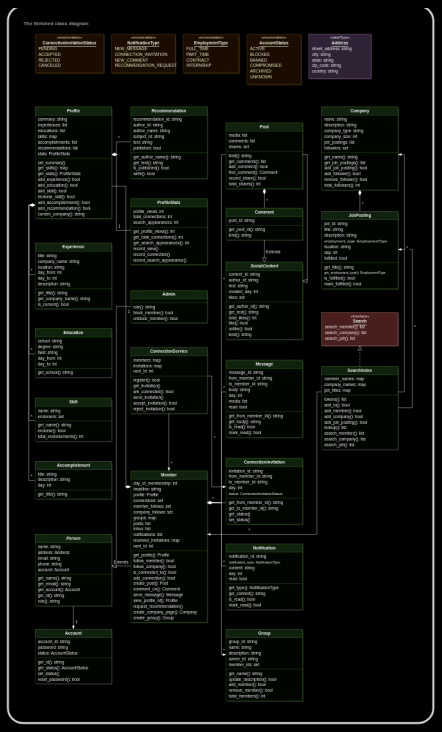
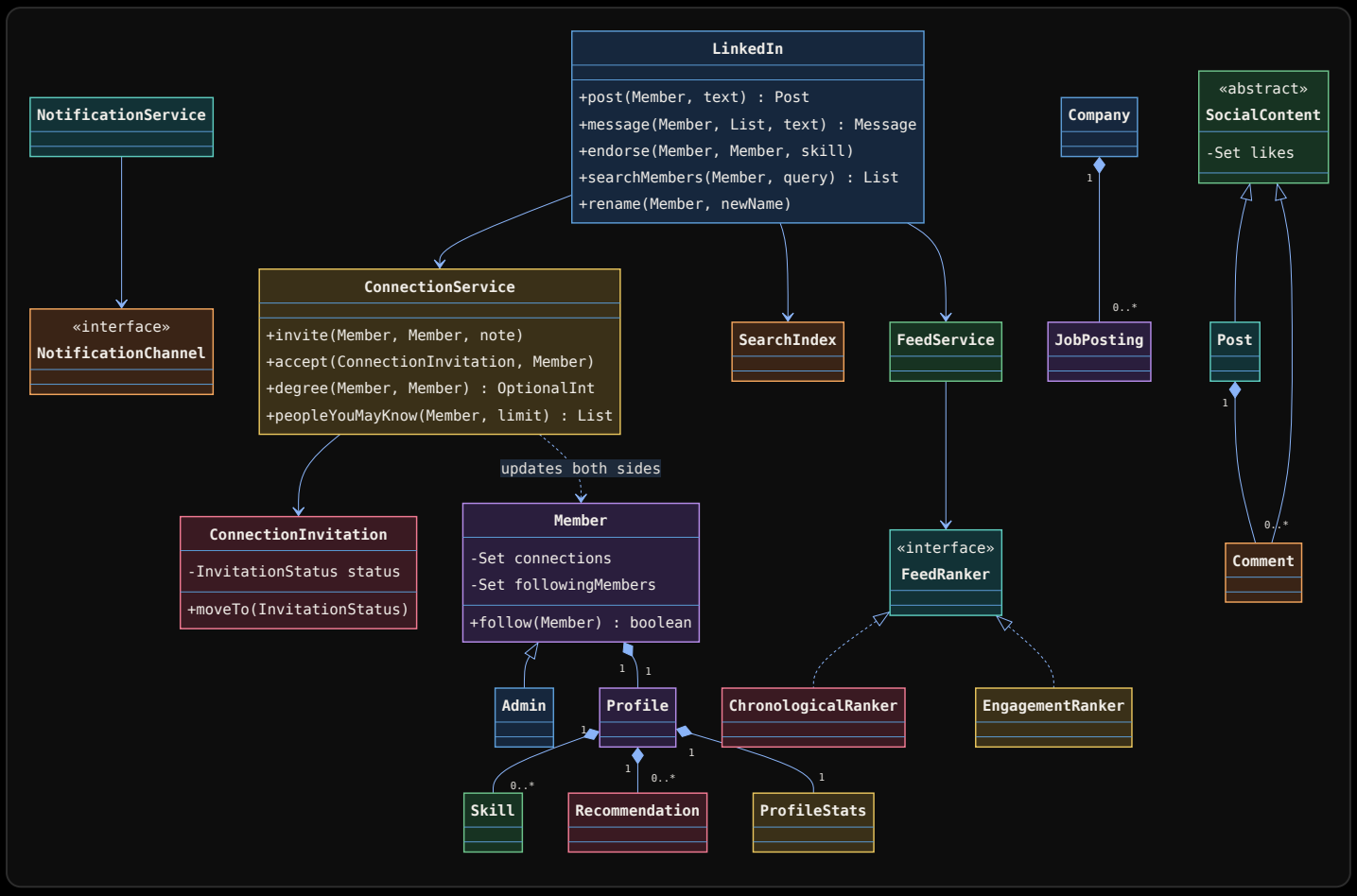


Figure — The finished LinkedIn class diagram (course diagram)

The course omits the feed and ranking, suggestions, degrees, job applications, privacy controls and member-to-member blocking, and admits two weak spots: no feed (pull at load vs copy on write — depends on typical connection count) and a search index that goes stale on rename. **Our Java design** fills these: a pull **feed** with a ranking Strategy, **BFS degrees**, **people you may know**, a **rename that updates the index**, job **applications**, and a connection count derived from the set.



Key flows

Invite and accept: `invite` checks not-self, not-connected, and nothing pending *either way*, creates a PENDING invitation and notifies the receiver. `accept` locks the graph, moves the invitation to ACCEPTED, and adds each member to the other's set together.

Invitation states: PENDING → ACCEPTED (receiver accepts) / REJECTED (receiver rejects) / CANCELED (sender withdraws); all final.

Degree of connection: BFS (all 1-hop neighbours, then 2-hop…) from the viewer, stopping at depth 3: Asha → Meera 1st, Karan 2nd, Zoya 3rd, Ravi out of network.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both under one lock	Service for two-object operation	Symmetry has one guardian	`member.accept(inv)` editing the other
Following = set on follower	Simplicity	One-sided, no agreement	Reusing invitations
Invitation with explicit states	**State** (enum + allowed transitions)	Accept-twice / accept-after-reject rejected	Boolean `accepted`
`SocialContent` base	Inheritance	Comments are liked too	Likes only on `Post`
One `Notification` with `type`	Field when only data differs	Delivery identical	Subclass per event
`NotificationChannel`	**Strategy**	In-app, email, push interchangeable	`if (email) … else …`
`FeedRanker`	**Strategy**	"Recent" vs "Top" at run time	Sorting inside the service
Index key → set	Robust inverted index	Both Sharmas found	Key → single member
`rename` tells the index	Sync derived data at the write point	Search never stale	Nightly rebuild
Count = `connections().size()`	Derive, don't store	Cannot disagree	Separate counter

Java implementation

Package `lld.ch18.linkedin` (`Profile`, `Content`, `Network`, `Pages`, `Services`, `LinkedIn`, `Main`). The heart is `Network.java`:

```java
// File: lld/ch18/linkedin/Network.java (excerpt)
enum InvitationStatus {
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}
// ... ConnectionInvitation:
    synchronized void moveTo(InvitationStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException("invitation is already " + status);
        status = next;
    }
// ...
final class ConnectionService {
    // ...
    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already
connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and " +
to + " is already pending");
        ConnectionInvitation inv = new ConnectionInvitation(from, to, note);
        invitations.put(inv.id(), inv);
        notifications.send(to, NotificationType.CONNECTION_INVITATION, from + " wants to connect: \"" + note +
"\"");
        return inv;
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }
    // ...
    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                             // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }
    // ... mutualConnections, peopleYouMayKnow
}
```

Full runnable code: code/lld/ch18/linkedin/ (see the Full Edition).

```
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Key points: Ravi's counter-invite is refused (pending *either* direction). A second accept fails because only PENDING can move. `peopleYouMayKnow` counts mutuals for 2nd-degree members, skips pending invitations, sorts by count then name. `endorse` needs a 1st-degree connection and rejects repeats via the endorser set; `Recommendation.write` accepts only the named author, once. `rename` removes old words before adding new ones. `FeedService` is a **pull** feed: Karan sees Asha's tip because he *follows* her.

Concurrency & edge cases

- **Symmetric connections:** all graph changes are `synchronized` on the one service. Distributed: write both edges in one transaction, or one row keyed by the ordered id pair.
- **Double invitations** either way refused while pending.
- **Likes, endorsements, applications:** id-keyed sets/maps with `synchronized` mutators.
- `Member` **collections** are concurrent and getters return copies.
- **Stale search:** handled by `rename` — "whatever changes the data tells the index" (events in production).
- **Blocked members:** writes refused, posts vanish from feeds. **Validation:** experience ending before it starts, blank posts rejected.

Extensions & follow-up questions

- **Feed for 900 million members — fan-out on write (push)**: append post ids to each connection's precomputed feed (fast reads, costly for huge followings). **Fan-out on read (pull)**: assemble on open (ours; cheap writes, slower reads). **Hybrid**: push for ordinary members, pull for very popular accounts, merged at read.
- **People you may know at scale** — compute offline (friends-of-friends + shared companies, schools), store, read live.
- **Connection limit** — 30,000; enforce in `accept`.
- **InMail** — a premium credit balance spent by `message`.
- **Who viewed my profile** — store view events, derive the count.
- **Job recommendations** — a Strategy matching skills and title to postings.

Real-world example

LinkedIn shows 1st/2nd/3rd-degree badges in search (our BFS depth) and "People You May Know" from shared connections. Messaging non-connections usually needs paid **InMail**. Its event stream gave rise to **Apache Kafka**, open-sourced in 2011.

In the interview

Minutes	What to do
0–5	Connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent`, invitation with state
15–25	**Say it:** accept changes two objects → `ConnectionService`; follow needs no service
25–35	Search (key → list), BFS degrees, people you may know
35–45	Feed pull/push/hybrid, notification Strategy, concurrency

30-second answer: "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both members, so it lives in a `ConnectionService` that updates both sides under one lock; following is just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is pull with a pluggable ranking Strategy, moving to a push/pull hybrid at scale."

- *Why not store* `connectionCount`*?* — It is `connections.size()`; a copy drifts.
- *2nd-degree badge for every search result quickly?* — Cache each member's 1st-degree set; a result is 2nd-degree if its set intersects yours.

Self-check

- **Q:** Why is accepting not a `Member` method? **A:** It updates two members; a service keeps both consistent.

- **Q:** Connect vs follow? **A:** Mutual with acceptance vs one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** It is requested first; the author fills it later.
- **Q:** Which feed model is `FeedService`, and its cost? **A:** Pull; cheap writes but every read scans connections' posts.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook

Facebook: **friends** (mutual), **follows** of people and **pages**, **groups**, posts. New challenges versus LinkedIn: **privacy** (who may see each post), a real **news feed**, **pages** with ratings, public/private **groups**, and **friend suggestions**.

Problem & clarifying questions

Question	Answer we assume
Friend vs follow?	Friendship mutual (request + accept); following one-sided
Who sees a post?	Author picks an **audience**: Public, Friends, custom **privacy list**, Only me
Privacy list?	A named subset of the author's friends ("Close friends")
Pages?	Created by members; followed; **recommendations** (1–5 rating + text)
Groups?	Public (join at once) or private (owner approves; posts visible only to members)
Search?	Members, groups, pages by name; posts by word — respecting privacy
Feed?	Friends, followed members/pages, joined groups — privacy-filtered, ranked
Extended?	Suggest people you may know

Requirements

Functional (twelve + extended): (1) profile — details, jobs, schooling, places; (2) search members, groups, pages; (3) friend requests; (4) follow without friendship; (5) create groups/pages, join groups, follow pages; (6) posts to friends; (7) comments; posts **and comments** liked or shared; (8) privacy lists; (9) messages; (10) page recommendations; (11) notifications for messages, friend requests, comments; (12) search posts by word; (13) **extended:** suggest connections.

Non-functional: privacy never leaked (feed, search, comments, likes, shares); fast feeds for hundreds of friends; symmetric friend graph; stable (non-reshuffling) suggestions and feeds.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search, friend requests, follow/unfollow, message friends, post/like/comment on visible posts, notifications, create page, recommend page. Activity *create post*: text, optional media, **choose the privacy list**, verify, publish to that audience, notify.

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows, groups, posts, privacy lists, inbox, notifications
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as set; posts own comments; may belong to a group/page or be a share
`FriendRequest` + `FriendService`	Friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)`
`Page` (+ `PageType`, `PageRecommendation`)	Followers, one recommendation per member, average rating
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, posts
`NotificationService` + `PushChannel` / `EmailChannel`	Routes each type to channels
`SearchIndex`	Names; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`; `SuggestionService` + `Suggestion`	Feed; friends-of-friends
`Facebook`	Facade

Class diagram

Five course stages plus the extended requirement:

1. **Profile** — `Work`, `Education`, `Place` in lists; every field has a **default** so a new account already works.
2. **Posts, privacy, messages, notifications** — fixes three old bugs: **store likers, not a like count** (an old constructor took the count); **comments get likes too**; **the post checks its own privacy list** — but it cannot know friendships (stage 5 supplies that). `Post` → `PrivacyList` is `0..1`; one list serves many posts. `Notification` abstract with a subclass per delivery method.
3. **Pages and groups** — *no member-count field* (an old version stored both count and members). Rule: **if a number can be computed by counting a collection, keep the collection and count it.**
4. **Actors and friend graph** — an old invitation forgot the *sender*. Acceptance changes two members → `FriendService`, which owns the graph and answers "friends?" and "pending?". Following is a set, no service.
5. **Search** — **`searchPost` takes the viewer**: privacy applies in search. Visibility needs the post's list *and* the friend graph; neither answers for the other. Keys map to collections.
6. **Extended — suggestions** — friends of friends **scored by mutual friends**; never yourself, a friend, or anyone with a request pending **in either direction**; sort by mutual count then id so equal scores don't reshuffle. Arrives as two new classes, no edits (OCP).

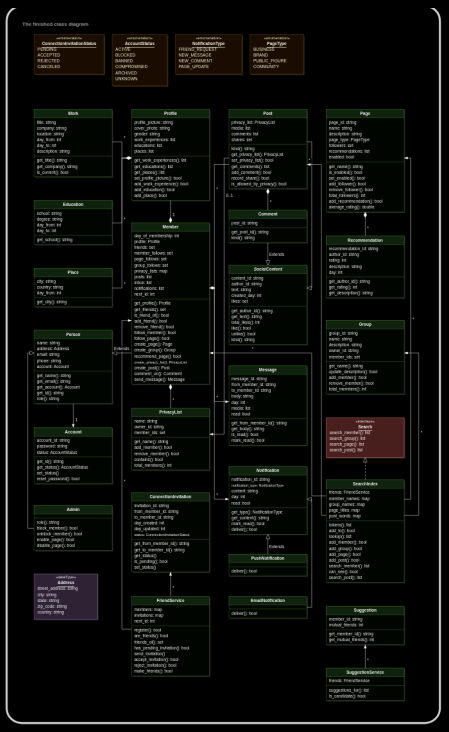
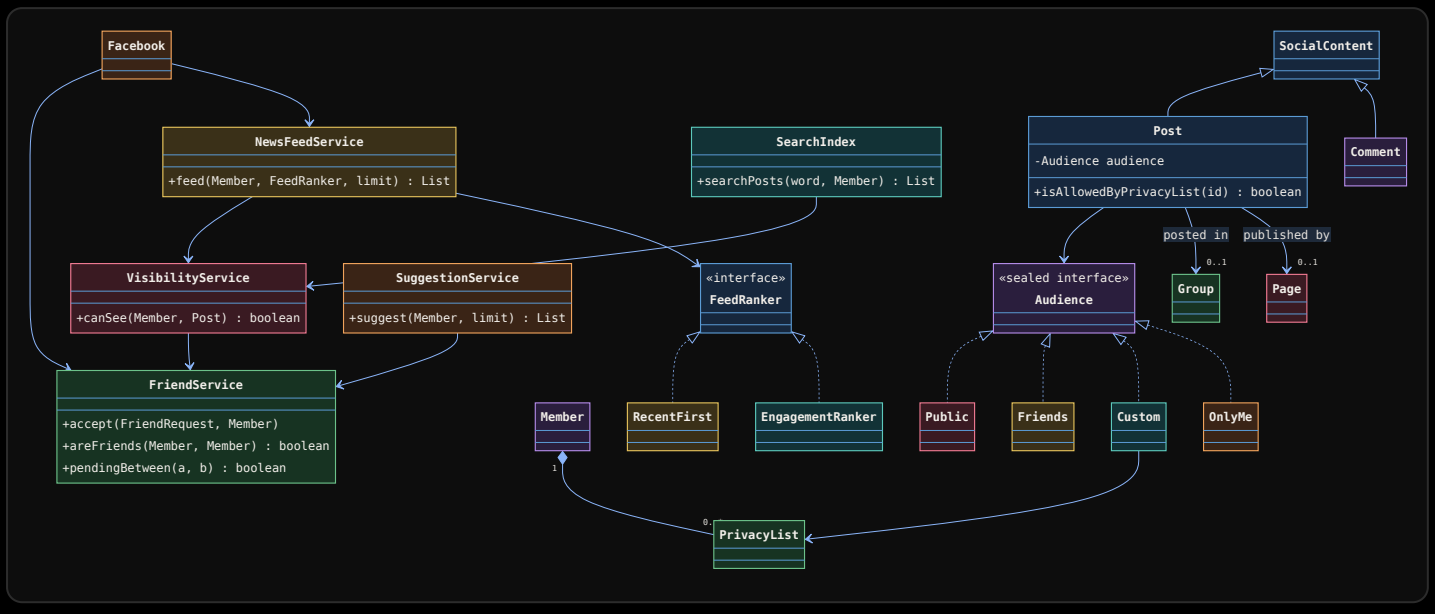


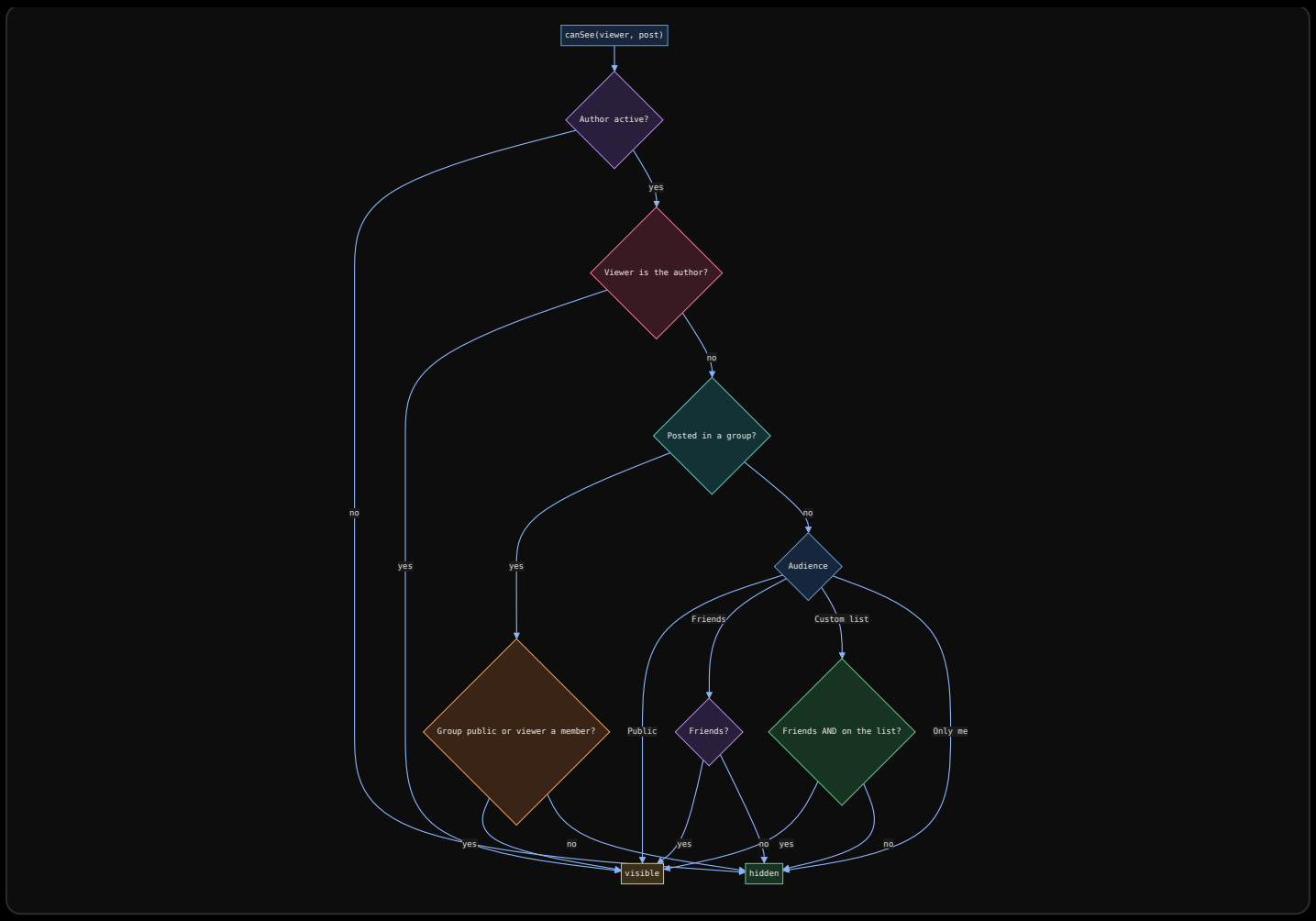
Figure — The finished Facebook class diagram (course diagram)

Our Java design adds a sealed `Audience`, one **`VisibilityService`** used by feed, search, comments, likes and shares, a **`NewsFeedService`** with ranking Strategies, private groups with approval, and page posts in the feed. The course deliberately omits the feed, member blocking, non-like reactions, tagging, events, marketplace, stories and albums with separate privacy; we add the feed and sketch the rest under Extensions.



Key flows

Can this viewer see this post? (feed, search, comment, like, share):



News feed (pull): friends + followed members + followed pages + joined groups → collect posts, drop own and duplicates → filter with `canSee` → sort with the ranker (recency, or engagement × affinity / age) → top N.

Friend suggestions: for each friend F, for each friend C of F: skip if C is me, already a friend, or pending either way; else `mutual[C] += 1`; sort by mutual desc, then id; take top N.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` everywhere	Single policy point (Specification-like)	Search and feed cannot leak	Checks per screen
Sealed `Audience` records	Algebraic data type, exhaustive `switch`	New audience forces every decision to handle it	Int/string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Information Expert	Each answers what it knows	Post holding the whole graph
`FriendService` owns the graph	Service for two-sided ops	Symmetry; "pending either way" in one place	`member.acceptFriend()`
Likes as a set; no stored counts	Store who; derive, don't store	No fake/double likes; counts can't drift	`likeCount`, constructor counts
`NotificationService` routes types → channels	**Strategy** + config map	Friend requests push + email; comments push only	`if/else` on type
`FeedRanker`	**Strategy**	Recent vs ranked	Ranking inside the feed
`SuggestionService` added without edits	OCP	Extended requirement = two new classes	Fields on `Member`
`Facebook`	**Facade**	One entry, status checks	UI calling every service

Java implementation

Package `lld.ch18.facebook` (`Model`, `PagesAndGroups`, `Services`, `Facebook`, `Main`).

```java
// File: lld/ch18/facebook/Model.java (excerpt)
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}
// ... in Post:
    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }

// File: lld/ch18/facebook/Services.java (excerpt)
final class VisibilityService {
    private final FriendService friends;
    VisibilityService(FriendService friends) { this.friends = friends; }

    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) &&
post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}
// ...
    List<Suggestion> suggest(Member me, int limit) {
        Set<Long> mine = friends.friendsOf(me.id());
        Map<Long, Integer> mutual = new HashMap<>();
        for (long f : mine) {
            for (long candidate : friends.friendsOf(f)) {
                if (candidate == me.id() || mine.contains(candidate) || friends.pendingBetween(me.id(),
candidate)) continue;
                mutual.merge(candidate, 1, Integer::sum);
            }
        }
        return mutual.entrySet().stream()
                .map(e -> new Suggestion(members.apply(e.getKey()), e.getValue()))
                .sorted(Comparator.comparingInt(Suggestion::mutualFriends).reversed()
                        .thenComparingLong(s -> s.member().id()))           // stable tie-break
                .limit(limit)
                .toList();
    }
```

Full runnable code: code/lld/ch18/facebook/ (see the Full Edition).

```
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
after inviting Grace: [Dave Paul(2 mutual)]
```

Key points: the visibility table is the course's key scenario — Bob is a friend *and* on "Close friends", Erin only a friend, Frank neither. Search respects privacy because `searchPosts` takes the viewer. `createPrivacyList` refuses non-friends. Comments, likes and shares also call `canSee` (otherwise "comment" is a side door). Pages allow one recommendation per member (`putIfAbsent`); count and average are

computed. A private group's join waits for approval and its posts are hidden from non-members, even in search. Once Frank is blocked his posts vanish from feeds. Notification routing is a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject`, `unfriend` `synchronized`; concurrent adjacency sets. In a DB, one row keyed by `(min(id), max(id))` makes symmetry automatic.
- **Pending both ways:** `pendingBetween(a, b)` for requests and suggestions.
- **Privacy changes after posting:** visibility computed at read time, so unfriending hides posts at once (a push feed must still re-check `canSee` on read).
- **Shares:** need to see the original; production should cap the share's audience at the original's.
- **Deterministic ordering:** suggestions tie-break by id, feeds by creation time; a logical `Tick` clock makes output repeatable.
- **Blocked/deleted authors:** `canSee` returns false.

Extensions & follow-up questions

- **Scale the feed** — fan-out on write vs on read; mix (push for normal users, pull for celebrities), always re-check privacy.
- **Better ranking** — affinity, content type, recency decay behind `FeedRanker`: early **EdgeRank** (affinity × weight × time decay).
- **Suggestions at scale** — precompute offline (the original diagram's `memberSuggestions` field), add shared groups/schools/work.
- **Member blocking** — per-member block set checked both ways by `canSee` and `sendRequest`.
- **Reactions** — `Map<memberId, Reaction>`, like Stack Overflow's vote map.
- **Tagging** — tagged ids on `Post`, notify, optionally widen audience.
- **Events, stories, marketplace, albums** — new `SocialContent` types with their own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006. Posts carry an audience selector, a profile is capped at **5,000 friends** (then followers or pages), and the graph is served by **TAO**, a distributed store for objects and associations.

In the interview

Minutes	What to do
0–5	Friend vs follow, audiences, pages, groups, feed
5–15	Profile, `SocialContent`, `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph; follow is a set
25–35	**Strongest point:** `canSee` consults post list + friend graph; every read path calls it
35–45	Feed models, suggestions, concurrency

> **30-second answer:** "Posts and comments share a likes set. A post has an audience — public, friends, a custom list, or only me. The post checks its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

- *How could a private post leak?* — Any read path that forgets privacy: search, feed, comments, shares, notifications.

- *Why the second sort key?* — Equal mutual counts would swap between loads; a self-reshuffling list looks broken.

Self-check

- **Q:** Erin is a friend but not on "Close friends" — can she see a close-friends post? **A:** No; friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Privacy applies inside search.
- **Q:** Who is excluded from suggestions? **A:** Yourself, existing friends, anyone pending either way.
- **Q:** Push vs pull feed weaknesses? **A:** Push is expensive for huge audiences; pull reads are slower.

TL;DR

Facebook = LinkedIn's structure + **privacy**: sealed `Audience`, privacy lists of friends, one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The feed gathers, filters, ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending both ways, stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page & group posts
User relationship	None (votes, reputation)	Team membership, officials	Connection + follow	Friendship + follow
Two-sided op → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Roles	Reputation privileges + Moderator/Admin	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
Derive, don't store	Score, tag counts	All stats from balls	Connection, follower counts	Member/follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator, extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners	Match observers	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets — cannot stop duplicates.
- Stored counts beside the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Index values that are a single object instead of a list.
- Modelling nouns but not rules (six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index mapping words to lists; and write each permission or privacy rule once, then call it from every read path.